

Understand the code

Student Attendance & Performance Tracker

Complete project file guide

Kotlin / Jetpack Compose / Google Apps Script / Google Sheets

78 text files and 11 binary resources are indexed. Complete numbered listings accompany explanations of the authored code. Build and dependency outputs are described separately.

Prepared for your presentation tomorrow. Read the first sections and the priority files first; use the rest as a reference.

How to use this guide

Start with the architecture and request walkthrough. Each file chapter gives its responsibility and behavior, followed by complete numbered source and line notes. The notes explain syntax and operations; the file overview connects them to the actual feature. The chapter index gives the starting PDF page for every file.

Snapshot and scope

This guide describes the cleaned project on 1 October 2026. It covers every authored Kotlin, JavaScript, test, XML, Gradle, configuration, and existing documentation file, plus wrapper infrastructure and binary assets. Blank lines retain their source numbers. Long lines wrap with a continuation marker. Non-BMP characters such as emoji are shown as Unicode escapes in listings only; source files are unchanged.

Generated and private files

Generated Android/Gradle outputs, .git internals, .idea workstation metadata, .kotlin caches, node_modules, and documentation build artifacts are not app source. They are inventoried by category instead of reproducing dependency internals. local.properties contains a machine-specific Android SDK path and is intentionally not printed. The generated Code.gs combines the fully explained authored backend with vendor cryptography; its minified dependency is not duplicated here.

What is verified

The debug APK builds. Android local tests and lint pass; lint warnings remain. All 27 backend tests pass. The device instrumentation template has not been executed. There is no verified live Google login or Apps Script deployment in these checks. Missing deployment URL and Google client ID prevent real login.

Overnight study priorities

Read MainActivity, AppNavigation, LoginScreen, NetworkModule, GoogleSheetsApi, AttendanceScreen, AssessmentsScreen, core.js, auth.js, and apps-script.js first. Then review profile forms, resources, and tests. Rehearse the attendance request aloud; do not attempt to memorize thousands of lines.

Architecture

Android screens are Kotlin Compose functions. AuthSession provides verified process-local identity. Retrofit describes API calls; TrackerInterceptor sends authenticated JSON POST bodies. Apps Script validates Google identity and approved role, applies academic rules, and appends records to Google Sheets. JSON responses return through Gson to observable UI state.

Roles

Admin manages profiles, faculty assignment, enrollment, and reports. Faculty works on assigned subjects, records attendance, and saves/publishes assessments. Students see their own attendance and published results. Client navigation is role-aware, and server checks independently enforce the same boundary.

Attendance request walkthrough

1. AttendanceScreen loads the scoped roster. 2. The teacher fills every status and a date/slot. 3. The screen derives a request ID and confirms. 4. Retrofit builds a request. 5. TrackerInterceptor inserts idToken and sends POST. 6. doPost verifies the signature and account, obtains a lock, and constructs createService. 7. The service validates assignment,

date, roster, statuses, and uniqueness. 8. sheetStore appends the audited snapshot. 9. The JSON envelope reaches the UI, which disables the submitted session.

Sheet schema

TrackerUsers columns: email, role, profileId, name, active, googleSubject. role is admin/faculty/student; active must represent true. Faculty/student profileId must match an active profile. TrackerRecords columns: kind, id, data, at, actor, action. Identifiers, actor, and payload are encoded so cell content stays data. The latest row for a kind/id is current state; earlier rows retain history.

Attendance and assessment rules

Attendance percentage is $(\text{present} + \text{late}) / (\text{present} + \text{late} + \text{absent}) * 100$; excused is excluded. A student with no counted sessions has no percentage. Historical sessions use their stored roster. Assessment scores can be zero; blank means pending; absent is distinct. Drafts can be edited with matching revisions; publication requires complete marks and locks the record.

Important Kotlin syntax

val is a read-only reference; var can be reassigned. data class supplies value-like objects. ? allows null; ?. is a safe call; ?: supplies a fallback. by delegates property access. suspend permits suspension. A lambda is a function value in braces. remember retains local composition state; rememberSaveable adds supported recreation restoration. Modifier chains describe layout. dp is density-independent size; sp is scaled text size.

Important JavaScript syntax

const and let bind values; === compares without type coercion. An arrow function can be a callback. map transforms, filter selects, and find returns a matching item. ... spreads fields or elements. JSON.parse decodes text and JSON.stringify encodes data. throw stops an operation. try/catch/finally handles failure and cleanup. A Map indexes the latest record state.

Production boundaries

Local tests do not prove Google OAuth configuration, Apps Script deployment access, real Sheets behavior, quota capacity, or real-device usability. Remaining work includes live acceptance, operations monitoring, backups and restore drills, pagination/indexing at scale, offline strategy, localization/accessibility review, and a controlled correction workflow. The current append-only store rereads its journal and is best suited to a bounded institutional workload.

Project file index

Page numbers refer to this PDF. PDF bookmarks also open each file chapter.

6	.gitignore
7	PRODUCTION_PLAN.md
9	README.md
11	app/.gitignore
12	app/build.gradle.kts
15	app/src/androidTest/java/com/example/studenttracker/ExampleInstrumentedTest.kt
16	app/src/main/AndroidManifest.xml
18	app/src/main/java/com/example/studenttracker/MainActivity.kt
20	app/src/main/java/com/example/studenttracker/StudentTrackerApp.kt
21	app/src/main/java/com/example/studenttracker/data/local/UserPreferences.kt
23	app/src/main/java/com/example/studenttracker/data/network/AcademicModels.kt
24	app/src/main/java/com/example/studenttracker/data/network/AuthSession.kt
25	app/src/main/java/com/example/studenttracker/data/network/GoogleSheetsApi.kt
32	app/src/main/java/com/example/studenttracker/data/network/NetworkModule.kt
35	app/src/main/java/com/example/studenttracker/domain/model/CsvExporter.kt
36	app/src/main/java/com/example/studenttracker/domain/model/UserRoleResponse.kt
37	app/src/main/java/com/example/studenttracker/presentation/academic/AcademicHubScreen.kt
39	app/src/main/java/com/example/studenttracker/presentation/academic/AssessmentsScreen.kt
43	app/src/main/java/com/example/studenttracker/presentation/academic/AttendanceScreen.kt
46	app/src/main/java/com/example/studenttracker/presentation/academic/EnrollmentScreen.kt
48	app/src/main/java/com/example/studenttracker/presentation/admin/AdminDashboardScreen.kt
50	app/src/main/java/com/example/studenttracker/presentation/admin/AttendanceReportScreen.kt
54	app/src/main/java/com/example/studenttracker/presentation/auth/LoginScreen.kt
59	app/src/main/java/com/example/studenttracker/presentation/components/DirectoryScreen.kt
61	app/src/main/java/com/example/studenttracker/presentation/components/SkeletonLoading.kt
63	app/src/main/java/com/example/studenttracker/presentation/components/WorkspaceComponents.kt
66	app/src/main/java/com/example/studenttracker/presentation/faculty/AddFacultyScreen.kt
73	app/src/main/java/com/example/studenttracker/presentation/faculty/EditFacultyScreen.kt
79	app/src/main/java/com/example/studenttracker/presentation/faculty/FacultyDashboardScreen.kt
80	app/src/main/java/com/example/studenttracker/presentation/faculty/FacultyDetailsScreen.kt
85	app/src/main/java/com/example/studenttracker/presentation/faculty/FacultyListScreen.kt
86	app/src/main/java/com/example/studenttracker/presentation/navigation/AppNavigation.kt
90	app/src/main/java/com/example/studenttracker/presentation/student/AddStudentScreen.kt
99	app/src/main/java/com/example/studenttracker/presentation/student/EditStudentScreen.kt
105	app/src/main/java/com/example/studenttracker/presentation/student/StudentDashboardScreen.kt
107	app/src/main/java/com/example/studenttracker/presentation/student/StudentDetailsScreen.kt
113	app/src/main/java/com/example/studenttracker/presentation/student/StudentListScreen.kt
114	app/src/main/java/com/example/studenttracker/presentation/subject/AddSubjectScreen.kt
120	app/src/main/java/com/example/studenttracker/presentation/subject/EditSubjectScreen.kt
126	app/src/main/java/com/example/studenttracker/presentation/subject/SubjectDetailsScreen.kt
131	app/src/main/java/com/example/studenttracker/presentation/subject/SubjectListScreen.kt
132	app/src/main/java/com/example/studenttracker/ui/theme/Color.kt
133	app/src/main/java/com/example/studenttracker/ui/theme/Theme.kt
135	app/src/main/java/com/example/studenttracker/ui/theme/Type.kt
136	app/src/main/keepRules/rules.keep

137 | app/src/main/res/drawable/ic_launcher_background.xml
142 | app/src/main/res/drawable/ic_launcher_foreground.xml
143 | app/src/main/res/mipmap-anydpi-v26/ic_launcher.xml
144 | app/src/main/res/mipmap-anydpi-v26/ic_launcher_round.xml
145 | app/src/main/res/values/colors.xml
146 | app/src/main/res/values/strings.xml
147 | app/src/main/res/values/themes.xml
148 | app/src/main/res/xml/backup_rules.xml
149 | app/src/main/res/xml/data_extraction_rules.xml
150 | app/src/main/res/xml/file_paths.xml
151 | app/src/test/java/com/example/studenttracker/CsvExporterTest.kt
152 | app/src/test/java/com/example/studenttracker/ExampleUnitTest.kt
153 | app/src/test/java/com/example/studenttracker/TrackerInterceptorTest.kt
155 | backend/README.md
158 | backend/package-lock.json
159 | backend/package.json
160 | backend/scripts/build.mjs
161 | backend/src/apps-script.js
165 | backend/src/auth.js
167 | backend/src/core.js
176 | backend/test/apps-script.test.cjs
179 | backend/test/auth.test.cjs
181 | backend/test/core.test.cjs
188 | build.gradle.kts
189 | docs/STAGING_CHECKLIST.md
191 | gradle.properties
192 | gradle.properties.example
193 | gradle/gradle-daemon-jvm.properties
194 | gradle/libs.versions.toml
195 | gradle/wrapper/gradle-wrapper.properties
196 | gradlew
202 | gradlew.bat
205 | settings.gradle.kts

01. .gitignore

.gitignore

Git exclusion patterns.

20 source lines. SHA-256: 8851fe3775f55df2

Build outputs, dependency directories, private local configuration, and transient IDE files are excluded from version control. Patterns do not delete those files.

Source lines 1-20

```
0001 | *.iml
0002 | .gradle
0003 | /local.properties
0004 | /.idea/caches
0005 | /.idea/libraries
0006 | /.idea/modules.xml
0007 | /.idea/workspace.xml
0008 | /.idea/navEditor.xml
0009 | /.idea/assetWizardSettings.xml
0010 | .DS_Store
0011 | /build
0012 | /captures
0013 | .externalNativeBuild
0014 | .cxx
0015 | local.properties
0016 | **/node_modules/
0017 | backend/dist/
0018 | backend/.clasp.json
0019 | .env
0020 | app/build/
```

L1: Infrastructure documentation or retained upstream notice; not an application operation.

L2-15: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L16: Infrastructure documentation or retained upstream notice; not an application operation.

L17-20: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

02. PRODUCTION_PLAN.md

PRODUCTION_PLAN.md

Production implementation scope and remaining work.

91 source lines. SHA-256: 53ac7a2bdbdcc322

Documents feature stages, deployment requirements, and validation expectations. Separate implemented local code from unverified operations and future production work.

Complete document text

001 # Production implementation plan

003 ## Objective

004 Complete the administrator → faculty → student workflow with trustworthy records,
005 then prove operational readiness before a public release. Existing IDE changes are
006 outside this work. No live data or cloud deployments are changed by local development.

008 ## Implementation order and acceptance criteria

010 1. **Build and authentication foundation**

011 - Reproducible Windows build, external environment configuration, typed API failures.

012 - Google identity verified on the server; deny unknown/inactive users and unsupported roles.

013 - All reads/writes authorized on the server; no authentication based on submitted email.

014 - Explicit expiration and logout; no credentials in logs, URLs, or device backups.

015 2. **Connected academic workflows**

016 - Real administrator counts and role-specific dashboards with loading/error/retry states.

017 - Explicit subject enrollment; faculty see assigned subjects and minimal roster details.

018 - Attendance by subject/date/session, complete roster validation, duplicate prevention.

019 - Assessments with validated marks, drafts and explicit publication; students see only their own published results.

020 - Attendance reports derived from submitted sessions, safe CSV export, empty-data handling.

021 3. **Academic lifecycle and record integrity**

022 - First-class terms, departments, programs, sections and subject offerings.

023 - Enrollment dates, transfers, electives, substitutions, promotion and archival.

024 - Configurable attendance/grade policy, weighted assessments, retests and exemptions.

025 - Versioned corrections with approval/locking; audit trail and safe deactivation.

026 - Validated bulk imports with preview and actionable row errors.

027 4. **Reliability and user experience**

028 - Durable offline drafts, retries with idempotency, conflict recovery and sync indicators.

029 - Stable record-ID navigation, process recreation and accessibility/device testing.

030 - Notifications, correction requests, date-filtered reports and printable progress reports.

031 - Pagination, measured response times and tested maximum institution size.

032 5. **Release gates**

033 - End-to-end tests against a separate staging backend and real Google credentials.

034 - Permission, concurrency, calculation, export and failure-path tests pass.

035 - Backup/restore drill, monitoring/alerts, retention policy and support runbook.

036 - Release signing, environment separation, migration/rollback rehearsal and device QA.

038 ## Backend decision

039 Google Sheets and Apps Script are compulsory, as confirmed by the owner.

040 Google recommends a supported server library for production ID-token verification;

041 the tokeninfo debugging endpoint is not a production authentication strategy.

042 Use a bundled RSA verification library inside Apps Script with Google's rotating public

043 certificates, issuer/audience/expiry checks and an explicit user allowlist.

044 The new backend must be configured and staged before
045 the updated Android client can connect; never silently fall back to the old email-only API.

047 ## Scope and evidence

048 Each implementation pass must update this file with delivered work, exact checks and
049 remaining release blockers. A successful debug build is not proof of production readiness.

051 ## Current status

052 - Initial connected implementation complete locally; no cloud resources were changed.
053 - Windows wrapper now supports the ampersand in the checkout path. Use JDK 21 or
054 Android Studio's bundled JDK; the workstation's default Java 8 cannot build this app.
055 - Authentication uses signed Google tokens in POST bodies, server role checks,
056 explicit expiration and logout. Credentials remain in memory. Stored role metadata
057 cannot authenticate a restarted app. The old email-only API is refused.
058 - Administrator counts, searchable directories, stable profile-ID navigation,
059 enrollment, faculty attendance/assessments, student progress and CSV reports are connected.
060 - Profile/enrollment/assessment revision conflicts, attendance uniqueness, retry
061 deduplication, complete roster validation and append-only audit metadata are implemented.
062 - Deactivation preserves records; form values survive rotation and all fields participate
063 in unsaved-change checks. Loading, errors and retries are explicit.
064 - The generated backend stays within Google Sheets and Apps Script. RSA verification
065 uses pinned node-forge 1.4.0 with the library's corrected signature validation.
066 - Deployment instructions: `backend/README.md`. Real acceptance checklist:
067 `docs/STAGING\_CHECKLIST.md`. Environment example: `gradle.properties.example`.

069 ## Verification evidence

070 - Android: `:app:assembleDebug :app:testDebugUnitTest :app:lintDebug` passed.
071 - Android unit tests: 9 passed (CSV handling, network credential transport/failures,
072 plus the existing example test). Lint: zero errors; 33 non-blocking warnings remain,
073 primarily dependency updates, version-catalog suggestions and unused resources.
074 - Backend: generated bundle built; 27 tests passed for permissions, RSA signatures,
075 X509 certificates/Unicode claims, dispatcher/storage, attendance/marks, conflicts
076 and retries.
077 - npm audit: zero reported vulnerabilities in the pinned backend dependency.
078 - Git whitespace check passed. Existing `.idea` modifications were not edited.

080 ## Remaining release blockers and next milestones

081 - Configure/deploy the version 2 backend to staging and supply the matching Web OAuth
082 client ID and deployment base URL to the Android build.
083 - Validate Google sign-in, the actual Apps Script runtime, key rotation, concurrency,
084 narrow/large screens, accessibility and interrupted submissions on devices.
085 - Rehearse a mapped legacy-data import, backup/restore, monitoring and rollback.
086 - Complete encrypted session renewal, durable offline drafts/sync, first-class academic
087 terms/offers, configurable policy/grades, approved published-record corrections,
088 bulk imports, notifications and report filters.
089 - Measure expected institution capacity; current journal projection reads are not paginated
090 and enrollment is guarded at 250 students per subject / 40,000-character aggregates.
091 - Public release remains pending these checks and milestones.

03. README.md

README.md

Project setup and usage documentation.

54 source lines. SHA-256: fb49a1fe4b398b51

Explains how to build, configure authentication, package/deploy Apps Script, and validate workflows. Follow the backend README for required Script Properties and sheet headers.

Complete document text

```
001 # Student Attendance & Performance Tracker
003 Native Android app built with Kotlin, Jetpack Compose and Material 3, backed by
004 Google Sheets and Google Apps Script. The project now includes the backend source
005 and a connected administrator, faculty and student workflow.
007 ## Implemented
009 - Google sign-in with server-verified signed ID tokens and an explicit account allowlist.
010 - Role-specific navigation, logout and session expiration.
011 - Live administrator counts and searchable student/faculty/subject directories.
012 - Create/edit/deactivate profiles with historical records retained and revision conflicts detected.
013 - Explicit subject enrollment managed by administrators.
014 - Faculty-assigned subject rosters and complete attendance submission by date/session.
015 - Present/absent/late/excused status, duplicate-session prevention and retry deduplication.
016 - Assessments with draft marks, validated scores and explicit publication.
017 - Student-only attendance and published results.
018 - Subject attendance reports and formula-safe, correctly quoted CSV sharing.
019 - Loading/error/retry states, unsaved-change guards and saved form values on rotation.
020 - Audited spreadsheet journal and serialized backend access.
022 ## Set up
024 Read [backend setup](backend/README.md) first. The Android client requires the
025 **version 2 backend** included here; it rejects the old email-only API.
027 Configure `TRACKER_BASE_URL` and `GOOGLE_WEB_CLIENT_ID` using environment variables
028 or your user-level Gradle properties. See [configuration example](gradle.properties.example).
029 The base URL must end with `/` and exclude `exec`. No live deployment URL or OAuth
030 configuration is assumed by the source code.
032 Open the project in Android Studio with its bundled JDK (or JDK 21) and sync Gradle.
033 For PowerShell builds on this workstation:
035 ```powershell
036 $env:JAVA_HOME = 'C:\Program Files\Android\Android Studio\jbr'
037 .\gradlew.bat :app:assembleDebug :app:testDebugUnitTest :app:lintDebug --console=plain
038 ```
040 Build the Apps Script bundle from `backend/` with `npm ci --ignore-scripts`,
041 `npm run build`, then `npm test`. The generated `backend/dist/Code.gs` is ready to
042 copy into your Apps Script editor; deployment and staging validation remain necessary.
044 ## Production status
046 This is a tested local implementation of the core workflow, not a completed public
047 release. Tokens currently remain in memory, so restarting the process requires
048 sign-in. Offline drafts/sync, academic-term lifecycle, configurable grading rules,
049 notifications, imports, published-record corrections, pagination and operational
050 release checks remain in [the production plan](PRODUCTION_PLAN.md).
```

052 Use [the staging checklist](docs/STAGING_CHECKLIST.md) to validate real Google
053 sign-in, Apps Script execution, device behavior and backup/restore before rollout.
054 Existing legacy spreadsheet tabs are not automatically migrated.

04. .gitignore

app/.gitignore

Git exclusion patterns.

1 source lines. SHA-256: 5c3174b74edc6f2b

Build outputs, dependency directories, private local configuration, and transient IDE files are excluded from version control. Patterns do not delete those files.

Source lines 1-1

0001 | /build

L1: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

05. build.gradle.kts

app/build.gradle.kts

Android module build settings and dependencies.

87 source lines. SHA-256: 31c3773073df950c

configString reads a Gradle property or environment variable, escapes it, and supplies BuildConfig strings. Missing institution values default to empty.

namespace and applicationId identify the app. compile/target SDK are 37, minimum SDK is 24, version is 1.0. Compose and BuildConfig generation are enabled.

Java source compatibility is 11. The build toolchain uses JDK 21. Release shrinking is currently disabled.

Dependencies include Compose, navigation, DataStore, Credential Manager/Google ID, Retrofit, Gson, OkHttp, JUnit, and MockWebServer. These are libraries used by the Kotlin files.

Source lines 1-32

```
0001 | plugins {
0002 |     alias(libs.plugins.android.application)
0003 |     alias(libs.plugins.kotlin.compose)
0004 | }
0005 |
0006 | fun configString(name: String): String = providers.gradleProperty(name)
0007 |     .orElse(providers.environmentVariable(name)).orElse("").get()
0008 |     .let { "\"" + it.replace("\\", "\\").replace("\"", "\\\"") + "\"" }
0009 |
0010 | android {
0011 |     namespace = "com.example.studenttracker"
0012 |     compileSdk = 37
0013 |
0014 |     defaultConfig {
0015 |         applicationId = "com.example.studenttracker"
0016 |         minSdk = 24
0017 |         targetSdk = 37
0018 |         versionCode = 1
0019 |         versionName = "1.0"
0020 |         buildConfigField("String", "TRACKER_BASE_URL", configString("TRACKER_BASE_URL"))
0021 |         buildConfigField("String", "GOOGLE_WEB_CLIENT_ID", configString("GOOGLE_WEB_CLIENT_ID"))
0022 |
0023 |         testInstrumentationRunner = "androidx.test.runner.AndroidJUnitRunner"
0024 |     }
0025 |
0026 |     buildTypes {
0027 |         release {
0028 |             isMinifyEnabled = false
0029 |         }
0030 |     }
0031 |
0032 |     packaging {
```

L1-3: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L4: Closes or continues the surrounding block, expression, or call.

L6: Defines this Kotlin operation and its parameters; suspend allows asynchronous calls.

L7-8: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L10: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L11-12: Assigns the displayed expression to a variable or named argument.

L14: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L15-19: Assigns the displayed expression to a variable or named argument.

L20-21: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L23: Assigns the displayed expression to a variable or named argument.

L24: Closes or continues the surrounding block, expression, or call.

L26-27: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L28: Assigns the displayed expression to a variable or named argument.

L29-30: Closes or continues the surrounding block, expression, or call.

L32: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

Source lines 33-64

```
0033 |         resources {
0034 |             excludes += setOf(
0035 |                 "META-INF/DEPENDENCIES",
0036 |                 "META-INF/LICENSE",
0037 |                 "META-INF/LICENSE.txt",
0038 |                 "META-INF/license.txt",
0039 |                 "META-INF/NOTICE",
0040 |                 "META-INF/NOTICE.txt",
0041 |                 "META-INF/notice.txt",
0042 |                 "META-INF/ASL2.0",
0043 |                 "META-INF/*.kotlin_module"
0044 |             )
0045 |         }
0046 |     }
0047 |     compileOptions {
0048 |         sourceCompatibility = JavaVersion.VERSION_11
0049 |         targetCompatibility = JavaVersion.VERSION_11
0050 |     }
0051 |     buildFeatures {
0052 |         compose = true
0053 |         buildConfig = true
0054 |     }
0055 | }
0056 |
0057 | dependencies {
0058 |     implementation(platform(libs.androidx.compose.bom))
0059 |     implementation(libs.androidx.activity.compose)
0060 |     implementation(libs.androidx.compose.material3)
0061 |     implementation(libs.androidx.compose.ui)
0062 |     implementation(libs.androidx.compose.ui.graphics)
0063 |     implementation(libs.androidx.compose.ui.tooling.preview)
0064 |     implementation("androidx.compose.material:material-icons-extended")
```

L33-43: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L44-46: Closes or continues the surrounding block, expression, or call.

L47: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L48-49: Assigns the displayed expression to a variable or named argument.

L50: Closes or continues the surrounding block, expression, or call.

L51: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L52-53: Assigns the displayed expression to a variable or named argument.

L54-55: Closes or continues the surrounding block, expression, or call.

L57-64: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

Source lines 65-87

```
0065 |     implementation(libs.androidx.core.ktx)
0066 |     implementation(libs.androidx.lifecycle.runtime.ktx)
0067 |     testImplementation(libs.junit)
0068 |     androidTestImplementation(platform(libs.androidx.compose.bom))
0069 |     androidTestImplementation(libs.androidx.compose.ui.test.junit4)
0070 |     androidTestImplementation(libs.androidx.espresso.core)
0071 |     androidTestImplementation(libs.androidx.junit)
0072 |     debugImplementation(libs.androidx.compose.ui.test.manifest)
0073 |     debugImplementation(libs.androidx.compose.ui.tooling)
0074 |
0075 |     implementation("com.squareup.retrofit2:retrofit:2.11.0")
0076 |     implementation("com.squareup.retrofit2:converter-gson:2.11.0")
0077 |     implementation("com.squareup.okhttp3:okhttp:4.12.0")
0078 |     testImplementation("com.squareup.okhttp3:mockwebserver:4.12.0")
0079 |
0080 |     implementation("androidx.credentials:credentials:1.3.0")
0081 |     implementation("androidx.credentials:credentials-play-services-auth:1.3.0")
0082 |     implementation("com.google.android.libraries.identity.googleid:googleid:1.1.1")
```

```
0083 |  
0084 |     implementation("androidx.navigation:navigation-compose:2.7.7")  
0085 |  
0086 |     implementation("androidx.datastore:datastore-preferences:1.1.1")  
0087 | }
```

L65-73: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L75-78: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L80-82: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L84: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L86: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L87: Closes or continues the surrounding block, expression, or call.

06. ExampleInstrumentedTest.kt

app/src/androidTest/java/com/example/studenttracker/ExampleInstrumentedTest.kt

Android Studio device-test template.

19 source lines. SHA-256: bd375c76a0040468

Uses AndroidJUnit4 and InstrumentationRegistry to assert the app package name on a device or emulator. It has not been run as part of the local checks reported here.

Source lines 1-19

```
0001 | package com.example.studenttracker
0002 |
0003 | import androidx.test.platform.app.InstrumentationRegistry
0004 | import androidx.test.ext.junit.runners.AndroidJUnit4
0005 |
0006 | import org.junit.Test
0007 | import org.junit.runner.RunWith
0008 |
0009 | import org.junit.Assert.*
0010 |
0011 | @RunWith(AndroidJUnit4::class)
0012 | class ExampleInstrumentedTest {
0013 |     @Test
0014 |     fun useAppContext() {
0015 |
0016 |         val appContext = InstrumentationRegistry.getInstrumentation().targetContext
0017 |         assertEquals("com.example.studenttracker", appContext.packageName)
0018 |     }
0019 | }
```

L1: Declares the Kotlin namespace; it organizes classes and functions.

L3-4: Imports the named library symbols used by this file.

L6-7: Imports the named library symbols used by this file.

L9: Imports the named library symbols used by this file.

L11: Applies the displayed annotation to the declaration.

L12: Declares the named type; constructor properties define its stored data or dependencies.

L13: Marks the next function as a test executed by the test runner.

L14: Defines this Kotlin operation and its parameters; suspend allows asynchronous calls.

L16: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L17: Checks the displayed actual value against the expected behavior; failure fails the test.

L18-19: Closes or continues the surrounding block, expression, or call.

07. AndroidManifest.xml

app/src/main/AndroidManifest.xml

Android application registration and platform permissions.

42 source lines. SHA-256: f1ac2ceaeef0f87d

INTERNET enables HTTPS calls. The manifest names StudentTrackerApp and the exported launcher MainActivity.

Cleartext traffic and backups are disabled. The FileProvider is not exported and grants temporary access using res/xml/file_paths.xml.

Source lines 1-32

```
0001 | <?xml version="1.0" encoding="utf-8"?>
0002 | <manifest xmlns:android="http://schemas.android.com/apk/res/android"
0003 |     xmlns:tools="http://schemas.android.com/tools">
0004 |
0005 |     <uses-permission android:name="android.permission.INTERNET" />
0006 |
0007 |     <application
0008 |         android:name=".StudentTrackerApp"
0009 |         android:allowBackup="false"
0010 |         android:usesCleartextTraffic="false"
0011 |         android:dataExtractionRules="@xml/data_extraction_rules"
0012 |         android:fullBackupContent="@xml/backup_rules"
0013 |         android:icon="@mipmap/ic_launcher"
0014 |         android:label="@string/app_name"
0015 |         android:roundIcon="@mipmap/ic_launcher_round"
0016 |         android:supportsRtl="true"
0017 |         android:theme="@style/Theme.StudentTracker">
0018 |         <activity
0019 |             android:name=".MainActivity"
0020 |             android:exported="true"
0021 |             android:label="@string/app_name"
0022 |             android:theme="@style/Theme.StudentTracker"
0023 |             android:windowSoftInputMode="adjustResize">
0024 |             <intent-filter>
0025 |                 <action android:name="android.intent.action.MAIN" />
0026 |
0027 |                 <category android:name="android.intent.category.LAUNCHER" />
0028 |             </intent-filter>
0029 |         </activity>
0030 |
0031 |         <provider
0032 |             android:name="androidx.core.content.FileProvider"
```

L1-2: Declares the XML resource or attribute shown here; the file overview explains its Android role.

L3: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L5: Declares the XML resource or attribute shown here; the file overview explains its Android role.

L7: Declares the XML resource or attribute shown here; the file overview explains its Android role.

L8-17: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L18: Declares the XML resource or attribute shown here; the file overview explains its Android role.

L19-23: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L24-25: Declares the XML resource or attribute shown here; the file overview explains its Android role.

L27-29: Declares the XML resource or attribute shown here; the file overview explains its Android role.

L31: Declares the XML resource or attribute shown here; the file overview explains its Android role.

L32: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

Source lines 33-42

```
0033 |         android:authorities="${applicationId}.provider"
0034 |         android:exported="false"
0035 |         android:grantUriPermissions="true">
0036 |         <meta-data
0037 |             android:name="android.support.FILE_PROVIDER_PATHS"
```



```
0038 |         android:resource="@xml/file_paths" />
0039 |     </provider>
0040 | </application>
0041 |
0042 | </manifest>
```

L33-35: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L36: Declares the XML resource or attribute shown here; the file overview explains its Android role.

L37-38: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L39-40: Declares the XML resource or attribute shown here; the file overview explains its Android role.

L42: Declares the XML resource or attribute shown here; the file overview explains its Android role.

08. MainActivity.kt

app/src/main/java/com/example/studenttracker/MainActivity.kt

Android entry point and session-driven UI.

36 source lines. SHA-256: 49654ec8af4468a7

onCreate calls the activity lifecycle, enables edge-to-edge drawing, and installs Compose content.

collectAsState observes AuthSession.user. A change causes Compose to redraw the navigation tree.

LaunchedEffect waits until the token expiry, then clears the session and saved metadata. key(email, role) rebuilds navigation for a new identity.

StudentTrackerTheme is called with darkTheme=true and dynamicColor=false, so this app uses its fixed dark appearance.

Source lines 1-32

```
0001 | package com.example.studenttracker
0002 |
0003 | import android.os.Bundle
0004 | import androidx.activity.ComponentActivity
0005 | import androidx.activity.compose.setContent
0006 | import androidx.activity.enableEdgeToEdge
0007 | import androidx.compose.runtime.*
0008 | import com.example.studenttracker.data.local.UserPreferences
0009 | import com.example.studenttracker.data.network.AuthSession
0010 | import com.example.studenttracker.presentation.navigation.AppNavigation
0011 | import com.example.studenttracker.ui.theme.StudentTrackerTheme
0012 | import kotlinx.coroutines.delay
0013 |
0014 | class MainActivity : ComponentActivity() {
0015 |     override fun onCreate(savedInstanceState: Bundle?) {
0016 |         super.onCreate(savedInstanceState)
0017 |         enableEdgeToEdge()
0018 |         setContent {
0019 |             StudentTrackerTheme(darkTheme = true, dynamicColor = false) {
0020 |                 val user by AuthSession.user.collectAsState()
0021 |                 val preferences = remember { UserPreferences(this@MainActivity) }
0022 |                 LaunchedEffect(user?.expiresAt) {
0023 |                     user?.let {
0024 |                         delay(((it.expiresAt * 1000) - System.currentTimeMillis()).coerceAtLeast(0))
0025 |                         AuthSession.clear()
0026 |                         preferences.clearSession()
0027 |                     }
0028 |                 }
0029 |
0030 |                 key(user?.email, user?.role) {
0031 |                     AppNavigation(initialRole = user?.role, userPreferences = preferences)
0032 |                 }
0033 |             }
0034 |         }
0035 |     }
0036 | }
```

L1: Declares the Kotlin namespace; it organizes classes and functions.

L3-12: Imports the named library symbols used by this file.

L14: Declares the named type; constructor properties define its stored data or dependencies.

L15: Defines this Kotlin operation and its parameters; suspend allows asynchronous calls.

L16-19: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L20-21: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L22: Runs suspend work when the displayed effect key changes; old work is cancelled.

L23-26: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L27-28: Closes or continues the surrounding block, expression, or call.

L30-31: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L32: Closes or continues the surrounding block, expression, or call.

Source lines 33-36

```
0033 |           }  
0034 |       }  
0035 |   }  
0036 | }
```

L33-36: Closes or continues the surrounding block, expression, or call.

09. StudentTrackerApp.kt

app/src/main/java/com/example/studenttracker/StudentTrackerApp.kt

Application lifecycle class referenced by AndroidManifest.

10 source lines. SHA-256: 6594f8f65b666c76

onCreate currently only delegates to Application. It is a startup extension point, not a dependency container or database initializer.

Source lines 1-10

```
0001 | package com.example.studenttracker
0002 |
0003 | import android.app.Application
0004 |
0005 | class StudentTrackerApp : Application() {
0006 |     override fun onCreate() {
0007 |         super.onCreate()
0008 |
0009 |     }
0010 | }
```

L1: Declares the Kotlin namespace; it organizes classes and functions.

L3: Imports the named library symbols used by this file.

L5: Declares the named type; constructor properties define its stored data or dependencies.

L6: Defines this Kotlin operation and its parameters; suspend allows asynchronous calls.

L7: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L9-10: Closes or continues the surrounding block, expression, or call.

10. UserPreferences.kt

app/src/main/java/com/example/studenttracker/data/local/UserPreferences.kt

Stores email and role metadata in Preferences DataStore.

38 source lines. SHA-256: c2c26d089a69c1ff

The Context delegate opens user_session. stringPreferencesKey defines the two metadata keys.

saveSession performs a suspend DataStore edit; clearSession first clears AuthSession and then preferences.

userRoleFlow maps stored metadata to a role string. Stored metadata is not proof of authorization; the verified in-memory session controls navigation.

Source lines 1-32

```
0001 | package com.example.studenttracker.data.local
0002 |
0003 | import android.content.Context
0004 | import androidx.datastore.core.DataStore
0005 | import androidx.datastore.preferences.core.Preferences
0006 | import androidx.datastore.preferences.core.edit
0007 | import androidx.datastore.preferences.core.stringPreferencesKey
0008 | import androidx.datastore.preferences.preferencesDataStore
0009 | import kotlinx.coroutines.flow.Flow
0010 | import kotlinx.coroutines.flow.map
0011 |
0012 | val Context.dataStore: DataStore<Preferences> by preferencesDataStore(name = "user_session")
0013 |
0014 | class UserPreferences(private val context: Context) {
0015 |
0016 |     companion object {
0017 |         val EMAIL_KEY = stringPreferencesKey("email")
0018 |         val ROLE_KEY = stringPreferencesKey("role")
0019 |     }
0020 |
0021 |     val userRoleFlow: Flow<String?> = context.dataStore.data.map { preferences ->
0022 |         preferences[ROLE_KEY]
0023 |     }
0024 |
0025 |     suspend fun saveSession(email: String, role: String) {
0026 |         context.dataStore.edit { preferences ->
0027 |             preferences[EMAIL_KEY] = email
0028 |             preferences[ROLE_KEY] = role
0029 |         }
0030 |     }
0031 |
0032 |     suspend fun clearSession() {
```

L1: Declares the Kotlin namespace; it organizes classes and functions.

L3-10: Imports the named library symbols used by this file.

L12: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L14: Declares the named type; constructor properties define its stored data or dependencies.

L16: Declares the named type; constructor properties define its stored data or dependencies.

L17-18: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L19: Closes or continues the surrounding block, expression, or call.

L21: Transforms, selects, or indexes the displayed collection using its callback.

L22: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L23: Closes or continues the surrounding block, expression, or call.

L25: Defines this Kotlin operation and its parameters; suspend allows asynchronous calls.

L26-28: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L29-30: Closes or continues the surrounding block, expression, or call.

L32: Defines this Kotlin operation and its parameters; suspend allows asynchronous calls.

Source lines 33-38

```
0033 |         com.example.studenttracker.data.network.AuthSession.clear()
0034 |         context.dataStore.edit { preferences ->
0035 |             preferences.clear()
0036 |         }
0037 |     }
0038 | }
```

L33-35: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L36-38: Closes or continues the surrounding block, expression, or call.

11. AcademicModels.kt

app/src/main/java/com/example/studenttracker/data/network/AcademicModels.kt

JSON contracts for academic workflows.

20 source lines. SHA-256: 63f5ce23cae5ee91

Dashboard and detail response classes describe backend envelopes. Default empty lists and values make omitted optional JSON fields predictable.

EnrollmentRequest includes student IDs and the current revision. AttendanceRequest includes date, slot, request ID, and complete student status entries.

Assessment and MarkEntry distinguish pending, absent, and scored marks. AssessmentRequest carries the expected revision and published flag.

ProgressResponse contains only the current student attendance summaries and published results returned by the server.

Source lines 1-20

```
0001 | package com.example.studenttracker.data.network
0002 |
0003 | data class DashboardCounts(val students: Int, val faculty: Int, val subjects: Int, val departments:
    > Int)
0004 | data class DashboardResponse(val dashboard: DashboardCounts)
0005 | data class StudentResponse(val student: Student)
0006 | data class FacultyResponse(val facultyMember: Faculty)
0007 | data class SubjectResponse(val subject: Subject)
0008 | data class RosterStudent(val studentId: String, val name: String)
0009 | data class RosterResponse(val students: List<RosterStudent>, val revision: Int)
0010 | data class EnrollmentRequest(val subjectCode: String, val studentIds: List<String>, val revision:
    > Int, val action: String = "saveEnrollment")
0011 | data class AttendanceEntry(val studentId: String, val status: String)
0012 | data class AttendanceRequest(val subjectCode: String, val date: String, val slot: String, val
    > requestId: String, val records: List<AttendanceEntry>, val action: String = "saveAttendance")
0013 | data class MarkEntry(val studentId: String, val status: String, val score: Double? = null, val name:
    > String = "")
0014 | data class Assessment(val assessmentId: String, val subjectCode: String, val title: String, val
    > maxMarks: Double, val marks: List<MarkEntry>, val published: Boolean, val revision: Int)
0015 | data class AssessmentsResponse(val assessments: List<Assessment>)
0016 | data class AssessmentRequest(val assessmentId: String, val subjectCode: String, val title: String,
    > val maxMarks: Double, val marks: List<MarkEntry>, val published: Boolean, val revision: Int, val
    > requestId: String, val action: String = "saveAssessment")
0017 | data class AssessmentResponse(val message: String, val assessment: Assessment)
0018 | data class SubjectAttendance(val subjectCode: String, val subjectName: String, val totalPresent:
    > String, val totalClasses: String, val percentage: String)
0019 | data class StudentResult(val assessmentId: String, val title: String, val subjectCode: String, val
    > maxMarks: Double, val score: Double?, val status: String)
0020 | data class ProgressResponse(val attendance: List<SubjectAttendance>, val results:
    > List<StudentResult>)
```

L1: Declares the Kotlin namespace; it organizes classes and functions.

L3-20: Declares the named type; constructor properties define its stored data or dependencies.

12. AuthSession.kt

app/src/main/java/com/example/studenttracker/data/network/AuthSession.kt

Process-local authentication state.

19 source lines. SHA-256: 0e7d8a30a27ac6f7

SignedInUser carries verified email, name, role, and token expiry.

MutableStateFlow is private; consumers receive a read-only StateFlow. begin sets the token before the role request, complete publishes the verified user, clear removes both.

The volatile token has a private setter. It is not written to disk; process restart requires sign-in.

Source lines 1-19

```
0001 | package com.example.studenttracker.data.network
0002 |
0003 | import kotlinx.coroutines.flow.MutableStateFlow
0004 | import kotlinx.coroutines.flow.asStateFlow
0005 |
0006 | data class SignedInUser(val email: String, val name: String, val role: String, val expiresAt: Long)
0007 |
0008 | object AuthSession {
0009 |     private val mutableUser = MutableStateFlow<SignedInUser?>(null)
0010 |     val user = mutableUser.asStateFlow()
0011 |     @Volatile var idToken: String? = null
0012 |     private set
0013 |     fun begin(token: String) { idToken = token }
0014 |     fun complete(user: SignedInUser) { mutableUser.value = user }
0015 |     fun clear() {
0016 |         idToken = null
0017 |         mutableUser.value = null
0018 |     }
0019 | }
```

L1: Declares the Kotlin namespace; it organizes classes and functions.

L3-4: Imports the named library symbols used by this file.

L6: Declares the named type; constructor properties define its stored data or dependencies.

L8: Declares the named type; constructor properties define its stored data or dependencies.

L9-10: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L11: Applies the displayed annotation to the declaration.

L12: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L13-15: Defines this Kotlin operation and its parameters; suspend allows asynchronous calls.

L16: Carries the Google identity token for server verification; the token itself does not assign a role.

L17: Assigns the displayed expression to a variable or named argument.

L18-19: Closes or continues the surrounding block, expression, or call.

13. GoogleSheetsApi.kt

app/src/main/java/com/example/studenttracker/data/network/GoogleSheetsApi.kt

Retrofit interface plus profile request and response data classes.

240 source lines. SHA-256: b64d6056df2f1cc6

Student stores identity, academic, contact, and guardian fields. Faculty stores employee and employment details.

Subject stores the assigned faculty and planned class count.

Add, update, and deactivate requests contain an action; update/deactivate requests carry revision numbers. Identifiers such as rollNumber, employeeId, and subjectCode are the record keys.

@GET and @Query describe read methods; @POST and @Body describe mutations. TrackerInterceptor converts both forms to authenticated JSON POST before transmission.

Every API method is suspend. Gson maps JSON fields into the Kotlin data classes. Response lists can be null; callers commonly use orEmpty().

Source lines 1-32

```
0001 | package com.example.studenttracker.data.network
0002 |
0003 | import com.example.studenttracker.domain.model.UserRoleResponse
0004 | import retrofit2.http.Body
0005 | import retrofit2.http.GET
0006 | import retrofit2.http.POST
0007 | import retrofit2.http.Query
0008 |
0009 | data class AddStudentRequest(
0010 |     val action: String = "addStudent",
0011 |     val rollNumber: String,
0012 |     val name: String,
0013 |     val email: String,
0014 |     val phone: String,
0015 |     val department: String,
0016 |     val batch: String,
0017 |     val year: String,
0018 |     val semester: String,
0019 |     val section: String,
0020 |     val parentName: String,
0021 |     val parentPhone: String,
0022 |     val parentEmail: String
0023 | )
0024 |
0025 | data class UpdateStudentRequest(
0026 |     val revision: Int = 0,
0027 |     val action: String = "updateStudent",
0028 |     val rollNumber: String,
0029 |     val name: String,
0030 |     val email: String,
0031 |     val phone: String,
0032 |     val department: String,
```

L1: Declares the Kotlin namespace; it organizes classes and functions.

L3-7: Imports the named library symbols used by this file.

L9: Declares the named type; constructor properties define its stored data or dependencies.

L10-22: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L23: Closes or continues the surrounding block, expression, or call.

L25: Declares the named type; constructor properties define its stored data or dependencies.

L26: Carries or checks the record version used to detect stale edits or matching retries.

L27-32: Declares the displayed value or mutable state; its initializer supplies the initial contents.

Source lines 33-64

```
0033 |     val batch: String,
```

```

0034 |     val year: String,
0035 |     val semester: String,
0036 |     val section: String,
0037 |     val parentName: String,
0038 |     val parentPhone: String,
0039 |     val parentEmail: String
0040 | )
0041 |
0042 | data class DeleteStudentRequest(
0043 |     val revision: Int = 0,
0044 |     val action: String = "deleteStudent",
0045 |     val rollNumber: String
0046 | )
0047 |
0048 | data class ApiResponse(
0049 |     val status: String,
0050 |     val message: String
0051 | )
0052 |
0053 | data class Student(
0054 |     val revision: Int = 0,
0055 |     val rollNumber: String,
0056 |     val name: String,
0057 |     val email: String,
0058 |     val phone: String,
0059 |     val department: String,
0060 |     val batch: String,
0061 |     val year: String,
0062 |     val semester: String,
0063 |     val section: String,
0064 |     val parentName: String,

```

L33-39: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L40: Closes or continues the surrounding block, expression, or call.

L42: Declares the named type; constructor properties define its stored data or dependencies.

L43: Carries or checks the record version used to detect stale edits or matching retries.

L44-45: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L46: Closes or continues the surrounding block, expression, or call.

L48: Declares the named type; constructor properties define its stored data or dependencies.

L49-50: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L51: Closes or continues the surrounding block, expression, or call.

L53: Declares the named type; constructor properties define its stored data or dependencies.

L54: Carries or checks the record version used to detect stale edits or matching retries.

L55-64: Declares the displayed value or mutable state; its initializer supplies the initial contents.

Source lines 65-96

```

0065 |     val parentPhone: String,
0066 |     val parentEmail: String
0067 | )
0068 |
0069 | data class StudentListResponse(
0070 |     val status: String,
0071 |     val students: List<Student>? = null,
0072 |     val message: String? = null
0073 | )
0074 |
0075 | data class AddFacultyRequest(
0076 |     val action: String = "addFaculty",
0077 |     val employeeId: String,
0078 |     val name: String,
0079 |     val email: String,
0080 |     val phone: String,
0081 |     val department: String,
0082 |     val designation: String,
0083 |     val joiningDate: String
0084 | )
0085 |

```

```

0086 | data class UpdateFacultyRequest(
0087 |     val revision: Int = 0,
0088 |     val action: String = "updateFaculty",
0089 |     val employeeId: String,
0090 |     val name: String,
0091 |     val email: String,
0092 |     val phone: String,
0093 |     val department: String,
0094 |     val designation: String,
0095 |     val joiningDate: String
0096 | )

```

L65-66: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L67: Closes or continues the surrounding block, expression, or call.

L69: Declares the named type; constructor properties define its stored data or dependencies.

L70-72: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L73: Closes or continues the surrounding block, expression, or call.

L75: Declares the named type; constructor properties define its stored data or dependencies.

L76-83: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L84: Closes or continues the surrounding block, expression, or call.

L86: Declares the named type; constructor properties define its stored data or dependencies.

L87: Carries or checks the record version used to detect stale edits or matching retries.

L88-95: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L96: Closes or continues the surrounding block, expression, or call.

Source lines 97-128

```

0097 |
0098 | data class DeleteFacultyRequest(
0099 |     val revision: Int = 0,
0100 |     val action: String = "deleteFaculty",
0101 |     val employeeId: String
0102 | )
0103 |
0104 | data class Faculty(
0105 |     val revision: Int = 0,
0106 |     val employeeId: String,
0107 |     val name: String,
0108 |     val email: String,
0109 |     val phone: String,
0110 |     val department: String,
0111 |     val designation: String,
0112 |     val joiningDate: String
0113 | )
0114 |
0115 | data class FacultyListResponse(
0116 |     val status: String,
0117 |     val faculty: List<Faculty>? = null,
0118 |     val message: String? = null
0119 | )
0120 |
0121 | data class AddSubjectRequest(
0122 |     val action: String = "addSubject",
0123 |     val subjectCode: String,
0124 |     val subjectName: String,
0125 |     val facultyId: String,
0126 |     val semester: String,
0127 |     val totalClasses: String
0128 | )

```

L98: Declares the named type; constructor properties define its stored data or dependencies.

L99: Carries or checks the record version used to detect stale edits or matching retries.

L100-101: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L102: Closes or continues the surrounding block, expression, or call.

L104: Declares the named type; constructor properties define its stored data or dependencies.

L105: Carries or checks the record version used to detect stale edits or matching retries.

L106-112: Declares the displayed value or mutable state; its initializer supplies the initial contents.
 L113: Closes or continues the surrounding block, expression, or call.
 L115: Declares the named type; constructor properties define its stored data or dependencies.
 L116-118: Declares the displayed value or mutable state; its initializer supplies the initial contents.
 L119: Closes or continues the surrounding block, expression, or call.
 L121: Declares the named type; constructor properties define its stored data or dependencies.
 L122-127: Declares the displayed value or mutable state; its initializer supplies the initial contents.
 L128: Closes or continues the surrounding block, expression, or call.

Source lines 129-160

```

0129 |
0130 | data class UpdateSubjectRequest(
0131 |     val revision: Int = 0,
0132 |     val action: String = "updateSubject",
0133 |     val subjectCode: String,
0134 |     val subjectName: String,
0135 |     val facultyId: String,
0136 |     val semester: String,
0137 |     val totalClasses: String
0138 | )
0139 |
0140 | data class DeleteSubjectRequest(
0141 |     val revision: Int = 0,
0142 |     val action: String = "deleteSubject",
0143 |     val subjectCode: String
0144 | )
0145 |
0146 | data class Subject(
0147 |     val revision: Int = 0,
0148 |     val subjectCode: String,
0149 |     val subjectName: String,
0150 |     val facultyId: String,
0151 |     val semester: String,
0152 |     val totalClasses: String
0153 | )
0154 |
0155 | data class SubjectListResponse(
0156 |     val status: String,
0157 |     val subjects: List<Subject>? = null,
0158 |     val message: String? = null
0159 | )
0160 |

```

L130: Declares the named type; constructor properties define its stored data or dependencies.
 L131: Carries or checks the record version used to detect stale edits or matching retries.
 L132-137: Declares the displayed value or mutable state; its initializer supplies the initial contents.
 L138: Closes or continues the surrounding block, expression, or call.
 L140: Declares the named type; constructor properties define its stored data or dependencies.
 L141: Carries or checks the record version used to detect stale edits or matching retries.
 L142-143: Declares the displayed value or mutable state; its initializer supplies the initial contents.
 L144: Closes or continues the surrounding block, expression, or call.
 L146: Declares the named type; constructor properties define its stored data or dependencies.
 L147: Carries or checks the record version used to detect stale edits or matching retries.
 L148-152: Declares the displayed value or mutable state; its initializer supplies the initial contents.
 L153: Closes or continues the surrounding block, expression, or call.
 L155: Declares the named type; constructor properties define its stored data or dependencies.
 L156-158: Declares the displayed value or mutable state; its initializer supplies the initial contents.
 L159: Closes or continues the surrounding block, expression, or call.

Source lines 161-192

```

0161 | data class AttendanceReportItem(

```

```

0162 |     val studentId: String,
0163 |     val name: String,
0164 |     val totalPresent: String,
0165 |     val totalClasses: String,
0166 |     val percentage: String
0167 | )
0168 |
0169 | data class AttendanceReportResponse(
0170 |     val status: String,
0171 |     val report: List<AttendanceReportItem>? = null,
0172 |     val message: String? = null
0173 | )
0174 |
0175 | interface GoogleSheetsApi {
0176 |     @GET("exec")
0177 |     suspend fun getStudent(@Query("rollNumber") rollNumber: String, @Query("action") action: String
0178 | > = "getStudent"): StudentResponse
0179 |     @GET("exec")
0180 |     suspend fun getFacultyMember(@Query("employeeId") employeeId: String, @Query("action") action:
0181 | > String = "getFacultyMember"): FacultyResponse
0182 |     @GET("exec")
0183 |     suspend fun getSubject(@Query("subjectCode") subjectCode: String, @Query("action") action:
0184 | > String = "getSubject"): SubjectResponse
0185 |     @GET("exec")
0186 |     suspend fun getDashboard(@Query("action") action: String = "getDashboard"): DashboardResponse
0187 |     @GET("exec")
0188 |     suspend fun getRoster(@Query("subjectCode") subjectCode: String, @Query("action") action: String
0189 | > = "getRoster"): RosterResponse
0190 |     @POST("exec")
0191 |     suspend fun saveEnrollment(@Body request: EnrollmentRequest): ApiResponse
0192 |     @POST("exec")
0193 |     suspend fun saveAttendance(@Body request: AttendanceRequest): ApiResponse
0194 |     @GET("exec")
0195 |     suspend fun getAssessments(@Query("subjectCode") subjectCode: String, @Query("action") action:
0196 | > String = "getAssessments"): AssessmentsResponse
0197 |     @POST("exec")

```

L161: Declares the named type; constructor properties define its stored data or dependencies.

L162-166: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L167: Closes or continues the surrounding block, expression, or call.

L169: Declares the named type; constructor properties define its stored data or dependencies.

L170-172: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L173: Closes or continues the surrounding block, expression, or call.

L175: Declares the named type; constructor properties define its stored data or dependencies.

L176: Declares the HTTP or JSON mapping for Retrofit/Gson.

L177: Defines this Kotlin operation and its parameters; suspend allows asynchronous calls.

L178: Declares the HTTP or JSON mapping for Retrofit/Gson.

L179: Defines this Kotlin operation and its parameters; suspend allows asynchronous calls.

L180: Declares the HTTP or JSON mapping for Retrofit/Gson.

L181: Defines this Kotlin operation and its parameters; suspend allows asynchronous calls.

L182: Declares the HTTP or JSON mapping for Retrofit/Gson.

L183: Defines this Kotlin operation and its parameters; suspend allows asynchronous calls.

L184: Declares the HTTP or JSON mapping for Retrofit/Gson.

L185: Defines this Kotlin operation and its parameters; suspend allows asynchronous calls.

L186: Declares the HTTP or JSON mapping for Retrofit/Gson.

L187: Defines this Kotlin operation and its parameters; suspend allows asynchronous calls.

L188: Declares the HTTP or JSON mapping for Retrofit/Gson.

L189: Defines this Kotlin operation and its parameters; suspend allows asynchronous calls.

L190: Declares the HTTP or JSON mapping for Retrofit/Gson.

L191: Defines this Kotlin operation and its parameters; suspend allows asynchronous calls.

L192: Declares the HTTP or JSON mapping for Retrofit/Gson.

Source lines 193-224

```
0193 | suspend fun saveAssessment(@Body request: AssessmentRequest): AssessmentResponse
0194 | @GET("exec")
0195 | suspend fun getMyProgress(@Query("action") action: String = "getMyProgress"): ProgressResponse
0196 | @GET("exec")
0197 | suspend fun getUserRole(@Query("action") action: String = "getUserRole"): UserRoleResponse
0198 |
0199 | @GET("exec")
0200 | suspend fun getStudents(@Query("action") action: String = "getStudents"): StudentListResponse
0201 |
0202 | @POST("exec")
0203 | suspend fun addStudent(@Body request: AddStudentRequest): ApiResponse
0204 |
0205 | @POST("exec")
0206 | suspend fun updateStudent(@Body request: UpdateStudentRequest): ApiResponse
0207 |
0208 | @POST("exec")
0209 | suspend fun deleteStudent(@Body request: DeleteStudentRequest): ApiResponse
0210 |
0211 | @GET("exec")
0212 | suspend fun getFaculty(@Query("action") action: String = "getFaculty"): FacultyListResponse
0213 |
0214 | @POST("exec")
0215 | suspend fun addFaculty(@Body request: AddFacultyRequest): ApiResponse
0216 |
0217 | @POST("exec")
0218 | suspend fun updateFaculty(@Body request: UpdateFacultyRequest): ApiResponse
0219 |
0220 | @POST("exec")
0221 | suspend fun deleteFaculty(@Body request: DeleteFacultyRequest): ApiResponse
0222 |
0223 | @GET("exec")
0224 | suspend fun getSubjects(@Query("action") action: String = "getSubjects"): SubjectListResponse
```

L193: Defines this Kotlin operation and its parameters; suspend allows asynchronous calls.

L194: Declares the HTTP or JSON mapping for Retrofit/Gson.

L195: Defines this Kotlin operation and its parameters; suspend allows asynchronous calls.

L196: Declares the HTTP or JSON mapping for Retrofit/Gson.

L197: Defines this Kotlin operation and its parameters; suspend allows asynchronous calls.

L199: Declares the HTTP or JSON mapping for Retrofit/Gson.

L200: Defines this Kotlin operation and its parameters; suspend allows asynchronous calls.

L202: Declares the HTTP or JSON mapping for Retrofit/Gson.

L203: Defines this Kotlin operation and its parameters; suspend allows asynchronous calls.

L205: Declares the HTTP or JSON mapping for Retrofit/Gson.

L206: Defines this Kotlin operation and its parameters; suspend allows asynchronous calls.

L208: Declares the HTTP or JSON mapping for Retrofit/Gson.

L209: Defines this Kotlin operation and its parameters; suspend allows asynchronous calls.

L211: Declares the HTTP or JSON mapping for Retrofit/Gson.

L212: Defines this Kotlin operation and its parameters; suspend allows asynchronous calls.

L214: Declares the HTTP or JSON mapping for Retrofit/Gson.

L215: Defines this Kotlin operation and its parameters; suspend allows asynchronous calls.

L217: Declares the HTTP or JSON mapping for Retrofit/Gson.

L218: Defines this Kotlin operation and its parameters; suspend allows asynchronous calls.

L220: Declares the HTTP or JSON mapping for Retrofit/Gson.

L221: Defines this Kotlin operation and its parameters; suspend allows asynchronous calls.

L223: Declares the HTTP or JSON mapping for Retrofit/Gson.

L224: Defines this Kotlin operation and its parameters; suspend allows asynchronous calls.

Source lines 225-240

```
0225 |
0226 | @POST("exec")
```

```

0227 | suspend fun addSubject(@Body request: AddSubjectRequest): ApiResponse
0228 |
0229 | @POST("exec")
0230 | suspend fun updateSubject(@Body request: UpdateSubjectRequest): ApiResponse
0231 |
0232 | @POST("exec")
0233 | suspend fun deleteSubject(@Body request: DeleteSubjectRequest): ApiResponse
0234 |
0235 | @GET("exec")
0236 | suspend fun getAttendanceReport(
0237 |     @Query("action") action: String = "getAttendanceReport",
0238 |     @Query("subjectCode") subjectCode: String
0239 | ): AttendanceReportResponse
0240 | }

```

L226: Declares the HTTP or JSON mapping for Retrofit/Gson.

L227: Defines this Kotlin operation and its parameters; suspend allows asynchronous calls.

L229: Declares the HTTP or JSON mapping for Retrofit/Gson.

L230: Defines this Kotlin operation and its parameters; suspend allows asynchronous calls.

L232: Declares the HTTP or JSON mapping for Retrofit/Gson.

L233: Defines this Kotlin operation and its parameters; suspend allows asynchronous calls.

L235: Declares the HTTP or JSON mapping for Retrofit/Gson.

L236: Defines this Kotlin operation and its parameters; suspend allows asynchronous calls.

L237-238: Declares the HTTP or JSON mapping for Retrofit/Gson.

L239: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L240: Closes or continues the surrounding block, expression, or call.

14. NetworkModule.kt

app/src/main/java/com/example/studenttracker/data/network/NetworkModule.kt

HTTP configuration, authenticated transport, and API error handling.

72 source lines. SHA-256: 3a14170a27c12e4b

TrackerApiException gives callers a code and readable message.

TrackerInterceptor requires the memory token, reads any JSON body, copies query fields into that body, adds idToken, removes the URL query, and performs a JSON POST.

The interceptor rejects failed HTTP responses, incompatible apiVersion, and failed backend envelopes. An UNAUTHENTICATED result clears the session.

isConfigured requires an Apps Script HTTPS base URL ending in / and a Google web client ID. OkHttp has finite timeouts and automatic connection retry disabled; Retrofit uses Gson.

Source lines 1-32

```
0001 | package com.example.studenttracker.data.network
0002 |
0003 | import com.example.studenttracker.BuildConfig
0004 | import com.google.gson.JsonObject
0005 | import com.google.gson.JsonParser
0006 | import okhttp3.Interceptor
0007 | import okhttp3.MediaType.Companion.toMediaType
0008 | import okhttp3.OkHttpClient
0009 | import okhttp3.RequestBody.Companion.toRequestBody
0010 | import okhttp3.Response
0011 | import okio.Buffer
0012 | import java.io.IOException
0013 | import retrofit2.Retrofit
0014 | import retrofit2.converter.gson.GsonConverterFactory
0015 | import java.util.concurrent.TimeUnit
0016 |
0017 | class TrackerApiException(val code: String, message: String) : IOException(message)
0018 |
0019 | class TrackerInterceptor : Interceptor {
0020 |     override fun intercept(chain: Interceptor.Chain): Response {
0021 |         val original = chain.request()
0022 |         val token = AuthSession.idToken ?: throw TrackerApiException("UNAUTHENTICATED", "Sign in to
> continue.")
0023 |         val payload = if (original.body != null) {
0024 |             val buffer = Buffer()
0025 |             original.body!!.writeTo(buffer)
0026 |             JsonParser.parseString(buffer.readUtf8()).asJsonObject
0027 |         } else JsonObject()
0028 |         original.url.queryParameterNames.forEach { name -> payload.addProperty(name,
> original.url.queryParameter(name)) }
0029 |         payload.addProperty("idToken", token)
0030 |         val request = original.newBuilder().url(original.url.newBuilder().query(null).build())
0031 |             .post(payload.toString().toRequestBody("application/json;
> charset=utf-8".toMediaType()))
0032 |             .build()
0033 |         val response = chain.proceed(request)
```

L1: Declares the Kotlin namespace; it organizes classes and functions.

L3-15: Imports the named library symbols used by this file.

L17: Declares the named type; constructor properties define its stored data or dependencies.

L19: Declares the named type; constructor properties define its stored data or dependencies.

L20: Defines this Kotlin operation and its parameters; suspend allows asynchronous calls.

L21: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L22: Stops the current operation by raising the displayed error.

L23-24: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L25-28: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L29: Carries the Google identity token for server verification; the token itself does not assign a role.

L30: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L31: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L32: Declares the displayed value or mutable state; its initializer supplies the initial contents.

Source lines 33-64

```
0033 |         try {
0034 |             if (!response.isSuccessful) throw TrackerApiException("HTTP", "The service is
> unavailable (${response.code}). Try again.")
0035 |             val envelope =
> JsonParser.parseString(response.peekBody(2_000_000).string()).asJsonObject
0036 |             if (envelope.get("apiVersion")?.asInt != 2) throw TrackerApiException("CONFIGURATION",
> "Deploy the version 2 Apps Script backend before using this app.")
0037 |             if (envelope.get("status")?.asString != "success") {
0038 |                 val code = envelope.get("code")?.asString ?: "API_ERROR"
0039 |                 if (code == "UNAUTHENTICATED") AuthSession.clear()
0040 |                 throw TrackerApiException(code, envelope.get("message")?.asString ?: "The request
> could not be completed.")
0041 |             }
0042 |             return response
0043 |         } catch (error: Exception) {
0044 |             response.close()
0045 |             if (error is IOException) throw error
0046 |             throw TrackerApiException("INVALID_RESPONSE", "The backend returned an invalid response.
> Check its deployment.")
0047 |         }
0048 |     }
0049 | }
0050 |
0051 | object NetworkModule {
0052 |     val isConfigured: Boolean get() =
> BuildConfig.TRACKER_BASE_URL.startsWith("https://script.google.com/macros/s/") &&
0053 |         BuildConfig.TRACKER_BASE_URL.endsWith("/") && BuildConfig.GOOGLE_WEB_CLIENT_ID.isNotBlank()
0054 |
0055 |     private val okHttpClient = OkHttpClient.Builder()
0056 |         .connectTimeout(20, TimeUnit.SECONDS)
0057 |         .readTimeout(60, TimeUnit.SECONDS)
0058 |         .callTimeout(75, TimeUnit.SECONDS)
0059 |         .retryOnConnectionFailure(false)
0060 |         .addInterceptor(TrackerInterceptor())
0061 |         .build()
0062 |
0063 |     val api: GoogleSheetsApi by lazy {
0064 |         check(isConfigured) { "Configure TRACKER_BASE_URL and GOOGLE_WEB_CLIENT_ID before signing
> in." }
0065 |     }
```

L33: Starts an operation whose failures are handled by the following catch/finally blocks.

L34: Branches on the displayed condition; the accepted branch performs the following operation.

L35: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L36-37: Branches on the displayed condition; the accepted branch performs the following operation.

L38: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L39: Branches on the displayed condition; the accepted branch performs the following operation.

L40: Stops the current operation by raising the displayed error.

L41: Closes or continues the surrounding block, expression, or call.

L42: Returns this expression as the operation result; subsequent statements in this path are skipped.

L43: Handles this exception type and translates it into the following failure behavior.

L44: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L45: Branches on the displayed condition; the accepted branch performs the following operation.

L46: Stops the current operation by raising the displayed error.

L47-49: Closes or continues the surrounding block, expression, or call.

L51: Declares the named type; constructor properties define its stored data or dependencies.

L52: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L53: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L55: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L56-61: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L63: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L64: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

Source lines 65-72

```
0065 |         Retrofit.Builder()
0066 |             .baseUrl(BuildConfig.TRACKER_BASE_URL)
0067 |             .client(okHttpClient)
0068 |             .addConverterFactory(GsonConverterFactory.create())
0069 |             .build()
0070 |             .create(GoogleSheetsApi::class.java)
0071 |         }
0072 |     }
```

L65-69: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L70: Declares the named type; constructor properties define its stored data or dependencies.

L71-72: Closes or continues the surrounding block, expression, or call.

15. CsvExporter.kt

app/src/main/java/com/example/studenttracker/domain/model/CsvExporter.kt

Safe CSV text construction.

11 source lines. SHA-256: b77d29e1179b8191

cell prefixes potentially executable spreadsheet formulas with an apostrophe, then quotes the field and doubles embedded quotes.

encode prepends a UTF-8 BOM and joins fields with commas and rows with CRLF. This handles embedded commas and newlines and improves spreadsheet compatibility.

Source lines 1-11

```
0001 | package com.example.studenttracker.domain.model
0002 |
0003 | object CsvExporter {
0004 |
0005 |     fun cell(value: String): String {
0006 |         val trimmed = value.trimStart()
0007 |         val safe = if (value.startsWith('\t') || value.startsWith('\r') || value.startsWith('\n') ||
> trimmed.firstOrNull() in listOf('=', '+', '-', '@')) "'$value" else value
0008 |         return "\"${safe.replace("\"", "\"\"")}\""
0009 |     }
0010 |     fun encode(rows: List<List<String>>): String = "\uFEFF" + rows.joinToString("\r\n") { row ->
> row.joinToString(", ", transform = ::cell) } + "\r\n"
0011 | }
```

L1: Declares the Kotlin namespace; it organizes classes and functions.

L3: Declares the named type; constructor properties define its stored data or dependencies.

L5: Defines this Kotlin operation and its parameters; suspend allows asynchronous calls.

L6-7: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L8: Returns this expression as the operation result; subsequent statements in this path are skipped.

L9: Closes or continues the surrounding block, expression, or call.

L10: Defines this Kotlin operation and its parameters; suspend allows asynchronous calls.

L11: Closes or continues the surrounding block, expression, or call.

16. UserRoleResponse.kt

app/src/main/java/com/example/studenttracker/domain/model/UserRoleResponse.kt

Response contract for verified login.

11 source lines. SHA-256: f5906662260bbf93

status, email, name, role, and expiresAt describe the approved backend identity. SerializedName binds JSON names explicitly. Login validates role and expiry before completing the session.

Source lines 1-11

```
0001 | package com.example.studenttracker.domain.model
0002 |
0003 | import com.google.gson.annotations.SerializedName
0004 |
0005 | data class UserRoleResponse(
0006 |     @SerializedName("status") val status: String,
0007 |     @SerializedName("email") val email: String,
0008 |     @SerializedName("name") val name: String?,
0009 |     @SerializedName("role") val role: String,
0010 |     val expiresAt: Long = 0
0011 | )
```

L1: Declares the Kotlin namespace; it organizes classes and functions.

L3: Imports the named library symbols used by this file.

L5: Declares the named type; constructor properties define its stored data or dependencies.

L6-9: Declares the HTTP or JSON mapping for Retrofit/Gson.

L10: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L11: Closes or continues the surrounding block, expression, or call.

17. AcademicHubScreen.kt

app/src/main/java/com/example/studenttracker/presentation/academic/AcademicHubScreen.kt

Subject workspace shared by admin and faculty.

38 source lines. SHA-256: f79fdabb056a8573

getSubjects is scoped by the backend. Faculty receives assigned subjects; admin receives active subjects.

Each subject offers attendance, report, and assessment actions. Admin additionally gets enrollment. Refresh reloads the server list.

Source lines 1-32

```
0001 | package com.example.studenttracker.presentation.academic
0002 |
0003 | import androidx.compose.foundation.layout.*
0004 | import androidx.compose.foundation.lazy.LazyColumn
0005 | import androidx.compose.foundation.lazy.items
0006 | import androidx.compose.material3.*
0007 | import androidx.compose.runtime.*
0008 | import androidx.compose.ui.Modifier
0009 | import androidx.compose.ui.unit.dp
0010 | import com.example.studenttracker.data.network.*
0011 | import com.example.studenttracker.presentation.components.*
0012 |
0013 | @Composable
0014 | fun AcademicHubScreen(isAdmin: Boolean, onOpen: (String, String) -> Unit, onBack: (() -> Unit)? =
    > null, onLogout: () -> Unit = {}) {
0015 |     val user by AuthSession.user.collectAsState()
0016 |     var refresh by remember { mutableIntStateOf(0) }
0017 |     WorkspacePage(if (isAdmin) "Teaching & enrollment" else "My teaching", onBack, actions = {
0018 |         TextButton(onClick = { refresh++ }) { Text("Refresh") }
0019 |         if (!isAdmin) TextButton(onClick = onLogout) { Text("Logout") }
0020 |     }) { padding ->
0021 |         Column(Modifier.fillMaxSize().padding(padding)) {
0022 |             RemoteContent(refresh, { NetworkModule.api.getSubjects().subjects.orEmpty() }) {
    > subjects ->
0023 |                 LazyColumn(contentPadding = PaddingValues(16.dp), verticalArrangement =
    > Arrangement.spacedBy(16.dp)) {
0024 |                     item { Text(if (isAdmin) "Manage enrollment and academic records by subject."
    > else "Welcome, ${user?.name.orEmpty()}. Choose an assigned subject.") }
0025 |                     if (subjects.isEmpty()) item { InfoCard("No subjects yet", if (isAdmin) "Create
    > subjects and assign faculty to get started." else "Your administrator needs to assign you to a
    > subject.") }
0026 |                     items(subjects, key = { it.subjectCode }) { subject ->
0027 |                         InfoCard(subject.subjectName, "${subject.subjectCode} · Semester
    > ${subject.semester}") {
0028 |                             if (isAdmin) OutlinedButton(onClick = { onOpen("enrollment",
    > subject.subjectCode) }, modifier = Modifier.fillMaxWidth()) { Text("Manage enrolled students") }
0029 |                             Button(onClick = { onOpen("attendance", subject.subjectCode) }, modifier
    > = Modifier.fillMaxWidth()) { Text("Mark attendance") }
0030 |                             OutlinedButton(onClick = { onOpen("report", subject.subjectCode) },
    > modifier = Modifier.fillMaxWidth()) { Text("Attendance report") }
0031 |                             OutlinedButton(onClick = { onOpen("assessments", subject.subjectCode) },
    > modifier = Modifier.fillMaxWidth()) { Text("Assessments & marks") }
0032 |                         }
```

L1: Declares the Kotlin namespace; it organizes classes and functions.

L3-11: Imports the named library symbols used by this file.

L13: Marks the following function as a Compose UI function.

L14: Defines this Kotlin operation and its parameters; suspend allows asynchronous calls.

L15-16: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L17: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L18: Connects a user interaction to the displayed state update or handler.

L19: Branches on the displayed condition; the accepted branch performs the following operation.

L20: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L21: Adds this UI element to the composition with the displayed content and parameters.
L22: Calls the named authenticated backend operation with the displayed request or key.
L23: Adds this UI element to the composition with the displayed content and parameters.
L24: Continues the surrounding call, condition, declaration, or data structure using the displayed value.
L25: Branches on the displayed condition; the accepted branch performs the following operation.
L26: Continues the surrounding call, condition, declaration, or data structure using the displayed value.
L27: Adds this UI element to the composition with the displayed content and parameters.
L28: Branches on the displayed condition; the accepted branch performs the following operation.
L29-31: Connects a user interaction to the displayed state update or handler.
L32: Closes or continues the surrounding block, expression, or call.

Source lines 33-38

```
0033 |           }  
0034 |         }  
0035 |       }  
0036 |     }  
0037 |   }  
0038 | }
```

L33-38: Closes or continues the surrounding block, expression, or call.

18. AssessmentsScreen.kt

app/src/main/java/com/example/studenttracker/presentation/academic/AssessmentsScreen.kt

Assessment listing, draft entry, and publication.

112 source lines. SHA-256: bd8357b9554175fd

The list loads assessments and roster. A selected ID opens AssessmentEditor; a new assessment gets a UUID.

The editor retains title, maximum marks, and a JSON map of entry values. Blank means pending, A means absent, and numeric values including 0 mean scored.

Validation checks a finite maximum from 1 to 10000 and each score within range. Publishing requires no pending entries and an explicit confirmation.

The request contains a payload-derived ID and expected revision. Published entries are displayed with editing disabled; server rules independently lock them.

Source lines 1-32

```
0001 | package com.example.studenttracker.presentation.academic
0002 |
0003 | import androidx.compose.foundation.layout.*
0004 | import androidx.compose.foundation.lazy.LazyColumn
0005 | import androidx.compose.foundation.lazy.items
0006 | import androidx.compose.foundation.text.KeyboardOptions
0007 | import androidx.compose.material3.*
0008 | import androidx.compose.runtime.*
0009 | import androidx.compose.runtime.saveable.rememberSaveable
0010 | import androidx.compose.ui.Modifier
0011 | import androidx.compose.ui.text.input.KeyboardType
0012 | import androidx.compose.ui.unit.dp
0013 | import com.example.studenttracker.data.network.*
0014 | import com.example.studenttracker.presentation.components.*
0015 | import com.google.gson.Gson
0016 | import kotlinx.coroutines.CancellationException
0017 | import kotlinx.coroutines.launch
0018 | import java.util.UUID
0019 |
0020 | @Composable
0021 | fun AssessmentsScreen(subjectCode: String, onBack: () -> Unit) {
0022 |     var refresh by remember { mutableIntStateOf(0) }
0023 |     var selectedId by rememberSaveable(subjectCode) { mutableStateOf<String?>(null) }
0024 |     RemoteContent(subjectCode to refresh, {
0025 |         > NetworkModule.api.getAssessments(subjectCode).assessments to
0026 |         > NetworkModule.api.getRoster(subjectCode).students }, onBack = onBack) { (assessments, students) ->
0027 |         if (selectedId != null) {
0028 |             val existing = assessments.find { it.assessmentId == selectedId }
0029 |             key(selectedId) {
0030 |                 AssessmentEditor(subjectCode, selectedId!!, existing, students) { selectedId = null;
0031 |                 refresh++ }
0032 |             }
0033 |         } else WorkspacePage("Assessments · $subjectCode", onBack, actions = { TextButton(onClick =
0034 |         { refresh++ }) { Text("Refresh") } }) { padding ->
0035 |         LazyColumn(Modifier.fillMaxSize().padding(padding), contentPadding =
0036 |         PaddingValues(16.dp), verticalArrangement = Arrangement.spacedBy(12.dp)) {
0037 |             item { Button(onClick = { selectedId = UUID.randomUUID().toString() }, enabled =
0038 |             students.isNotEmpty(), modifier = Modifier.fillMaxWidth()) { Text("Create assessment") } }
0039 |         }
0040 |     }
0041 | }
```

L1: Declares the Kotlin namespace; it organizes classes and functions.

L3-18: Imports the named library symbols used by this file.

L20: Marks the following function as a Compose UI function.

L21: Defines this Kotlin operation and its parameters; suspend allows asynchronous calls.

L22: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L23: Keeps this UI value across supported activity recreation; it is not server persistence.

L24: Calls the named authenticated backend operation with the displayed request or key.

L25: Branches on the displayed condition; the accepted branch performs the following operation.

L26: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L27-28: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L29: Closes or continues the surrounding block, expression, or call.

L30: Connects a user interaction to the displayed state update or handler.

L31: Adds this UI element to the composition with the displayed content and parameters.

L32: Connects a user interaction to the displayed state update or handler.

Source lines 33-64

```
0033 |             if (students.isEmpty()) item { Text("Enroll students before creating an
> assessment.") }
0034 |             if (assessments.isEmpty()) item { InfoCard("No assessments yet", "Create an
> assessment, enter marks, then publish results to students.") }
0035 |             items(assessments, key = { it.assessmentId }) { assessment ->
0036 |                 InfoCard(assessment.title, "Out of ${assessment.maxMarks} · ${if
> (assessment.published) "Published" else "Draft"}") {
0037 |                     OutlinedButton(onClick = { selectedId = assessment.assessmentId }) { Text(if
> (assessment.published) "View results" else "Continue entering marks") }
0038 |                 }
0039 |             }
0040 |         }
0041 |     }
0042 | }
0043 | }
0044 |
0045 | @OptIn(ExperimentalLayoutApi::class)
0046 | @Composable
0047 | private fun AssessmentEditor(subjectCode: String, assessmentId: String, existing: Assessment?,
> currentRoster: List<RosterStudent>, onBack: () -> Unit) {
0048 |     val roster = if (existing?.published == true) existing.marks.map { RosterStudent(it.studentId,
> it.name) } else currentRoster
0049 |     var title by rememberSaveable { mutableStateOf(existing?.title.orEmpty()) }
0050 |     var maxMarks by rememberSaveable { mutableStateOf(existing?.maxMarks?.toString() ?: "100") }
0051 |     val initial = roster.associate { student ->
0052 |         val mark = existing?.marks?.find { it.studentId == student.studentId }
0053 |         student.studentId to when (mark?.status) { "absent" -> "A"; "scored" ->
> mark.score.toString(); else -> "" }
0054 |     }
0055 |     var valuesJson by rememberSaveable { mutableStateOf(Gson().toJson(initial)) }
0056 |     val values = remember(valuesJson) { Gson().fromJson(valuesJson, Map::class.java).map {
> it.key.toString() to it.value.toString() }.toMap() }
0057 |     var saving by remember { mutableStateOf(false) }
0058 |     var error by remember { mutableStateOf<String?>(null) }
0059 |     var confirm by remember { mutableStateOf(false) }
0060 |     val locked = existing?.published == true
0061 |     val dirty = title != existing?.title.orEmpty() || maxMarks != (existing?.maxMarks?.toString() ?:
> "100") || values != initial
0062 |     val back = guardedBack(dirty && !locked, saving, onBack)
0063 |     val scope = rememberCoroutineScope()
0064 |     fun save(publish: Boolean) {
```

L33-34: Branches on the displayed condition; the accepted branch performs the following operation.

L35: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L36: Adds this UI element to the composition with the displayed content and parameters.

L37: Connects a user interaction to the displayed state update or handler.

L38-43: Closes or continues the surrounding block, expression, or call.

L45: Applies the displayed annotation to the declaration.

L46: Marks the following function as a Compose UI function.

L47: Defines this Kotlin operation and its parameters; suspend allows asynchronous calls.

L48: Transforms, selects, or indexes the displayed collection using its callback.

L49-50: Keeps this UI value across supported activity recreation; it is not server persistence.

L51: Transforms, selects, or indexes the displayed collection using its callback.

L52: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L53: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L54: Closes or continues the surrounding block, expression, or call.

L55: Keeps this UI value across supported activity recreation; it is not server persistence.

L56: Declares the named type; constructor properties define its stored data or dependencies.

L57-59: Creates observable local UI state; assignment triggers recomposition.

L60-62: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L63: Obtains a coroutine scope tied to this composable for button-triggered asynchronous work.

L64: Defines this Kotlin operation and its parameters; suspend allows asynchronous calls.

Source lines 65-96

```
0065 |         error = null; confirm = false
0066 |         val maximum = maxMarks.toDoubleOrNull()
0067 |         if (title.isBlank() || maximum == null || !maximum.isFinite() || maximum < 1 || maximum >
> 10000) {
0068 |             error = "Enter a title and maximum marks between 1 and 10,000."; return
0069 |         }
0070 |         val marks = roster.map { student ->
0071 |             val value = values[student.studentId].orEmpty().trim()
0072 |             MarkEntry(student.studentId, when { value.equals("A", true) -> "absent"; value.isBlank()
> -> "pending"; else -> "scored" }, value.toDoubleOrNull())
0073 |         }
0074 |         if (marks.any { it.status == "scored" } && (it.score == null || !it.score.isFinite() ||
> it.score < 0 || it.score > maximum) ) {
0075 |             error = "Each mark must be between 0 and $maximum. Use A for absent or leave blank for
> pending."; return
0076 |         }
0077 |         if (publish && marks.any { it.status == "pending" }) { error = "Complete every student's
> marks before publishing."; return }
0078 |         val fingerprint = Gson().toJson(listOf(assessmentId, title.trim(), maximum, marks, publish,
> existing?.revision ?: 0))
0079 |         val requestId = UUID.nameUUIDFromBytes(fingerprint.toByteArray(Charsets.UTF_8)).toString()
0080 |         saving = true
0081 |         scope.launch {
0082 |             try {
0083 |                 NetworkModule.api.saveAssessment(AssessmentRequest(assessmentId, subjectCode,
> title.trim(), maximum, marks, publish, existing?.revision ?: 0, requestId))
0084 |                 onBack()
0085 |             } catch (e: CancellationException) { throw e }
0086 |             catch (e: Exception) { error = e.message }
0087 |             finally { saving = false }
0088 |         }
0089 |     }
0090 |     if (confirm) AlertDialog(onDismissRequest = { confirm = false }, title = { Text("Publish
> results?") },
0091 |         text = { Text("Students will be able to see their own marks. Published results are locked in
> this version.") },
0092 |         confirmButton = { TextButton(onClick = { save(true) }) { Text("Publish") } },
0093 |         dismissButton = { TextButton(onClick = { confirm = false }) { Text("Review") } })
0094 |     WorkspacePage(if (locked) "Published results" else "Enter marks", back) { padding ->
0095 |         Column(Modifier.fillMaxSize().padding(padding).padding(16.dp), verticalArrangement =
> Arrangement.spacedBy(12.dp)) {
0096 |             OutlinedTextField(title, { title = it }, enabled = !locked && !saving, label = {
> Text("Assessment title") }, modifier = Modifier.fillMaxWidth())
```

L65: Assigns the displayed expression to a variable or named argument.

L66: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L67: Branches on the displayed condition; the accepted branch performs the following operation.

L68: Assigns the displayed expression to a variable or named argument.

L69: Closes or continues the surrounding block, expression, or call.

L70: Transforms, selects, or indexes the displayed collection using its callback.

L71: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L72: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L73: Closes or continues the surrounding block, expression, or call.

L74: Branches on the displayed condition; the accepted branch performs the following operation.

L75: Assigns the displayed expression to a variable or named argument.

L76: Closes or continues the surrounding block, expression, or call.

L77: Branches on the displayed condition; the accepted branch performs the following operation.

L78: Serializes structured values to JSON for transport, saveable state, or journal storage.

L79: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L80: Assigns the displayed expression to a variable or named argument.

L81: Starts asynchronous work in the UI scope so the button handler can return promptly.

L82: Starts an operation whose failures are handled by the following catch/finally blocks.

L83: Calls the named authenticated backend operation with the displayed request or key.

L84: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L85: Preserves coroutine cancellation rather than presenting it as an ordinary application error.

L86: Handles this exception type and translates it into the following failure behavior.

L87: Runs cleanup whether the preceding operation succeeded or failed.

L88-89: Closes or continues the surrounding block, expression, or call.

L90: Branches on the displayed condition; the accepted branch performs the following operation.

L91: Assigns the displayed expression to a variable or named argument.

L92-93: Connects a user interaction to the displayed state update or handler.

L94: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L95-96: Adds this UI element to the composition with the displayed content and parameters.

Source lines 97-112

```

0097 |         OutlinedTextField(maxMarks, { maxMarks = it }, enabled = !locked && !saving, label = {
> Text("Maximum marks") }, modifier = Modifier.fillMaxWidth(), keyboardOptions =
> KeyboardOptions(keyboardType = KeyboardType.Decimal))
0098 |         Text("Enter 0 for zero marks, A for absent, or leave blank for pending.", style =
> MaterialTheme.typography.bodySmall)
0099 |         error?.let { Text(it, color = MaterialTheme.colorScheme.error) }
0100 |         LazyColumn(Modifier.weight(1f), verticalArrangement = Arrangement.spacedBy(10.dp)) {
0101 |             items(roster, key = { it.studentId }) { student ->
0102 |                 OutlinedTextField(values[student.studentId].orEmpty(), { valuesJson =
> Gson().toJson(values + (student.studentId to it)) },
0103 |                 label = { Text(student.name) }, supportingText = { Text(student.studentId)
> }, modifier = Modifier.fillMaxWidth(), enabled = !locked && !saving, singleLine = true)
0104 |             }
0105 |         }
0106 |         if (!locked) FlowRow(horizontalArrangement = Arrangement.spacedBy(12.dp)) {
0107 |             OutlinedButton(enabled = !saving, onClick = { save(false) }) { Text(if (saving)
> "Saving..." else "Save draft") }
0108 |             Button(enabled = !saving && roster.isNotEmpty(), onClick = { confirm = true }) {
> Text("Review & publish") }
0109 |         }
0110 |     }
0111 | }
0112 | }

```

L97-98: Adds this UI element to the composition with the displayed content and parameters.

L99: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L100: Adds this UI element to the composition with the displayed content and parameters.

L101: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L102: Serializes structured values to JSON for transport, saveable state, or journal storage.

L103: Configures UI layout, sizing, spacing, or appearance; it does not modify academic records.

L104-105: Closes or continues the surrounding block, expression, or call.

L106: Branches on the displayed condition; the accepted branch performs the following operation.

L107-108: Connects a user interaction to the displayed state update or handler.

L109-112: Closes or continues the surrounding block, expression, or call.

19. AttendanceScreen.kt

app/src/main/java/com/example/studenttracker/presentation/academic/AttendanceScreen.kt

Full-roster class-session attendance entry.

79 source lines. SHA-256: 9378b0612ef5b168

Loads the assigned subject roster and keeps date, slot, chosen statuses, submission state, and errors in Compose state.

The roster must be nonempty and every student must receive present, absent, late, or excused. Mark all present fills the map.

Submission builds entries in roster order. A deterministic UUID is derived from the request contents; date and slot are also validated by the backend.

A confirmation dialog precedes saving. Busy or completed sessions disable editing; errors remain visible. This screen does not implement a persistent offline queue.

Source lines 1-32

```
0001 | package com.example.studenttracker.presentation.academic
0002 |
0003 | import androidx.compose.foundation.layout.*
0004 | import androidx.compose.foundation.lazy.LazyColumn
0005 | import androidx.compose.foundation.lazy.items
0006 | import androidx.compose.material3.*
0007 | import androidx.compose.runtime.*
0008 | import androidx.compose.runtime.saveable.rememberSaveable
0009 | import androidx.compose.ui.Modifier
0010 | import androidx.compose.ui.unit.dp
0011 | import com.example.studenttracker.data.network.*
0012 | import com.example.studenttracker.presentation.components.*
0013 | import com.google.gson.Gson
0014 | import kotlinx.coroutines.CancellationException
0015 | import kotlinx.coroutines.launch
0016 | import java.text.SimpleDateFormat
0017 | import java.util.Date
0018 | import java.util.Locale
0019 | import java.util.UUID
0020 |
0021 | @OptIn(ExperimentalLayoutApi::class)
0022 | @Composable
0023 | fun AttendanceScreen(subjectCode: String, onBack: () -> Unit) {
0024 |     RemoteContent(subjectCode, { NetworkModule.api.getRoster(subjectCode) }, onBack = onBack) {
0025 |         > roster ->
0026 |             var date by rememberSaveable { mutableStateOf(SimpleDateFormat("yyyy-MM-dd",
0027 |                 Locale.US).format(Date())) }
0028 |             var slot by rememberSaveable { mutableStateOf("") }
0029 |             var statusesJson by rememberSaveable { mutableStateOf("{}") }
0030 |             val statuses = remember(statusesJson) { Gson().fromJson(statusesJson, Map::class.java).map {
0031 |                 > it.key.toString() to it.value.toString() }.toMap() }
0032 |             var saving by remember { mutableStateOf(false) }
0033 |             var message by remember { mutableStateOf<String?>(null) }
0034 |             var submitted by rememberSaveable { mutableStateOf(false) }
0035 |             var confirm by remember { mutableStateOf(false) }
```

L1: Declares the Kotlin namespace; it organizes classes and functions.

L3-19: Imports the named library symbols used by this file.

L21: Applies the displayed annotation to the declaration.

L22: Marks the following function as a Compose UI function.

L23: Defines this Kotlin operation and its parameters; suspend allows asynchronous calls.

L24: Calls the named authenticated backend operation with the displayed request or key.

L25-27: Keeps this UI value across supported activity recreation; it is not server persistence.

L28: Declares the named type; constructor properties define its stored data or dependencies.

L29-30: Creates observable local UI state; assignment triggers recomposition.

L31: Keeps this UI value across supported activity recreation; it is not server persistence.

L32: Creates observable local UI state; assignment triggers recomposition.

Source lines 33-64

```
0033 |         val scope = rememberCoroutineScope()
0034 |         val back = guardedBack(!submitted && (statuses.isNotEmpty() || slot.isNotBlank()), saving,
> onBack)
0035 |         val complete = roster.students.isNotEmpty() && roster.students.all { statuses[it.studentId]
> != null }
0036 |         fun submit() {
0037 |             saving = true; message = null; confirm = false
0038 |             val entries = roster.students.map { AttendanceEntry(it.studentId,
> statuses.getValue(it.studentId)) }
0039 |             val fingerprint = Gson().toJson(listOf(subjectCode, date, slot.trim(), entries))
0040 |             val requestId =
> UUID.nameUUIDFromBytes(fingerprint.toByteArray(Charsets.UTF_8)).toString()
0041 |             scope.launch {
0042 |                 try { message = NetworkModule.api.saveAttendance(AttendanceRequest(subjectCode,
> date, slot.trim(), requestId, entries)).message; submitted = true }
0043 |                 catch (e: CancellationException) { throw e }
0044 |                 catch (e: Exception) { message = e.message }
0045 |                 finally { saving = false }
0046 |             }
0047 |         }
0048 |         if (confirm) AlertDialog(onDismissRequest = { confirm = false }, title = { Text("Submit
> attendance?") },
0049 |             text = { Text("$subjectCode · $date · $slot\n${roster.students.size} students. Submitted
> sessions are locked against duplicate entry.") },
0050 |             confirmButton = { TextButton(onClick = { submit() }) { Text("Submit") } },
0051 |             dismissButton = { TextButton(onClick = { confirm = false }) { Text("Review") } })
0052 |             WorkspacePage("Attendance · $subjectCode", back) { padding ->
0053 |                 Column(Modifier.fillMaxSize().padding(padding).padding(16.dp), verticalArrangement =
> Arrangement.spacedBy(12.dp)) {
0054 |                     Row(horizontalArrangement = Arrangement.spacedBy(8.dp)) {
0055 |                         OutlinedTextField(date, { date = it }, enabled = !saving && !submitted, label =
> { Text("Date · YYYY-MM-DD") }, modifier = Modifier.weight(1f), singleLine = true)
0056 |                         OutlinedTextField(slot, { slot = it }, enabled = !saving && !submitted, label =
> { Text("Session / period") }, modifier = Modifier.weight(1f), singleLine = true)
0057 |                     }
0058 |                     if (roster.students.isEmpty()) Text("No students are enrolled. Ask your
> administrator to set up enrollment.")
0059 |                     else Text("${roster.students.count { statuses[it.studentId] != null
> }}/${roster.students.size} marked · Late counts as present; excused is excluded.", style =
> MaterialTheme.typography.bodySmall)
0060 |                     OutlinedButton(enabled = !saving && !submitted && roster.students.isNotEmpty(),
> onClick = { statusesJson = Gson().toJson(roster.students.associate { it.studentId to "present" }) })
> { Text("Mark all present") }
0061 |                     message?.let { Text(it, color = if (submitted) MaterialTheme.colorScheme.primary
> else MaterialTheme.colorScheme.error) }
0062 |                     LazyColumn(Modifier.weight(1f), verticalArrangement = Arrangement.spacedBy(10.dp)) {
0063 |                         items(roster.students, key = { it.studentId }) { student ->
0064 |                             InfoCard(student.name, student.studentId) {
```

L33: Obtains a coroutine scope tied to this composable for button-triggered asynchronous work.

L34-35: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L36: Defines this Kotlin operation and its parameters; suspend allows asynchronous calls.

L37: Assigns the displayed expression to a variable or named argument.

L38: Transforms, selects, or indexes the displayed collection using its callback.

L39: Serializes structured values to JSON for transport, saveable state, or journal storage.

L40: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L41: Starts asynchronous work in the UI scope so the button handler can return promptly.

L42: Starts an operation whose failures are handled by the following catch/finally blocks.

L43: Preserves coroutine cancellation rather than presenting it as an ordinary application error.

L44: Handles this exception type and translates it into the following failure behavior.

L45: Runs cleanup whether the preceding operation succeeded or failed.

L46-47: Closes or continues the surrounding block, expression, or call.

L48: Branches on the displayed condition; the accepted branch performs the following operation.

L49: Assigns the displayed expression to a variable or named argument.

L50-51: Connects a user interaction to the displayed state update or handler.

L52: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L53-56: Adds this UI element to the composition with the displayed content and parameters.

L57: Closes or continues the surrounding block, expression, or call.

L58: Branches on the displayed condition; the accepted branch performs the following operation.

L59: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L60: Serializes structured values to JSON for transport, saveable state, or journal storage.

L61: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L62: Adds this UI element to the composition with the displayed content and parameters.

L63: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L64: Adds this UI element to the composition with the displayed content and parameters.

Source lines 65-79

```
0065 |                 FlowRow(horizontalArrangement = Arrangement.spacedBy(6.dp)) {
0066 |                     listOf("present", "absent", "late", "excused").forEach { status ->
0067 |                         FilterChip(selected = statuses[student.studentId] == status,
> enabled = !saving && !submitted,
0068 |                                     onClick = { statusesJson = Gson().toJson(statuses +
> (student.studentId to status)) }, label = { Text(status.replaceFirstChar { it.uppercase() }) })
0069 |                         }
0070 |                     }
0071 |                 }
0072 |             }
0073 |         }
0074 |         if (submitted) Button(onClick = onBack, modifier = Modifier.fillMaxWidth()) {
> Text("Done") }
0075 |         else Button(onClick = { confirm = true }, enabled = complete && date.isNotBlank() &&
> slot.isNotBlank() && !saving, modifier = Modifier.fillMaxWidth()) { Text(if (saving) "Submitting..."
> else "Review & submit") }
0076 |     }
0077 | }
0078 | }
0079 | }
```

L65: Adds this UI element to the composition with the displayed content and parameters.

L66-67: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L68: Serializes structured values to JSON for transport, saveable state, or journal storage.

L69-73: Closes or continues the surrounding block, expression, or call.

L74: Branches on the displayed condition; the accepted branch performs the following operation.

L75: Connects a user interaction to the displayed state update or handler.

L76-79: Closes or continues the surrounding block, expression, or call.

20. EnrollmentScreen.kt

app/src/main/java/com/example/studenttracker/presentation/academic/EnrollmentScreen.kt

Administrator subject roster editor.

56 source lines. SHA-256: 06c3f25d48b75c50

Loads the subject, student directory, and current roster. Selected IDs are saveable state; query text filters candidates.

Checkboxes add or remove student IDs. The dirty guard compares selection membership against the original roster.

Save sends EnrollmentRequest with the current revision. The backend validates active students, uniqueness, capacity, and stale revisions.

Source lines 1-32

```
0001 | package com.example.studenttracker.presentation.academic
0002 |
0003 | import androidx.compose.foundation.layout.*
0004 | import androidx.compose.foundation.lazy.LazyColumn
0005 | import androidx.compose.foundation.lazy.items
0006 | import androidx.compose.material3.*
0007 | import androidx.compose.runtime.*
0008 | import androidx.compose.runtime.saveable.rememberSaveable
0009 | import androidx.compose.ui.Modifier
0010 | import androidx.compose.ui.unit.dp
0011 | import com.example.studenttracker.data.network.*
0012 | import com.example.studenttracker.presentation.components.*
0013 | import kotlinx.coroutines.CancellationException
0014 | import kotlinx.coroutines.launch
0015 |
0016 | @Composable
0017 | fun EnrollmentScreen(subjectCode: String, onBack: () -> Unit) {
0018 |     RemoteContent(subjectCode, { NetworkModule.api.getStudents().students.orEmpty() to
0019 | > NetworkModule.api.getRoster(subjectCode) }, onBack = onBack) { (students, roster) ->
0019 |         var selected by rememberSaveable(subjectCode) {
0020 | > mutableStateOf<List<String>>(roster.students.map { it.studentId }) }
0020 |         var query by rememberSaveable { mutableStateOf("") }
0021 |         var saving by remember { mutableStateOf(false) }
0022 |         var error by remember { mutableStateOf<String?>(null) }
0023 |         val scope = rememberCoroutineScope()
0024 |         val back = guardedBack(selected.toSet() != roster.students.map { it.studentId }.toSet(),
0025 | > saving, onBack)
0025 |         WorkspacePage("Enrollment · $subjectCode", back) { padding ->
0026 |             Column(Modifier.fillMaxSize().padding(padding).padding(16.dp), verticalArrangement =
0027 | > Arrangement.spacedBy(12.dp)) {
0027 |                 Text("${selected.size} students selected · maximum 250")
0028 |                 Text("Only enrolled students appear in attendance and marks. Earlier submitted
0029 | > records are retained when enrollment changes.", style = MaterialTheme.typography.bodySmall)
0029 |                 OutlinedTextField(query, { query = it }, label = { Text("Search name, roll number or
0030 | > department") }, modifier = Modifier.fillMaxWidth(), singleLine = true)
0030 |                 error?.let { Text(it, color = MaterialTheme.colorScheme.error) }
0031 |                 LazyColumn(Modifier.weight(1f)) {
0032 |                     items(students.filter { "${it.name} ${it.rollNumber}
0033 | > ${it.department}".contains(query, true) }, key = { it.rollNumber }) { student ->
```

L1: Declares the Kotlin namespace; it organizes classes and functions.

L3-14: Imports the named library symbols used by this file.

L16: Marks the following function as a Compose UI function.

L17: Defines this Kotlin operation and its parameters; suspend allows asynchronous calls.

L18: Calls the named authenticated backend operation with the displayed request or key.

L19-20: Keeps this UI value across supported activity recreation; it is not server persistence.

L21-22: Creates observable local UI state; assignment triggers recomposition.

L23: Obtains a coroutine scope tied to this composable for button-triggered asynchronous work.

L24: Transforms, selects, or indexes the displayed collection using its callback.

L25: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L26-29: Adds this UI element to the composition with the displayed content and parameters.

L30: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L31: Adds this UI element to the composition with the displayed content and parameters.

L32: Transforms, selects, or indexes the displayed collection using its callback.

Source lines 33-56

```
0033 |                 Row(Modifier.fillMaxWidth(), horizontalArrangement =
> Arrangement.spacedBy(8.dp)) {
0034 |                     Checkbox(student.rollNumber in selected, enabled = !saving,
> onCheckedChange = { checked ->
0035 |                         selected = if (checked) selected + student.rollNumber else selected
> - student.rollNumber
0036 |                     })
0037 |                 Column(Modifier.padding(vertical = 8.dp)) {
0038 |                     Text(student.name)
0039 |                     Text("${student.rollNumber} · ${student.department} ·
> ${student.section}", style = MaterialTheme.typography.bodySmall)
0040 |                 }
0041 |             }
0042 |         }
0043 |     }
0044 |     Button(enabled = !saving && selected.size <= 250, modifier =
> Modifier.fillMaxWidth(), onClick = {
0045 |         saving = true; error = null
0046 |         scope.launch {
0047 |             try { NetworkModule.api.saveEnrollment(EnrollmentRequest(subjectCode,
> selected, roster.revision)); onBack() }
0048 |             catch (e: CancellationException) { throw e }
0049 |             catch (e: Exception) { error = e.message }
0050 |             finally { saving = false }
0051 |         }
0052 |     }) { Text(if (saving) "Saving..." else "Save enrollment") }
0053 | }
0054 | }
0055 | }
0056 | }
```

L33: Adds this UI element to the composition with the displayed content and parameters.

L34: Connects a user interaction to the displayed state update or handler.

L35: Assigns the displayed expression to a variable or named argument.

L36: Closes or continues the surrounding block, expression, or call.

L37-39: Adds this UI element to the composition with the displayed content and parameters.

L40-43: Closes or continues the surrounding block, expression, or call.

L44: Connects a user interaction to the displayed state update or handler.

L45: Assigns the displayed expression to a variable or named argument.

L46: Starts asynchronous work in the UI scope so the button handler can return promptly.

L47: Starts an operation whose failures are handled by the following catch/finally blocks.

L48: Preserves coroutine cancellation rather than presenting it as an ordinary application error.

L49: Handles this exception type and translates it into the following failure behavior.

L50: Runs cleanup whether the preceding operation succeeded or failed.

L51: Closes or continues the surrounding block, expression, or call.

L52: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L53-56: Closes or continues the surrounding block, expression, or call.

21. AdminDashboardScreen.kt

app/src/main/java/com/example/studenttracker/presentation/admin/AdminDashboardScreen.kt

Administrator home and current backend counts.

52 source lines. SHA-256: ca3d3e249bf4e5e9

RemoteContent calls getDashboard and shows active student, faculty, subject, and department counts. Refresh changes the load key.

Cards navigate to profile management, subject management, enrollment, and reports. The header shows the verified email and logout.

Source lines 1-32

```
0001 | package com.example.studenttracker.presentation.admin
0002 |
0003 | import androidx.compose.foundation.layout.*
0004 | import androidx.compose.foundation.lazy.LazyColumn
0005 | import androidx.compose.material3.*
0006 | import androidx.compose.runtime.*
0007 | import androidx.compose.ui.Modifier
0008 | import androidx.compose.ui.unit.dp
0009 | import androidx.navigation.NavHostController
0010 | import com.example.studenttracker.data.local.UserPreferences
0011 | import com.example.studenttracker.data.network.*
0012 | import com.example.studenttracker.presentation.components.*
0013 | import kotlinx.coroutines.launch
0014 |
0015 | @Composable
0016 | fun AdminDashboardScreen(navController: NavHostController, userPreferences: UserPreferences) {
0017 |     val scope = rememberCoroutineScope()
0018 |     val user by AuthSession.user.collectAsState()
0019 |     var refresh by remember { mutableIntStateOf(0) }
0020 |     WorkspacePage("Administration", actions = {
0021 |         TextButton(onClick = { refresh++ }) { Text("Refresh") }
0022 |         TextButton(onClick = { scope.launch { userPreferences.clearSession() } }) { Text("Logout") }
0023 |     }) { padding ->
0024 |         LazyColumn(Modifier.fillMaxSize().padding(padding), contentPadding = PaddingValues(16.dp),
0025 |             > verticalArrangement = Arrangement.spacedBy(16.dp)) {
0026 |             item { Text("Welcome, ${user?.name.orEmpty()}", style =
0027 |                 > MaterialTheme.typography.headlineSmall) }
0028 |             item { Text(user?.email.orEmpty(), color = MaterialTheme.colorScheme.onSurfaceVariant) }
0029 |             item {
0030 |                 RemoteContent(refresh, { NetworkModule.api.getDashboard().dashboard }) { counts ->
0031 |                     InfoCard("Institution overview", "Current active records") {
0032 |                         Text("${counts.students} students · ${counts.faculty} faculty")
0033 |                         Text("${counts.subjects} subjects · ${counts.departments} departments")
0034 |                     }
0035 |                 }
0036 |             }
0037 |         }
0038 |     }
```

L1: Declares the Kotlin namespace; it organizes classes and functions.

L3-13: Imports the named library symbols used by this file.

L15: Marks the following function as a Compose UI function.

L16: Defines this Kotlin operation and its parameters; suspend allows asynchronous calls.

L17: Obtains a coroutine scope tied to this composable for button-triggered asynchronous work.

L18-19: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L20: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L21: Connects a user interaction to the displayed state update or handler.

L22: Starts asynchronous work in the UI scope so the button handler can return promptly.

L23: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L24: Adds this UI element to the composition with the displayed content and parameters.

L25-27: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L28: Calls the named authenticated backend operation with the displayed request or key.

L29-31: Adds this UI element to the composition with the displayed content and parameters.

L32: Closes or continues the surrounding block, expression, or call.

Source lines 33-52

```
0033 |         }
0034 |     }
0035 |     item { InfoCard("Academic workspace", "Enroll students, mark class attendance and manage
> assessments.") {
0036 |         Button(onClick = { navController.navigate("academic") }, modifier =
> Modifier.fillMaxWidth()) { Text("Open academic workspace") }
0037 |     } }
0038 |     item { InfoCard("Students", "Manage profiles, academic details and guardian contacts.")
> {
0039 |         Button(onClick = { navController.navigate("student_list") }) { Text("Manage
> students") }
0040 |         OutlinedButton(onClick = { navController.navigate("add_student") }) { Text("Add
> student") }
0041 |     } }
0042 |     item { InfoCard("Faculty", "Manage faculty profiles and subject assignments.") {
0043 |         Button(onClick = { navController.navigate("faculty_list") }) { Text("Manage
> faculty") }
0044 |         OutlinedButton(onClick = { navController.navigate("add_faculty") }) { Text("Add
> faculty") }
0045 |     } }
0046 |     item { InfoCard("Subjects & reports") {
0047 |         Button(onClick = { navController.navigate("subject_list") }) { Text("Manage
> subjects") }
0048 |         OutlinedButton(onClick = { navController.navigate("attendance_report") }) {
> Text("Attendance reports & export") }
0049 |     } }
0050 | }
0051 | }
0052 | }
```

L33-34: Closes or continues the surrounding block, expression, or call.

L35: Declares the named type; constructor properties define its stored data or dependencies.

L36: Connects a user interaction to the displayed state update or handler.

L37-38: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L39-40: Connects a user interaction to the displayed state update or handler.

L41-42: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L43-44: Connects a user interaction to the displayed state update or handler.

L45-46: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L47-48: Connects a user interaction to the displayed state update or handler.

L49: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L50-52: Closes or continues the surrounding block, expression, or call.

22. AttendanceReportScreen.kt

app/src/main/java/com/example/studenttracker/presentation/admin/AttendanceReportScreen.kt

Subject attendance report and Android CSV sharing.

102 source lines. SHA-256: d57504eced0b8d2a

The subject menu uses the backend-scoped list. The selected code and refresh counter key the report load.

The report and export retain the same subject snapshot. A percentage below 75 is highlighted; this threshold is currently fixed in the UI.

attendanceCsv writes a sanitized temporary filename inside cache/exports, removes exports older than 24 hours, and uses CsvExporter.

FileProvider grants read access only to the export URI. ACTION_SEND, ClipData, and the grant flag allow a selected app to read the CSV.

Source lines 1-32

```
0001 | package com.example.studenttracker.presentation.admin
0002 |
0003 | import android.content.ClipData
0004 | import android.content.Context
0005 | import android.content.Intent
0006 | import androidx.compose.foundation.layout.*
0007 | import androidx.compose.foundation.lazy.LazyColumn
0008 | import androidx.compose.foundation.lazy.items
0009 | import androidx.compose.material3.*
0010 | import androidx.compose.runtime.*
0011 | import androidx.compose.runtime.saveable.rememberSaveable
0012 | import androidx.compose.ui.Modifier
0013 | import androidx.compose.ui.platform.LocalContext
0014 | import androidx.compose.ui.unit.dp
0015 | import androidx.core.content.FileProvider
0016 | import com.example.studenttracker.data.network.*
0017 | import com.example.studenttracker.domain.model.CsvExporter
0018 | import com.example.studenttracker.presentation.components.*
0019 | import kotlinx.coroutines.CancellationException
0020 | import kotlinx.coroutines.Dispatchers
0021 | import kotlinx.coroutines.launch
0022 | import kotlinx.coroutines.withContext
0023 | import java.io.File
0024 |
0025 | @OptIn(ExperimentalMaterial3Api::class)
0026 | @Composable
0027 | fun AttendanceReportScreen(onNavigateBack: () -> Unit, subjectCode: String? = null) {
0028 |     var refresh by remember { mutableIntStateOf(0) }
0029 |     var selectedCode by rememberSaveable { mutableStateOf(subjectCode.orEmpty()) }
0030 |     var expanded by remember { mutableStateOf(false) }
0031 |     val context = LocalContext.current
0032 |     val scope = rememberCoroutineScope()
```

L1: Declares the Kotlin namespace; it organizes classes and functions.

L3-23: Imports the named library symbols used by this file.

L25: Applies the displayed annotation to the declaration.

L26: Marks the following function as a Compose UI function.

L27: Defines this Kotlin operation and its parameters; suspend allows asynchronous calls.

L28: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L29: Keeps this UI value across supported activity recreation; it is not server persistence.

L30: Creates observable local UI state; assignment triggers recomposition.

L31: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L32: Obtains a coroutine scope tied to this composable for button-triggered asynchronous work.

Source lines 33-64

```

0033 |     var exportError by remember { mutableStateOf<String?>(null) }
0034 |     var exporting by remember { mutableStateOf(false) }
0035 |     WorkspacePage("Attendance reports", onNavigateBack, actions = {
0036 |         TextButton(onClick = { refresh++ }, enabled = !exporting) { Text("Refresh") }
0037 |     }) { padding ->
0038 |         Column(Modifier.fillMaxSize().padding(padding).padding(16.dp), verticalArrangement =
> Arrangement.spacedBy(16.dp)) {
0039 |             RemoteContent(refresh, { NetworkModule.api.getSubjects().subjects.orEmpty() }) {
> subjects ->
0040 |                 if (subjects.isEmpty()) Text("No subjects are available yet.")
0041 |                 else ExposedDropdownMenuBox(expanded, { expanded = !expanded }) {
0042 |                     OutlinedTextField(subjects.find { it.subjectCode == selectedCode }?.let {
> "${it.subjectCode} · ${it.subjectName}" }?: "Select subject",
0043 |                     onValueChange = {}, readOnly = true, modifier =
> Modifier.fillMaxWidth().menuAnchor(ExposedDropdownMenuAnchorType.PrimaryNotEditable),
0044 |                     label = { Text("Subject") }, trailingIcon = {
> ExposedDropdownMenuDefaults.TrailingIcon(expanded) })
0045 |                     ExposedDropdownMenu(expanded, { expanded = false }) {
0046 |                         subjects.forEach { subject -> DropdownMenuItem(text = {
> Text("${subject.subjectCode} · ${subject.subjectName}") }, onClick = { selectedCode =
> subject.subjectCode; expanded = false; exportError = null }) }
0047 |                     }
0048 |                 }
0049 |             }
0050 |             if (selectedCode.isNotBlank()) {
0051 |                 val reportCode = selectedCode
0052 |                 RemoteContent(reportCode to refresh, {
> NetworkModule.api.getAttendanceReport(subjectCode = reportCode).report.orEmpty() }) { report ->
0053 |                     if (report.isEmpty()) Text("No attendance records or enrolled students for this
> subject.")
0054 |                     else {
0055 |                         Text("Submitted sessions only · Late counts as present; excused is
> excluded.", style = MaterialTheme.typography.bodySmall)
0056 |                         OutlinedButton(enabled = !exporting, onClick = {
0057 |                             exporting = true; exportError = null
0058 |                             scope.launch {
0059 |                                 try {
0060 |                                     val file = withContext(Dispatchers.IO) { attendanceCsv(context,
> reportCode, report) }
0061 |                                     val uri = FileProvider.getUriForFile(context,
> "${context.packageName}.provider", file)
0062 |                                     val intent = Intent(Intent.ACTION_SEND).apply {
0063 |                                         type = "text/csv"
0064 |                                         putExtra(Intent.EXTRA_STREAM, uri)

```

L33-34: Creates observable local UI state; assignment triggers recomposition.

L35: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L36: Connects a user interaction to the displayed state update or handler.

L37: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L38: Adds this UI element to the composition with the displayed content and parameters.

L39: Calls the named authenticated backend operation with the displayed request or key.

L40: Branches on the displayed condition; the accepted branch performs the following operation.

L41: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L42: Adds this UI element to the composition with the displayed content and parameters.

L43: Connects a user interaction to the displayed state update or handler.

L44: Assigns the displayed expression to a variable or named argument.

L45: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L46: Connects a user interaction to the displayed state update or handler.

L47-49: Closes or continues the surrounding block, expression, or call.

L50: Branches on the displayed condition; the accepted branch performs the following operation.

L51: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L52: Calls the named authenticated backend operation with the displayed request or key.

L53: Branches on the displayed condition; the accepted branch performs the following operation.

L54: Closes or continues the surrounding block, expression, or call.

L55: Adds this UI element to the composition with the displayed content and parameters.

L56: Connects a user interaction to the displayed state update or handler.

L57: Assigns the displayed expression to a variable or named argument.

L58: Starts asynchronous work in the UI scope so the button handler can return promptly.

L59: Starts an operation whose failures are handled by the following catch/finally blocks.

L60: Runs this block in the chosen coroutine dispatcher, often IO for network/file work.

L61-62: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L63: Assigns the displayed expression to a variable or named argument.

L64: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

Source lines 65-96

```
0065 |                                     clipData = ClipData.newRawUri("Attendance report", uri)
0066 |                                     addFlags(Intent.FLAG_GRANT_READ_URI_PERMISSION)
0067 |                                     }
0068 |                                     context.startActivity(Intent.createChooser(intent, "Share
> attendance report"))
0069 |                                     } catch (e: CancellationException) { throw e }
0070 |                                     catch (e: Exception) { exportError = "Unable to export the report.
> Please try again." }
0071 |                                     finally { exporting = false }
0072 |                                     }
0073 |                                     }) { Text(if (exporting) "Preparing export..." else "Export CSV") }
0074 |                                     exportError?.let { Text(it, color = MaterialTheme.colorScheme.error) }
0075 |                                     LazyColumn(Modifier.fillMaxSize(), verticalArrangement =
> Arrangement.spacedBy(10.dp)) {
0076 |                                     items(report, key = { it.studentId }) { item ->
0077 |                                     InfoCard(item.name, item.studentId) {
0078 |                                     val percentage = item.percentage.toFloatOrNull()
0079 |                                     Text(if (percentage == null) "No counted sessions" else
> "${item.percentage}% · ${item.totalPresent}/${item.totalClasses} classes",
0080 |                                     color = if (percentage != null && percentage < 75)
> MaterialTheme.colorScheme.error else MaterialTheme.colorScheme.onSurface)
0081 |                                     }
0082 |                                     }
0083 |                                     }
0084 |                                     }
0085 |                                     }
0086 |                                     } else Text("Select a subject to view attendance.")
0087 |                                     }
0088 |                                     }
0089 |                                     }
0090 |
0091 | private fun attendanceCsv(context: Context, code: String, report: List<AttendanceReportItem>): File
> {
0092 |     val directory = File(context.cacheDir, "exports").apply { check(isDirectory || mkdirs()) }
0093 |
0094 |     directory.listFiles()?.filter { it.isFile && it.lastModified() < System.currentTimeMillis() -
> 86_400_000 }?.forEach { it.delete() }
0095 |     val safeCode = code.replace(Regex("[^A-Za-z0-9_-]"), "_").take(80)
0096 |     val file = File.createTempFile("Attendance_${safeCode}_", ".csv", directory)
```

L65: Assigns the displayed expression to a variable or named argument.

L66: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L67: Closes or continues the surrounding block, expression, or call.

L68: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L69: Preserves coroutine cancellation rather than presenting it as an ordinary application error.

L70: Handles this exception type and translates it into the following failure behavior.

L71: Runs cleanup whether the preceding operation succeeded or failed.

L72: Closes or continues the surrounding block, expression, or call.

L73-74: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L75: Adds this UI element to the composition with the displayed content and parameters.

L76: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L77: Adds this UI element to the composition with the displayed content and parameters.

L78: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L79: Adds this UI element to the composition with the displayed content and parameters.

L80: Assigns the displayed expression to a variable or named argument.

L81-85: Closes or continues the surrounding block, expression, or call.

L86: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L87-89: Closes or continues the surrounding block, expression, or call.

L91: Defines this Kotlin operation and its parameters; suspend allows asynchronous calls.

L92: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L94: Transforms, selects, or indexes the displayed collection using its callback.

L95-96: Declares the displayed value or mutable state; its initializer supplies the initial contents.

Source lines 97-102

```
0097 |     val rows = listOf(listOf("Student_ID", "Name", "Total_Present", "Total_Classes", "Percentage"))
      |     > + report.map {
0098 |         listOf(it.studentId, it.name, it.totalPresent, it.totalClasses, it.percentage.ifBlank {
      |         > "N/A" })
0099 |     }
0100 |     file.writeText(CsvExporter.encode(rows), Charsets.UTF_8)
0101 |     return file
0102 | }
```

L97: Transforms, selects, or indexes the displayed collection using its callback.

L98: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L99: Closes or continues the surrounding block, expression, or call.

L100: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L101: Returns this expression as the operation result; subsequent statements in this path are skipped.

L102: Closes or continues the surrounding block, expression, or call.

23. LoginScreen.kt

app/src/main/java/com/example/studenttracker/presentation/auth/LoginScreen.kt

Google authentication interface and verification handshake.

176 source lines. SHA-256: 99c0e123fd87a89c

CredentialManager requests a Google ID token for BuildConfig.GOOGLE_WEB_CLIENT_ID. The configured endpoint must be present before the request begins.

initiateGoogleSignIn launches asynchronous work, parses the returned custom Google credential, begins the temporary token session, and requests getUserRole on Dispatchers.IO.

The server response must have a supported role and future expiry. The app stores metadata, publishes SignedInUser, and invokes onLoginSuccess.

Cancellation is rethrown. Missing credentials, user cancellation, and other exceptions clear the temporary session and display errors. finally ends the loading state.

Source lines 1-32

```
0001 | package com.example.studenttracker.presentation.auth
0002 |
0003 | import androidx.compose.foundation.background
0004 | import androidx.compose.foundation.layout.*
0005 | import androidx.compose.foundation.shape.RoundedCornerShape
0006 | import androidx.compose.material3.*
0007 | import androidx.compose.runtime.Composable
0008 | import androidx.compose.runtime.rememberCoroutineScope
0009 | import androidx.compose.ui.Alignment
0010 | import androidx.compose.ui.Modifier
0011 | import androidx.compose.ui.graphics.Color
0012 | import androidx.compose.ui.platform.LocalContext
0013 | import androidx.compose.ui.text.font.FontWeight
0014 | import androidx.compose.ui.text.style.TextAlign
0015 | import androidx.compose.ui.unit.dp
0016 | import androidx.compose.ui.unit.sp
0017 | import androidx.credentials.CredentialManager
0018 | import androidx.credentials.CustomCredential
0019 | import androidx.credentials.GetCredentialRequest
0020 | import androidx.credentials.exceptions.GetCredentialException
0021 | import androidx.credentials.exceptions.NoCredentialException
0022 | import com.google.android.identity.identity.googleid.GetGoogleIdOption
0023 | import com.google.android.identity.identity.googleid.GoogleIdTokenCredential
0024 | import kotlin.coroutines.launch
0025 |
0026 | import com.example.studenttracker.BuildConfig
0027 | import com.example.studenttracker.data.network.AuthSession
0028 | import com.example.studenttracker.data.network.SignedInUser
0029 | import com.example.studenttracker.data.network.NetworkModule
0030 | import kotlin.coroutines.CancellationException
0031 | import kotlin.coroutines.Dispatchers
0032 | import kotlin.coroutines.withContext
```

L1: Declares the Kotlin namespace; it organizes classes and functions.

L3-24: Imports the named library symbols used by this file.

L26-32: Imports the named library symbols used by this file.

Source lines 33-64

```
0033 | import androidx.compose.runtime.mutableStateOf
0034 | import androidx.compose.runtime.remember
0035 | import androidx.compose.runtime.getValue
0036 | import androidx.compose.runtime.setValue
0037 |
0038 | @Composable
0039 | fun LoginScreen(
0040 |     userPreferences: com.example.studenttracker.data.local.UserPreferences,
0041 |     onLoginSuccess: (String) -> Unit
```

```

0042 | ) {
0043 |     val context = LocalContext.current
0044 |     val coroutineScope = rememberCoroutineScope()
0045 |     val credentialManager = CredentialManager.create(context)
0046 |
0047 |     var isLoading by remember { mutableStateOf(false) }
0048 |
0049 |     var loginError by remember { mutableStateOf<String?>(null) }
0050 |     val webClientId = BuildConfig.GOOGLE_WEB_CLIENT_ID
0051 |
0052 |     fun initiateGoogleSignIn() {
0053 |         if (!NetworkModule.isConfigured) {
0054 |             loginError = "This app has not been connected to your institution yet. Contact the
> administrator."
0055 |             return
0056 |         }
0057 |         loginError = null
0058 |         coroutineScope.launch {
0059 |             try {
0060 |                 isLoading = true
0061 |                 val googleIdOption: GetGoogleIdOption = GetGoogleIdOption.Builder()
0062 |                     .setFilterByAuthorizedAccounts(false)
0063 |                     .setServerClientId(webClientId)
0064 |                     .setAutoSelectEnabled(false)

```

L33-36: Imports the named library symbols used by this file.

L38: Marks the following function as a Compose UI function.

L39: Defines this Kotlin operation and its parameters; suspend allows asynchronous calls.

L40-42: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L43: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L44: Obtains a coroutine scope tied to this composable for button-triggered asynchronous work.

L45: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L47: Creates observable local UI state; assignment triggers recomposition.

L49: Creates observable local UI state; assignment triggers recomposition.

L50: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L52: Defines this Kotlin operation and its parameters; suspend allows asynchronous calls.

L53: Branches on the displayed condition; the accepted branch performs the following operation.

L54: Assigns the displayed expression to a variable or named argument.

L55: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L56: Closes or continues the surrounding block, expression, or call.

L57: Assigns the displayed expression to a variable or named argument.

L58: Starts asynchronous work in the UI scope so the button handler can return promptly.

L59: Starts an operation whose failures are handled by the following catch/finally blocks.

L60: Assigns the displayed expression to a variable or named argument.

L61: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L62-64: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

Source lines 65-96

```

0065 |         .build()
0066 |
0067 |         val request: GetCredentialRequest = GetCredentialRequest.Builder()
0068 |             .addCredentialOption(googleIdOption)
0069 |             .build()
0070 |
0071 |         val result = credentialManager.getCredential(
0072 |             request = request,
0073 |             context = context
0074 |         )
0075 |
0076 |         val credential = result.credential
0077 |         if (credential is CustomCredential && credential.type ==
> GoogleIdTokenCredential.TYPE_GOOGLE_ID_TOKEN_CREDENTIAL) {
0078 |             val googleIdTokenCredential =

```

```

    > GoogleIdTokenCredential.createFrom(credential.data)
0079 |         AuthSession.begin(googleIdTokenCredential.idToken)
0080 |         val response = withContext(Dispatchers.IO) { NetworkModule.api.getUserRole() }
0081 |         check(response.status == "success" && response.role in setOf("admin", "faculty",
    > "student")) && response.expiresAt > System.currentTimeMillis() / 1000) {
0082 |             "Your account does not have an active role. Contact the administrator."
0083 |         }
0084 |         userPreferences.saveSession(response.email, response.role)
0085 |         AuthSession.complete(SignedInUser(response.email, response.name.orEmpty(),
    > response.role, response.expiresAt))
0086 |             onLoginSuccess(response.role)
0087 |         } else {
0088 |             error("Google returned an unsupported credential. Try signing in again.")
0089 |         }
0090 |     } catch (e: CancellationException) {
0091 |         AuthSession.clear()
0092 |         throw e
0093 |     } catch (e: NoCredentialException) {
0094 |         AuthSession.clear()
0095 |         loginError = "No Google account is available. Add an account to your device and try
    > again."
0096 |     } catch (e: GetCredentialException) {

```

L65: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L67: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L68-69: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L71: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L72-73: Assigns the displayed expression to a variable or named argument.

L74: Closes or continues the surrounding block, expression, or call.

L76: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L77: Branches on the displayed condition; the accepted branch performs the following operation.

L78: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L79: Carries the Google identity token for server verification; the token itself does not assign a role.

L80: Runs this block in the chosen coroutine dispatcher, often IO for network/file work.

L81-82: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L83: Closes or continues the surrounding block, expression, or call.

L84-87: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L88: Stops the current operation by raising the displayed error.

L89: Closes or continues the surrounding block, expression, or call.

L90: Preserves coroutine cancellation rather than presenting it as an ordinary application error.

L91: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L92: Stops the current operation by raising the displayed error.

L93: Handles this exception type and translates it into the following failure behavior.

L94: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L95: Assigns the displayed expression to a variable or named argument.

L96: Handles this exception type and translates it into the following failure behavior.

Source lines 97-128

```

0097 |         AuthSession.clear()
0098 |         loginError = "Sign-in was cancelled or unavailable. Please try again."
0099 |     } catch (e: Exception) {
0100 |         AuthSession.clear()
0101 |         loginError = e.message ?: "Unable to sign in. Please try again."
0102 |     } finally {
0103 |         isLoading = false
0104 |     }
0105 | }
0106 | }
0107 |
0108 | Box(
0109 |     modifier = Modifier
0110 |         .fillMaxSize()

```



```

0111 |         .safeDrawingPadding()
0112 |         .background(Color(0xFF121212)),
0113 |         contentAlignment = Alignment.Center
0114 |     ) {
0115 |         Column(
0116 |             horizontalAlignment = Alignment.CenterHorizontally,
0117 |             modifier = Modifier.padding(32.dp)
0118 |         ) {
0119 |             Box(
0120 |                 modifier = Modifier
0121 |                     .size(120.dp)
0122 |                     .background(Color(0xFF1E1E1E), shape = RoundedCornerShape(24.dp)),
0123 |                 contentAlignment = Alignment.Center
0124 |             ) {
0125 |                 Text(text = "\U0001f393", fontSize = 64.sp)
0126 |             }
0127 |             Spacer(modifier = Modifier.height(32.dp))
0128 |             Text(

```

L97: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L98: Assigns the displayed expression to a variable or named argument.

L99: Handles this exception type and translates it into the following failure behavior.

L100: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L101: Assigns the displayed expression to a variable or named argument.

L102: Runs cleanup whether the preceding operation succeeded or failed.

L103: Assigns the displayed expression to a variable or named argument.

L104-106: Closes or continues the surrounding block, expression, or call.

L108: Adds this UI element to the composition with the displayed content and parameters.

L109-110: Configures UI layout, sizing, spacing, or appearance; it does not modify academic records.

L111: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L112: Configures UI layout, sizing, spacing, or appearance; it does not modify academic records.

L113: Assigns the displayed expression to a variable or named argument.

L114: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L115: Adds this UI element to the composition with the displayed content and parameters.

L116: Assigns the displayed expression to a variable or named argument.

L117: Configures UI layout, sizing, spacing, or appearance; it does not modify academic records.

L118: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L119: Adds this UI element to the composition with the displayed content and parameters.

L120-122: Configures UI layout, sizing, spacing, or appearance; it does not modify academic records.

L123: Assigns the displayed expression to a variable or named argument.

L124: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L125: Adds this UI element to the composition with the displayed content and parameters.

L126: Closes or continues the surrounding block, expression, or call.

L127-128: Adds this UI element to the composition with the displayed content and parameters.

Source lines 129-160

```

0129 |         text = "Welcome to\nStudent Tracker",
0130 |         style = MaterialTheme.typography.headlineLarge,
0131 |         color = Color.White,
0132 |         fontWeight = FontWeight.Bold,
0133 |         textAlign = TextAlign.Center
0134 |     )
0135 |     Spacer(modifier = Modifier.height(16.dp))
0136 |     Text(
0137 |         text = "Log in with your institutional or personal Google account to continue.",
0138 |         style = MaterialTheme.typography.bodyMedium,
0139 |         color = Color.Gray,
0140 |         textAlign = TextAlign.Center
0141 |     )
0142 |     Spacer(modifier = Modifier.height(24.dp))
0143 |     loginError?.let { Text(it, color = MaterialTheme.colorScheme.error, textAlign =

```

```

    > TextAlign.Center) }
0144 |         Spacer(modifier = Modifier.height(24.dp))
0145 |         Button(
0146 |             onClick = { initiateGoogleSignIn() },
0147 |             enabled = !isLoading,
0148 |             colors = ButtonDefaults.buttonColors(
0149 |                 containerColor = Color.White,
0150 |                 contentColor = Color.Black,
0151 |                 disabledContainerColor = Color.LightGray
0152 |             ),
0153 |             shape = RoundedCornerShape(12.dp),
0154 |             modifier = Modifier
0155 |                 .fillMaxWidth()
0156 |                 .height(56.dp)
0157 |         ) {
0158 |             if (isLoading) {
0159 |                 CircularProgressIndicator(
0160 |                     modifier = Modifier.size(24.dp),

```

L129-133: Assigns the displayed expression to a variable or named argument.

L134: Closes or continues the surrounding block, expression, or call.

L135-136: Adds this UI element to the composition with the displayed content and parameters.

L137-140: Assigns the displayed expression to a variable or named argument.

L141: Closes or continues the surrounding block, expression, or call.

L142: Adds this UI element to the composition with the displayed content and parameters.

L143: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L144-145: Adds this UI element to the composition with the displayed content and parameters.

L146: Connects a user interaction to the displayed state update or handler.

L147-151: Assigns the displayed expression to a variable or named argument.

L152: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L153: Assigns the displayed expression to a variable or named argument.

L154-156: Configures UI layout, sizing, spacing, or appearance; it does not modify academic records.

L157: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L158: Branches on the displayed condition; the accepted branch performs the following operation.

L159: Adds this UI element to the composition with the displayed content and parameters.

L160: Configures UI layout, sizing, spacing, or appearance; it does not modify academic records.

Source lines 161-176

```

0161 |             color = Color.Black,
0162 |             strokeWidth = 2.dp
0163 |         )
0164 |         Spacer(modifier = Modifier.width(12.dp))
0165 |         Text("Signing in...", fontSize = 16.sp, fontWeight = FontWeight.SemiBold)
0166 |     } else {
0167 |         Text(
0168 |             text = "Sign in with Google",
0169 |             fontSize = 16.sp,
0170 |             fontWeight = FontWeight.SemiBold
0171 |         )
0172 |     }
0173 | }
0174 | }
0175 | }
0176 | }

```

L161-162: Assigns the displayed expression to a variable or named argument.

L163: Closes or continues the surrounding block, expression, or call.

L164-165: Adds this UI element to the composition with the displayed content and parameters.

L166: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L167: Adds this UI element to the composition with the displayed content and parameters.

L168-170: Assigns the displayed expression to a variable or named argument.

L171-176: Closes or continues the surrounding block, expression, or call.

24. DirectoryScreen.kt

app/src/main/java/com/example/studenttracker/presentation/components/DirectoryScreen.kt

Reusable searchable record directory.

41 source lines. SHA-256: 1c218149c52f13af

DirectoryRecord adapts different profile models into id, title, and subtitle.

The search matches the combined fields ignoring case. LazyColumn uses stable record IDs as keys; cards invoke onOpen.

The refresh counter triggers RemoteContent reload. Loading, empty results, errors, retry, and add actions are handled consistently.

Source lines 1-32

```
0001 | package com.example.studenttracker.presentation.components
0002 |
0003 | import androidx.compose.foundation.layout.*
0004 | import androidx.compose.foundation.lazy.LazyColumn
0005 | import androidx.compose.foundation.lazy.items
0006 | import androidx.compose.material3.*
0007 | import androidx.compose.runtime.*
0008 | import androidx.compose.runtime.saveable.rememberSaveable
0009 | import androidx.compose.ui.Modifier
0010 | import androidx.compose.ui.unit.dp
0011 |
0012 | data class DirectoryRecord(val id: String, val title: String, val subtitle: String)
0013 |
0014 | @Composable
0015 | fun DirectoryScreen(title: String, onBackPressed: () -> Unit, onAdd: () -> Unit, onOpen: (String) -> Unit,
    > load: suspend () -> List<DirectoryRecord>) {
0016 |     var refresh by remember { mutableIntStateOf(0) }
0017 |     var query by rememberSaveable { mutableStateOf("") }
0018 |     WorkspacePage(title, onBackPressed, actions = {
0019 |         TextButton(onClick = { refresh++ }) { Text("Refresh") }
0020 |         TextButton(onClick = onAdd) { Text("Add") }
0021 |     }) { padding ->
0022 |         Column(Modifier.fillMaxSize().padding(padding)) {
0023 |             OutlinedTextField(query, { query = it }, label = { Text("Search $title") }, modifier =
    > Modifier.fillMaxWidth().padding(16.dp), singleLine = true)
0024 |             RemoteContent(refresh, load) { records ->
0025 |                 val matches = records.filter { "${it.id} ${it.title}
    > ${it.subtitle}".contains(query.trim(), ignoreCase = true) }
0026 |                 LazyColumn(contentPadding = PaddingValues(16.dp), verticalArrangement =
    > Arrangement.spacedBy(12.dp)) {
0027 |                     if (matches.isEmpty()) item { Text(if (records.isEmpty()) "No records yet. Tap
    > Add to get started." else "No matching records.") }
0028 |                     items(matches, key = { it.id }) { record ->
0029 |                         Card(onClick = { onOpen(record.id) }, modifier = Modifier.fillMaxWidth()) {
0030 |                             Column(Modifier.padding(16.dp), verticalArrangement =
    > Arrangement.spacedBy(6.dp)) {
0031 |                                 Text(record.title, style = MaterialTheme.typography.titleMedium)
0032 |                                 Text(record.id, color = MaterialTheme.colorScheme.primary)
```

L1: Declares the Kotlin namespace; it organizes classes and functions.

L3-10: Imports the named library symbols used by this file.

L12: Declares the named type; constructor properties define its stored data or dependencies.

L14: Marks the following function as a Compose UI function.

L15: Defines this Kotlin operation and its parameters; suspend allows asynchronous calls.

L16: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L17: Keeps this UI value across supported activity recreation; it is not server persistence.

L18: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L19-20: Connects a user interaction to the displayed state update or handler.

L21: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L22-23: Adds this UI element to the composition with the displayed content and parameters.

L24: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L25: Transforms, selects, or indexes the displayed collection using its callback.

L26: Adds this UI element to the composition with the displayed content and parameters.

L27: Branches on the displayed condition; the accepted branch performs the following operation.

L28: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L29: Connects a user interaction to the displayed state update or handler.

L30-32: Adds this UI element to the composition with the displayed content and parameters.

Source lines 33-41

```
0033 |                                     Text(record.subtitle, style = MaterialTheme.typography.bodySmall)
0034 |                                     }
0035 |                                 }
0036 |                             }
0037 |                         }
0038 |                     }
0039 |                 }
0040 |             }
0041 | }
```

L33: Adds this UI element to the composition with the displayed content and parameters.

L34-41: Closes or continues the surrounding block, expression, or call.

25. SkeletonLoading.kt

app/src/main/java/com/example/studenttracker/presentation/components/SkeletonLoading.kt

Reusable placeholder animation left in the project.

40 source lines. SHA-256: bedd4e48df60054b

SkeletonLoadingCard uses an infinite transition to animate the opacity of rounded gray shapes.

The current directory screens use RemoteContent loading indicators; this helper is not proof that shimmer is displayed in every live screen.

Source lines 1-32

```
0001 | package com.example.studenttracker.presentation.components
0002 |
0003 | import androidx.compose.animation.core.*
0004 | import androidx.compose.foundation.background
0005 | import androidx.compose.foundation.layout.Box
0006 | import androidx.compose.foundation.layout.fillMaxWidth
0007 | import androidx.compose.foundation.layout.height
0008 | import androidx.compose.foundation.shape.RoundedCornerShape
0009 | import androidx.compose.runtime.Composable
0010 | import androidx.compose.runtime.getValue
0011 | import androidx.compose.ui.Modifier
0012 | import androidx.compose.ui.draw.clip
0013 | import androidx.compose.ui.graphics.Color
0014 | import androidx.compose.ui.unit.Dp
0015 | import androidx.compose.ui.unit.dp
0016 |
0017 | @Composable
0018 | fun SkeletonLoadingCard(
0019 |     modifier: Modifier = Modifier,
0020 |     height: Dp = 100.dp
0021 | ) {
0022 |     val infiniteTransition = rememberInfiniteTransition(label = "skeleton_transition")
0023 |     val alpha by infiniteTransition.animateFloat(
0024 |         initialValue = 0.2f,
0025 |         targetValue = 0.7f,
0026 |         animationSpec = infiniteRepeatable(
0027 |             animation = tween(1000, easing = LinearEasing),
0028 |             repeatMode = RepeatMode.Reverse
0029 |         ),
0030 |         label = "skeleton_alpha"
0031 |     )
0032 | }
```

L1: Declares the Kotlin namespace; it organizes classes and functions.

L3-15: Imports the named library symbols used by this file.

L17: Marks the following function as a Compose UI function.

L18: Defines this Kotlin operation and its parameters; suspend allows asynchronous calls.

L19: Configures UI layout, sizing, spacing, or appearance; it does not modify academic records.

L20-21: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L22-23: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L24-28: Assigns the displayed expression to a variable or named argument.

L29: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L30: Assigns the displayed expression to a variable or named argument.

L31: Closes or continues the surrounding block, expression, or call.

Source lines 33-40

```
0033 |     Box(
0034 |         modifier = modifier
0035 |         .fillMaxWidth()
0036 |         .height(height)
0037 |         .clip(RoundedCornerShape(12.dp))
```

```
0038 |         .background(Color.Gray.copy(alpha = alpha))
0039 |     )
0040 | }
```

L33: Adds this UI element to the composition with the displayed content and parameters.

L34: Assigns the displayed expression to a variable or named argument.

L35-38: Configures UI layout, sizing, spacing, or appearance; it does not modify academic records.

L39-40: Closes or continues the surrounding block, expression, or call.

26. WorkspaceComponents.kt

app/src/main/java/com/example/studenttracker/presentation/components/WorkspaceComponents.kt

Shared page, asynchronous content, cards, and back-navigation guard.

73 source lines. SHA-256: 4e701fdeda93608

WorkspacePage wraps a Scaffold and top bar, supports optional back navigation and caller-supplied action buttons.

RemoteContent remembers data, loading, error, and retry state keyed by the record identity. LaunchedEffect executes the suspend loader; cancellation is rethrown; retry increments an attempt counter.

InfoCard wraps a title and optional subtitle around caller content. guardedBack prevents leaving during a write and asks before discarding unsaved changes.

Source lines 1-32

```
0001 | package com.example.studenttracker.presentation.components
0002 |
0003 | import androidx.activity.compose.BackHandler
0004 | import androidx.compose.foundation.layout.*
0005 | import androidx.compose.material.icons.Icons
0006 | import androidx.compose.material.icons.automirrored.filled.ArrowBack
0007 | import androidx.compose.material3.*
0008 | import androidx.compose.runtime.*
0009 | import androidx.compose.ui.Modifier
0010 | import androidx.compose.ui.unit.dp
0011 | import kotlinx.coroutines.CancellationException
0012 |
0013 | @OptIn(ExperimentalMaterial3Api::class)
0014 | @Composable
0015 | fun WorkspacePage(title: String, onBack: (() -> Unit)? = null, actions: @Composable RowScope.() ->
    > Unit = {}, content: @Composable (PaddingValues) -> Unit) {
0016 |     Scaffold(topBar = {
0017 |         TopAppBar(title = { Text(title) }, navigationIcon = {
0018 |             if (onBack != null) IconButton(onClick = onBack) {
0019 |                 Icon(Icons.AutoMirrored.Filled.ArrowBack, "Back")
0020 |             }, actions = actions)
0021 |         }, content = content)
0022 |     }
0023 | ) @Composable
0024 | fun <T : Any> RemoteContent(key: Any = Unit, load: suspend () -> T, onBack: (() -> Unit)? = null,
    > content: @Composable (T) -> Unit) {
0025 |     var data by remember(key) { mutableStateOf<T?>(null) }
0026 |     var error by remember(key) { mutableStateOf<String?>(null) }
0027 |     var attempt by remember(key) { mutableIntStateOf(0) }
0028 |     var loading by remember(key) { mutableStateOf(true) }
0029 |     LaunchedEffect(key, attempt) {
0030 |         loading = true
0031 |         error = null
0032 |         try { data = load() }
```

L1: Declares the Kotlin namespace; it organizes classes and functions.

L3-11: Imports the named library symbols used by this file.

L13: Applies the displayed annotation to the declaration.

L14: Marks the following function as a Compose UI function.

L15: Defines this Kotlin operation and its parameters; suspend allows asynchronous calls.

L16: Adds this UI element to the composition with the displayed content and parameters.

L17: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L18: Branches on the displayed condition; the accepted branch performs the following operation.

L19-20: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L21: Closes or continues the surrounding block, expression, or call.

L23: Marks the following function as a Compose UI function.

L24: Defines this Kotlin operation and its parameters; suspend allows asynchronous calls.

L25-26: Creates observable local UI state; assignment triggers recomposition.

L27: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L28: Creates observable local UI state; assignment triggers recomposition.

L29: Runs suspend work when the displayed effect key changes; old work is cancelled.

L30-31: Assigns the displayed expression to a variable or named argument.

L32: Starts an operation whose failures are handled by the following catch/finally blocks.

Source lines 33-64

```
0033 |         catch (e: CancellationException) { throw e }
0034 |         catch (e: Exception) { error = e.message ?: "Unable to load data. Please try again." }
0035 |         finally { loading = false }
0036 |     }
0037 |     when {
0038 |         loading -> Column(Modifier.fillMaxWidth().safeDrawingPadding().padding(24.dp),
0039 |         > verticalArrangement = Arrangement.spacedBy(12.dp)) {
0040 |             if (onBack != null) TextButton(onClick = onBack) { Text("Back") }
0041 |             LinearProgressIndicator(Modifier.fillMaxWidth())
0042 |             Text("Loading records...")
0043 |         }
0044 |         error != null -> Column(Modifier.safeDrawingPadding().padding(24.dp), verticalArrangement =
0045 |         > Arrangement.spacedBy(12.dp)) {
0046 |             if (onBack != null) TextButton(onClick = onBack) { Text("Back") }
0047 |             Text(error!!, color = MaterialTheme.colorScheme.error)
0048 |             Button(onClick = { attempt++ }) { Text("Retry") }
0049 |         }
0050 |     }
0051 | }
0052 | @Composable
0053 | fun InfoCard(title: String, subtitle: String? = null, content: @Composable ColumnScope.() -> Unit =
0054 | > {} ) {
0055 |     Card(Modifier.fillMaxWidth()) {
0056 |         Column(Modifier.padding(16.dp), verticalArrangement = Arrangement.spacedBy(10.dp)) {
0057 |             Text(title, style = MaterialTheme.typography.titleMedium)
0058 |             subtitle?.let { Text(it, style = MaterialTheme.typography.bodyMedium, color =
0059 |             > MaterialTheme.colorScheme.onSurfaceVariant) }
0060 |             content()
0061 |         }
0062 |     }
0063 | }
0064 | @Composable
0065 | fun guardedBack(dirty: Boolean, busy: Boolean, onBack: () -> Unit): () -> Unit {
```

L33: Preserves coroutine cancellation rather than presenting it as an ordinary application error.

L34: Handles this exception type and translates it into the following failure behavior.

L35: Runs cleanup whether the preceding operation succeeded or failed.

L36: Closes or continues the surrounding block, expression, or call.

L37: Branches on the displayed condition; the accepted branch performs the following operation.

L38: Configures UI layout, sizing, spacing, or appearance; it does not modify academic records.

L39: Branches on the displayed condition; the accepted branch performs the following operation.

L40-41: Adds this UI element to the composition with the displayed content and parameters.

L42: Closes or continues the surrounding block, expression, or call.

L43: Configures UI layout, sizing, spacing, or appearance; it does not modify academic records.

L44: Branches on the displayed condition; the accepted branch performs the following operation.

L45: Adds this UI element to the composition with the displayed content and parameters.

L46: Connects a user interaction to the displayed state update or handler.

L47: Closes or continues the surrounding block, expression, or call.

L48: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L49-50: Closes or continues the surrounding block, expression, or call.

L52: Marks the following function as a Compose UI function.

L53: Defines this Kotlin operation and its parameters; suspend allows asynchronous calls.

L54-56: Adds this UI element to the composition with the displayed content and parameters.

L57-58: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L59-61: Closes or continues the surrounding block, expression, or call.

L63: Marks the following function as a Compose UI function.

L64: Defines this Kotlin operation and its parameters; suspend allows asynchronous calls.

Source lines 65-73

```
0065 |     var confirm by remember { mutableStateOf(false) }
0066 |     val back = { if (!busy) { if (dirty) confirm = true else onBack() } }
0067 |     BackHandler(enabled = dirty || busy, onBack = back)
0068 |     if (confirm) AlertDialog(onDismissRequest = { confirm = false }, title = { Text("Discard unsaved
    > changes?") },
0069 |         text = { Text("Changes on this screen have not been saved.") },
0070 |         confirmButton = { TextButton(onClick = onBack) { Text("Discard") } },
0071 |         dismissButton = { TextButton(onClick = { confirm = false }) { Text("Keep editing") } })
0072 |     return back
0073 | }
```

L65: Creates observable local UI state; assignment triggers recomposition.

L66: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L67: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L68: Branches on the displayed condition; the accepted branch performs the following operation.

L69: Assigns the displayed expression to a variable or named argument.

L70-71: Connects a user interaction to the displayed state update or handler.

L72: Returns this expression as the operation result; subsequent statements in this path are skipped.

L73: Closes or continues the surrounding block, expression, or call.

27. AddFacultyScreen.kt

app/src/main/java/com/example/studenttracker/presentation/faculty/AddFacultyScreen.kt

Create a faculty profile.

214 source lines. SHA-256: 63c4896e497718b3

rememberSaveable holds each form field. Derived dirty state includes every editable field; BackHandler and the discard dialog guard unsaved changes.

The form starts empty; the identifier is supplied when creating the record.

The save handler trims input, validates required values and applicable email formats, then launches the API request. Busy state disables fields and navigation; success shows feedback and returns.

Cancellation is rethrown. Other failures show a message; finally restores the busy flag. Backend validation remains authoritative.

CustomTextField wraps OutlinedTextField and the appropriate keyboard type. Modifier chains size, padding, background, and alignment without changing backend data.

Source lines 1-32

```
0001 | package com.example.studenttracker.presentation.faculty
0002 |
0003 | import android.widget.Toast
0004 | import androidx.compose.foundation.layout.*
0005 | import androidx.compose.foundation.rememberScrollState
0006 | import androidx.compose.foundation.shape.RoundedCornerShape
0007 | import androidx.compose.foundation.text.KeyboardOptions
0008 | import androidx.compose.foundation.verticalScroll
0009 | import androidx.compose.material.icons.Icons
0010 | import androidx.compose.material.icons.filled.ArrowBack
0011 | import androidx.compose.material3.*
0012 | import androidx.compose.runtime.*
0013 | import androidx.compose.runtime.saveable.rememberSaveable
0014 | import androidx.compose.ui.Modifier
0015 | import androidx.compose.ui.graphics.Color
0016 | import androidx.compose.ui.platform.LocalContext
0017 | import androidx.compose.ui.text.font.FontWeight
0018 | import androidx.compose.ui.text.input.KeyboardType
0019 | import androidx.compose.ui.unit.dp
0020 | import com.example.studenttracker.data.network.AddFacultyRequest
0021 | import com.example.studenttracker.data.network.NetworkModule
0022 | import kotlinx.coroutines.launch
0023 |
0024 | @OptIn(ExperimentalMaterial3Api::class)
0025 | @Composable
0026 | fun AddFacultyScreen(
0027 |     onNavigateBack: () -> Unit
0028 | ) {
0029 |     val context = LocalContext.current
0030 |
0031 |     var employeeId by rememberSaveable { mutableStateOf("") }
0032 |     var name by rememberSaveable { mutableStateOf("") }
```

L1: Declares the Kotlin namespace; it organizes classes and functions.

L3-22: Imports the named library symbols used by this file.

L24: Applies the displayed annotation to the declaration.

L25: Marks the following function as a Compose UI function.

L26: Defines this Kotlin operation and its parameters; suspend allows asynchronous calls.

L27-28: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L29: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L31-32: Keeps this UI value across supported activity recreation; it is not server persistence.

Source lines 33-64

```

0033 |     var email by rememberSaveable { mutableStateOf("") }
0034 |     var phone by rememberSaveable { mutableStateOf("") }
0035 |     var department by rememberSaveable { mutableStateOf("") }
0036 |     var designation by rememberSaveable { mutableStateOf("") }
0037 |     var joiningDate by rememberSaveable { mutableStateOf("") }
0038 |
0039 |     var isError by remember { mutableStateOf(false) }
0040 |     var isSaving by remember { mutableStateOf(false) }
0041 |     val coroutineScope = rememberCoroutineScope()
0042 |
0043 |     fun validateAndSave() {
0044 |         if (employeeId.isBlank() || name.isBlank() || email.isBlank() || department.isBlank() ||
> designation.isBlank()) {
0045 |             isError = true
0046 |             Toast.makeText(context, "Please fill in all required fields", Toast.LENGTH_SHORT).show()
0047 |             return
0048 |         }
0049 |
0050 |         isSaving = true
0051 |         coroutineScope.launch {
0052 |             try {
0053 |                 val request = AddFacultyRequest(
0054 |                     employeeId = employeeId.trim(),
0055 |                     name = name.trim(),
0056 |                     email = email.trim(),
0057 |                     phone = phone.trim(),
0058 |                     department = department.trim(),
0059 |                     designation = designation.trim(),
0060 |                     joiningDate = joiningDate.trim()
0061 |                 )
0062 |                 val response = NetworkModule.api.addFaculty(request)
0063 |                 if (response.status == "success") {
0064 |                     Toast.makeText(context, "Faculty added successfully!",
> Toast.LENGTH_SHORT).show()

```

L33-37: Keeps this UI value across supported activity recreation; it is not server persistence.

L39-40: Creates observable local UI state; assignment triggers recomposition.

L41: Obtains a coroutine scope tied to this composable for button-triggered asynchronous work.

L43: Defines this Kotlin operation and its parameters; suspend allows asynchronous calls.

L44: Branches on the displayed condition; the accepted branch performs the following operation.

L45: Assigns the displayed expression to a variable or named argument.

L46-47: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L48: Closes or continues the surrounding block, expression, or call.

L50: Assigns the displayed expression to a variable or named argument.

L51: Starts asynchronous work in the UI scope so the button handler can return promptly.

L52: Starts an operation whose failures are handled by the following catch/finally blocks.

L53: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L54-60: Assigns the displayed expression to a variable or named argument.

L61: Closes or continues the surrounding block, expression, or call.

L62: Calls the named authenticated backend operation with the displayed request or key.

L63: Branches on the displayed condition; the accepted branch performs the following operation.

L64: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

Source lines 65-96

```

0065 |         onBackPressed()
0066 |     } else {
0067 |         Toast.makeText(context, "Error: ${response.message}", Toast.LENGTH_LONG).show()
0068 |     }
0069 | } catch (e: kotlinx.coroutines.CancellationException) { throw e
0070 | } catch (e: Exception) {
0071 |     Toast.makeText(context, "Failed to save: ${e.message}", Toast.LENGTH_LONG).show()
0072 | } finally {
0073 |     isSaving = false
0074 | }
0075 | }

```

```

0076 |     }
0077 |
0078 |     var showUnsavedDialog by remember { mutableStateOf(false) }
0079 |     val hasUnsavedChanges = employeeId.isNotBlank() || name.isNotBlank() || email.isNotBlank() ||
    > phone.isNotBlank() || department.isNotBlank() || designation.isNotBlank() ||
    > joiningDate.isNotBlank()
0080 |
0081 |     androidx.activity.compose.BackHandler(enabled = hasUnsavedChanges || isSaving) {
0082 |         if (!isSaving) showUnsavedDialog = true
0083 |     }
0084 |
0085 |     val handleBackPressed = {
0086 |         if (isSaving) {
0087 |
0088 |         } else if (hasUnsavedChanges) {
0089 |             showUnsavedDialog = true
0090 |         } else {
0091 |             onBackPressed()
0092 |         }
0093 |     }
0094 |
0095 |     if (showUnsavedDialog) {
0096 |         AlertDialog(

```

L65-67: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L68: Closes or continues the surrounding block, expression, or call.

L69: Preserves coroutine cancellation rather than presenting it as an ordinary application error.

L70: Handles this exception type and translates it into the following failure behavior.

L71: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L72: Runs cleanup whether the preceding operation succeeded or failed.

L73: Assigns the displayed expression to a variable or named argument.

L74-76: Closes or continues the surrounding block, expression, or call.

L78: Creates observable local UI state; assignment triggers recomposition.

L79: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L81: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L82: Branches on the displayed condition; the accepted branch performs the following operation.

L83: Closes or continues the surrounding block, expression, or call.

L85: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L86: Branches on the displayed condition; the accepted branch performs the following operation.

L88: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L89: Assigns the displayed expression to a variable or named argument.

L90-91: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L92-93: Closes or continues the surrounding block, expression, or call.

L95: Branches on the displayed condition; the accepted branch performs the following operation.

L96: Adds this UI element to the composition with the displayed content and parameters.

Source lines 97-128

```

0097 |         onDismissRequest = { showUnsavedDialog = false },
0098 |         title = { Text("Discard Changes?", color = Color.White) },
0099 |         text = { Text("You have unsaved changes. Are you sure you want to go back?", color =
    > Color.LightGray) },
0100 |         containerColor = Color(0xFF1E1E1E),
0101 |         confirmButton = {
0102 |             TextButton(onClick = { onBackPressed() }) { Text("Discard", color = Color.Red) }
0103 |         },
0104 |         dismissButton = {
0105 |             TextButton(onClick = { showUnsavedDialog = false }) { Text("Cancel", color =
    > Color.Gray) }
0106 |         }
0107 |     )
0108 | }
0109 |
0110 | Scaffold(

```

```

0111 |         topBar = {
0112 |             TopAppBar(
0113 |                 title = { Text("Add New Faculty", color = Color.White) },
0114 |                 navigationIcon = {
0115 |                     IconButton(onClick = handleBackPressed) {
0116 |                         Icon(Icons.Default.ArrowBack, contentDescription = "Back", tint =
> Color.White)
0117 |                     }
0118 |                 },
0119 |                 colors = TopAppBarDefaults.topAppBarColors(containerColor = Color(0xFF1E1E1E))
0120 |             )
0121 |         },
0122 |         containerColor = Color(0xFF121212)
0123 |     ) { paddingValues ->
0124 |         Column(
0125 |             modifier = Modifier
0126 |                 .fillMaxSize()
0127 |                 .padding(paddingValues)
0128 |                 .padding(16.dp)

```

L97-101: Assigns the displayed expression to a variable or named argument.

L102: Connects a user interaction to the displayed state update or handler.

L103: Closes or continues the surrounding block, expression, or call.

L104: Assigns the displayed expression to a variable or named argument.

L105: Connects a user interaction to the displayed state update or handler.

L106-108: Closes or continues the surrounding block, expression, or call.

L110: Adds this UI element to the composition with the displayed content and parameters.

L111: Assigns the displayed expression to a variable or named argument.

L112: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L113-114: Assigns the displayed expression to a variable or named argument.

L115: Connects a user interaction to the displayed state update or handler.

L116: Adds this UI element to the composition with the displayed content and parameters.

L117-118: Closes or continues the surrounding block, expression, or call.

L119: Assigns the displayed expression to a variable or named argument.

L120-121: Closes or continues the surrounding block, expression, or call.

L122: Assigns the displayed expression to a variable or named argument.

L123: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L124: Adds this UI element to the composition with the displayed content and parameters.

L125-128: Configures UI layout, sizing, spacing, or appearance; it does not modify academic records.

Source lines 129-160

```

0129 |         .verticalScroll(rememberScrollState()),
0130 |         verticalArrangement = Arrangement.spacedBy(16.dp)
0131 |     ) {
0132 |         Text("Faculty Details", color = Color(0xFF03DAC5), style =
> MaterialTheme.typography.titleMedium)
0133 |
0134 |         OutlinedTextField(
0135 |             enabled = !isSaving,
0136 |             value = employeeId, onValueChange = { employeeId = it },
0137 |             label = { Text("Employee ID *") },
0138 |             modifier = Modifier.fillMaxWidth(),
0139 |             isError = isError && employeeId.isBlank()
0140 |         )
0141 |
0142 |         OutlinedTextField(
0143 |             enabled = !isSaving,
0144 |             value = name, onValueChange = { name = it },
0145 |             label = { Text("Full Name *") },
0146 |             modifier = Modifier.fillMaxWidth(),
0147 |             isError = isError && name.isBlank()
0148 |         )
0149 |
0150 |         OutlinedTextField(

```

```

0151 |         enabled = !isSaving,
0152 |         value = email, onValueChange = { email = it },
0153 |         label = { Text("Email *") },
0154 |         keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Email),
0155 |         modifier = Modifier.fillMaxWidth(),
0156 |         isError = isError && email.isBlank()
0157 |     )
0158 |
0159 |     OutlinedTextField(
0160 |         enabled = !isSaving,

```

L129: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L130: Assigns the displayed expression to a variable or named argument.

L131: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L132: Adds this UI element to the composition with the displayed content and parameters.

L134: Adds this UI element to the composition with the displayed content and parameters.

L135: Assigns the displayed expression to a variable or named argument.

L136: Connects a user interaction to the displayed state update or handler.

L137: Assigns the displayed expression to a variable or named argument.

L138: Configures UI layout, sizing, spacing, or appearance; it does not modify academic records.

L139: Assigns the displayed expression to a variable or named argument.

L140: Closes or continues the surrounding block, expression, or call.

L142: Adds this UI element to the composition with the displayed content and parameters.

L143: Assigns the displayed expression to a variable or named argument.

L144: Connects a user interaction to the displayed state update or handler.

L145: Assigns the displayed expression to a variable or named argument.

L146: Configures UI layout, sizing, spacing, or appearance; it does not modify academic records.

L147: Assigns the displayed expression to a variable or named argument.

L148: Closes or continues the surrounding block, expression, or call.

L150: Adds this UI element to the composition with the displayed content and parameters.

L151: Assigns the displayed expression to a variable or named argument.

L152: Connects a user interaction to the displayed state update or handler.

L153-154: Assigns the displayed expression to a variable or named argument.

L155: Configures UI layout, sizing, spacing, or appearance; it does not modify academic records.

L156: Assigns the displayed expression to a variable or named argument.

L157: Closes or continues the surrounding block, expression, or call.

L159: Adds this UI element to the composition with the displayed content and parameters.

L160: Assigns the displayed expression to a variable or named argument.

Source lines 161-192

```

0161 |         value = phone, onValueChange = { phone = it },
0162 |         label = { Text("Phone Number") },
0163 |         keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Phone),
0164 |         modifier = Modifier.fillMaxWidth()
0165 |     )
0166 |
0167 |     HorizontalDivider(modifier = Modifier.padding(vertical = 8.dp), color = Color.DarkGray)
0168 |     Text("Professional Info", color = Color(0xFF03DAC5), style =
> MaterialTheme.typography.titleMedium)
0169 |
0170 |     Row(modifier = Modifier.fillMaxWidth(), horizontalArrangement =
> Arrangement.spacedBy(16.dp)) {
0171 |         OutlinedTextField(
0172 |             enabled = !isSaving,
0173 |             value = department, onValueChange = { department = it },
0174 |             label = { Text("Department *") },
0175 |             modifier = Modifier.weight(1f),
0176 |             isError = isError && department.isBlank()
0177 |         )
0178 |         OutlinedTextField(

```

```

0179 |         enabled = !isSaving,
0180 |         value = designation, onValueChange = { designation = it },
0181 |         label = { Text("Designation *") },
0182 |         modifier = Modifier.weight(1f),
0183 |         isError = isError && designation.isBlank()
0184 |     )
0185 | }
0186 |
0187 | OutlinedTextField(
0188 |     enabled = !isSaving,
0189 |     value = joiningDate, onValueChange = { joiningDate = it },
0190 |     label = { Text("Joining Date (YYYY-MM-DD)") },
0191 |     modifier = Modifier.fillMaxWidth()
0192 | )

```

L161: Connects a user interaction to the displayed state update or handler.

L162-163: Assigns the displayed expression to a variable or named argument.

L164: Configures UI layout, sizing, spacing, or appearance; it does not modify academic records.

L165: Closes or continues the surrounding block, expression, or call.

L167: Configures UI layout, sizing, spacing, or appearance; it does not modify academic records.

L168: Adds this UI element to the composition with the displayed content and parameters.

L170-171: Adds this UI element to the composition with the displayed content and parameters.

L172: Assigns the displayed expression to a variable or named argument.

L173: Connects a user interaction to the displayed state update or handler.

L174: Assigns the displayed expression to a variable or named argument.

L175: Configures UI layout, sizing, spacing, or appearance; it does not modify academic records.

L176: Assigns the displayed expression to a variable or named argument.

L177: Closes or continues the surrounding block, expression, or call.

L178: Adds this UI element to the composition with the displayed content and parameters.

L179: Assigns the displayed expression to a variable or named argument.

L180: Connects a user interaction to the displayed state update or handler.

L181: Assigns the displayed expression to a variable or named argument.

L182: Configures UI layout, sizing, spacing, or appearance; it does not modify academic records.

L183: Assigns the displayed expression to a variable or named argument.

L184-185: Closes or continues the surrounding block, expression, or call.

L187: Adds this UI element to the composition with the displayed content and parameters.

L188: Assigns the displayed expression to a variable or named argument.

L189: Connects a user interaction to the displayed state update or handler.

L190: Assigns the displayed expression to a variable or named argument.

L191: Configures UI layout, sizing, spacing, or appearance; it does not modify academic records.

L192: Closes or continues the surrounding block, expression, or call.

Source lines 193-214

```

0193 |
0194 |         Spacer(modifier = Modifier.height(24.dp))
0195 |
0196 |         Button(
0197 |             onClick = { validateAndSave() },
0198 |             enabled = !isSaving,
0199 |             modifier = Modifier.fillMaxWidth().height(50.dp),
0200 |             shape = RoundedCornerShape(12.dp),
0201 |             colors = ButtonDefaults.buttonColors(containerColor = Color(0xFF03DAC5),
> disabledContainerColor = Color.DarkGray)
0202 |         ) {
0203 |             if (isSaving) {
0204 |                 CircularProgressIndicator(modifier = Modifier.size(20.dp), color = Color.White,
> strokeWidth = 2.dp)
0205 |                 Spacer(modifier = Modifier.width(8.dp))
0206 |                 Text("Saving...", color = Color.Black)
0207 |             } else {

```

```

0208 |         Text("Save Faculty", color = Color.Black, fontWeight = FontWeight.Bold, style =
      > MaterialTheme.typography.titleMedium)
0209 |     }
0210 | }
0211 |     Spacer(modifier = Modifier.height(40.dp))
0212 | }
0213 | }
0214 | }

```

L194: Adds this UI element to the composition with the displayed content and parameters.

L196: Adds this UI element to the composition with the displayed content and parameters.

L197: Connects a user interaction to the displayed state update or handler.

L198: Assigns the displayed expression to a variable or named argument.

L199: Configures UI layout, sizing, spacing, or appearance; it does not modify academic records.

L200-201: Assigns the displayed expression to a variable or named argument.

L202: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L203: Branches on the displayed condition; the accepted branch performs the following operation.

L204-206: Adds this UI element to the composition with the displayed content and parameters.

L207: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L208: Adds this UI element to the composition with the displayed content and parameters.

L209-210: Closes or continues the surrounding block, expression, or call.

L211: Adds this UI element to the composition with the displayed content and parameters.

L212-214: Closes or continues the surrounding block, expression, or call.

28. EditFacultyScreen.kt

app/src/main/java/com/example/studenttracker/presentation/faculty/EditFacultyScreen.kt

Edit a faculty profile.

181 source lines. SHA-256: 7258ae359de0276a

rememberSaveable holds each form field. Derived dirty state includes every editable field; BackHandler and the discard dialog guard unsaved changes.

Initial values come from the typed server record; the identifier stays immutable and the expected revision is sent.

The save handler trims input, validates required values and applicable email formats, then launches the API request. Busy state disables fields and navigation; success shows feedback and returns.

Cancellation is rethrown. Other failures show a message; finally restores the busy flag. Backend validation remains authoritative.

CustomTextField wraps OutlinedTextField and the appropriate keyboard type. Modifier chains size, padding, background, and alignment without changing backend data.

Source lines 1-32

```
0001 | package com.example.studenttracker.presentation.faculty
0002 |
0003 | import android.widget.Toast
0004 | import androidx.compose.foundation.layout.*
0005 | import androidx.compose.foundation.rememberScrollState
0006 | import androidx.compose.foundation.shape.RoundedCornerShape
0007 | import androidx.compose.foundation.text.KeyboardOptions
0008 | import androidx.compose.foundation.verticalScroll
0009 | import androidx.compose.material.icons.Icons
0010 | import androidx.compose.material.icons.filled.ArrowBack
0011 | import androidx.compose.material3.*
0012 | import androidx.compose.runtime.*
0013 | import androidx.compose.runtime.saveable.rememberSaveable
0014 | import androidx.compose.ui.Modifier
0015 | import androidx.compose.ui.graphics.Color
0016 | import androidx.compose.ui.platform.LocalContext
0017 | import androidx.compose.ui.text.font.FontWeight
0018 | import androidx.compose.ui.text.input.KeyboardType
0019 | import androidx.compose.ui.unit.dp
0020 | import com.example.studenttracker.data.network.Faculty
0021 | import com.example.studenttracker.data.network.NetworkModule
0022 | import com.example.studenttracker.data.network.UpdateFacultyRequest
0023 | import kotlin.coroutines.launch
0024 |
0025 | @OptIn(ExperimentalMaterial3Api::class)
0026 | @Composable
0027 | fun EditFacultyScreen(
0028 |     faculty: Faculty,
0029 |     onNavigateBack: () -> Unit
0030 | ) {
0031 |     val context = LocalContext.current
0032 |     val coroutineScope = rememberCoroutineScope()
```

L1: Declares the Kotlin namespace; it organizes classes and functions.

L3-23: Imports the named library symbols used by this file.

L25: Applies the displayed annotation to the declaration.

L26: Marks the following function as a Compose UI function.

L27: Defines this Kotlin operation and its parameters; suspend allows asynchronous calls.

L28-30: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L31: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L32: Obtains a coroutine scope tied to this composable for button-triggered asynchronous work.

Source lines 33-64

```

0033 |
0034 |     var name by rememberSaveable { mutableStateOf(faculty.name) }
0035 |     var email by rememberSaveable { mutableStateOf(faculty.email) }
0036 |     var phone by rememberSaveable { mutableStateOf(faculty.phone) }
0037 |     var department by rememberSaveable { mutableStateOf(faculty.department) }
0038 |     var designation by rememberSaveable { mutableStateOf(faculty.designation) }
0039 |     var joiningDate by rememberSaveable { mutableStateOf(faculty.joiningDate) }
0040 |
0041 |     var isLoading by remember { mutableStateOf(false) }
0042 |
0043 |     var showUnsavedDialog by remember { mutableStateOf(false) }
0044 |     val hasUnsavedChanges = name != faculty.name || email != faculty.email || phone != faculty.phone
    > || department != faculty.department || designation != faculty.designation || joiningDate !=
    > faculty.joiningDate
0045 |
0046 |     androidx.activity.compose.BackHandler(enabled = hasUnsavedChanges || isLoading) {
0047 |         if (!isLoading) showUnsavedDialog = true
0048 |     }
0049 |
0050 |     val handleBackPress = {
0051 |         if (isLoading) {
0052 |
0053 |         } else if (hasUnsavedChanges) {
0054 |             showUnsavedDialog = true
0055 |         } else {
0056 |             onBackPressed()
0057 |         }
0058 |     }
0059 |
0060 |     if (showUnsavedDialog) {
0061 |         AlertDialog(
0062 |             onDismissRequest = { showUnsavedDialog = false },
0063 |             title = { Text("Discard Changes?", color = Color.White) },
0064 |             text = { Text("You have unsaved changes. Are you sure you want to go back?", color =
    > Color.LightGray) },

```

L34-39: Keeps this UI value across supported activity recreation; it is not server persistence.

L41: Creates observable local UI state; assignment triggers recomposition.

L43: Creates observable local UI state; assignment triggers recomposition.

L44: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L46: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L47: Branches on the displayed condition; the accepted branch performs the following operation.

L48: Closes or continues the surrounding block, expression, or call.

L50: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L51: Branches on the displayed condition; the accepted branch performs the following operation.

L53: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L54: Assigns the displayed expression to a variable or named argument.

L55-56: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L57-58: Closes or continues the surrounding block, expression, or call.

L60: Branches on the displayed condition; the accepted branch performs the following operation.

L61: Adds this UI element to the composition with the displayed content and parameters.

L62-64: Assigns the displayed expression to a variable or named argument.

Source lines 65-96

```

0065 |         containerColor = Color(0xFF1E1E1E),
0066 |         confirmButton = {
0067 |             TextButton(onClick = { onBackPressed() }) { Text("Discard", color = Color.Red) }
0068 |         },
0069 |         dismissButton = {
0070 |             TextButton(onClick = { showUnsavedDialog = false }) { Text("Cancel", color =
    > Color.Gray) }
0071 |         }
0072 |     )
0073 | }
0074 |

```

```

0075 | Scaffold(
0076 |     topBar = {
0077 |         TopAppBar(
0078 |             title = { Text("Edit Faculty", color = Color.White) },
0079 |             navigationIcon = {
0080 |                 IconButton(onClick = handleBackPressed) {
0081 |                     Icon(Icons.Default.ArrowBack, contentDescription = "Back", tint =
> Color.White)
0082 |                 }
0083 |             },
0084 |             colors = TopAppBarDefaults.topAppBarColors(containerColor = Color(0xFF1E1E1E))
0085 |         )
0086 |     },
0087 |     containerColor = Color(0xFF121212)
0088 | ) { paddingValues ->
0089 |     Column(
0090 |         modifier = Modifier
0091 |             .fillMaxSize()
0092 |             .padding(paddingValues)
0093 |             .padding(16.dp)
0094 |             .verticalScroll(rememberScrollState()),
0095 |         verticalArrangement = Arrangement.spacedBy(16.dp)
0096 |     ) {

```

L65-66: Assigns the displayed expression to a variable or named argument.

L67: Connects a user interaction to the displayed state update or handler.

L68: Closes or continues the surrounding block, expression, or call.

L69: Assigns the displayed expression to a variable or named argument.

L70: Connects a user interaction to the displayed state update or handler.

L71-73: Closes or continues the surrounding block, expression, or call.

L75: Adds this UI element to the composition with the displayed content and parameters.

L76: Assigns the displayed expression to a variable or named argument.

L77: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L78-79: Assigns the displayed expression to a variable or named argument.

L80: Connects a user interaction to the displayed state update or handler.

L81: Adds this UI element to the composition with the displayed content and parameters.

L82-83: Closes or continues the surrounding block, expression, or call.

L84: Assigns the displayed expression to a variable or named argument.

L85-86: Closes or continues the surrounding block, expression, or call.

L87: Assigns the displayed expression to a variable or named argument.

L88: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L89: Adds this UI element to the composition with the displayed content and parameters.

L90-93: Configures UI layout, sizing, spacing, or appearance; it does not modify academic records.

L94: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L95: Assigns the displayed expression to a variable or named argument.

L96: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

Source lines 97-128

```

0097 |         Text("Employee ID: ${faculty.employeeId} (Cannot be changed)", color = Color.Gray, style
> = MaterialTheme.typography.bodyMedium)
0098 |
0099 |         Text("Professional Info", color = Color(0xFF03DAC5), fontWeight = FontWeight.Bold)
0100 |         CustomTextField(value = name, onValueChange = { name = it }, label = "Full Name",
> enabled = !isLoading)
0101 |         CustomTextField(value = email, onValueChange = { email = it }, label = "Email",
> keyboardType = KeyboardType.Email, enabled = !isLoading)
0102 |         CustomTextField(value = phone, onValueChange = { phone = it }, label = "Phone Number",
> keyboardType = KeyboardType.Phone, enabled = !isLoading)
0103 |
0104 |         Row(horizontalArrangement = Arrangement.spacedBy(8.dp)) {
0105 |             CustomTextField(value = department, onValueChange = { department = it }, label =
> "Department", modifier = Modifier.weight(1f), enabled = !isLoading)

```

```

0106 |         CustomTextField(value = designation, onValueChange = { designation = it }, label =
> "Designation", modifier = Modifier.weight(1f), enabled = !isLoading)
0107 |     }
0108 |
0109 |         CustomTextField(value = joiningDate, onValueChange = { joiningDate = it }, label =
> "Joining Date", enabled = !isLoading)
0110 |
0111 |         Spacer(modifier = Modifier.height(16.dp))
0112 |
0113 |         Button(
0114 |             onClick = {
0115 |                 if (name.isBlank() || email.isBlank() || department.isBlank() ||
> designation.isBlank()) {
0116 |                     Toast.makeText(context, "All required fields must be filled",
> Toast.LENGTH_SHORT).show()
0117 |                     return@Button
0118 |                 }
0119 |
0120 |                 isLoading = true
0121 |                 coroutineScope.launch {
0122 |                     try {
0123 |                         val request = UpdateFacultyRequest(
0124 |                             revision = faculty.revision,
0125 |                             employeeId = faculty.employeeId,
0126 |                             name = name.trim(),
0127 |                             email = email.trim(),
0128 |                             phone = phone.trim(),

```

L97: Adds this UI element to the composition with the displayed content and parameters.

L99: Adds this UI element to the composition with the displayed content and parameters.

L100-102: Connects a user interaction to the displayed state update or handler.

L104: Adds this UI element to the composition with the displayed content and parameters.

L105-106: Connects a user interaction to the displayed state update or handler.

L107: Closes or continues the surrounding block, expression, or call.

L109: Connects a user interaction to the displayed state update or handler.

L111: Adds this UI element to the composition with the displayed content and parameters.

L113: Adds this UI element to the composition with the displayed content and parameters.

L114: Connects a user interaction to the displayed state update or handler.

L115: Branches on the displayed condition; the accepted branch performs the following operation.

L116-117: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L118: Closes or continues the surrounding block, expression, or call.

L120: Assigns the displayed expression to a variable or named argument.

L121: Starts asynchronous work in the UI scope so the button handler can return promptly.

L122: Starts an operation whose failures are handled by the following catch/finally blocks.

L123: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L124: Carries or checks the record version used to detect stale edits or matching retries.

L125-128: Assigns the displayed expression to a variable or named argument.

Source lines 129-160

```

0129 |         department = department.trim(),
0130 |         designation = designation.trim(),
0131 |         joiningDate = joiningDate.trim()
0132 |     )
0133 |     val response = NetworkModule.api.updateFaculty(request)
0134 |     if (response.status == "success") {
0135 |         Toast.makeText(context, "Faculty updated successfully",
> Toast.LENGTH_SHORT).show()
0136 |         onBackPressed()
0137 |     } else {
0138 |         Toast.makeText(context, "Error: ${response.message}",
> Toast.LENGTH_LONG).show()
0139 |     }
0140 |     } catch (e: kotlin.coroutines.CancellationException) { throw e
0141 |     } catch (e: Exception) {

```

```

0142 |         Toast.makeText(context, "Failed to update: ${e.message}",
> Toast.LENGTH_LONG).show()
0143 |     } finally {
0144 |         isLoading = false
0145 |     }
0146 | }
0147 | },
0148 | modifier = Modifier.fillMaxWidth().height(50.dp),
0149 | shape = RoundedCornerShape(12.dp),
0150 | colors = ButtonDefaults.buttonColors(containerColor = Color(0xFFBB86FC)),
0151 | enabled = !isLoading
0152 | ) {
0153 |     if (isLoading) {
0154 |         CircularProgressIndicator(color = Color.White, modifier = Modifier.size(24.dp))
0155 |     } else {
0156 |         Text("Update Faculty", color = Color.Black, fontWeight = FontWeight.Bold, style
> = MaterialTheme.typography.titleMedium)
0157 |     }
0158 | }
0159 |     Spacer(modifier = Modifier.height(32.dp))
0160 | }

```

L129-131: Assigns the displayed expression to a variable or named argument.

L132: Closes or continues the surrounding block, expression, or call.

L133: Calls the named authenticated backend operation with the displayed request or key.

L134: Branches on the displayed condition; the accepted branch performs the following operation.

L135-138: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L139: Closes or continues the surrounding block, expression, or call.

L140: Preserves coroutine cancellation rather than presenting it as an ordinary application error.

L141: Handles this exception type and translates it into the following failure behavior.

L142: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L143: Runs cleanup whether the preceding operation succeeded or failed.

L144: Assigns the displayed expression to a variable or named argument.

L145-147: Closes or continues the surrounding block, expression, or call.

L148: Configures UI layout, sizing, spacing, or appearance; it does not modify academic records.

L149-151: Assigns the displayed expression to a variable or named argument.

L152: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L153: Branches on the displayed condition; the accepted branch performs the following operation.

L154: Adds this UI element to the composition with the displayed content and parameters.

L155: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L156: Adds this UI element to the composition with the displayed content and parameters.

L157-158: Closes or continues the surrounding block, expression, or call.

L159: Adds this UI element to the composition with the displayed content and parameters.

L160: Closes or continues the surrounding block, expression, or call.

Source lines 161-181

```

0161 |     }
0162 | }
0163 |
0164 | @Composable
0165 | fun CustomTextField(
0166 |     value: String,
0167 |     onChange: (String) -> Unit,
0168 |     label: String,
0169 |     modifier: Modifier = Modifier,
0170 |     keyboardType: KeyboardType = KeyboardType.Text,
0171 |     enabled: Boolean = true
0172 | ) {
0173 |     OutlinedTextField(
0174 |         enabled = enabled,
0175 |         value = value,
0176 |         onChange = onChange,

```

```

0177 |         label = { Text(label) },
0178 |         keyboardOptions = KeyboardOptions(keyboardType = keyboardType),
0179 |         modifier = modifier.fillMaxWidth()
0180 |     )
0181 | }

```

L161-162: Closes or continues the surrounding block, expression, or call.

L164: Marks the following function as a Compose UI function.

L165: Defines this Kotlin operation and its parameters; suspend allows asynchronous calls.

L166: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L167: Connects a user interaction to the displayed state update or handler.

L168: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L169: Configures UI layout, sizing, spacing, or appearance; it does not modify academic records.

L170-172: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L173: Adds this UI element to the composition with the displayed content and parameters.

L174-175: Assigns the displayed expression to a variable or named argument.

L176: Connects a user interaction to the displayed state update or handler.

L177-179: Assigns the displayed expression to a variable or named argument.

L180-181: Closes or continues the surrounding block, expression, or call.

29. FacultyDashboardScreen.kt

app/src/main/java/com/example/studenttracker/presentation/faculty/FacultyDashboardScreen.kt

Faculty entry point.

9 source lines. SHA-256: 1f5dd1199181ed02

This small wrapper delegates to AcademicHubScreen with isAdmin=false and supplies navigation and logout callbacks. It avoids duplicating the subject workspace.

Source lines 1-9

```
0001 | package com.example.studenttracker.presentation.faculty
0002 |
0003 | import androidx.compose.runtime.Composable
0004 | import com.example.studenttracker.presentation.academic.AcademicHubScreen
0005 |
0006 | @Composable
0007 | fun FacultyDashboardScreen(onOpen: (String, String) -> Unit, onLogout: () -> Unit) {
0008 |     AcademicHubScreen(isAdmin = false, onOpen = onOpen, onLogout = onLogout)
0009 | }
```

L1: Declares the Kotlin namespace; it organizes classes and functions.

L3-4: Imports the named library symbols used by this file.

L6: Marks the following function as a Compose UI function.

L7: Defines this Kotlin operation and its parameters; suspend allows asynchronous calls.

L8: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L9: Closes or continues the surrounding block, expression, or call.

30. FacultyDetailsScreen.kt

app/src/main/java/com/example/studenttracker/presentation/faculty/FacultyDetailsScreen.kt

Read-only faculty profile details and actions.

163 source lines. SHA-256: e511d36f20bed58f

The typed record supplies the display values; cards group related fields. Small DetailRow helpers display icon, label, and value.

Edit invokes the provided route callback. Deactivate opens confirmation and sends the record key with its expected revision.

Busy state prevents repeated actions. Success returns to the previous screen; failure shows feedback. Deactivation preserves historical academic data.

Source lines 1-32

```
0001 | package com.example.studenttracker.presentation.faculty
0002 |
0003 | import android.widget.Toast
0004 | import androidx.compose.foundation.background
0005 | import androidx.compose.foundation.layout.*
0006 | import androidx.compose.foundation.rememberScrollState
0007 | import androidx.compose.foundation.shape.CircleShape
0008 | import androidx.compose.foundation.shape.RoundedCornerShape
0009 | import androidx.compose.foundation.verticalScroll
0010 | import androidx.compose.material.icons.Icons
0011 | import androidx.compose.material.icons.filled.*
0012 | import androidx.compose.material3.*
0013 | import androidx.compose.runtime.*
0014 | import androidx.compose.ui.Alignment
0015 | import androidx.compose.ui.Modifier
0016 | import androidx.compose.ui.draw.clip
0017 | import androidx.compose.ui.graphics.Color
0018 | import androidx.compose.ui.platform.LocalContext
0019 | import androidx.compose.ui.text.font.FontWeight
0020 | import androidx.compose.ui.unit.dp
0021 | import androidx.compose.ui.unit.sp
0022 | import com.example.studenttracker.data.network.DeleteFacultyRequest
0023 | import com.example.studenttracker.data.network.Faculty
0024 | import com.example.studenttracker.data.network.NetworkModule
0025 | import kotlinx.coroutines.launch
0026 |
0027 | @OptIn(ExperimentalMaterial3Api::class)
0028 | @Composable
0029 | fun FacultyDetailsScreen(
0030 |     faculty: Faculty,
0031 |     onNavigateBack: () -> Unit,
0032 |     onNavigateToEdit: () -> Unit
```

L1: Declares the Kotlin namespace; it organizes classes and functions.

L3-25: Imports the named library symbols used by this file.

L27: Applies the displayed annotation to the declaration.

L28: Marks the following function as a Compose UI function.

L29: Defines this Kotlin operation and its parameters; suspend allows asynchronous calls.

L30-32: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

Source lines 33-64

```
0033 | ) {
0034 |     val context = LocalContext.current
0035 |     val coroutineScope = rememberCoroutineScope()
0036 |     var showDeleteDialog by remember { mutableStateOf(false) }
0037 |     var isDeleting by remember { mutableStateOf(false) }
0038 |
0039 |     if (showDeleteDialog) {
0040 |         AlertDialog(
```



```

0041 |         onDismissRequest = { showDeleteDialog = false },
0042 |         title = { Text("Deactivate Faculty", color = Color.White) },
0043 |         text = { Text("Are you sure you want to remove ${faculty.name}? The profile will be
> deactivated; historical records will be retained.", color = Color.LightGray) },
0044 |         containerColor = Color(0xFF1E1E1E),
0045 |         confirmButton = {
0046 |             TextButton(
0047 |                 onClick = {
0048 |                     isDeleting = true
0049 |                     coroutineScope.launch {
0050 |                         try {
0051 |                             val response =
> NetworkModule.api.deleteFaculty(DeleteFacultyRequest(revision = faculty.revision, employeeId =
> faculty.employeeId))
0052 |                             if (response.status == "success") {
0053 |                                 Toast.makeText(context, "Faculty deactivated",
> Toast.LENGTH_SHORT).show()
0054 |                                 showDeleteDialog = false
0055 |                                 onBackPressed()
0056 |                             } else {
0057 |                                 Toast.makeText(context, "Error: ${response.message}",
> Toast.LENGTH_LONG).show()
0058 |                             }
0059 |                         } catch (e: kotlinx.coroutines.CancellationException) { throw e
0060 |                         } catch (e: Exception) {
0061 |                             Toast.makeText(context, "Failed to delete: ${e.message}",
> Toast.LENGTH_LONG).show()
0062 |                         } finally {
0063 |                             isDeleting = false
0064 |                             showDeleteDialog = false

```

L33: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L34: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L35: Obtains a coroutine scope tied to this composable for button-triggered asynchronous work.

L36-37: Creates observable local UI state; assignment triggers recomposition.

L39: Branches on the displayed condition; the accepted branch performs the following operation.

L40: Adds this UI element to the composition with the displayed content and parameters.

L41-45: Assigns the displayed expression to a variable or named argument.

L46: Adds this UI element to the composition with the displayed content and parameters.

L47: Connects a user interaction to the displayed state update or handler.

L48: Assigns the displayed expression to a variable or named argument.

L49: Starts asynchronous work in the UI scope so the button handler can return promptly.

L50: Starts an operation whose failures are handled by the following catch/finally blocks.

L51: Calls the named authenticated backend operation with the displayed request or key.

L52: Branches on the displayed condition; the accepted branch performs the following operation.

L53: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L54: Assigns the displayed expression to a variable or named argument.

L55-57: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L58: Closes or continues the surrounding block, expression, or call.

L59: Preserves coroutine cancellation rather than presenting it as an ordinary application error.

L60: Handles this exception type and translates it into the following failure behavior.

L61: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L62: Runs cleanup whether the preceding operation succeeded or failed.

L63-64: Assigns the displayed expression to a variable or named argument.

Source lines 65-96

```

0065 |         }
0066 |     }
0067 | }
0068 | ) {
0069 |     if (isDeleting) {
0070 |         CircularProgressIndicator(modifier = Modifier.size(20.dp), color =

```

```

    > Color.Red)
0071 |         } else {
0072 |             Text("Delete", color = Color.Red, fontWeight = FontWeight.Bold)
0073 |         }
0074 |     }
0075 | },
0076 |     dismissButton = {
0077 |         TextButton(onClick = { showDeleteDialog = false }) {
0078 |             Text("Cancel", color = Color.Gray)
0079 |         }
0080 |     }
0081 | )
0082 | }
0083 |
0084 | Scaffold(
0085 |     topBar = {
0086 |         TopAppBar(
0087 |             title = { Text("Faculty Profile", color = Color.White) },
0088 |             navigationIcon = {
0089 |                 IconButton(onClick = onNavigateBack) {
0090 |                     Icon(Icons.Default.ArrowBack, contentDescription = "Back", tint =
    > Color.White)
0091 |                 }
0092 |             },
0093 |             actions = {
0094 |                 IconButton(onClick = onNavigateToEdit, enabled = !isDeleting) {
0095 |                     Icon(Icons.Default.Edit, contentDescription = "Edit", tint = Color.White)
0096 |                 }

```

L65-67: Closes or continues the surrounding block, expression, or call.

L68: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L69: Branches on the displayed condition; the accepted branch performs the following operation.

L70: Adds this UI element to the composition with the displayed content and parameters.

L71: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L72: Adds this UI element to the composition with the displayed content and parameters.

L73-75: Closes or continues the surrounding block, expression, or call.

L76: Assigns the displayed expression to a variable or named argument.

L77: Connects a user interaction to the displayed state update or handler.

L78: Adds this UI element to the composition with the displayed content and parameters.

L79-82: Closes or continues the surrounding block, expression, or call.

L84: Adds this UI element to the composition with the displayed content and parameters.

L85: Assigns the displayed expression to a variable or named argument.

L86: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L87-88: Assigns the displayed expression to a variable or named argument.

L89: Connects a user interaction to the displayed state update or handler.

L90: Adds this UI element to the composition with the displayed content and parameters.

L91-92: Closes or continues the surrounding block, expression, or call.

L93: Assigns the displayed expression to a variable or named argument.

L94: Connects a user interaction to the displayed state update or handler.

L95: Adds this UI element to the composition with the displayed content and parameters.

L96: Closes or continues the surrounding block, expression, or call.

Source lines 97-128

```

0097 |         IconButton(onClick = { showDeleteDialog = true }, enabled = !isDeleting) {
0098 |             Icon(Icons.Default.Delete, contentDescription = "Delete", tint = Color.Red)
0099 |         }
0100 |     },
0101 |     colors = TopAppBarDefaults.topAppBarColors(containerColor = Color(0xFF1E1E1E))
0102 | )
0103 | },
0104 |     containerColor = Color(0xFF121212)
0105 | ) { paddingValues ->

```

```

0106 |         Column(
0107 |             modifier = Modifier
0108 |                 .fillMaxSize()
0109 |                 .padding(paddingValues)
0110 |                 .verticalScroll(rememberScrollState()),
0111 |             horizontalAlignment = Alignment.CenterHorizontally
0112 |         ) {
0113 |             Spacer(modifier = Modifier.height(32.dp))
0114 |             Box(
0115 |                 modifier = Modifier
0116 |                     .size(100.dp)
0117 |                     .clip(CircleShape)
0118 |                     .background(Color(0xFF03DAC5).copy(alpha = 0.2f)),
0119 |                 contentAlignment = Alignment.Center
0120 |             ) {
0121 |                 Icon(Icons.Default.Person, contentDescription = null, tint = Color(0xFF03DAC5),
> modifier = Modifier.size(50.dp))
0122 |             }
0123 |             Spacer(modifier = Modifier.height(16.dp))
0124 |             Text(faculty.name, style = MaterialTheme.typography.headlineMedium, color = Color.White,
> fontWeight = FontWeight.Bold)
0125 |             Text(faculty.employeeId, style = MaterialTheme.typography.bodyLarge, color = Color.Gray)
0126 |
0127 |             Spacer(modifier = Modifier.height(32.dp))
0128 |

```

L97: Connects a user interaction to the displayed state update or handler.

L98: Adds this UI element to the composition with the displayed content and parameters.

L99-100: Closes or continues the surrounding block, expression, or call.

L101: Assigns the displayed expression to a variable or named argument.

L102-103: Closes or continues the surrounding block, expression, or call.

L104: Assigns the displayed expression to a variable or named argument.

L105: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L106: Adds this UI element to the composition with the displayed content and parameters.

L107-109: Configures UI layout, sizing, spacing, or appearance; it does not modify academic records.

L110: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L111: Assigns the displayed expression to a variable or named argument.

L112: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L113-114: Adds this UI element to the composition with the displayed content and parameters.

L115-118: Configures UI layout, sizing, spacing, or appearance; it does not modify academic records.

L119: Assigns the displayed expression to a variable or named argument.

L120: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L121: Adds this UI element to the composition with the displayed content and parameters.

L122: Closes or continues the surrounding block, expression, or call.

L123-125: Adds this UI element to the composition with the displayed content and parameters.

L127: Adds this UI element to the composition with the displayed content and parameters.

Source lines 129-160

```

0129 |         Card(
0130 |             modifier = Modifier
0131 |                 .fillMaxWidth()
0132 |                 .padding(horizontal = 16.dp),
0133 |             colors = CardDefaults.cardColors(containerColor = Color(0xFF1E1E1E)),
0134 |             shape = RoundedCornerShape(16.dp)
0135 |         ) {
0136 |             Column(modifier = Modifier.padding(16.dp), verticalArrangement =
> Arrangement.spacedBy(16.dp)) {
0137 |                 DetailRow(Icons.Default.Email, "Email", faculty.email)
0138 |                 DetailRow(Icons.Default.Phone, "Phone", faculty.phone.takeIf { it.isNotBlank() }
> ?: "N/A")
0139 |                 DetailRow(Icons.Default.Business, "Department", faculty.department)
0140 |                 DetailRow(Icons.Default.Work, "Designation", faculty.designation)
0141 |                 DetailRow(Icons.Default.DateRange, "Joining Date", faculty.joiningDate.takeIf {

```

```

    > it.isNotBlank() } ?: "N/A")
0142 |         }
0143 |     }
0144 | }
0145 | }
0146 | }
0147 |
0148 | @Composable
0149 | fun DetailRow(icon: androidx.compose.ui.graphics.vector.ImageVector, label: String, value: String) {
0150 |     Row(verticalAlignment = Alignment.CenterVertically, modifier = Modifier.fillMaxWidth()) {
0151 |         Box(
0152 |             modifier = Modifier.size(40.dp).clip(CircleShape).background(Color.DarkGray.copy(alpha =
    > 0.3f)),
0153 |             contentAlignment = Alignment.Center
0154 |         ) {
0155 |             Icon(icon, contentDescription = null, tint = Color(0xFFBB86FC), modifier =
    > Modifier.size(20.dp))
0156 |         }
0157 |         Spacer(modifier = Modifier.width(16.dp))
0158 |         Column {
0159 |             Text(label, color = Color.Gray, fontSize = 12.sp)
0160 |             Text(value, color = Color.White, fontSize = 16.sp, fontWeight = FontWeight.SemiBold)

```

L129: Adds this UI element to the composition with the displayed content and parameters.

L130-132: Configures UI layout, sizing, spacing, or appearance; it does not modify academic records.

L133-134: Assigns the displayed expression to a variable or named argument.

L135: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L136: Adds this UI element to the composition with the displayed content and parameters.

L137-141: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L142-146: Closes or continues the surrounding block, expression, or call.

L148: Marks the following function as a Compose UI function.

L149: Defines this Kotlin operation and its parameters; suspend allows asynchronous calls.

L150-151: Adds this UI element to the composition with the displayed content and parameters.

L152: Configures UI layout, sizing, spacing, or appearance; it does not modify academic records.

L153: Assigns the displayed expression to a variable or named argument.

L154: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L155: Adds this UI element to the composition with the displayed content and parameters.

L156: Closes or continues the surrounding block, expression, or call.

L157: Adds this UI element to the composition with the displayed content and parameters.

L158: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L159-160: Adds this UI element to the composition with the displayed content and parameters.

Source lines 161-163

```

0161 |     }
0162 | }
0163 | }

```

L161-163: Closes or continues the surrounding block, expression, or call.

31. FacultyListScreen.kt

app/src/main/java/com/example/studenttracker/presentation/faculty/FacultyListScreen.kt

Faculty directory adapter.

13 source lines. SHA-256: 2a4bdac19174334f

Loads records with `getFaculty`, maps the stable identifier and display fields into `DirectoryRecord`, and delegates searching and list rendering to `DirectoryScreen`.

Caller callbacks determine which record or add form opens.

Source lines 1-13

```
0001 | package com.example.studenttracker.presentation.faculty
0002 |
0003 | import androidx.compose.runtime.Composable
0004 | import com.example.studenttracker.data.network.NetworkModule
0005 | import com.example.studenttracker.presentation.components.DirectoryRecord
0006 | import com.example.studenttracker.presentation.components.DirectoryScreen
0007 |
0008 | @Composable
0009 | fun FacultyListScreen(onNavigateBack: () -> Unit, onFacultyClick: (String) -> Unit,
    > onNavigateToAddFaculty: () -> Unit) {
0010 |     DirectoryScreen("Faculty", onNavigateBack, onNavigateToAddFaculty, onFacultyClick) {
0011 |         NetworkModule.api.getFaculty().faculty.orEmpty().map { DirectoryRecord(it.employeeId,
    > it.name, "${it.department} · ${it.designation}") }
0012 |     }
0013 | }
```

L1: Declares the Kotlin namespace; it organizes classes and functions.

L3-6: Imports the named library symbols used by this file.

L8: Marks the following function as a Compose UI function.

L9: Defines this Kotlin operation and its parameters; suspend allows asynchronous calls.

L10: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L11: Calls the named authenticated backend operation with the displayed request or key.

L12-13: Closes or continues the surrounding block, expression, or call.

32. AppNavigation.kt

app/src/main/java/com/example/studenttracker/presentation/navigation/AppNavigation.kt

Role-specific Compose navigation and record loading.

105 source lines. SHA-256: fcae19202b191ecd

rememberNavController owns the route stack. The verified role determines the initial destination and which graph entries are available.

Admin has directory/add/detail/edit/enrollment routes; faculty has assigned subjects, attendance, reports, and assessments; student has personal progress.

Identifiers are URI-encoded for routes. Detail and edit destinations load the current backend record through RemoteContent, rather than reading a global selected record.

Logout clears preferences and AuthSession. UI routing improves usability; backend checks provide the actual authorization boundary.

Source lines 1-32

```
0001 | package com.example.studenttracker.presentation.navigation
0002 |
0003 | import android.net.Uri
0004 | import androidx.compose.runtime.Composable
0005 | import androidx.compose.runtime.rememberCoroutineScope
0006 | import androidx.navigation.NavHostController
0007 | import androidx.navigation.compose.*
0008 | import com.example.studenttracker.data.local.UserPreferences
0009 | import com.example.studenttracker.data.network.NetworkModule
0010 | import com.example.studenttracker.presentation.academic.*
0011 | import com.example.studenttracker.presentation.admin.*
0012 | import com.example.studenttracker.presentation.auth.LoginScreen
0013 | import com.example.studenttracker.presentation.components.RemoteContent
0014 | import com.example.studenttracker.presentation.faculty.*
0015 | import com.example.studenttracker.presentation.student.*
0016 | import com.example.studenttracker.presentation.subject.*
0017 | import kotlinx.coroutines.launch
0018 |
0019 | @Composable
0020 | fun AppNavigation(navController: NavHostController = rememberNavController(), initialRole: String?,
    > userPreferences: UserPreferences) {
0021 |     val scope = rememberCoroutineScope()
0022 |     val logout: () -> Unit = { scope.launch { userPreferences.clearSession() }; Unit }
0023 |     val back: () -> Unit = { navController.popBackStack(); Unit }
0024 |     val start = when (initialRole) { "admin" -> "admin_dashboard"; "faculty" -> "faculty_dashboard";
    > "student" -> "student_dashboard"; else -> "login" }
0025 |     val openAcademic: (String, String) -> Unit = { page, code ->
    > navController.navigate("$page/${Uri.encode(code)}") }
0026 |     NavHost(navController, startDestination = start) {
0027 |         if (initialRole == null) {
0028 |             composable("login") { LoginScreen(userPreferences, onLoginSuccess = {}) }
0029 |         }
0030 |         if (initialRole == "student") {
0031 |             composable("student_dashboard") { StudentDashboardScreen(onLogout = logout) }
0032 |         }
    }
```

L1: Declares the Kotlin namespace; it organizes classes and functions.

L3-17: Imports the named library symbols used by this file.

L19: Marks the following function as a Compose UI function.

L20: Defines this Kotlin operation and its parameters; suspend allows asynchronous calls.

L21: Obtains a coroutine scope tied to this composable for button-triggered asynchronous work.

L22: Starts asynchronous work in the UI scope so the button handler can return promptly.

L23-25: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L26: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L27: Branches on the displayed condition; the accepted branch performs the following operation.

L28: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L29: Closes or continues the surrounding block, expression, or call.

L30: Branches on the displayed condition; the accepted branch performs the following operation.

L31: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L32: Closes or continues the surrounding block, expression, or call.

Source lines 33-64

```
0033 |         if (initialRole == "faculty") {
0034 |             composable("faculty_dashboard") { FacultyDashboardScreen(onOpen = openAcademic, onLogout
> = logout) }
0035 |         }
0036 |         if (initialRole == "faculty" || initialRole == "admin") {
0037 |             composable("academic") { AcademicHubScreen(isAdmin = initialRole == "admin", onOpen =
> openAcademic, onBack = back) }
0038 |             composable("report/{subjectCode}") { entry -> AttendanceReportScreen(back,
> entry.arguments?.getString("subjectCode")) }
0039 |             composable("attendance/{subjectCode}") { entry ->
> AttendanceScreen(entry.arguments?.getString("subjectCode").orEmpty(), back) }
0040 |             composable("assessments/{subjectCode}") { entry ->
> AssessmentsScreen(entry.arguments?.getString("subjectCode").orEmpty(), back) }
0041 |         }
0042 |         if (initialRole == "admin") {
0043 |             composable("admin_dashboard") { AdminDashboardScreen(navController, userPreferences) }
0044 |             composable("attendance_report") { AttendanceReportScreen(back) }
0045 |             composable("enrollment/{subjectCode}") { entry ->
> EnrollmentScreen(entry.arguments?.getString("subjectCode").orEmpty(), back) }
0046 |             composable("student_list") {
0047 |                 StudentListScreen(onNavigateBack = back,
0048 |                 onStudentClick = { id ->
> navController.navigate("student_details/${Uri.encode(id)}") },
0049 |                 onNavigateToAddStudent = { navController.navigate("add_student") }) }
0050 |             }
0051 |             composable("add_student") { AddStudentScreen(back) }
0052 |             composable("student_details/{id}") { entry ->
0053 |                 val id = entry.arguments?.getString("id").orEmpty()
0054 |                 RemoteContent(key = id, load = { NetworkModule.api.getStudent(id).student }, onBack
> = back) { record ->
0055 |                     StudentDetailsScreen(student = record, onNavigateBack = back,
0056 |                     onNavigateToEdit = {
> navController.navigate("edit_student/${Uri.encode(id)}") }) }
0057 |                 }
0058 |             }
0059 |             composable("edit_student/{id}") { entry ->
0060 |                 val id = entry.arguments?.getString("id").orEmpty()
0061 |                 RemoteContent(key = id, load = { NetworkModule.api.getStudent(id).student }, onBack
> = back) { record ->
0062 |                     EditStudentScreen(student = record, onNavigateBack = back)
0063 |                 }
0064 |             }
```

L33: Branches on the displayed condition; the accepted branch performs the following operation.

L34: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L35: Closes or continues the surrounding block, expression, or call.

L36: Branches on the displayed condition; the accepted branch performs the following operation.

L37-40: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L41: Closes or continues the surrounding block, expression, or call.

L42: Branches on the displayed condition; the accepted branch performs the following operation.

L43-47: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L48-49: Assigns the displayed expression to a variable or named argument.

L50: Closes or continues the surrounding block, expression, or call.

L51-52: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L53: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L54: Calls the named authenticated backend operation with the displayed request or key.

L55: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L56: Assigns the displayed expression to a variable or named argument.

L57-58: Closes or continues the surrounding block, expression, or call.

L59: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L60: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L61: Calls the named authenticated backend operation with the displayed request or key.

L62: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L63-64: Closes or continues the surrounding block, expression, or call.

Source lines 65-96

```
0065 |         composable("faculty_list") {
0066 |             FacultyListScreen(onNavigateBack = back,
0067 |                 onFacultyClick = { id ->
0068 |                     navController.navigate("${Uri.encode(id)}") },
0069 |                     onNavigateToAddFaculty = { navController.navigate("add_faculty") })
0070 |         }
0071 |         composable("add_faculty") { AddFacultyScreen(back) }
0072 |         composable("faculty_details/{id}") { entry ->
0073 |             val id = entry.arguments?.getString("id").orEmpty()
0074 |             RemoteContent(key = id, load = {
0075 |                 NetworkModule.api.getFacultyMember(id).facultyMember }, onBack = back) { record ->
0076 |                 FacultyDetailsScreen(faculty = record, onNavigateBack = back,
0077 |                     onNavigateToEdit = {
0078 |                         navController.navigate("edit_faculty/${Uri.encode(id)}") })
0079 |             }
0080 |         }
0081 |         composable("edit_faculty/{id}") { entry ->
0082 |             val id = entry.arguments?.getString("id").orEmpty()
0083 |             RemoteContent(key = id, load = {
0084 |                 NetworkModule.api.getFacultyMember(id).facultyMember }, onBack = back) { record ->
0085 |                 EditFacultyScreen(faculty = record, onNavigateBack = back)
0086 |             }
0087 |         }
0088 |         composable("subject_list") {
0089 |             SubjectListScreen(onNavigateBack = back,
0090 |                 onSubjectClick = { id ->
0091 |                     navController.navigate("${Uri.encode(id)}") },
0092 |                     onNavigateToAddSubject = { navController.navigate("add_subject") })
0093 |             }
0094 |             composable("add_subject") { AddSubjectScreen(back) }
0095 |             composable("subject_details/{id}") { entry ->
0096 |                 val id = entry.arguments?.getString("id").orEmpty()
0097 |                 RemoteContent(key = id, load = { NetworkModule.api.getSubject(id).subject }, onBack
0098 |                     = back) { record ->
0099 |                     SubjectDetailsScreen(subject = record, onNavigateBack = back,
0100 |                         onNavigateToEdit = {
0101 |                             navController.navigate("edit_subject/${Uri.encode(id)}") })
0102 |                     }
0103 |             }
0104 |         }
```

L65-66: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L67-68: Assigns the displayed expression to a variable or named argument.

L69: Closes or continues the surrounding block, expression, or call.

L70-71: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L72: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L73: Calls the named authenticated backend operation with the displayed request or key.

L74: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L75: Assigns the displayed expression to a variable or named argument.

L76-77: Closes or continues the surrounding block, expression, or call.

L78: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L79: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L80: Calls the named authenticated backend operation with the displayed request or key.

L81: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L82-83: Closes or continues the surrounding block, expression, or call.

L84-85: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L86-87: Assigns the displayed expression to a variable or named argument.

L88: Closes or continues the surrounding block, expression, or call.

L89-90: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L91: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L92: Calls the named authenticated backend operation with the displayed request or key.

L93: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L94: Assigns the displayed expression to a variable or named argument.

L95-96: Closes or continues the surrounding block, expression, or call.

Source lines 97-105

```
0097 |             composable("edit_subject/{id}") { entry ->
0098 |                 val id = entry.arguments?.getString("id").orEmpty()
0099 |                 RemoteContent(key = id, load = { NetworkModule.api.getSubject(id).subject }, onBack
    > = back) { record ->
0100 |                     EditSubjectScreen(subject = record, onNavigateBack = back)
0101 |                 }
0102 |             }
0103 |         }
0104 |     }
0105 | }
```

L97: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L98: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L99: Calls the named authenticated backend operation with the displayed request or key.

L100: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L101-105: Closes or continues the surrounding block, expression, or call.

33. AddStudentScreen.kt

app/src/main/java/com/example/studenttracker/presentation/student/AddStudentScreen.kt

Create a student profile.

303 source lines. SHA-256: 3412b5e18a7f9fe2

rememberSaveable holds each form field. Derived dirty state includes every editable field; BackHandler and the discard dialog guard unsaved changes.

The form starts empty; the identifier is supplied when creating the record.

The save handler trims input, validates required values and applicable email formats, then launches the API request. Busy state disables fields and navigation; success shows feedback and returns.

Cancellation is rethrown. Other failures show a message; finally restores the busy flag. Backend validation remains authoritative.

CustomTextField wraps OutlinedTextField and the appropriate keyboard type. Modifier chains size, padding, background, and alignment without changing backend data.

Source lines 1-32

```
0001 | package com.example.studenttracker.presentation.student
0002 |
0003 | import android.widget.Toast
0004 | import androidx.compose.foundation.layout.*
0005 | import androidx.compose.foundation.rememberScrollState
0006 | import androidx.compose.foundation.text.KeyboardOptions
0007 | import androidx.compose.foundation.verticalScroll
0008 | import androidx.compose.material.icons.Icons
0009 | import androidx.compose.material.icons.filled.ArrowBack
0010 | import androidx.compose.material3.*
0011 | import androidx.compose.runtime.*
0012 | import androidx.compose.runtime.saveable.rememberSaveable
0013 | import androidx.compose.ui.Modifier
0014 | import androidx.compose.ui.graphics.Color
0015 | import androidx.compose.ui.platform.LocalContext
0016 | import androidx.compose.ui.text.input.KeyboardType
0017 | import androidx.compose.ui.unit.dp
0018 | import kotlinx.coroutines.launch
0019 |
0020 | @OptIn(ExperimentalMaterial3Api::class)
0021 | @Composable
0022 | fun AddStudentScreen(
0023 |     onNavigateBack: () -> Unit
0024 | ) {
0025 |     val context = LocalContext.current
0026 |
0027 |     var rollNumber by rememberSaveable { mutableStateOf("") }
0028 |     var name by rememberSaveable { mutableStateOf("") }
0029 |     var email by rememberSaveable { mutableStateOf("") }
0030 |     var phone by rememberSaveable { mutableStateOf("") }
0031 |     var parentName by rememberSaveable { mutableStateOf("") }
0032 |     var parentPhone by rememberSaveable { mutableStateOf("") }
```

L1: Declares the Kotlin namespace; it organizes classes and functions.

L3-18: Imports the named library symbols used by this file.

L20: Applies the displayed annotation to the declaration.

L21: Marks the following function as a Compose UI function.

L22: Defines this Kotlin operation and its parameters; suspend allows asynchronous calls.

L23-24: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L25: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L27-32: Keeps this UI value across supported activity recreation; it is not server persistence.

Source lines 33-64

```

0033 |     var parentEmail by rememberSaveable { mutableStateOf("") }
0034 |     var department by rememberSaveable { mutableStateOf("") }
0035 |     var year by rememberSaveable { mutableStateOf("") }
0036 |     var semester by rememberSaveable { mutableStateOf("") }
0037 |     var section by rememberSaveable { mutableStateOf("") }
0038 |     var batch by rememberSaveable { mutableStateOf("") }
0039 |
0040 |     var isError by remember { mutableStateOf(false) }
0041 |
0042 |     val coroutineScope = rememberCoroutineScope()
0043 |     var isSaving by remember { mutableStateOf(false) }
0044 |
0045 |     fun validateAndSave() {
0046 |         val cleanRoll = rollNumber.trim()
0047 |         val cleanName = name.trim()
0048 |         val cleanEmail = email.trim()
0049 |         val cleanDept = department.trim()
0050 |
0051 |         if (cleanRoll.isBlank() || cleanName.isBlank() || cleanEmail.isBlank() ||
> cleanDept.isBlank()) {
0052 |             isError = true
0053 |             Toast.makeText(context, "Please fill in all required fields", Toast.LENGTH_SHORT).show()
0054 |             return
0055 |         }
0056 |         if (!android.util.Patterns.EMAIL_ADDRESS.matcher(cleanEmail).matches()) {
0057 |             Toast.makeText(context, "Invalid Email Format (e.g., student@college.edu)",
> Toast.LENGTH_SHORT).show()
0058 |             return
0059 |         }
0060 |
0061 |         isSaving = true
0062 |         coroutineScope.launch {
0063 |             try {
0064 |                 val api = com.example.studenttracker.data.network.NetworkModule.api

```

L33-38: Keeps this UI value across supported activity recreation; it is not server persistence.

L40: Creates observable local UI state; assignment triggers recomposition.

L42: Obtains a coroutine scope tied to this composable for button-triggered asynchronous work.

L43: Creates observable local UI state; assignment triggers recomposition.

L45: Defines this Kotlin operation and its parameters; suspend allows asynchronous calls.

L46-49: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L51: Branches on the displayed condition; the accepted branch performs the following operation.

L52: Assigns the displayed expression to a variable or named argument.

L53-54: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L55: Closes or continues the surrounding block, expression, or call.

L56: Branches on the displayed condition; the accepted branch performs the following operation.

L57-58: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L59: Closes or continues the surrounding block, expression, or call.

L61: Assigns the displayed expression to a variable or named argument.

L62: Starts asynchronous work in the UI scope so the button handler can return promptly.

L63: Starts an operation whose failures are handled by the following catch/finally blocks.

L64: Declares the displayed value or mutable state; its initializer supplies the initial contents.

Source lines 65-96

```

0065 |         val request = com.example.studenttracker.data.network.AddStudentRequest(
0066 |             rollNumber = cleanRoll,
0067 |             name = cleanName,
0068 |             email = cleanEmail,
0069 |             phone = phone.trim(),
0070 |             department = cleanDept,
0071 |             batch = batch.trim(),
0072 |             year = year.trim(),
0073 |             semester = semester.trim(),
0074 |             section = section.trim(),
0075 |             parentName = parentName.trim(),

```

```

0076 |         parentPhone = parentPhone.trim(),
0077 |         parentEmail = parentEmail.trim()
0078 |     )
0079 |
0080 |     val response = api.addStudent(request)
0081 |     if (response.status == "success") {
0082 |         Toast.makeText(context, "Student saved successfully!",
> Toast.LENGTH_SHORT).show()
0083 |         onBackPressed()
0084 |     } else {
0085 |         Toast.makeText(context, "Error: ${response.message}", Toast.LENGTH_LONG).show()
0086 |     }
0087 | } catch (e: kotlinx.coroutines.CancellationException) { throw e
0088 | } catch (e: Exception) {
0089 |     Toast.makeText(context, "Failed to save: ${e.message}", Toast.LENGTH_LONG).show()
0090 | } finally {
0091 |     isSaving = false
0092 | }
0093 | }
0094 | }
0095 |
0096 | fun resetForm() {

```

L65: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L66-77: Assigns the displayed expression to a variable or named argument.

L78: Closes or continues the surrounding block, expression, or call.

L80: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L81: Branches on the displayed condition; the accepted branch performs the following operation.

L82-85: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L86: Closes or continues the surrounding block, expression, or call.

L87: Preserves coroutine cancellation rather than presenting it as an ordinary application error.

L88: Handles this exception type and translates it into the following failure behavior.

L89: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L90: Runs cleanup whether the preceding operation succeeded or failed.

L91: Assigns the displayed expression to a variable or named argument.

L92-94: Closes or continues the surrounding block, expression, or call.

L96: Defines this Kotlin operation and its parameters; suspend allows asynchronous calls.

Source lines 97-128

```

0097 |     rollNumber = ""
0098 |     name = ""
0099 |     email = ""
0100 |     phone = ""
0101 |     parentName = ""
0102 |     parentPhone = ""
0103 |     parentEmail = ""
0104 |     department = ""
0105 |     year = ""
0106 |     semester = ""
0107 |     section = ""
0108 |     batch = ""
0109 |     isError = false
0110 | }
0111 |
0112 | var showUnsavedDialog by remember { mutableStateOf(false) }
0113 | val hasUnsavedChanges = rollNumber.isNotBlank() || name.isNotBlank() || email.isNotBlank() ||
> phone.isNotBlank() || department.isNotBlank() || batch.isNotBlank() || year.isNotBlank() ||
> semester.isNotBlank() || section.isNotBlank() || parentName.isNotBlank() || parentPhone.isNotBlank()
> || parentEmail.isNotBlank()
0114 |
0115 | androidx.activity.compose.BackHandler(enabled = hasUnsavedChanges || isSaving) {
0116 |     if (!isSaving) showUnsavedDialog = true
0117 | }
0118 |
0119 | val handleBackPress = {
0120 |     if (isSaving) {

```

```

0121 |
0122 |         } else if (hasUnsavedChanges) {
0123 |             showUnsavedDialog = true
0124 |         } else {
0125 |             onNavigateBack()
0126 |         }
0127 |     }
0128 |

```

L97-109: Assigns the displayed expression to a variable or named argument.

L110: Closes or continues the surrounding block, expression, or call.

L112: Creates observable local UI state; assignment triggers recomposition.

L113: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L115: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L116: Branches on the displayed condition; the accepted branch performs the following operation.

L117: Closes or continues the surrounding block, expression, or call.

L119: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L120: Branches on the displayed condition; the accepted branch performs the following operation.

L122: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L123: Assigns the displayed expression to a variable or named argument.

L124-125: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L126-127: Closes or continues the surrounding block, expression, or call.

Source lines 129-160

```

0129 |     if (showUnsavedDialog) {
0130 |         AlertDialog(
0131 |             onDismissRequest = { showUnsavedDialog = false },
0132 |             title = { Text("Discard Changes?", color = Color.White) },
0133 |             text = { Text("You have unsaved changes. Are you sure you want to go back?", color =
> Color.LightGray) },
0134 |             containerColor = Color(0xFF1E1E1E),
0135 |             confirmButton = {
0136 |                 TextButton(onClick = { onNavigateBack() }) { Text("Discard", color = Color.Red) }
0137 |             },
0138 |             dismissButton = {
0139 |                 TextButton(onClick = { showUnsavedDialog = false }) { Text("Cancel", color =
> Color.Gray) }
0140 |             }
0141 |         )
0142 |     }
0143 |
0144 |     Scaffold(
0145 |         topBar = {
0146 |             TopAppBar(
0147 |                 title = { Text("Add New Student", color = Color.White) },
0148 |                 navigationIcon = {
0149 |                     IconButton(onClick = handleBackPressed) {
0150 |                         Icon(Icons.Default.ArrowBack, contentDescription = "Back", tint =
> Color.White)
0151 |                     }
0152 |                 },
0153 |                 colors = TopAppBarDefaults.topAppBarColors(containerColor = Color(0xFF1E1E1E))
0154 |             )
0155 |         },
0156 |         containerColor = Color(0xFF121212)
0157 |     ) { paddingValues ->
0158 |         Column(
0159 |             modifier = Modifier
0160 |                 .fillMaxSize()

```

L129: Branches on the displayed condition; the accepted branch performs the following operation.

L130: Adds this UI element to the composition with the displayed content and parameters.

L131-135: Assigns the displayed expression to a variable or named argument.

L136: Connects a user interaction to the displayed state update or handler.

L137: Closes or continues the surrounding block, expression, or call.

L138: Assigns the displayed expression to a variable or named argument.

L139: Connects a user interaction to the displayed state update or handler.

L140-142: Closes or continues the surrounding block, expression, or call.

L144: Adds this UI element to the composition with the displayed content and parameters.

L145: Assigns the displayed expression to a variable or named argument.

L146: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L147-148: Assigns the displayed expression to a variable or named argument.

L149: Connects a user interaction to the displayed state update or handler.

L150: Adds this UI element to the composition with the displayed content and parameters.

L151-152: Closes or continues the surrounding block, expression, or call.

L153: Assigns the displayed expression to a variable or named argument.

L154-155: Closes or continues the surrounding block, expression, or call.

L156: Assigns the displayed expression to a variable or named argument.

L157: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L158: Adds this UI element to the composition with the displayed content and parameters.

L159-160: Configures UI layout, sizing, spacing, or appearance; it does not modify academic records.

Source lines 161-192

```

0161 |             .padding(paddingValues)
0162 |             .padding(16.dp)
0163 |             .verticalScroll(rememberScrollState()),
0164 |             verticalArrangement = Arrangement.spacedBy(16.dp)
0165 |         ) {
0166 |             Text("Student Details", color = Color(0xFFBB86FC), style =
> MaterialTheme.typography.titleMedium)
0167 |
0168 |             OutlinedTextField(
0169 |                 enabled = !isSaving,
0170 |                 value = rollNumber, onValueChange = { rollNumber = it },
0171 |                 label = { Text("Roll Number *") },
0172 |                 modifier = Modifier.fillMaxWidth(),
0173 |                 isError = isError && rollNumber.isBlank()
0174 |             )
0175 |
0176 |             OutlinedTextField(
0177 |                 enabled = !isSaving,
0178 |                 value = name, onValueChange = { name = it },
0179 |                 label = { Text("Full Name *") },
0180 |                 modifier = Modifier.fillMaxWidth(),
0181 |                 isError = isError && name.isBlank()
0182 |             )
0183 |
0184 |             OutlinedTextField(
0185 |                 enabled = !isSaving,
0186 |                 value = email, onValueChange = { email = it },
0187 |                 label = { Text("Email *") },
0188 |                 keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Email),
0189 |                 modifier = Modifier.fillMaxWidth(),
0190 |                 isError = isError && email.isBlank()
0191 |             )
0192 |

```

L161-162: Configures UI layout, sizing, spacing, or appearance; it does not modify academic records.

L163: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L164: Assigns the displayed expression to a variable or named argument.

L165: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L166: Adds this UI element to the composition with the displayed content and parameters.

L168: Adds this UI element to the composition with the displayed content and parameters.

L169: Assigns the displayed expression to a variable or named argument.

L170: Connects a user interaction to the displayed state update or handler.

L171: Assigns the displayed expression to a variable or named argument.

L172: Configures UI layout, sizing, spacing, or appearance; it does not modify academic records.

L173: Assigns the displayed expression to a variable or named argument.

L174: Closes or continues the surrounding block, expression, or call.

L176: Adds this UI element to the composition with the displayed content and parameters.

L177: Assigns the displayed expression to a variable or named argument.

L178: Connects a user interaction to the displayed state update or handler.

L179: Assigns the displayed expression to a variable or named argument.

L180: Configures UI layout, sizing, spacing, or appearance; it does not modify academic records.

L181: Assigns the displayed expression to a variable or named argument.

L182: Closes or continues the surrounding block, expression, or call.

L184: Adds this UI element to the composition with the displayed content and parameters.

L185: Assigns the displayed expression to a variable or named argument.

L186: Connects a user interaction to the displayed state update or handler.

L187-188: Assigns the displayed expression to a variable or named argument.

L189: Configures UI layout, sizing, spacing, or appearance; it does not modify academic records.

L190: Assigns the displayed expression to a variable or named argument.

L191: Closes or continues the surrounding block, expression, or call.

Source lines 193-224

```

0193 |         OutlinedTextField(
0194 |             enabled = !isSaving,
0195 |             value = phone, onValueChange = { phone = it },
0196 |             label = { Text("Phone Number") },
0197 |             keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Phone),
0198 |             modifier = Modifier.fillMaxWidth()
0199 |         )
0200 |
0201 |         HorizontalDivider(modifier = Modifier.padding(vertical = 8.dp), color = Color.DarkGray)
0202 |         Text("Academic Info", color = Color(0xFFB86FC), style =
> MaterialTheme.typography.titleMedium)
0203 |
0204 |         Row(modifier = Modifier.fillMaxWidth(), horizontalArrangement =
> Arrangement.spacedBy(16.dp)) {
0205 |             OutlinedTextField(
0206 |                 enabled = !isSaving,
0207 |                 value = department, onValueChange = { department = it },
0208 |                 label = { Text("Department *") },
0209 |                 modifier = Modifier.weight(1f),
0210 |                 isError = isError && department.isBlank()
0211 |             )
0212 |             OutlinedTextField(
0213 |                 enabled = !isSaving,
0214 |                 value = batch, onValueChange = { batch = it },
0215 |                 label = { Text("Batch") },
0216 |                 modifier = Modifier.weight(1f)
0217 |             )
0218 |         }
0219 |
0220 |         Row(modifier = Modifier.fillMaxWidth(), horizontalArrangement =
> Arrangement.spacedBy(16.dp)) {
0221 |             OutlinedTextField(
0222 |                 enabled = !isSaving,
0223 |                 value = year, onValueChange = { year = it },
0224 |                 label = { Text("Year") },

```

L193: Adds this UI element to the composition with the displayed content and parameters.

L194: Assigns the displayed expression to a variable or named argument.

L195: Connects a user interaction to the displayed state update or handler.

L196-197: Assigns the displayed expression to a variable or named argument.

L198: Configures UI layout, sizing, spacing, or appearance; it does not modify academic records.

L199: Closes or continues the surrounding block, expression, or call.

L201: Configures UI layout, sizing, spacing, or appearance; it does not modify academic records.

L202: Adds this UI element to the composition with the displayed content and parameters.

L204-205: Adds this UI element to the composition with the displayed content and parameters.

L206: Assigns the displayed expression to a variable or named argument.

L207: Connects a user interaction to the displayed state update or handler.

L208: Assigns the displayed expression to a variable or named argument.

L209: Configures UI layout, sizing, spacing, or appearance; it does not modify academic records.

L210: Assigns the displayed expression to a variable or named argument.

L211: Closes or continues the surrounding block, expression, or call.

L212: Adds this UI element to the composition with the displayed content and parameters.

L213: Assigns the displayed expression to a variable or named argument.

L214: Connects a user interaction to the displayed state update or handler.

L215: Assigns the displayed expression to a variable or named argument.

L216: Configures UI layout, sizing, spacing, or appearance; it does not modify academic records.

L217-218: Closes or continues the surrounding block, expression, or call.

L220-221: Adds this UI element to the composition with the displayed content and parameters.

L222: Assigns the displayed expression to a variable or named argument.

L223: Connects a user interaction to the displayed state update or handler.

L224: Assigns the displayed expression to a variable or named argument.

Source lines 225-256

```

0225 |             modifier = Modifier.weight(1f)
0226 |         )
0227 |         OutlinedTextField(
0228 |             enabled = !isSaving,
0229 |             value = semester, onValueChange = { semester = it },
0230 |             label = { Text("Semester") },
0231 |             modifier = Modifier.weight(1f)
0232 |         )
0233 |         OutlinedTextField(
0234 |             enabled = !isSaving,
0235 |             value = section, onValueChange = { section = it },
0236 |             label = { Text("Section") },
0237 |             modifier = Modifier.weight(1f)
0238 |         )
0239 |     }
0240 |
0241 |     HorizontalDivider(modifier = Modifier.padding(vertical = 8.dp), color = Color.DarkGray)
0242 |     Text("Parent/Guardian Info", color = Color(0xFFB86FC), style =
> MaterialTheme.typography.titleMedium)
0243 |
0244 |     OutlinedTextField(
0245 |         enabled = !isSaving,
0246 |         value = parentName, onValueChange = { parentName = it },
0247 |         label = { Text("Parent Name") },
0248 |         modifier = Modifier.fillMaxWidth()
0249 |     )
0250 |
0251 |     OutlinedTextField(
0252 |         enabled = !isSaving,
0253 |         value = parentPhone, onValueChange = { parentPhone = it },
0254 |         label = { Text("Parent Phone") },
0255 |         keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Phone),
0256 |         modifier = Modifier.fillMaxWidth()

```

L225: Configures UI layout, sizing, spacing, or appearance; it does not modify academic records.

L226: Closes or continues the surrounding block, expression, or call.

L227: Adds this UI element to the composition with the displayed content and parameters.

L228: Assigns the displayed expression to a variable or named argument.

L229: Connects a user interaction to the displayed state update or handler.

L230: Assigns the displayed expression to a variable or named argument.

L231: Configures UI layout, sizing, spacing, or appearance; it does not modify academic records.

L232: Closes or continues the surrounding block, expression, or call.

L233: Adds this UI element to the composition with the displayed content and parameters.

L234: Assigns the displayed expression to a variable or named argument.

L235: Connects a user interaction to the displayed state update or handler.

L236: Assigns the displayed expression to a variable or named argument.

L237: Configures UI layout, sizing, spacing, or appearance; it does not modify academic records.

L238-239: Closes or continues the surrounding block, expression, or call.

L241: Configures UI layout, sizing, spacing, or appearance; it does not modify academic records.

L242: Adds this UI element to the composition with the displayed content and parameters.

L244: Adds this UI element to the composition with the displayed content and parameters.

L245: Assigns the displayed expression to a variable or named argument.

L246: Connects a user interaction to the displayed state update or handler.

L247: Assigns the displayed expression to a variable or named argument.

L248: Configures UI layout, sizing, spacing, or appearance; it does not modify academic records.

L249: Closes or continues the surrounding block, expression, or call.

L251: Adds this UI element to the composition with the displayed content and parameters.

L252: Assigns the displayed expression to a variable or named argument.

L253: Connects a user interaction to the displayed state update or handler.

L254-255: Assigns the displayed expression to a variable or named argument.

L256: Configures UI layout, sizing, spacing, or appearance; it does not modify academic records.

Source lines 257-288

```

0257 |         )
0258 |
0259 |         OutlinedTextField(
0260 |             enabled = !isSaving,
0261 |             value = parentEmail, onValueChange = { parentEmail = it },
0262 |             label = { Text("Parent Email") },
0263 |             keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Email),
0264 |             modifier = Modifier.fillMaxWidth()
0265 |         )
0266 |
0267 |         Spacer(modifier = Modifier.height(24.dp))
0268 |
0269 |         Row(
0270 |             modifier = Modifier.fillMaxWidth(),
0271 |             horizontalArrangement = Arrangement.SpaceBetween
0272 |         ) {
0273 |             OutlinedButton(
0274 |                 onClick = { resetForm() },
0275 |                 enabled = !isSaving,
0276 |                 colors = ButtonDefaults.outlinedButtonColors(contentColor = Color.White)
0277 |             ) {
0278 |                 Text("Reset")
0279 |             }
0280 |
0281 |             Row(horizontalArrangement = Arrangement.spacedBy(16.dp)) {
0282 |                 TextButton(onClick = handleBackPress) {
0283 |                     Text("Cancel", color = Color.Gray)
0284 |                 }
0285 |                 Button(
0286 |                     onClick = { validateAndSave() },
0287 |                     enabled = !isSaving,
0288 |                     colors = ButtonDefaults.buttonColors(containerColor = Color(0xFFBB86FC),
> disabledContainerColor = Color.DarkGray)

```

L257: Closes or continues the surrounding block, expression, or call.

L259: Adds this UI element to the composition with the displayed content and parameters.

L260: Assigns the displayed expression to a variable or named argument.

L261: Connects a user interaction to the displayed state update or handler.

L262-263: Assigns the displayed expression to a variable or named argument.

L264: Configures UI layout, sizing, spacing, or appearance; it does not modify academic records.

L265: Closes or continues the surrounding block, expression, or call.

L267: Adds this UI element to the composition with the displayed content and parameters.

L269: Adds this UI element to the composition with the displayed content and parameters.

L270: Configures UI layout, sizing, spacing, or appearance; it does not modify academic records.

L271: Assigns the displayed expression to a variable or named argument.

L272: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L273: Adds this UI element to the composition with the displayed content and parameters.

L274: Connects a user interaction to the displayed state update or handler.

L275-276: Assigns the displayed expression to a variable or named argument.

L277: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L278: Adds this UI element to the composition with the displayed content and parameters.

L279: Closes or continues the surrounding block, expression, or call.

L281: Adds this UI element to the composition with the displayed content and parameters.

L282: Connects a user interaction to the displayed state update or handler.

L283: Adds this UI element to the composition with the displayed content and parameters.

L284: Closes or continues the surrounding block, expression, or call.

L285: Adds this UI element to the composition with the displayed content and parameters.

L286: Connects a user interaction to the displayed state update or handler.

L287-288: Assigns the displayed expression to a variable or named argument.

Source lines 289-303

```

0289 |         ) {
0290 |             if (isSaving) {
0291 |                 CircularProgressIndicator(modifier = Modifier.size(20.dp), color =
    > Color.White, strokeWidth = 2.dp)
0292 |                 Spacer(modifier = Modifier.width(8.dp))
0293 |                 Text("Saving...", color = Color.White)
0294 |             } else {
0295 |                 Text("Save Student", color = Color.Black)
0296 |             }
0297 |         }
0298 |     }
0299 | }
0300 |     Spacer(modifier = Modifier.height(40.dp))
0301 | }
0302 | }
0303 | }

```

L289: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L290: Branches on the displayed condition; the accepted branch performs the following operation.

L291-293: Adds this UI element to the composition with the displayed content and parameters.

L294: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L295: Adds this UI element to the composition with the displayed content and parameters.

L296-299: Closes or continues the surrounding block, expression, or call.

L300: Adds this UI element to the composition with the displayed content and parameters.

L301-303: Closes or continues the surrounding block, expression, or call.

34. EditStudentScreen.kt

app/src/main/java/com/example/studenttracker/presentation/student/EditStudentScreen.kt

Edit a student profile.

202 source lines. SHA-256: 197d1f415065166b

rememberSaveable holds each form field. Derived dirty state includes every editable field; BackHandler and the discard dialog guard unsaved changes.

Initial values come from the typed server record; the identifier stays immutable and the expected revision is sent.

The save handler trims input, validates required values and applicable email formats, then launches the API request. Busy state disables fields and navigation; success shows feedback and returns.

Cancellation is rethrown. Other failures show a message; finally restores the busy flag. Backend validation remains authoritative.

CustomTextField wraps OutlinedTextField and the appropriate keyboard type. Modifier chains size, padding, background, and alignment without changing backend data.

Source lines 1-32

```
0001 | package com.example.studenttracker.presentation.student
0002 |
0003 | import android.widget.Toast
0004 | import androidx.compose.foundation.layout.*
0005 | import androidx.compose.foundation.rememberScrollState
0006 | import androidx.compose.foundation.shape.RoundedCornerShape
0007 | import androidx.compose.foundation.text.KeyboardOptions
0008 | import androidx.compose.foundation.verticalScroll
0009 | import androidx.compose.material.icons.Icons
0010 | import androidx.compose.material.icons.filled.ArrowBack
0011 | import androidx.compose.material3.*
0012 | import androidx.compose.runtime.*
0013 | import androidx.compose.runtime.saveable.rememberSaveable
0014 | import androidx.compose.ui.Modifier
0015 | import androidx.compose.ui.graphics.Color
0016 | import androidx.compose.ui.platform.LocalContext
0017 | import androidx.compose.ui.text.font.FontWeight
0018 | import androidx.compose.ui.text.input.KeyboardType
0019 | import androidx.compose.ui.unit.dp
0020 | import com.example.studenttracker.data.network.NetworkModule
0021 | import com.example.studenttracker.data.network.Student
0022 | import com.example.studenttracker.data.network.UpdateStudentRequest
0023 | import kotlin.coroutines.launch
0024 |
0025 | @OptIn(ExperimentalMaterial3Api::class)
0026 | @Composable
0027 | fun EditStudentScreen(
0028 |     student: Student,
0029 |     onNavigateBack: () -> Unit
0030 | ) {
0031 |     val context = LocalContext.current
0032 |     val coroutineScope = rememberCoroutineScope()
```

L1: Declares the Kotlin namespace; it organizes classes and functions.

L3-23: Imports the named library symbols used by this file.

L25: Applies the displayed annotation to the declaration.

L26: Marks the following function as a Compose UI function.

L27: Defines this Kotlin operation and its parameters; suspend allows asynchronous calls.

L28-30: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L31: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L32: Obtains a coroutine scope tied to this composable for button-triggered asynchronous work.

Source lines 33-64

```

0033 |
0034 |     var name by rememberSaveable { mutableStateOf(student.name) }
0035 |     var email by rememberSaveable { mutableStateOf(student.email) }
0036 |     var phone by rememberSaveable { mutableStateOf(student.phone) }
0037 |     var department by rememberSaveable { mutableStateOf(student.department) }
0038 |     var batch by rememberSaveable { mutableStateOf(student.batch) }
0039 |     var year by rememberSaveable { mutableStateOf(student.year) }
0040 |     var semester by rememberSaveable { mutableStateOf(student.semester) }
0041 |     var section by rememberSaveable { mutableStateOf(student.section) }
0042 |     var parentName by rememberSaveable { mutableStateOf(student.parentName) }
0043 |     var parentPhone by rememberSaveable { mutableStateOf(student.parentPhone) }
0044 |     var parentEmail by rememberSaveable { mutableStateOf(student.parentEmail) }
0045 |
0046 |     var isLoading by remember { mutableStateOf(false) }
0047 |
0048 |     var showUnsavedDialog by remember { mutableStateOf(false) }
0049 |     val hasUnsavedChanges = name != student.name || email != student.email || phone != student.phone
    > || department != student.department || batch != student.batch || year != student.year || semester
    > != student.semester || section != student.section || parentName != student.parentName || parentPhone
    > != student.parentPhone || parentEmail != student.parentEmail
0050 |
0051 |     androidx.activity.compose.BackHandler(enabled = hasUnsavedChanges || isLoading) {
0052 |         if (!isLoading) showUnsavedDialog = true
0053 |     }
0054 |
0055 |     val handleBackPress = {
0056 |         if (isLoading) {
0057 |
0058 |         } else if (hasUnsavedChanges) {
0059 |             showUnsavedDialog = true
0060 |         } else {
0061 |             onBackPressed()
0062 |         }
0063 |     }
0064 |

```

L34-44: Keeps this UI value across supported activity recreation; it is not server persistence.

L46: Creates observable local UI state; assignment triggers recomposition.

L48: Creates observable local UI state; assignment triggers recomposition.

L49: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L51: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L52: Branches on the displayed condition; the accepted branch performs the following operation.

L53: Closes or continues the surrounding block, expression, or call.

L55: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L56: Branches on the displayed condition; the accepted branch performs the following operation.

L58: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L59: Assigns the displayed expression to a variable or named argument.

L60-61: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L62-63: Closes or continues the surrounding block, expression, or call.

Source lines 65-96

```

0065 |     if (showUnsavedDialog) {
0066 |         AlertDialog(
0067 |             onDismissRequest = { showUnsavedDialog = false },
0068 |             title = { Text("Discard Changes?", color = Color.White) },
0069 |             text = { Text("You have unsaved changes. Are you sure you want to go back?", color =
    > Color.LightGray) },
0070 |             containerColor = Color(0xFF1E1E1E),
0071 |             confirmButton = {
0072 |                 TextButton(onClick = { onBackPressed() }) { Text("Discard", color = Color.Red) }
0073 |             },
0074 |             dismissButton = {
0075 |                 TextButton(onClick = { showUnsavedDialog = false }) { Text("Cancel", color =
    > Color.Gray) }
0076 |             }
0077 |         )
0078 |     }

```

```

0079 |
0080 |     Scaffold(
0081 |         topBar = {
0082 |             TopAppBar(
0083 |                 title = { Text("Edit Student", color = Color.White) },
0084 |                 navigationIcon = {
0085 |                     IconButton(onClick = handleBackPressed) {
0086 |                         Icon(Icons.Default.ArrowBack, contentDescription = "Back", tint =
> Color.White)
0087 |                     }
0088 |                 },
0089 |                 colors = TopAppBarDefaults.topAppBarColors(containerColor = Color(0xFF1E1E1E))
0090 |             )
0091 |         },
0092 |         containerColor = Color(0xFF121212)
0093 |     ) { paddingValues ->
0094 |         Column(
0095 |             modifier = Modifier
0096 |                 .fillMaxSize()

```

L65: Branches on the displayed condition; the accepted branch performs the following operation.

L66: Adds this UI element to the composition with the displayed content and parameters.

L67-71: Assigns the displayed expression to a variable or named argument.

L72: Connects a user interaction to the displayed state update or handler.

L73: Closes or continues the surrounding block, expression, or call.

L74: Assigns the displayed expression to a variable or named argument.

L75: Connects a user interaction to the displayed state update or handler.

L76-78: Closes or continues the surrounding block, expression, or call.

L80: Adds this UI element to the composition with the displayed content and parameters.

L81: Assigns the displayed expression to a variable or named argument.

L82: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L83-84: Assigns the displayed expression to a variable or named argument.

L85: Connects a user interaction to the displayed state update or handler.

L86: Adds this UI element to the composition with the displayed content and parameters.

L87-88: Closes or continues the surrounding block, expression, or call.

L89: Assigns the displayed expression to a variable or named argument.

L90-91: Closes or continues the surrounding block, expression, or call.

L92: Assigns the displayed expression to a variable or named argument.

L93: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L94: Adds this UI element to the composition with the displayed content and parameters.

L95-96: Configures UI layout, sizing, spacing, or appearance; it does not modify academic records.

Source lines 97-128

```

0097 |         .padding(paddingValues)
0098 |         .padding(16.dp)
0099 |         .verticalScroll(rememberScrollState()),
0100 |         verticalArrangement = Arrangement.spacedBy(16.dp)
0101 |     ) {
0102 |         Text("Roll Number: ${student.rollNumber} (Cannot be changed)", color = Color.Gray, style
> = MaterialTheme.typography.bodyMedium)
0103 |
0104 |         Text("Personal Info", color = Color(0xFF03DAC5), fontWeight = FontWeight.Bold)
0105 |         CustomTextField(value = name, onValueChange = { name = it }, label = "Full Name",
> enabled = !isLoading)
0106 |         CustomTextField(value = email, onValueChange = { email = it }, label = "Email",
> keyboardType = KeyboardType.Email, enabled = !isLoading)
0107 |         CustomTextField(value = phone, onValueChange = { phone = it }, label = "Phone Number",
> keyboardType = KeyboardType.Phone, enabled = !isLoading)
0108 |
0109 |         Text("Academic Info", color = Color(0xFF03DAC5), fontWeight = FontWeight.Bold)
0110 |         CustomTextField(value = department, onValueChange = { department = it }, label =
> "Department (e.g., B.Tech CSE)", enabled = !isLoading)
0111 |         Row(horizontalArrangement = Arrangement.spacedBy(8.dp)) {

```

```

0112 |         CustomTextField(value = batch, onValueChange = { batch = it }, label = "Batch (e.g.,
> 2023)", modifier = Modifier.weight(1f), keyboardType = KeyboardType.Number, enabled = !isLoading)
0113 |         CustomTextField(value = year, onValueChange = { year = it }, label = "Year",
> modifier = Modifier.weight(1f), keyboardType = KeyboardType.Number, enabled = !isLoading)
0114 |     }
0115 |     Row(horizontalArrangement = Arrangement.spacedBy(8.dp)) {
0116 |         CustomTextField(value = semester, onValueChange = { semester = it }, label =
> "Semester", modifier = Modifier.weight(1f), keyboardType = KeyboardType.Number, enabled =
> !isLoading)
0117 |         CustomTextField(value = section, onValueChange = { section = it }, label =
> "Section", modifier = Modifier.weight(1f), enabled = !isLoading)
0118 |     }
0119 |
0120 |     Text("Parent/Guardian Info", color = Color(0xFF03DAC5), fontWeight = FontWeight.Bold)
0121 |     CustomTextField(value = parentName, onValueChange = { parentName = it }, label = "Parent
> Name", enabled = !isLoading)
0122 |     CustomTextField(value = parentPhone, onValueChange = { parentPhone = it }, label =
> "Parent Phone", keyboardType = KeyboardType.Phone, enabled = !isLoading)
0123 |     CustomTextField(value = parentEmail, onValueChange = { parentEmail = it }, label =
> "Parent Email", keyboardType = KeyboardType.Email, enabled = !isLoading)
0124 |
0125 |     Spacer(modifier = Modifier.height(16.dp))
0126 |
0127 |     Button(
0128 |         onClick = {

```

L97-98: Configures UI layout, sizing, spacing, or appearance; it does not modify academic records.

L99: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L100: Assigns the displayed expression to a variable or named argument.

L101: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L102: Adds this UI element to the composition with the displayed content and parameters.

L104: Adds this UI element to the composition with the displayed content and parameters.

L105-107: Connects a user interaction to the displayed state update or handler.

L109: Adds this UI element to the composition with the displayed content and parameters.

L110: Connects a user interaction to the displayed state update or handler.

L111: Adds this UI element to the composition with the displayed content and parameters.

L112-113: Connects a user interaction to the displayed state update or handler.

L114: Closes or continues the surrounding block, expression, or call.

L115: Adds this UI element to the composition with the displayed content and parameters.

L116-117: Connects a user interaction to the displayed state update or handler.

L118: Closes or continues the surrounding block, expression, or call.

L120: Adds this UI element to the composition with the displayed content and parameters.

L121-123: Connects a user interaction to the displayed state update or handler.

L125: Adds this UI element to the composition with the displayed content and parameters.

L127: Adds this UI element to the composition with the displayed content and parameters.

L128: Connects a user interaction to the displayed state update or handler.

Source lines 129-160

```

0129 |         if (name.isBlank() || email.isBlank() || department.isBlank()) {
0130 |             Toast.makeText(context, "Name, Email, and Department are required",
> Toast.LENGTH_SHORT).show()
0131 |             return@Button
0132 |         }
0133 |
0134 |         isLoading = true
0135 |         coroutineScope.launch {
0136 |             try {
0137 |                 val request = UpdateStudentRequest(
0138 |                     revision = student.revision,
0139 |                     rollNumber = student.rollNumber,
0140 |                     name = name.trim(),
0141 |                     email = email.trim(),
0142 |                     phone = phone.trim(),

```

```

0143 |                 department = department.trim(),
0144 |                 batch = batch.trim(),
0145 |                 year = year.trim(),
0146 |                 semester = semester.trim(),
0147 |                 section = section.trim(),
0148 |                 parentName = parentName.trim(),
0149 |                 parentPhone = parentPhone.trim(),
0150 |                 parentEmail = parentEmail.trim()
0151 |             )
0152 |             val response = NetworkModule.api.updateStudent(request)
0153 |             if (response.status == "success") {
0154 |                 Toast.makeText(context, "Student updated successfully",
> Toast.LENGTH_SHORT).show()
0155 |                 onBackPressed()
0156 |             } else {
0157 |                 Toast.makeText(context, "Error: ${response.message}",
> Toast.LENGTH_LONG).show()
0158 |             }
0159 |             } catch (e: kotlinx.coroutines.CancellationException) { throw e
0160 |             } catch (e: Exception) {

```

L129: Branches on the displayed condition; the accepted branch performs the following operation.

L130-131: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L132: Closes or continues the surrounding block, expression, or call.

L134: Assigns the displayed expression to a variable or named argument.

L135: Starts asynchronous work in the UI scope so the button handler can return promptly.

L136: Starts an operation whose failures are handled by the following catch/finally blocks.

L137: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L138: Carries or checks the record version used to detect stale edits or matching retries.

L139-150: Assigns the displayed expression to a variable or named argument.

L151: Closes or continues the surrounding block, expression, or call.

L152: Calls the named authenticated backend operation with the displayed request or key.

L153: Branches on the displayed condition; the accepted branch performs the following operation.

L154-157: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L158: Closes or continues the surrounding block, expression, or call.

L159: Preserves coroutine cancellation rather than presenting it as an ordinary application error.

L160: Handles this exception type and translates it into the following failure behavior.

Source lines 161-192

```

0161 |                 Toast.makeText(context, "Failed to update: ${e.message}",
> Toast.LENGTH_LONG).show()
0162 |             } finally {
0163 |                 isLoading = false
0164 |             }
0165 |         }
0166 |     },
0167 |     modifier = Modifier
0168 |         .fillMaxWidth()
0169 |         .height(50.dp),
0170 |     shape = RoundedCornerShape(12.dp),
0171 |     colors = ButtonDefaults.buttonColors(containerColor = Color(0xFFBB86FC)),
0172 |     enabled = !isLoading
0173 | ) {
0174 |     if (isLoading) {
0175 |         CircularProgressIndicator(color = Color.White, modifier = Modifier.size(24.dp))
0176 |     } else {
0177 |         Text("Update Student", color = Color.Black, fontWeight = FontWeight.Bold, style
> = MaterialTheme.typography.titleMedium)
0178 |     }
0179 | }
0180 | Spacer(modifier = Modifier.height(32.dp))
0181 | }
0182 | }
0183 | }
0184 |

```

```

0185 | @Composable
0186 | fun CustomTextField(
0187 |     value: String,
0188 |     onValueChange: (String) -> Unit,
0189 |     label: String,
0190 |     modifier: Modifier = Modifier,
0191 |     keyboardType: KeyboardType = KeyboardType.Text,
0192 |     enabled: Boolean = true

```

L161: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L162: Runs cleanup whether the preceding operation succeeded or failed.

L163: Assigns the displayed expression to a variable or named argument.

L164-166: Closes or continues the surrounding block, expression, or call.

L167-169: Configures UI layout, sizing, spacing, or appearance; it does not modify academic records.

L170-172: Assigns the displayed expression to a variable or named argument.

L173: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L174: Branches on the displayed condition; the accepted branch performs the following operation.

L175: Adds this UI element to the composition with the displayed content and parameters.

L176: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L177: Adds this UI element to the composition with the displayed content and parameters.

L178-179: Closes or continues the surrounding block, expression, or call.

L180: Adds this UI element to the composition with the displayed content and parameters.

L181-183: Closes or continues the surrounding block, expression, or call.

L185: Marks the following function as a Compose UI function.

L186: Defines this Kotlin operation and its parameters; suspend allows asynchronous calls.

L187: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L188: Connects a user interaction to the displayed state update or handler.

L189: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L190: Configures UI layout, sizing, spacing, or appearance; it does not modify academic records.

L191-192: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

Source lines 193-202

```

0193 | ) {
0194 |     OutlinedTextField(
0195 |         enabled = enabled,
0196 |         value = value,
0197 |         onValueChange = onValueChange,
0198 |         label = { Text(label) },
0199 |         keyboardOptions = KeyboardOptions(keyboardType = keyboardType),
0200 |         modifier = modifier.fillMaxWidth()
0201 |     )
0202 | }

```

L193: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L194: Adds this UI element to the composition with the displayed content and parameters.

L195-196: Assigns the displayed expression to a variable or named argument.

L197: Connects a user interaction to the displayed state update or handler.

L198-200: Assigns the displayed expression to a variable or named argument.

L201-202: Closes or continues the surrounding block, expression, or call.

35. StudentDashboardScreen.kt

app/src/main/java/com/example/studenttracker/presentation/student/StudentDashboardScreen.kt

Current student attendance and published results.

44 source lines. SHA-256: ee32b58986477ab8

getMyProgress returns server-scoped data. The UI never requests a separate arbitrary student identity.

Attendance cards show counted sessions and a bounded progress indicator. Empty percentage means no counted sessions, rather than zero attendance.

Published result cards distinguish absent from scored results. Refresh and logout are available; draft assessment marks are not shown.

Source lines 1-32

```
0001 | package com.example.studenttracker.presentation.student
0002 |
0003 | import androidx.compose.foundation.layout.*
0004 | import androidx.compose.foundation.lazy.LazyColumn
0005 | import androidx.compose.material3.*
0006 | import androidx.compose.runtime.*
0007 | import androidx.compose.ui.Modifier
0008 | import androidx.compose.ui.unit.dp
0009 | import com.example.studenttracker.data.network.*
0010 | import com.example.studenttracker.presentation.components.*
0011 |
0012 | @Composable
0013 | fun StudentDashboardScreen(onLogout: () -> Unit) {
0014 |     val user by AuthSession.user.collectAsState()
0015 |     var refresh by remember { mutableIntStateOf(0) }
0016 |     WorkspacePage("My progress", actions = {
0017 |         TextButton(onClick = { refresh++ }) { Text("Refresh") }
0018 |         TextButton(onClick = onLogout) { Text("Logout") }
0019 |     }) { padding ->
0020 |         Column(Modifier.fillMaxSize().padding(padding)) {
0021 |             RemoteContent(refresh, { NetworkModule.api.getMyProgress() }) { progress ->
0022 |                 LazyColumn(contentPadding = PaddingValues(16.dp), verticalArrangement =
0023 |                 > Arrangement.spacedBy(16.dp)) {
0024 |                     item { Text("Welcome, ${user?.name.orEmpty()}", style =
0025 |                     > MaterialTheme.typography.headlineSmall) }
0026 |                     item { Text("Attendance", style = MaterialTheme.typography.titleLarge) }
0027 |                     if (progress.attendance.isEmpty()) item { InfoCard("No attendance yet", "Your
0028 |                     > enrolled subjects and submitted sessions will appear here.") }
0029 |                     progress.attendance.forEach { record -> item(key =
0030 |                     > "attendance-${record.subjectCode}") {
0031 |                         InfoCard(record.subjectName, record.subjectCode) {
0032 |                             val percentage = record.percentage.toFloatOrNull()
0033 |                             Text(if (percentage == null) "No counted classes yet" else
0034 |                             > "${record.percentage}% · ${record.totalPresent}/${record.totalClasses} classes")
0035 |                             if (percentage != null) LinearProgressIndicator(progress = { (percentage
0036 |                             > / 100).coerceIn(0f, 1f) }, modifier = Modifier.fillMaxWidth())
0037 |                         }
0038 |                     }
0039 |                 }
0040 |             }
0041 |         }
0042 |     }
0043 | }
```

L1: Declares the Kotlin namespace; it organizes classes and functions.

L3-10: Imports the named library symbols used by this file.

L12: Marks the following function as a Compose UI function.

L13: Defines this Kotlin operation and its parameters; suspend allows asynchronous calls.

L14-15: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L16: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L17-18: Connects a user interaction to the displayed state update or handler.

L19: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L20: Adds this UI element to the composition with the displayed content and parameters.

L21: Calls the named authenticated backend operation with the displayed request or key.

L22: Adds this UI element to the composition with the displayed content and parameters.

L23-24: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L25: Branches on the displayed condition; the accepted branch performs the following operation.

L26: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L27: Adds this UI element to the composition with the displayed content and parameters.

L28: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L29: Adds this UI element to the composition with the displayed content and parameters.

L30: Branches on the displayed condition; the accepted branch performs the following operation.

L31: Closes or continues the surrounding block, expression, or call.

L32: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

Source lines 33-44

```
0033 |           item { Text("Published results", style = MaterialTheme.typography.titleLarge) }
0034 |           if (progress.results.isEmpty()) item { InfoCard("No published results", "Results
> appear after your faculty publishes them.") }
0035 |           progress.results.forEach { result -> item(key = "result-${result.assessmentId}")
> {
0036 |               InfoCard(result.title, result.subjectCode) {
0037 |                   Text(if (result.status == "absent") "Absent" else "${result.score} /
> ${result.maxMarks}", style = MaterialTheme.typography.headlineSmall)
0038 |               }
0039 |           } }
0040 |       }
0041 |   }
0042 | }
0043 | }
0044 | }
```

L33: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L34: Branches on the displayed condition; the accepted branch performs the following operation.

L35: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L36-37: Adds this UI element to the composition with the displayed content and parameters.

L38: Closes or continues the surrounding block, expression, or call.

L39: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L40-44: Closes or continues the surrounding block, expression, or call.

36. StudentDetailsScreen.kt

app/src/main/java/com/example/studenttracker/presentation/student/StudentDetailsScreen.kt

Read-only student profile details and actions.

201 source lines. SHA-256: d2d394007b0c21bc

The typed record supplies the display values; cards group related fields. Small DetailRow helpers display icon, label, and value.

Edit invokes the provided route callback. Deactivate opens confirmation and sends the record key with its expected revision.

Busy state prevents repeated actions. Success returns to the previous screen; failure shows feedback. Deactivation preserves historical academic data.

Source lines 1-32

```
0001 | package com.example.studenttracker.presentation.student
0002 |
0003 | import androidx.compose.foundation.background
0004 | import androidx.compose.foundation.layout.*
0005 | import androidx.compose.foundation.rememberScrollState
0006 | import androidx.compose.foundation.shape.CircleShape
0007 | import androidx.compose.foundation.shape.RoundedCornerShape
0008 | import androidx.compose.foundation.verticalScroll
0009 | import androidx.compose.material.icons.Icons
0010 | import androidx.compose.material.icons.filled.ArrowBack
0011 | import androidx.compose.material.icons.filled.Email
0012 | import androidx.compose.material.icons.filled.Person
0013 | import androidx.compose.material.icons.filled.Phone
0014 | import androidx.compose.material3.*
0015 | import androidx.compose.material.icons.filled.Delete
0016 | import androidx.compose.material.icons.filled.Edit
0017 | import androidx.compose.runtime.*
0018 | import androidx.compose.ui.platform.LocalContext
0019 | import android.widget.Toast
0020 | import kotlinx.coroutines.launch
0021 | import androidx.compose.ui.Alignment
0022 | import androidx.compose.ui.Modifier
0023 | import androidx.compose.ui.draw.clip
0024 | import androidx.compose.ui.graphics.Color
0025 | import androidx.compose.ui.graphics.vector.ImageVector
0026 | import androidx.compose.ui.text.font.FontWeight
0027 | import androidx.compose.ui.unit.dp
0028 | import com.example.studenttracker.data.network.Student
0029 |
0030 | @OptIn(ExperimentalMaterial3Api::class)
0031 | @Composable
0032 | fun StudentDetailsScreen(
```

L1: Declares the Kotlin namespace; it organizes classes and functions.

L3-28: Imports the named library symbols used by this file.

L30: Applies the displayed annotation to the declaration.

L31: Marks the following function as a Compose UI function.

L32: Defines this Kotlin operation and its parameters; suspend allows asynchronous calls.

Source lines 33-64

```
0033 |     student: Student,
0034 |     onNavigateBack: () -> Unit,
0035 |     onNavigateToEdit: () -> Unit
0036 | ) {
0037 |     val context = LocalContext.current
0038 |     val coroutineScope = rememberCoroutineScope()
0039 |     var showDeleteDialog by remember { mutableStateOf(false) }
0040 |     var isDeleting by remember { mutableStateOf(false) }
0041 |
```

```

0042 |         if (showDeleteDialog) {
0043 |             AlertDialog(
0044 |                 onDismissRequest = { showDeleteDialog = false },
0045 |                 title = { Text("Deactivate Student") },
0046 |                 text = { Text("Are you sure you want to delete ${student.name} (${student.rollNumber})?
> The profile will be deactivated. Historical records will be retained.") },
0047 |                 confirmButton = {
0048 |                     TextButton(
0049 |                         onClick = {
0050 |                             isDeleting = true
0051 |                             coroutineScope.launch {
0052 |                                 try {
0053 |                                     val api = com.example.studenttracker.data.network.NetworkModule.api
0054 |                                     val response =
> api.deleteStudent(com.example.studenttracker.data.network.DeleteStudentRequest(revision =
> student.revision, rollNumber = student.rollNumber))
0055 |                                     if (response.status == "success") {
0056 |                                         Toast.makeText(context, "Student deactivated successfully",
> Toast.LENGTH_SHORT).show()
0057 |                                         showDeleteDialog = false
0058 |                                         onBackPressed()
0059 |                                     } else {
0060 |                                         Toast.makeText(context, "Error: ${response.message}",
> Toast.LENGTH_SHORT).show()
0061 |                                     }
0062 |                                 } catch (e: kotlinx.coroutines.CancellationException) { throw e
0063 |                                 } catch (e: Exception) {
0064 |                                     Toast.makeText(context, "Failed to delete: ${e.toString()}",
> Toast.LENGTH_LONG).show()

```

L33-36: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L37: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L38: Obtains a coroutine scope tied to this composable for button-triggered asynchronous work.

L39-40: Creates observable local UI state; assignment triggers recomposition.

L42: Branches on the displayed condition; the accepted branch performs the following operation.

L43: Adds this UI element to the composition with the displayed content and parameters.

L44-47: Assigns the displayed expression to a variable or named argument.

L48: Adds this UI element to the composition with the displayed content and parameters.

L49: Connects a user interaction to the displayed state update or handler.

L50: Assigns the displayed expression to a variable or named argument.

L51: Starts asynchronous work in the UI scope so the button handler can return promptly.

L52: Starts an operation whose failures are handled by the following catch/finally blocks.

L53: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L54: Carries or checks the record version used to detect stale edits or matching retries.

L55: Branches on the displayed condition; the accepted branch performs the following operation.

L56: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L57: Assigns the displayed expression to a variable or named argument.

L58-60: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L61: Closes or continues the surrounding block, expression, or call.

L62: Preserves coroutine cancellation rather than presenting it as an ordinary application error.

L63: Handles this exception type and translates it into the following failure behavior.

L64: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

Source lines 65-96

```

0065 |                                     } finally {
0066 |                                         isDeleting = false
0067 |                                     }
0068 |                                 }
0069 |                             },
0070 |                             enabled = !isDeleting
0071 |                         ) {
0072 |                             if (isDeleting) {

```

```

0073 |         CircularProgressIndicator(modifier = Modifier.size(16.dp), strokeWidth =
    > 2.dp)
0074 |     } else {
0075 |         Text("Deactivate", color = Color.Red)
0076 |     }
0077 | }
0078 | },
0079 | dismissButton = {
0080 |     TextButton(onClick = { showDeleteDialog = false }, enabled = !isDeleting) {
0081 |         Text("Cancel", color = Color.Gray)
0082 |     }
0083 | },
0084 | containerColor = Color(0xFF1E1E1E),
0085 | titleContentColor = Color.White,
0086 | textContentColor = Color.LightGray
0087 | )
0088 | }
0089 |
0090 | Scaffold(
0091 |     topBar = {
0092 |         TopAppBar(
0093 |             title = { Text("Student Profile", color = Color.White) },
0094 |             navigationIcon = {
0095 |                 IconButton(onClick = onNavigateBack) {
0096 |                     Icon(Icons.Default.ArrowBack, contentDescription = "Back", tint =
    > Color.White)

```

L65: Runs cleanup whether the preceding operation succeeded or failed.

L66: Assigns the displayed expression to a variable or named argument.

L67-69: Closes or continues the surrounding block, expression, or call.

L70: Assigns the displayed expression to a variable or named argument.

L71: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L72: Branches on the displayed condition; the accepted branch performs the following operation.

L73: Adds this UI element to the composition with the displayed content and parameters.

L74: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L75: Adds this UI element to the composition with the displayed content and parameters.

L76-78: Closes or continues the surrounding block, expression, or call.

L79: Assigns the displayed expression to a variable or named argument.

L80: Connects a user interaction to the displayed state update or handler.

L81: Adds this UI element to the composition with the displayed content and parameters.

L82-83: Closes or continues the surrounding block, expression, or call.

L84-86: Assigns the displayed expression to a variable or named argument.

L87-88: Closes or continues the surrounding block, expression, or call.

L90: Adds this UI element to the composition with the displayed content and parameters.

L91: Assigns the displayed expression to a variable or named argument.

L92: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L93-94: Assigns the displayed expression to a variable or named argument.

L95: Connects a user interaction to the displayed state update or handler.

L96: Adds this UI element to the composition with the displayed content and parameters.

Source lines 97-128

```

0097 |     }
0098 | },
0099 | actions = {
0100 |     IconButton(onClick = onNavigateToEdit, enabled = !isDeleting) {
0101 |         Icon(Icons.Default.Edit, contentDescription = "Edit", tint = Color.White)
0102 |     }
0103 |     IconButton(onClick = { showDeleteDialog = true }, enabled = !isDeleting) {
0104 |         Icon(Icons.Default.Delete, contentDescription = "Delete", tint =
    > Color(0xFFCF6679))
0105 |     }
0106 | },

```

```

0107 |         colors = TopAppBarDefaults.topAppBarColors(containerColor = Color(0xFF1E1E1E))
0108 |     )
0109 | },
0110 |     containerColor = Color(0xFF121212)
0111 | ) { paddingValues ->
0112 |
0113 |     Column(
0114 |         modifier = Modifier
0115 |             .fillMaxSize()
0116 |             .padding(paddingValues)
0117 |             .padding(16.dp)
0118 |             .verticalScroll(rememberScrollState()),
0119 |         horizontalAlignment = Alignment.CenterHorizontally
0120 |     ) {
0121 |
0122 |         Box(
0123 |             modifier = Modifier
0124 |                 .size(100.dp)
0125 |                 .clip(CircleShape)
0126 |                 .background(Color(0xFFBB86FC).copy(alpha = 0.2f)),
0127 |             contentAlignment = Alignment.Center
0128 |         ) {

```

L97-98: Closes or continues the surrounding block, expression, or call.

L99: Assigns the displayed expression to a variable or named argument.

L100: Connects a user interaction to the displayed state update or handler.

L101: Adds this UI element to the composition with the displayed content and parameters.

L102: Closes or continues the surrounding block, expression, or call.

L103: Connects a user interaction to the displayed state update or handler.

L104: Adds this UI element to the composition with the displayed content and parameters.

L105-106: Closes or continues the surrounding block, expression, or call.

L107: Assigns the displayed expression to a variable or named argument.

L108-109: Closes or continues the surrounding block, expression, or call.

L110: Assigns the displayed expression to a variable or named argument.

L111: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L113: Adds this UI element to the composition with the displayed content and parameters.

L114-117: Configures UI layout, sizing, spacing, or appearance; it does not modify academic records.

L118: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L119: Assigns the displayed expression to a variable or named argument.

L120: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L122: Adds this UI element to the composition with the displayed content and parameters.

L123-126: Configures UI layout, sizing, spacing, or appearance; it does not modify academic records.

L127: Assigns the displayed expression to a variable or named argument.

L128: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

Source lines 129-160

```

0129 |         Icon(Icons.Default.Person, contentDescription = null, tint = Color(0xFFBB86FC),
0130 |             > modifier = Modifier.size(60.dp))
0131 |     }
0132 |     Spacer(modifier = Modifier.height(16.dp))
0133 |     Text(student.name, color = Color.White, style = MaterialTheme.typography.headlineMedium,
0134 |         > fontWeight = FontWeight.Bold)
0135 |     Text(student.rollNumber, color = Color.Gray, style =
0136 |         > MaterialTheme.typography.titleMedium)
0137 |
0138 |     Spacer(modifier = Modifier.height(24.dp))
0139 |
0140 |     SectionCard(title = "Academic Information") {
0141 |         DetailRow("Department", student.department)
0142 |         DetailRow("Batch", student.batch)
0143 |         DetailRow("Year", student.year)
0144 |         DetailRow("Semester", student.semester)
0145 |         DetailRow("Section", student.section)

```

```

0143 |         }
0144 |
0145 |         Spacer(modifier = Modifier.height(16.dp))
0146 |
0147 |         SectionCard(title = "Contact Information") {
0148 |             DetailIconRow(Icons.Default.Email, "Email", student.email)
0149 |             DetailIconRow(Icons.Default.Phone, "Phone", student.phone)
0150 |         }
0151 |
0152 |         Spacer(modifier = Modifier.height(16.dp))
0153 |
0154 |         SectionCard(title = "Parent Information") {
0155 |             DetailRow("Name", student.parentName)
0156 |             DetailIconRow(Icons.Default.Phone, "Phone", student.parentPhone)
0157 |             DetailIconRow(Icons.Default.Email, "Email", student.parentEmail)
0158 |         }
0159 |
0160 |         Spacer(modifier = Modifier.height(32.dp))

```

L129: Adds this UI element to the composition with the displayed content and parameters.

L130: Closes or continues the surrounding block, expression, or call.

L131-133: Adds this UI element to the composition with the displayed content and parameters.

L135: Adds this UI element to the composition with the displayed content and parameters.

L137-142: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L143: Closes or continues the surrounding block, expression, or call.

L145: Adds this UI element to the composition with the displayed content and parameters.

L147-149: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L150: Closes or continues the surrounding block, expression, or call.

L152: Adds this UI element to the composition with the displayed content and parameters.

L154-157: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L158: Closes or continues the surrounding block, expression, or call.

L160: Adds this UI element to the composition with the displayed content and parameters.

Source lines 161-192

```

0161 |         }
0162 |     }
0163 | }
0164 |
0165 | @Composable
0166 | fun SectionCard(title: String, content: @Composable ColumnScope.() -> Unit) {
0167 |     Card(
0168 |         modifier = Modifier.fillMaxWidth(),
0169 |         colors = CardDefaults.cardColors(containerColor = Color(0xFF1E1E1E)),
0170 |         shape = RoundedCornerShape(12.dp)
0171 |     ) {
0172 |         Column(modifier = Modifier.padding(16.dp)) {
0173 |             Text(title, color = Color(0xFF03DAC5), fontWeight = FontWeight.Bold, style =
0174 | > MaterialTheme.typography.titleMedium)
0175 |             Spacer(modifier = Modifier.height(12.dp))
0176 |             content()
0177 |         }
0178 |     }
0179 | }
0180 | @Composable
0181 | fun DetailRow(label: String, value: String) {
0182 |     Row(
0183 |         modifier = Modifier.fillMaxWidth().padding(vertical = 4.dp),
0184 |         horizontalArrangement = Arrangement.SpaceBetween
0185 |     ) {
0186 |         Text(label, color = Color.Gray, style = MaterialTheme.typography.bodyMedium)
0187 |         Text(if (value.isBlank()) "N/A" else value, color = Color.White, style =
0188 | > MaterialTheme.typography.bodyMedium, fontWeight = FontWeight.Medium)
0189 |     }
0190 | }
0191 | @Composable

```

```
0192 | fun DetailIconRow(icon: ImageVector, label: String, value: String) {
```

L161-163: Closes or continues the surrounding block, expression, or call.

L165: Marks the following function as a Compose UI function.

L166: Defines this Kotlin operation and its parameters; suspend allows asynchronous calls.

L167: Adds this UI element to the composition with the displayed content and parameters.

L168: Configures UI layout, sizing, spacing, or appearance; it does not modify academic records.

L169-170: Assigns the displayed expression to a variable or named argument.

L171: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L172-174: Adds this UI element to the composition with the displayed content and parameters.

L175: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L176-178: Closes or continues the surrounding block, expression, or call.

L180: Marks the following function as a Compose UI function.

L181: Defines this Kotlin operation and its parameters; suspend allows asynchronous calls.

L182: Adds this UI element to the composition with the displayed content and parameters.

L183: Configures UI layout, sizing, spacing, or appearance; it does not modify academic records.

L184: Assigns the displayed expression to a variable or named argument.

L185: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L186-187: Adds this UI element to the composition with the displayed content and parameters.

L188-189: Closes or continues the surrounding block, expression, or call.

L191: Marks the following function as a Compose UI function.

L192: Defines this Kotlin operation and its parameters; suspend allows asynchronous calls.

Source lines 193-201

```
0193 |     Row(  
0194 |         modifier = Modifier.fillMaxWidth().padding(vertical = 6.dp),  
0195 |         verticalAlignment = Alignment.CenterVertically  
0196 |     ) {  
0197 |         Icon(icon, contentDescription = label, tint = Color.Gray, modifier = Modifier.size(20.dp))  
0198 |         Spacer(modifier = Modifier.width(12.dp))  
0199 |         Text(if (value.isBlank()) "N/A" else value, color = Color.White, style =  
    > MaterialTheme.typography.bodyMedium)  
0200 |     }  
0201 | }
```

L193: Adds this UI element to the composition with the displayed content and parameters.

L194: Configures UI layout, sizing, spacing, or appearance; it does not modify academic records.

L195: Assigns the displayed expression to a variable or named argument.

L196: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L197-199: Adds this UI element to the composition with the displayed content and parameters.

L200-201: Closes or continues the surrounding block, expression, or call.

37. StudentListScreen.kt

app/src/main/java/com/example/studenttracker/presentation/student/StudentListScreen.kt

Student directory adapter.

13 source lines. SHA-256: b55f8d88ac484acd

Loads records with `getStudents`, maps the stable identifier and display fields into `DirectoryRecord`, and delegates searching and list rendering to `DirectoryScreen`.

Caller callbacks determine which record or add form opens.

Source lines 1-13

```
0001 | package com.example.studenttracker.presentation.student
0002 |
0003 | import androidx.compose.runtime.Composable
0004 | import com.example.studenttracker.data.network.NetworkModule
0005 | import com.example.studenttracker.presentation.components.DirectoryRecord
0006 | import com.example.studenttracker.presentation.components.DirectoryScreen
0007 |
0008 | @Composable
0009 | fun StudentListScreen(onNavigateBack: () -> Unit, onStudentClick: (String) -> Unit,
    > onNavigateToAddStudent: () -> Unit) {
0010 |     DirectoryScreen("Students", onNavigateBack, onNavigateToAddStudent, onStudentClick) {
0011 |         NetworkModule.api.getStudents().students.orEmpty().map { DirectoryRecord(it.rollNumber,
    > it.name, "${it.department} · Semester ${it.semester} · ${it.section}") }
0012 |     }
0013 | }
```

L1: Declares the Kotlin namespace; it organizes classes and functions.

L3-6: Imports the named library symbols used by this file.

L8: Marks the following function as a Compose UI function.

L9: Defines this Kotlin operation and its parameters; suspend allows asynchronous calls.

L10: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L11: Calls the named authenticated backend operation with the displayed request or key.

L12-13: Closes or continues the surrounding block, expression, or call.

38. AddSubjectScreen.kt

app/src/main/java/com/example/studenttracker/presentation/subject/AddSubjectScreen.kt

Create a subject profile.

196 source lines. SHA-256: 22e325dfcf13dbdf

rememberSaveable holds each form field. Derived dirty state includes every editable field; BackHandler and the discard dialog guard unsaved changes.

The form starts empty; the identifier is supplied when creating the record.

The save handler trims input, validates required values and applicable email formats, then launches the API request. Busy state disables fields and navigation; success shows feedback and returns.

Cancellation is rethrown. Other failures show a message; finally restores the busy flag. Backend validation remains authoritative.

CustomTextField wraps OutlinedTextField and the appropriate keyboard type. Modifier chains size, padding, background, and alignment without changing backend data.

Source lines 1-32

```
0001 | package com.example.studenttracker.presentation.subject
0002 |
0003 | import android.widget.Toast
0004 | import androidx.compose.foundation.layout.*
0005 | import androidx.compose.foundation.rememberScrollState
0006 | import androidx.compose.foundation.shape.RoundedCornerShape
0007 | import androidx.compose.foundation.text.KeyboardOptions
0008 | import androidx.compose.foundation.verticalScroll
0009 | import androidx.compose.material.icons.Icons
0010 | import androidx.compose.material.icons.filled.ArrowBack
0011 | import androidx.compose.material3.*
0012 | import androidx.compose.runtime.*
0013 | import androidx.compose.runtime.saveable.rememberSaveable
0014 | import androidx.compose.ui.Modifier
0015 | import androidx.compose.ui.graphics.Color
0016 | import androidx.compose.ui.platform.LocalContext
0017 | import androidx.compose.ui.text.font.FontWeight
0018 | import androidx.compose.ui.text.input.KeyboardType
0019 | import androidx.compose.ui.unit.dp
0020 | import com.example.studenttracker.data.network.AddSubjectRequest
0021 | import com.example.studenttracker.data.network.NetworkModule
0022 | import kotlinx.coroutines.launch
0023 |
0024 | @OptIn(ExperimentalMaterial3Api::class)
0025 | @Composable
0026 | fun AddSubjectScreen(
0027 |     onNavigateBack: () -> Unit
0028 | ) {
0029 |     val context = LocalContext.current
0030 |
0031 |     var subjectCode by rememberSaveable { mutableStateOf("") }
0032 |     var subjectName by rememberSaveable { mutableStateOf("") }
```

L1: Declares the Kotlin namespace; it organizes classes and functions.

L3-22: Imports the named library symbols used by this file.

L24: Applies the displayed annotation to the declaration.

L25: Marks the following function as a Compose UI function.

L26: Defines this Kotlin operation and its parameters; suspend allows asynchronous calls.

L27-28: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L29: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L31-32: Keeps this UI value across supported activity recreation; it is not server persistence.

Source lines 33-64

```

0033 |     var facultyId by rememberSaveable { mutableStateOf("") }
0034 |     var semester by rememberSaveable { mutableStateOf("") }
0035 |     var totalClasses by rememberSaveable { mutableStateOf("") }
0036 |
0037 |     var isError by remember { mutableStateOf(false) }
0038 |     var isSaving by remember { mutableStateOf(false) }
0039 |     val coroutineScope = rememberCoroutineScope()
0040 |
0041 |     fun validateAndSave() {
0042 |         if (subjectCode.isBlank() || subjectName.isBlank() || facultyId.isBlank() ||
> semester.isBlank() || totalClasses.isBlank()) {
0043 |             isError = true
0044 |             Toast.makeText(context, "Please fill in all fields", Toast.LENGTH_SHORT).show()
0045 |             return
0046 |         }
0047 |
0048 |         isSaving = true
0049 |         coroutineScope.launch {
0050 |             try {
0051 |                 val request = AddSubjectRequest(
0052 |                     subjectCode = subjectCode.trim(),
0053 |                     subjectName = subjectName.trim(),
0054 |                     facultyId = facultyId.trim(),
0055 |                     semester = semester.trim(),
0056 |                     totalClasses = totalClasses.trim()
0057 |                 )
0058 |                 val response = NetworkModule.api.addSubject(request)
0059 |                 if (response.status == "success") {
0060 |                     Toast.makeText(context, "Subject added successfully!",
> Toast.LENGTH_SHORT).show()
0061 |                     onBackPressed()
0062 |                 } else {
0063 |                     Toast.makeText(context, "Error: ${response.message}", Toast.LENGTH_LONG).show()
0064 |                 }

```

L33-35: Keeps this UI value across supported activity recreation; it is not server persistence.

L37-38: Creates observable local UI state; assignment triggers recomposition.

L39: Obtains a coroutine scope tied to this composable for button-triggered asynchronous work.

L41: Defines this Kotlin operation and its parameters; suspend allows asynchronous calls.

L42: Branches on the displayed condition; the accepted branch performs the following operation.

L43: Assigns the displayed expression to a variable or named argument.

L44-45: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L46: Closes or continues the surrounding block, expression, or call.

L48: Assigns the displayed expression to a variable or named argument.

L49: Starts asynchronous work in the UI scope so the button handler can return promptly.

L50: Starts an operation whose failures are handled by the following catch/finally blocks.

L51: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L52-56: Assigns the displayed expression to a variable or named argument.

L57: Closes or continues the surrounding block, expression, or call.

L58: Calls the named authenticated backend operation with the displayed request or key.

L59: Branches on the displayed condition; the accepted branch performs the following operation.

L60-63: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L64: Closes or continues the surrounding block, expression, or call.

Source lines 65-96

```

0065 |         } catch (e: kotlinx.coroutines.CancellationException) { throw e
0066 |         } catch (e: Exception) {
0067 |             Toast.makeText(context, "Failed to save: ${e.message}", Toast.LENGTH_LONG).show()
0068 |         } finally {
0069 |             isSaving = false
0070 |         }
0071 |     }
0072 | }
0073 |

```

```

0074 |     var showUnsavedDialog by remember { mutableStateOf(false) }
0075 |     val hasUnsavedChanges = subjectCode.isNotBlank() || subjectName.isNotBlank() ||
    > facultyId.isNotBlank() || semester.isNotBlank() || totalClasses.isNotBlank()
0076 |
0077 |     androidx.activity.compose.BackHandler(enabled = hasUnsavedChanges || isSaving) {
0078 |         if (!isSaving) showUnsavedDialog = true
0079 |     }
0080 |
0081 |     val handleBackPress = {
0082 |         if (isSaving) {
0083 |
0084 |         } else if (hasUnsavedChanges) {
0085 |             showUnsavedDialog = true
0086 |         } else {
0087 |             onBackPressed()
0088 |         }
0089 |     }
0090 |
0091 |     if (showUnsavedDialog) {
0092 |         AlertDialog(
0093 |             onDismissRequest = { showUnsavedDialog = false },
0094 |             title = { Text("Discard Changes?", color = Color.White) },
0095 |             text = { Text("You have unsaved changes. Are you sure you want to go back?", color =
    > Color.LightGray) },
0096 |             containerColor = Color(0xFF1E1E1E),

```

L65: Preserves coroutine cancellation rather than presenting it as an ordinary application error.

L66: Handles this exception type and translates it into the following failure behavior.

L67: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L68: Runs cleanup whether the preceding operation succeeded or failed.

L69: Assigns the displayed expression to a variable or named argument.

L70-72: Closes or continues the surrounding block, expression, or call.

L74: Creates observable local UI state; assignment triggers recomposition.

L75: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L77: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L78: Branches on the displayed condition; the accepted branch performs the following operation.

L79: Closes or continues the surrounding block, expression, or call.

L81: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L82: Branches on the displayed condition; the accepted branch performs the following operation.

L84: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L85: Assigns the displayed expression to a variable or named argument.

L86-87: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L88-89: Closes or continues the surrounding block, expression, or call.

L91: Branches on the displayed condition; the accepted branch performs the following operation.

L92: Adds this UI element to the composition with the displayed content and parameters.

L93-96: Assigns the displayed expression to a variable or named argument.

Source lines 97-128

```

0097 |         confirmButton = {
0098 |             TextButton(onClick = { onBackPressed() }) { Text("Discard", color = Color.Red) }
0099 |         },
0100 |         dismissButton = {
0101 |             TextButton(onClick = { showUnsavedDialog = false }) { Text("Cancel", color =
    > Color.Gray) }
0102 |         }
0103 |     )
0104 | }
0105 |
0106 | Scaffold(
0107 |     topBar = {
0108 |         TopAppBar(
0109 |             title = { Text("Add New Subject", color = Color.White) },
0110 |             navigationIcon = {

```

```

0111 |             IconButton(onClick = handleBackPress) {
0112 |                 Icon(Icons.Default.ArrowBack, contentDescription = "Back", tint =
> Color.White)
0113 |             }
0114 |         },
0115 |         colors = TopAppBarDefaults.topAppBarColors(containerColor = Color(0xFF1E1E1E))
0116 |     )
0117 | },
0118 |     containerColor = Color(0xFF121212)
0119 | ) { paddingValues ->
0120 |     Column(
0121 |         modifier = Modifier
0122 |             .fillMaxSize()
0123 |             .padding(paddingValues)
0124 |             .padding(16.dp)
0125 |             .verticalScroll(rememberScrollState()),
0126 |         verticalArrangement = Arrangement.spacedBy(16.dp)
0127 |     ) {
0128 |         Text("Subject Details", color = Color(0xFF3700B3), style =
> MaterialTheme.typography.titleMedium)

```

L97: Assigns the displayed expression to a variable or named argument.

L98: Connects a user interaction to the displayed state update or handler.

L99: Closes or continues the surrounding block, expression, or call.

L100: Assigns the displayed expression to a variable or named argument.

L101: Connects a user interaction to the displayed state update or handler.

L102-104: Closes or continues the surrounding block, expression, or call.

L106: Adds this UI element to the composition with the displayed content and parameters.

L107: Assigns the displayed expression to a variable or named argument.

L108: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L109-110: Assigns the displayed expression to a variable or named argument.

L111: Connects a user interaction to the displayed state update or handler.

L112: Adds this UI element to the composition with the displayed content and parameters.

L113-114: Closes or continues the surrounding block, expression, or call.

L115: Assigns the displayed expression to a variable or named argument.

L116-117: Closes or continues the surrounding block, expression, or call.

L118: Assigns the displayed expression to a variable or named argument.

L119: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L120: Adds this UI element to the composition with the displayed content and parameters.

L121-124: Configures UI layout, sizing, spacing, or appearance; it does not modify academic records.

L125: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L126: Assigns the displayed expression to a variable or named argument.

L127: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L128: Adds this UI element to the composition with the displayed content and parameters.

Source lines 129-160

```

0129 |
0130 |         OutlinedTextField(
0131 |             enabled = !isSaving,
0132 |             value = subjectCode, onValueChange = { subjectCode = it },
0133 |             label = { Text("Subject Code (e.g., CS101) **") },
0134 |             modifier = Modifier.fillMaxWidth(),
0135 |             isError = isError && subjectCode.isBlank()
0136 |         )
0137 |
0138 |         OutlinedTextField(
0139 |             enabled = !isSaving,
0140 |             value = subjectName, onValueChange = { subjectName = it },
0141 |             label = { Text("Subject Name **") },
0142 |             modifier = Modifier.fillMaxWidth(),
0143 |             isError = isError && subjectName.isBlank()
0144 |         )

```

```

0145 |
0146 |         HorizontalDivider(modifier = Modifier.padding(vertical = 8.dp), color = Color.DarkGray)
0147 |         Text("Assignment & Curriculum", color = Color(0xFF3700B3), style =
    > MaterialTheme.typography.titleMedium)
0148 |
0149 |         OutlinedTextField(
0150 |             enabled = !isSaving,
0151 |             value = facultyId, onValueChange = { facultyId = it },
0152 |             label = { Text("Assigned Faculty ID *") },
0153 |             modifier = Modifier.fillMaxWidth(),
0154 |             isError = isError && facultyId.isBlank()
0155 |         )
0156 |
0157 |         Row(modifier = Modifier.fillMaxWidth(), horizontalArrangement =
    > Arrangement.spacedBy(16.dp)) {
0158 |             OutlinedTextField(
0159 |                 enabled = !isSaving,
0160 |                 value = semester, onValueChange = { semester = it },

```

L130: Adds this UI element to the composition with the displayed content and parameters.

L131: Assigns the displayed expression to a variable or named argument.

L132: Connects a user interaction to the displayed state update or handler.

L133: Assigns the displayed expression to a variable or named argument.

L134: Configures UI layout, sizing, spacing, or appearance; it does not modify academic records.

L135: Assigns the displayed expression to a variable or named argument.

L136: Closes or continues the surrounding block, expression, or call.

L138: Adds this UI element to the composition with the displayed content and parameters.

L139: Assigns the displayed expression to a variable or named argument.

L140: Connects a user interaction to the displayed state update or handler.

L141: Assigns the displayed expression to a variable or named argument.

L142: Configures UI layout, sizing, spacing, or appearance; it does not modify academic records.

L143: Assigns the displayed expression to a variable or named argument.

L144: Closes or continues the surrounding block, expression, or call.

L146: Configures UI layout, sizing, spacing, or appearance; it does not modify academic records.

L147: Adds this UI element to the composition with the displayed content and parameters.

L149: Adds this UI element to the composition with the displayed content and parameters.

L150: Assigns the displayed expression to a variable or named argument.

L151: Connects a user interaction to the displayed state update or handler.

L152: Assigns the displayed expression to a variable or named argument.

L153: Configures UI layout, sizing, spacing, or appearance; it does not modify academic records.

L154: Assigns the displayed expression to a variable or named argument.

L155: Closes or continues the surrounding block, expression, or call.

L157-158: Adds this UI element to the composition with the displayed content and parameters.

L159: Assigns the displayed expression to a variable or named argument.

L160: Connects a user interaction to the displayed state update or handler.

Source lines 161-192

```

0161 |             label = { Text("Semester *") },
0162 |             keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Number),
0163 |             modifier = Modifier.weight(1f),
0164 |             isError = isError && semester.isBlank()
0165 |         )
0166 |         OutlinedTextField(
0167 |             enabled = !isSaving,
0168 |             value = totalClasses, onValueChange = { totalClasses = it },
0169 |             label = { Text("Total Classes *") },
0170 |             keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Number),
0171 |             modifier = Modifier.weight(1f),
0172 |             isError = isError && totalClasses.isBlank()
0173 |         )

```

```

0174 |         }
0175 |
0176 |         Spacer(modifier = Modifier.height(24.dp))
0177 |
0178 |         Button(
0179 |             onClick = { validateAndSave() },
0180 |             enabled = !isSaving,
0181 |             modifier = Modifier.fillMaxWidth().height(50.dp),
0182 |             shape = RoundedCornerShape(12.dp),
0183 |             colors = ButtonDefaults.buttonColors(containerColor = Color(0xFF3700B3),
> disabledContainerColor = Color.DarkGray)
0184 |         ) {
0185 |             if (isSaving) {
0186 |                 CircularProgressIndicator(modifier = Modifier.size(20.dp), color = Color.White,
> strokeWidth = 2.dp)
0187 |                 Spacer(modifier = Modifier.width(8.dp))
0188 |                 Text("Saving...", color = Color.White)
0189 |             } else {
0190 |                 Text("Save Subject", color = Color.White, fontWeight = FontWeight.Bold, style =
> MaterialTheme.typography.titleMedium)
0191 |             }
0192 |         }

```

L161-162: Assigns the displayed expression to a variable or named argument.

L163: Configures UI layout, sizing, spacing, or appearance; it does not modify academic records.

L164: Assigns the displayed expression to a variable or named argument.

L165: Closes or continues the surrounding block, expression, or call.

L166: Adds this UI element to the composition with the displayed content and parameters.

L167: Assigns the displayed expression to a variable or named argument.

L168: Connects a user interaction to the displayed state update or handler.

L169-170: Assigns the displayed expression to a variable or named argument.

L171: Configures UI layout, sizing, spacing, or appearance; it does not modify academic records.

L172: Assigns the displayed expression to a variable or named argument.

L173-174: Closes or continues the surrounding block, expression, or call.

L176: Adds this UI element to the composition with the displayed content and parameters.

L178: Adds this UI element to the composition with the displayed content and parameters.

L179: Connects a user interaction to the displayed state update or handler.

L180: Assigns the displayed expression to a variable or named argument.

L181: Configures UI layout, sizing, spacing, or appearance; it does not modify academic records.

L182-183: Assigns the displayed expression to a variable or named argument.

L184: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L185: Branches on the displayed condition; the accepted branch performs the following operation.

L186-188: Adds this UI element to the composition with the displayed content and parameters.

L189: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L190: Adds this UI element to the composition with the displayed content and parameters.

L191-192: Closes or continues the surrounding block, expression, or call.

Source lines 193-196

```

0193 |         Spacer(modifier = Modifier.height(40.dp))
0194 |     }
0195 | }
0196 | }

```

L193: Adds this UI element to the composition with the displayed content and parameters.

L194-196: Closes or continues the surrounding block, expression, or call.

39. EditSubjectScreen.kt

app/src/main/java/com/example/studenttracker/presentation/subject/EditSubjectScreen.kt

Edit a subject profile.

173 source lines. SHA-256: cf7e965ff257b1ab

rememberSaveable holds each form field. Derived dirty state includes every editable field; BackHandler and the discard dialog guard unsaved changes.

Initial values come from the typed server record; the identifier stays immutable and the expected revision is sent.

The save handler trims input, validates required values and applicable email formats, then launches the API request. Busy state disables fields and navigation; success shows feedback and returns.

Cancellation is rethrown. Other failures show a message; finally restores the busy flag. Backend validation remains authoritative.

CustomTextField wraps OutlinedTextField and the appropriate keyboard type. Modifier chains size, padding, background, and alignment without changing backend data.

Source lines 1-32

```
0001 | package com.example.studenttracker.presentation.subject
0002 |
0003 | import android.widget.Toast
0004 | import androidx.compose.foundation.layout.*
0005 | import androidx.compose.foundation.rememberScrollState
0006 | import androidx.compose.foundation.shape.RoundedCornerShape
0007 | import androidx.compose.foundation.text.KeyboardOptions
0008 | import androidx.compose.foundation.verticalScroll
0009 | import androidx.compose.material.icons.Icons
0010 | import androidx.compose.material.icons.filled.ArrowBack
0011 | import androidx.compose.material3.*
0012 | import androidx.compose.runtime.*
0013 | import androidx.compose.runtime.saveable.rememberSaveable
0014 | import androidx.compose.ui.Modifier
0015 | import androidx.compose.ui.graphics.Color
0016 | import androidx.compose.ui.platform.LocalContext
0017 | import androidx.compose.ui.text.font.FontWeight
0018 | import androidx.compose.ui.text.input.KeyboardType
0019 | import androidx.compose.ui.unit.dp
0020 | import com.example.studenttracker.data.network.Subject
0021 | import com.example.studenttracker.data.network.NetworkModule
0022 | import com.example.studenttracker.data.network.UpdateSubjectRequest
0023 | import kotlin.coroutines.launch
0024 |
0025 | @OptIn(ExperimentalMaterial3Api::class)
0026 | @Composable
0027 | fun EditSubjectScreen(
0028 |     subject: Subject,
0029 |     onNavigateBack: () -> Unit
0030 | ) {
0031 |     val context = LocalContext.current
0032 |     val coroutineScope = rememberCoroutineScope()
```

L1: Declares the Kotlin namespace; it organizes classes and functions.

L3-23: Imports the named library symbols used by this file.

L25: Applies the displayed annotation to the declaration.

L26: Marks the following function as a Compose UI function.

L27: Defines this Kotlin operation and its parameters; suspend allows asynchronous calls.

L28-30: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L31: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L32: Obtains a coroutine scope tied to this composable for button-triggered asynchronous work.

Source lines 33-64


```

0033 |
0034 |     var subjectName by rememberSaveable { mutableStateOf(subject.subjectName) }
0035 |     var facultyId by rememberSaveable { mutableStateOf(subject.facultyId) }
0036 |     var semester by rememberSaveable { mutableStateOf(subject.semester) }
0037 |     var totalClasses by rememberSaveable { mutableStateOf(subject.totalClasses) }
0038 |
0039 |     var isLoading by remember { mutableStateOf(false) }
0040 |
0041 |     var showUnsavedDialog by remember { mutableStateOf(false) }
0042 |     val hasUnsavedChanges = subjectName != subject.subjectName || facultyId != subject.facultyId ||
> semester != subject.semester || totalClasses != subject.totalClasses
0043 |
0044 |     androidx.activity.compose.BackHandler(enabled = hasUnsavedChanges || isLoading) {
0045 |         if (!isLoading) showUnsavedDialog = true
0046 |     }
0047 |
0048 |     val handleBackPressed = {
0049 |         if (isLoading) {
0050 |
0051 |         } else if (hasUnsavedChanges) {
0052 |             showUnsavedDialog = true
0053 |         } else {
0054 |             onBackPressed()
0055 |         }
0056 |     }
0057 |
0058 |     if (showUnsavedDialog) {
0059 |         AlertDialog(
0060 |             onDismissRequest = { showUnsavedDialog = false },
0061 |             title = { Text("Discard Changes?", color = Color.White) },
0062 |             text = { Text("You have unsaved changes. Are you sure you want to go back?", color =
> Color.LightGray) },
0063 |             containerColor = Color(0xFF1E1E1E),
0064 |             confirmButton = {

```

L34-37: Keeps this UI value across supported activity recreation; it is not server persistence.

L39: Creates observable local UI state; assignment triggers recomposition.

L41: Creates observable local UI state; assignment triggers recomposition.

L42: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L44: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L45: Branches on the displayed condition; the accepted branch performs the following operation.

L46: Closes or continues the surrounding block, expression, or call.

L48: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L49: Branches on the displayed condition; the accepted branch performs the following operation.

L51: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L52: Assigns the displayed expression to a variable or named argument.

L53-54: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L55-56: Closes or continues the surrounding block, expression, or call.

L58: Branches on the displayed condition; the accepted branch performs the following operation.

L59: Adds this UI element to the composition with the displayed content and parameters.

L60-64: Assigns the displayed expression to a variable or named argument.

Source lines 65-96

```

0065 |         TextButton(onClick = { onBackPressed() }) { Text("Discard", color = Color.Red) }
0066 |     },
0067 |     dismissButton = {
0068 |         TextButton(onClick = { showUnsavedDialog = false }) { Text("Cancel", color =
> Color.Gray) }
0069 |     }
0070 | )
0071 | }
0072 |
0073 | Scaffold(
0074 |     topBar = {
0075 |         TopAppBar(

```

```

0076 |         title = { Text("Edit Subject", color = Color.White) },
0077 |         navigationIcon = {
0078 |             IconButton(onClick = handleBackPressed) {
0079 |                 Icon(Icons.Default.ArrowBack, contentDescription = "Back", tint =
> Color.White)
0080 |             }
0081 |         },
0082 |         colors = TopAppBarDefaults.topAppBarColors(containerColor = Color(0xFF1E1E1E))
0083 |     )
0084 | },
0085 |     containerColor = Color(0xFF121212)
0086 | ) { paddingValues ->
0087 |     Column(
0088 |         modifier = Modifier
0089 |             .fillMaxSize()
0090 |             .padding(paddingValues)
0091 |             .padding(16.dp)
0092 |             .verticalScroll(rememberScrollState()),
0093 |         verticalArrangement = Arrangement.spacedBy(16.dp)
0094 |     ) {
0095 |         Text("Subject Code: ${subject.subjectCode} (Cannot be changed)", color = Color.Gray,
> style = MaterialTheme.typography.bodyMedium)
0096 |     }

```

L65: Connects a user interaction to the displayed state update or handler.

L66: Closes or continues the surrounding block, expression, or call.

L67: Assigns the displayed expression to a variable or named argument.

L68: Connects a user interaction to the displayed state update or handler.

L69-71: Closes or continues the surrounding block, expression, or call.

L73: Adds this UI element to the composition with the displayed content and parameters.

L74: Assigns the displayed expression to a variable or named argument.

L75: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L76-77: Assigns the displayed expression to a variable or named argument.

L78: Connects a user interaction to the displayed state update or handler.

L79: Adds this UI element to the composition with the displayed content and parameters.

L80-81: Closes or continues the surrounding block, expression, or call.

L82: Assigns the displayed expression to a variable or named argument.

L83-84: Closes or continues the surrounding block, expression, or call.

L85: Assigns the displayed expression to a variable or named argument.

L86: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L87: Adds this UI element to the composition with the displayed content and parameters.

L88-91: Configures UI layout, sizing, spacing, or appearance; it does not modify academic records.

L92: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L93: Assigns the displayed expression to a variable or named argument.

L94: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L95: Adds this UI element to the composition with the displayed content and parameters.

Source lines 97-128

```

0097 |         CustomTextField(value = subjectName, onValueChange = { subjectName = it }, label =
> "Subject Name", enabled = !isLoading)
0098 |         CustomTextField(value = facultyId, onValueChange = { facultyId = it }, label = "Assigned
> Faculty ID", enabled = !isLoading)
0099 |
0100 |         Row(horizontalArrangement = Arrangement.spacedBy(8.dp)) {
0101 |             CustomTextField(value = semester, onValueChange = { semester = it }, label =
> "Semester", modifier = Modifier.weight(1f), keyboardType = TextInputType.Number, enabled =
> !isLoading)
0102 |             CustomTextField(value = totalClasses, onValueChange = { totalClasses = it }, label =
> "Total Classes", modifier = Modifier.weight(1f), keyboardType = TextInputType.Number, enabled =
> !isLoading)
0103 |         }
0104 |

```

```

0105 |         Spacer(modifier = Modifier.height(16.dp))
0106 |
0107 |         Button(
0108 |             onClick = {
0109 |                 if (subjectName.isBlank() || facultyId.isBlank() || semester.isBlank() ||
> totalClasses.isBlank()) {
0110 |                     Toast.makeText(context, "All required fields must be filled",
> Toast.LENGTH_SHORT).show()
0111 |                     return@Button
0112 |                 }
0113 |
0114 |                 isLoading = true
0115 |                 coroutineScope.launch {
0116 |                     try {
0117 |                         val request = UpdateSubjectRequest(
0118 |                             revision = subject.revision,
0119 |                             subjectCode = subject.subjectCode,
0120 |                             subjectName = subjectName.trim(),
0121 |                             facultyId = facultyId.trim(),
0122 |                             semester = semester.trim(),
0123 |                             totalClasses = totalClasses.trim()
0124 |                         )
0125 |                         val response = NetworkModule.api.updateSubject(request)
0126 |                         if (response.status == "success") {
0127 |                             Toast.makeText(context, "Subject updated successfully",
> Toast.LENGTH_SHORT).show()
0128 |                             onNavigateBack()

```

L97-98: Connects a user interaction to the displayed state update or handler.

L100: Adds this UI element to the composition with the displayed content and parameters.

L101-102: Connects a user interaction to the displayed state update or handler.

L103: Closes or continues the surrounding block, expression, or call.

L105: Adds this UI element to the composition with the displayed content and parameters.

L107: Adds this UI element to the composition with the displayed content and parameters.

L108: Connects a user interaction to the displayed state update or handler.

L109: Branches on the displayed condition; the accepted branch performs the following operation.

L110-111: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L112: Closes or continues the surrounding block, expression, or call.

L114: Assigns the displayed expression to a variable or named argument.

L115: Starts asynchronous work in the UI scope so the button handler can return promptly.

L116: Starts an operation whose failures are handled by the following catch/finally blocks.

L117: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L118: Carries or checks the record version used to detect stale edits or matching retries.

L119-123: Assigns the displayed expression to a variable or named argument.

L124: Closes or continues the surrounding block, expression, or call.

L125: Calls the named authenticated backend operation with the displayed request or key.

L126: Branches on the displayed condition; the accepted branch performs the following operation.

L127-128: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

Source lines 129-160

```

0129 |             } else {
0130 |                 Toast.makeText(context, "Error: ${response.message}",
> Toast.LENGTH_LONG).show()
0131 |             }
0132 |             } catch (e: kotlinx.coroutines.CancellationException) { throw e
0133 |             } catch (e: Exception) {
0134 |                 Toast.makeText(context, "Failed to update: ${e.message}",
> Toast.LENGTH_LONG).show()
0135 |             } finally {
0136 |                 isLoading = false
0137 |             }
0138 |         }
0139 |     },

```

```

0140 |         modifier = Modifier.fillMaxWidth().height(50.dp),
0141 |         shape = RoundedCornerShape(12.dp),
0142 |         colors = ButtonDefaults.buttonColors(containerColor = Color(0xFF3700B3)),
0143 |         enabled = !isLoading
0144 |     ) {
0145 |         if (isLoading) {
0146 |             CircularProgressIndicator(color = Color.White, modifier = Modifier.size(24.dp))
0147 |         } else {
0148 |             Text("Update Subject", color = Color.White, fontWeight = FontWeight.Bold, style
> = MaterialTheme.typography.titleMedium)
0149 |         }
0150 |     }
0151 |     Spacer(modifier = Modifier.height(32.dp))
0152 | }
0153 | }
0154 | }
0155 |
0156 | @Composable
0157 | fun CustomTextField(
0158 |     value: String,
0159 |     onChange: (String) -> Unit,
0160 |     label: String,

```

L129-130: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L131: Closes or continues the surrounding block, expression, or call.

L132: Preserves coroutine cancellation rather than presenting it as an ordinary application error.

L133: Handles this exception type and translates it into the following failure behavior.

L134: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L135: Runs cleanup whether the preceding operation succeeded or failed.

L136: Assigns the displayed expression to a variable or named argument.

L137-139: Closes or continues the surrounding block, expression, or call.

L140: Configures UI layout, sizing, spacing, or appearance; it does not modify academic records.

L141-143: Assigns the displayed expression to a variable or named argument.

L144: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L145: Branches on the displayed condition; the accepted branch performs the following operation.

L146: Adds this UI element to the composition with the displayed content and parameters.

L147: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L148: Adds this UI element to the composition with the displayed content and parameters.

L149-150: Closes or continues the surrounding block, expression, or call.

L151: Adds this UI element to the composition with the displayed content and parameters.

L152-154: Closes or continues the surrounding block, expression, or call.

L156: Marks the following function as a Compose UI function.

L157: Defines this Kotlin operation and its parameters; suspend allows asynchronous calls.

L158: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L159: Connects a user interaction to the displayed state update or handler.

L160: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

Source lines 161-173

```

0161 |     modifier: Modifier = Modifier,
0162 |     keyboardType: KeyboardType = KeyboardType.Text,
0163 |     enabled: Boolean = true
0164 | ) {
0165 |     OutlinedTextField(
0166 |         enabled = enabled,
0167 |         value = value,
0168 |         onChange = onChange,
0169 |         label = { Text(label) },
0170 |         keyboardOptions = KeyboardOptions(keyboardType = keyboardType),
0171 |         modifier = modifier.fillMaxWidth()
0172 |     )
0173 | }

```

L161: Configures UI layout, sizing, spacing, or appearance; it does not modify academic records.

L162-164: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L165: Adds this UI element to the composition with the displayed content and parameters.

L166-167: Assigns the displayed expression to a variable or named argument.

L168: Connects a user interaction to the displayed state update or handler.

L169-171: Assigns the displayed expression to a variable or named argument.

L172-173: Closes or continues the surrounding block, expression, or call.

40. SubjectDetailsScreen.kt

app/src/main/java/com/example/studenttracker/presentation/subject/SubjectDetailsScreen.kt

Read-only subject profile details and actions.

161 source lines. SHA-256: 4f22894c28929603

The typed record supplies the display values; cards group related fields. Small DetailRow helpers display icon, label, and value.

Edit invokes the provided route callback. Deactivate opens confirmation and sends the record key with its expected revision.

Busy state prevents repeated actions. Success returns to the previous screen; failure shows feedback. Deactivation preserves historical academic data.

Source lines 1-32

```
0001 | package com.example.studenttracker.presentation.subject
0002 |
0003 | import android.widget.Toast
0004 | import androidx.compose.foundation.background
0005 | import androidx.compose.foundation.layout.*
0006 | import androidx.compose.foundation.rememberScrollState
0007 | import androidx.compose.foundation.shape.CircleShape
0008 | import androidx.compose.foundation.shape.RoundedCornerShape
0009 | import androidx.compose.foundation.verticalScroll
0010 | import androidx.compose.material.icons.Icons
0011 | import androidx.compose.material.icons.filled.*
0012 | import androidx.compose.material3.*
0013 | import androidx.compose.runtime.*
0014 | import androidx.compose.ui.Alignment
0015 | import androidx.compose.ui.Modifier
0016 | import androidx.compose.ui.draw.clip
0017 | import androidx.compose.ui.graphics.Color
0018 | import androidx.compose.ui.platform.LocalContext
0019 | import androidx.compose.ui.text.font.FontWeight
0020 | import androidx.compose.ui.unit.dp
0021 | import androidx.compose.ui.unit.sp
0022 | import com.example.studenttracker.data.network.DeleteSubjectRequest
0023 | import com.example.studenttracker.data.network.Subject
0024 | import com.example.studenttracker.data.network.NetworkModule
0025 | import kotlin.coroutines.launch
0026 |
0027 | @OptIn(ExperimentalMaterial3Api::class)
0028 | @Composable
0029 | fun SubjectDetailsScreen(
0030 |     subject: Subject,
0031 |     onNavigateBack: () -> Unit,
0032 |     onNavigateToEdit: () -> Unit
```

L1: Declares the Kotlin namespace; it organizes classes and functions.

L3-25: Imports the named library symbols used by this file.

L27: Applies the displayed annotation to the declaration.

L28: Marks the following function as a Compose UI function.

L29: Defines this Kotlin operation and its parameters; suspend allows asynchronous calls.

L30-32: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

Source lines 33-64

```
0033 | ) {
0034 |     val context = LocalContext.current
0035 |     val coroutineScope = rememberCoroutineScope()
0036 |     var showDeleteDialog by remember { mutableStateOf(false) }
0037 |     var isDeleting by remember { mutableStateOf(false) }
0038 |
0039 |     if (showDeleteDialog) {
0040 |         AlertDialog(
```

```

0041 |         onDismissRequest = { showDeleteDialog = false },
0042 |         title = { Text("Deactivate Subject", color = Color.White) },
0043 |         text = { Text("Are you sure you want to remove ${subject.subjectName}? The profile will
> be deactivated; historical records will be retained.", color = Color.LightGray) },
0044 |         containerColor = Color(0xFF1E1E1E),
0045 |         confirmButton = {
0046 |             TextButton(
0047 |                 onClick = {
0048 |                     isDeleting = true
0049 |                     coroutineScope.launch {
0050 |                         try {
0051 |                             val response =
> NetworkModule.api.deleteSubject(DeleteSubjectRequest(revision = subject.revision, subjectCode =
> subject.subjectCode))
0052 |                             if (response.status == "success") {
0053 |                                 Toast.makeText(context, "Subject deactivated",
> Toast.LENGTH_SHORT).show()
0054 |                                 showDeleteDialog = false
0055 |                                 onBackPressed()
0056 |                             } else {
0057 |                                 Toast.makeText(context, "Error: ${response.message}",
> Toast.LENGTH_LONG).show()
0058 |                             }
0059 |                         } catch (e: kotlinx.coroutines.CancellationException) { throw e
0060 |                         } catch (e: Exception) {
0061 |                             Toast.makeText(context, "Failed to delete: ${e.message}",
> Toast.LENGTH_LONG).show()
0062 |                         } finally {
0063 |                             isDeleting = false
0064 |                             showDeleteDialog = false

```

L33: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L34: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L35: Obtains a coroutine scope tied to this composable for button-triggered asynchronous work.

L36-37: Creates observable local UI state; assignment triggers recomposition.

L39: Branches on the displayed condition; the accepted branch performs the following operation.

L40: Adds this UI element to the composition with the displayed content and parameters.

L41-45: Assigns the displayed expression to a variable or named argument.

L46: Adds this UI element to the composition with the displayed content and parameters.

L47: Connects a user interaction to the displayed state update or handler.

L48: Assigns the displayed expression to a variable or named argument.

L49: Starts asynchronous work in the UI scope so the button handler can return promptly.

L50: Starts an operation whose failures are handled by the following catch/finally blocks.

L51: Calls the named authenticated backend operation with the displayed request or key.

L52: Branches on the displayed condition; the accepted branch performs the following operation.

L53: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L54: Assigns the displayed expression to a variable or named argument.

L55-57: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L58: Closes or continues the surrounding block, expression, or call.

L59: Preserves coroutine cancellation rather than presenting it as an ordinary application error.

L60: Handles this exception type and translates it into the following failure behavior.

L61: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L62: Runs cleanup whether the preceding operation succeeded or failed.

L63-64: Assigns the displayed expression to a variable or named argument.

Source lines 65-96

```

0065 |         }
0066 |     }
0067 | }
0068 | ) {
0069 |     if (isDeleting) {
0070 |         CircularProgressIndicator(modifier = Modifier.size(20.dp), color =

```

```

    > Color.Red)
0071 |         } else {
0072 |             Text("Delete", color = Color.Red, fontWeight = FontWeight.Bold)
0073 |         }
0074 |     }
0075 | },
0076 |     dismissButton = {
0077 |         TextButton(onClick = { showDeleteDialog = false }) {
0078 |             Text("Cancel", color = Color.Gray)
0079 |         }
0080 |     }
0081 | )
0082 | }
0083 |
0084 | Scaffold(
0085 |     topBar = {
0086 |         TopAppBar(
0087 |             title = { Text("Subject Details", color = Color.White) },
0088 |             navigationIcon = {
0089 |                 IconButton(onClick = onNavigateBack) {
0090 |                     Icon(Icons.Default.ArrowBack, contentDescription = "Back", tint =
    > Color.White)
0091 |                 }
0092 |             },
0093 |             actions = {
0094 |                 IconButton(onClick = onNavigateToEdit, enabled = !isDeleting) {
0095 |                     Icon(Icons.Default.Edit, contentDescription = "Edit", tint = Color.White)
0096 |                 }

```

L65-67: Closes or continues the surrounding block, expression, or call.

L68: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L69: Branches on the displayed condition; the accepted branch performs the following operation.

L70: Adds this UI element to the composition with the displayed content and parameters.

L71: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L72: Adds this UI element to the composition with the displayed content and parameters.

L73-75: Closes or continues the surrounding block, expression, or call.

L76: Assigns the displayed expression to a variable or named argument.

L77: Connects a user interaction to the displayed state update or handler.

L78: Adds this UI element to the composition with the displayed content and parameters.

L79-82: Closes or continues the surrounding block, expression, or call.

L84: Adds this UI element to the composition with the displayed content and parameters.

L85: Assigns the displayed expression to a variable or named argument.

L86: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L87-88: Assigns the displayed expression to a variable or named argument.

L89: Connects a user interaction to the displayed state update or handler.

L90: Adds this UI element to the composition with the displayed content and parameters.

L91-92: Closes or continues the surrounding block, expression, or call.

L93: Assigns the displayed expression to a variable or named argument.

L94: Connects a user interaction to the displayed state update or handler.

L95: Adds this UI element to the composition with the displayed content and parameters.

L96: Closes or continues the surrounding block, expression, or call.

Source lines 97-128

```

0097 |         IconButton(onClick = { showDeleteDialog = true }, enabled = !isDeleting) {
0098 |             Icon(Icons.Default.Delete, contentDescription = "Delete", tint = Color.Red)
0099 |         }
0100 |     },
0101 |     colors = TopAppBarDefaults.topAppBarColors(containerColor = Color(0xFF1E1E1E))
0102 | )
0103 | },
0104 |     containerColor = Color(0xFF121212)
0105 | ) { paddingValues ->

```



```

0106 |         Column(
0107 |             modifier = Modifier
0108 |                 .fillMaxSize()
0109 |                 .padding(paddingValues)
0110 |                 .verticalScroll(rememberScrollState()),
0111 |             horizontalAlignment = Alignment.CenterHorizontally
0112 |         ) {
0113 |             Spacer(modifier = Modifier.height(32.dp))
0114 |             Box(
0115 |                 modifier = Modifier
0116 |                     .size(100.dp)
0117 |                     .clip(CircleShape)
0118 |                     .background(Color(0xFF3700B3).copy(alpha = 0.2f)),
0119 |                 contentAlignment = Alignment.Center
0120 |             ) {
0121 |                 Icon(Icons.Default.Book, contentDescription = null, tint = Color(0xFF3700B3),
> modifier = Modifier.size(50.dp))
0122 |             }
0123 |             Spacer(modifier = Modifier.height(16.dp))
0124 |             Text(subject.subjectName, style = MaterialTheme.typography.headlineMedium, color =
> Color.White, fontWeight = FontWeight.Bold)
0125 |             Text(subject.subjectCode, style = MaterialTheme.typography.bodyLarge, color =
> Color.Gray)
0126 |
0127 |             Spacer(modifier = Modifier.height(32.dp))
0128 |

```

L97: Connects a user interaction to the displayed state update or handler.

L98: Adds this UI element to the composition with the displayed content and parameters.

L99-100: Closes or continues the surrounding block, expression, or call.

L101: Assigns the displayed expression to a variable or named argument.

L102-103: Closes or continues the surrounding block, expression, or call.

L104: Assigns the displayed expression to a variable or named argument.

L105: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L106: Adds this UI element to the composition with the displayed content and parameters.

L107-109: Configures UI layout, sizing, spacing, or appearance; it does not modify academic records.

L110: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L111: Assigns the displayed expression to a variable or named argument.

L112: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L113-114: Adds this UI element to the composition with the displayed content and parameters.

L115-118: Configures UI layout, sizing, spacing, or appearance; it does not modify academic records.

L119: Assigns the displayed expression to a variable or named argument.

L120: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L121: Adds this UI element to the composition with the displayed content and parameters.

L122: Closes or continues the surrounding block, expression, or call.

L123-125: Adds this UI element to the composition with the displayed content and parameters.

L127: Adds this UI element to the composition with the displayed content and parameters.

Source lines 129-160

```

0129 |         Card(
0130 |             modifier = Modifier
0131 |                 .fillMaxWidth()
0132 |                 .padding(horizontal = 16.dp),
0133 |             colors = CardDefaults.cardColors(containerColor = Color(0xFF1E1E1E)),
0134 |             shape = RoundedCornerShape(16.dp)
0135 |         ) {
0136 |             Column(modifier = Modifier.padding(16.dp), verticalArrangement =
> Arrangement.spacedBy(16.dp)) {
0137 |                 DetailRow(Icons.Default.Person, "Assigned Faculty ID", subject.facultyId)
0138 |                 DetailRow(Icons.Default.Timeline, "Semester", subject.semester)
0139 |                 DetailRow(Icons.Default.Class, "Total Classes", subject.totalClasses)
0140 |             }
0141 |         }

```

```

0142 |         }
0143 |     }
0144 | }
0145 |
0146 | @Composable
0147 | fun DetailRow(icon: androidx.compose.ui.graphics.vector.ImageVector, label: String, value: String) {
0148 |     Row(verticalAlignment = Alignment.CenterVertically, modifier = Modifier.fillMaxWidth()) {
0149 |         Box(
0150 |             modifier = Modifier.size(40.dp).clip(CircleShape).background(Color.DarkGray.copy(alpha =
> 0.3f)),
0151 |             contentAlignment = Alignment.Center
0152 |         ) {
0153 |             Icon(icon, contentDescription = null, tint = Color(0xFF3700B3), modifier =
> Modifier.size(20.dp))
0154 |         }
0155 |         Spacer(modifier = Modifier.width(16.dp))
0156 |         Column {
0157 |             Text(label, color = Color.Gray, fontSize = 12.sp)
0158 |             Text(value, color = Color.White, fontSize = 16.sp, fontWeight = FontWeight.SemiBold)
0159 |         }
0160 |     }

```

L129: Adds this UI element to the composition with the displayed content and parameters.

L130-132: Configures UI layout, sizing, spacing, or appearance; it does not modify academic records.

L133-134: Assigns the displayed expression to a variable or named argument.

L135: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L136: Adds this UI element to the composition with the displayed content and parameters.

L137-139: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L140-144: Closes or continues the surrounding block, expression, or call.

L146: Marks the following function as a Compose UI function.

L147: Defines this Kotlin operation and its parameters; suspend allows asynchronous calls.

L148-149: Adds this UI element to the composition with the displayed content and parameters.

L150: Configures UI layout, sizing, spacing, or appearance; it does not modify academic records.

L151: Assigns the displayed expression to a variable or named argument.

L152: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L153: Adds this UI element to the composition with the displayed content and parameters.

L154: Closes or continues the surrounding block, expression, or call.

L155: Adds this UI element to the composition with the displayed content and parameters.

L156: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L157-158: Adds this UI element to the composition with the displayed content and parameters.

L159-160: Closes or continues the surrounding block, expression, or call.

Source lines 161-161

```

0161 | }

```

L161: Closes or continues the surrounding block, expression, or call.

41. SubjectListScreen.kt

app/src/main/java/com/example/studenttracker/presentation/subject/SubjectListScreen.kt

Subject directory adapter.

13 source lines. SHA-256: 6773b0a8336a6006

Loads records with `getSubjects`, maps the stable identifier and display fields into `DirectoryRecord`, and delegates searching and list rendering to `DirectoryScreen`.

Caller callbacks determine which record or add form opens.

Source lines 1-13

```
0001 | package com.example.studenttracker.presentation.subject
0002 |
0003 | import androidx.compose.runtime.Composable
0004 | import com.example.studenttracker.data.network.NetworkModule
0005 | import com.example.studenttracker.presentation.components.DirectoryRecord
0006 | import com.example.studenttracker.presentation.components.DirectoryScreen
0007 |
0008 | @Composable
0009 | fun SubjectListScreen(onNavigateBack: () -> Unit, onSubjectClick: (String) -> Unit,
    > onNavigateToAddSubject: () -> Unit) {
0010 |     DirectoryScreen("Subjects", onNavigateBack, onNavigateToAddSubject, onSubjectClick) {
0011 |         NetworkModule.api.getSubjects().subjects.orEmpty().map { DirectoryRecord(it.subjectCode,
    > it.subjectName, "Semester ${it.semester} · Faculty ${it.facultyId}") }
0012 |     }
0013 | }
```

L1: Declares the Kotlin namespace; it organizes classes and functions.

L3-6: Imports the named library symbols used by this file.

L8: Marks the following function as a Compose UI function.

L9: Defines this Kotlin operation and its parameters; suspend allows asynchronous calls.

L10: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L11: Calls the named authenticated backend operation with the displayed request or key.

L12-13: Closes or continues the surrounding block, expression, or call.

42. Color.kt

app/src/main/java/com/example/studenttracker/ui/theme/Color.kt

Named Compose color constants.

11 source lines. SHA-256: 2fe08e07dd2c2bd1

These hexadecimal ARGB colors feed the fixed light and dark color schemes. They do not themselves draw any UI.

Source lines 1-11

```
0001 | package com.example.studenttracker.ui.theme
0002 |
0003 | import androidx.compose.ui.graphics.Color
0004 |
0005 | val Purple80 = Color(0xFFD0BCFF)
0006 | val PurpleGrey80 = Color(0xFFCCC2DC)
0007 | val Pink80 = Color(0xFFE8B88C)
0008 |
0009 | val Purple40 = Color(0xFF6650a4)
0010 | val PurpleGrey40 = Color(0xFF625b71)
0011 | val Pink40 = Color(0xFF7D5260)
```

L1: Declares the Kotlin namespace; it organizes classes and functions.

L3: Imports the named library symbols used by this file.

L5-7: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L9-11: Declares the displayed value or mutable state; its initializer supplies the initial contents.

43. Theme.kt

app/src/main/java/com/example/studenttracker/ui/theme/Theme.kt

Material 3 color and typography configuration.

49 source lines. SHA-256: 4dfd3fe8fc24e79b

DarkColorScheme and LightColorScheme provide fixed palettes. On Android S or later, the optional dynamic-color branch can use wallpaper-derived colors.

StudentTrackerTheme wraps caller content in MaterialTheme. MainActivity explicitly disables dynamic colors and selects dark mode.

Source lines 1-32

```
0001 | package com.example.studenttracker.ui.theme
0002 |
0003 | import android.app.Activity
0004 | import android.os.Build
0005 | import androidx.compose.foundation.isSystemInDarkTheme
0006 | import androidx.compose.material3.MaterialTheme
0007 | import androidx.compose.material3.darkColorScheme
0008 | import androidx.compose.material3.dynamicDarkColorScheme
0009 | import androidx.compose.material3.dynamicLightColorScheme
0010 | import androidx.compose.material3.lightColorScheme
0011 | import androidx.compose.runtime.Composable
0012 | import androidx.compose.ui.platform.LocalContext
0013 |
0014 | private val DarkColorScheme = darkColorScheme(
0015 |     primary = Purple80,
0016 |     secondary = PurpleGrey80,
0017 |     tertiary = Pink80
0018 | )
0019 |
0020 | private val LightColorScheme = lightColorScheme(
0021 |     primary = Purple40,
0022 |     secondary = PurpleGrey40,
0023 |     tertiary = Pink40
0024 | )
0025 | )
0026 |
0027 | @Composable
0028 | fun StudentTrackerTheme(
0029 |     darkTheme: Boolean = isSystemInDarkTheme(),
0030 |
0031 |     dynamicColor: Boolean = true,
0032 |     content: @Composable () -> Unit
```

L1: Declares the Kotlin namespace; it organizes classes and functions.

L3-12: Imports the named library symbols used by this file.

L14: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L15-17: Assigns the displayed expression to a variable or named argument.

L18: Closes or continues the surrounding block, expression, or call.

L20: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L21-23: Assigns the displayed expression to a variable or named argument.

L25: Closes or continues the surrounding block, expression, or call.

L27: Marks the following function as a Compose UI function.

L28: Defines this Kotlin operation and its parameters; suspend allows asynchronous calls.

L29: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L31-32: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

Source lines 33-49

```
0033 | ) {
0034 |     val colorScheme = when {
```

```

0035 |         dynamicColor && Build.VERSION.SDK_INT >= Build.VERSION_CODES.S -> {
0036 |             val context = LocalContext.current
0037 |             if (darkTheme) dynamicDarkColorScheme(context) else dynamicLightColorScheme(context)
0038 |         }
0039 |
0040 |         darkTheme -> DarkColorScheme
0041 |         else -> LightColorScheme
0042 |     }
0043 |
0044 |     MaterialTheme(
0045 |         colorScheme = colorScheme,
0046 |         typography = Typography,
0047 |         content = content
0048 |     )
0049 | }

```

L33: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L34: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L35: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L36: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L37: Branches on the displayed condition; the accepted branch performs the following operation.

L38: Closes or continues the surrounding block, expression, or call.

L40-41: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L42: Closes or continues the surrounding block, expression, or call.

L44: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L45-47: Assigns the displayed expression to a variable or named argument.

L48-49: Closes or continues the surrounding block, expression, or call.

44. Type.kt

app/src/main/java/com/example/studenttracker/ui/theme/Type.kt

Material text style definition.

18 source lines. SHA-256: 7266e2ac41e35664

Typography sets bodyLarge to the default font family with normal weight, 16sp size, 24sp line height, and 0.5sp letter spacing. sp respects font scaling.

Source lines 1-18

```
0001 | package com.example.studenttracker.ui.theme
0002 |
0003 | import androidx.compose.material3.Typography
0004 | import androidx.compose.ui.text.TextStyle
0005 | import androidx.compose.ui.text.font.FontFamily
0006 | import androidx.compose.ui.text.font.FontWeight
0007 | import androidx.compose.ui.unit.sp
0008 |
0009 | val Typography = Typography(
0010 |     bodyLarge = TextStyle(
0011 |         fontFamily = FontFamily.Default,
0012 |         fontWeight = FontWeight.Normal,
0013 |         fontSize = 16.sp,
0014 |         lineHeight = 24.sp,
0015 |         letterSpacing = 0.5.sp
0016 |     )
0017 |
0018 | )
```

L1: Declares the Kotlin namespace; it organizes classes and functions.

L3-7: Imports the named library symbols used by this file.

L9: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L10-15: Assigns the displayed expression to a variable or named argument.

L16: Closes or continues the surrounding block, expression, or call.

L18: Closes or continues the surrounding block, expression, or call.

45. rules.keep

app/src/main/keepRules/rules.keep

Optional Android keep-rule file.

0 source lines. SHA-256: e3b0c44298fc1c14

Currently empty. No custom keep directive remains after removing explanatory comments. Release minification is disabled in the current app build.

This file is empty. There is no executable code to explain.

46. ic_launcher_background.xml

app/src/main/res/drawable/ic_launcher_background.xml

Vector drawable launcher artwork.

170 source lines. SHA-256: ed423c73a6f40a4d

width/height specify the nominal size; viewport coordinates define the drawing space. pathData is the sequence of move, line, curve, and close operations.

fillColor paints enclosed shapes; strokeColor and strokeWidth draw outlines. These numerical geometry values describe the icon rather than an application workflow.

Source lines 1-32

```
0001 | <?xml version="1.0" encoding="utf-8"?>
0002 | <vector xmlns:android="http://schemas.android.com/apk/res/android"
0003 |     android:width="108dp"
0004 |     android:height="108dp"
0005 |     android:viewportWidth="108"
0006 |     android:viewportHeight="108">
0007 |     <path
0008 |         android:fillColor="#3DDC84"
0009 |         android:pathData="M0,0h108v108h-108z" />
0010 |     <path
0011 |         android:fillColor="#00000000"
0012 |         android:pathData="M9,0L9,108"
0013 |         android:strokeWidth="0.8"
0014 |         android:strokeColor="#33FFFFFF" />
0015 |     <path
0016 |         android:fillColor="#00000000"
0017 |         android:pathData="M19,0L19,108"
0018 |         android:strokeWidth="0.8"
0019 |         android:strokeColor="#33FFFFFF" />
0020 |     <path
0021 |         android:fillColor="#00000000"
0022 |         android:pathData="M29,0L29,108"
0023 |         android:strokeWidth="0.8"
0024 |         android:strokeColor="#33FFFFFF" />
0025 |     <path
0026 |         android:fillColor="#00000000"
0027 |         android:pathData="M39,0L39,108"
0028 |         android:strokeWidth="0.8"
0029 |         android:strokeColor="#33FFFFFF" />
0030 |     <path
0031 |         android:fillColor="#00000000"
0032 |         android:pathData="M49,0L49,108"
```

L1-2: Declares the XML resource or attribute shown here; the file overview explains its Android role.

L3-6: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L7: Declares the XML resource or attribute shown here; the file overview explains its Android role.

L8-9: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L10: Declares the XML resource or attribute shown here; the file overview explains its Android role.

L11-14: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L15: Declares the XML resource or attribute shown here; the file overview explains its Android role.

L16-19: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L20: Declares the XML resource or attribute shown here; the file overview explains its Android role.

L21-24: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L25: Declares the XML resource or attribute shown here; the file overview explains its Android role.

L26-29: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L30: Declares the XML resource or attribute shown here; the file overview explains its Android role.

L31-32: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

Source lines 33-64

```

0033 |         android:strokeWidth="0.8"
0034 |         android:strokeColor="#33FFFFFF" />
0035 |     <path
0036 |         android:fillColor="#00000000"
0037 |         android:pathData="M59,0L59,108"
0038 |         android:strokeWidth="0.8"
0039 |         android:strokeColor="#33FFFFFF" />
0040 |     <path
0041 |         android:fillColor="#00000000"
0042 |         android:pathData="M69,0L69,108"
0043 |         android:strokeWidth="0.8"
0044 |         android:strokeColor="#33FFFFFF" />
0045 |     <path
0046 |         android:fillColor="#00000000"
0047 |         android:pathData="M79,0L79,108"
0048 |         android:strokeWidth="0.8"
0049 |         android:strokeColor="#33FFFFFF" />
0050 |     <path
0051 |         android:fillColor="#00000000"
0052 |         android:pathData="M89,0L89,108"
0053 |         android:strokeWidth="0.8"
0054 |         android:strokeColor="#33FFFFFF" />
0055 |     <path
0056 |         android:fillColor="#00000000"
0057 |         android:pathData="M99,0L99,108"
0058 |         android:strokeWidth="0.8"
0059 |         android:strokeColor="#33FFFFFF" />
0060 |     <path
0061 |         android:fillColor="#00000000"
0062 |         android:pathData="M0,9L108,9"
0063 |         android:strokeWidth="0.8"
0064 |         android:strokeColor="#33FFFFFF" />

```

L33-34: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L35: Declares the XML resource or attribute shown here; the file overview explains its Android role.

L36-39: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L40: Declares the XML resource or attribute shown here; the file overview explains its Android role.

L41-44: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L45: Declares the XML resource or attribute shown here; the file overview explains its Android role.

L46-49: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L50: Declares the XML resource or attribute shown here; the file overview explains its Android role.

L51-54: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L55: Declares the XML resource or attribute shown here; the file overview explains its Android role.

L56-59: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L60: Declares the XML resource or attribute shown here; the file overview explains its Android role.

L61-64: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

Source lines 65-96

```

0065 |     <path
0066 |         android:fillColor="#00000000"
0067 |         android:pathData="M0,19L108,19"
0068 |         android:strokeWidth="0.8"
0069 |         android:strokeColor="#33FFFFFF" />
0070 |     <path
0071 |         android:fillColor="#00000000"
0072 |         android:pathData="M0,29L108,29"
0073 |         android:strokeWidth="0.8"
0074 |         android:strokeColor="#33FFFFFF" />
0075 |     <path
0076 |         android:fillColor="#00000000"
0077 |         android:pathData="M0,39L108,39"
0078 |         android:strokeWidth="0.8"
0079 |         android:strokeColor="#33FFFFFF" />
0080 |     <path
0081 |         android:fillColor="#00000000"
0082 |         android:pathData="M0,49L108,49"
0083 |         android:strokeWidth="0.8"

```

```

0084 |         android:strokeColor="#33FFFFFF" />
0085 |     <path
0086 |         android:fillColor="#00000000"
0087 |         android:pathData="M0,59L108,59"
0088 |         android:strokeWidth="0.8"
0089 |         android:strokeColor="#33FFFFFF" />
0090 |     <path
0091 |         android:fillColor="#00000000"
0092 |         android:pathData="M0,69L108,69"
0093 |         android:strokeWidth="0.8"
0094 |         android:strokeColor="#33FFFFFF" />
0095 |     <path
0096 |         android:fillColor="#00000000"

```

L65: Declares the XML resource or attribute shown here; the file overview explains its Android role.

L66-69: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L70: Declares the XML resource or attribute shown here; the file overview explains its Android role.

L71-74: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L75: Declares the XML resource or attribute shown here; the file overview explains its Android role.

L76-79: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L80: Declares the XML resource or attribute shown here; the file overview explains its Android role.

L81-84: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L85: Declares the XML resource or attribute shown here; the file overview explains its Android role.

L86-89: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L90: Declares the XML resource or attribute shown here; the file overview explains its Android role.

L91-94: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L95: Declares the XML resource or attribute shown here; the file overview explains its Android role.

L96: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

Source lines 97-128

```

0097 |         android:pathData="M0,79L108,79"
0098 |         android:strokeWidth="0.8"
0099 |         android:strokeColor="#33FFFFFF" />
0100 |     <path
0101 |         android:fillColor="#00000000"
0102 |         android:pathData="M0,89L108,89"
0103 |         android:strokeWidth="0.8"
0104 |         android:strokeColor="#33FFFFFF" />
0105 |     <path
0106 |         android:fillColor="#00000000"
0107 |         android:pathData="M0,99L108,99"
0108 |         android:strokeWidth="0.8"
0109 |         android:strokeColor="#33FFFFFF" />
0110 |     <path
0111 |         android:fillColor="#00000000"
0112 |         android:pathData="M19,29L89,29"
0113 |         android:strokeWidth="0.8"
0114 |         android:strokeColor="#33FFFFFF" />
0115 |     <path
0116 |         android:fillColor="#00000000"
0117 |         android:pathData="M19,39L89,39"
0118 |         android:strokeWidth="0.8"
0119 |         android:strokeColor="#33FFFFFF" />
0120 |     <path
0121 |         android:fillColor="#00000000"
0122 |         android:pathData="M19,49L89,49"
0123 |         android:strokeWidth="0.8"
0124 |         android:strokeColor="#33FFFFFF" />
0125 |     <path
0126 |         android:fillColor="#00000000"
0127 |         android:pathData="M19,59L89,59"
0128 |         android:strokeWidth="0.8"

```

L97-99: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L100: Declares the XML resource or attribute shown here; the file overview explains its Android role.

L101-104: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L105: Declares the XML resource or attribute shown here; the file overview explains its Android role.

L106-109: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L110: Declares the XML resource or attribute shown here; the file overview explains its Android role.

L111-114: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L115: Declares the XML resource or attribute shown here; the file overview explains its Android role.

L116-119: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L120: Declares the XML resource or attribute shown here; the file overview explains its Android role.

L121-124: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L125: Declares the XML resource or attribute shown here; the file overview explains its Android role.

L126-128: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

Source lines 129-160

```

0129 |         android:strokeColor="#33FFFFFF" />
0130 |     <path
0131 |         android:fillColor="#00000000"
0132 |         android:pathData="M19,69L89,69"
0133 |         android:strokeWidth="0.8"
0134 |         android:strokeColor="#33FFFFFF" />
0135 |     <path
0136 |         android:fillColor="#00000000"
0137 |         android:pathData="M19,79L89,79"
0138 |         android:strokeWidth="0.8"
0139 |         android:strokeColor="#33FFFFFF" />
0140 |     <path
0141 |         android:fillColor="#00000000"
0142 |         android:pathData="M29,19L29,89"
0143 |         android:strokeWidth="0.8"
0144 |         android:strokeColor="#33FFFFFF" />
0145 |     <path
0146 |         android:fillColor="#00000000"
0147 |         android:pathData="M39,19L39,89"
0148 |         android:strokeWidth="0.8"
0149 |         android:strokeColor="#33FFFFFF" />
0150 |     <path
0151 |         android:fillColor="#00000000"
0152 |         android:pathData="M49,19L49,89"
0153 |         android:strokeWidth="0.8"
0154 |         android:strokeColor="#33FFFFFF" />
0155 |     <path
0156 |         android:fillColor="#00000000"
0157 |         android:pathData="M59,19L59,89"
0158 |         android:strokeWidth="0.8"
0159 |         android:strokeColor="#33FFFFFF" />
0160 |     <path

```

L129: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L130: Declares the XML resource or attribute shown here; the file overview explains its Android role.

L131-134: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L135: Declares the XML resource or attribute shown here; the file overview explains its Android role.

L136-139: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L140: Declares the XML resource or attribute shown here; the file overview explains its Android role.

L141-144: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L145: Declares the XML resource or attribute shown here; the file overview explains its Android role.

L146-149: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L150: Declares the XML resource or attribute shown here; the file overview explains its Android role.

L151-154: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L155: Declares the XML resource or attribute shown here; the file overview explains its Android role.

L156-159: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L160: Declares the XML resource or attribute shown here; the file overview explains its Android role.

Source lines 161-170

```

0161 |         android:fillColor="#00000000"
0162 |         android:pathData="M69,19L69,89"
0163 |         android:strokeWidth="0.8"
0164 |         android:strokeColor="#33FFFFFF" />
0165 |     <path
0166 |         android:fillColor="#00000000"
0167 |         android:pathData="M79,19L79,89"
0168 |         android:strokeWidth="0.8"
0169 |         android:strokeColor="#33FFFFFF" />
0170 | </vector>

```

L161-164: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L165: Declares the XML resource or attribute shown here; the file overview explains its Android role.

L166-169: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L170: Declares the XML resource or attribute shown here; the file overview explains its Android role.

47. ic_launcher_foreground.xml

app/src/main/res/drawable/ic_launcher_foreground.xml

Vector drawable launcher artwork.

30 source lines. SHA-256: 5c78c0bf44feefc9

width/height specify the nominal size; viewport coordinates define the drawing space. pathData is the sequence of move, line, curve, and close operations.

fillColor paints enclosed shapes; strokeColor and strokeWidth draw outlines. These numerical geometry values describe the icon rather than an application workflow.

Source lines 1-30

```
0001 | <vector xmlns:android="http://schemas.android.com/apk/res/android"
0002 |     xmlns:aapt="http://schemas.android.com/aapt"
0003 |     android:width="108dp"
0004 |     android:height="108dp"
0005 |     android:viewportWidth="108"
0006 |     android:viewportHeight="108">
0007 |     <path android:pathData="M31,63.928c0,0 6.4,-11 12.1,-13.1c7.2,-2.6 26,-1.4
> 26,-1.4l38.1,38.1l107,108.928l-32,-1l31,63.928z">
0008 |         <aapt:attr name="android:fillColor">
0009 |             <gradient
0010 |                 android:endX="85.84757"
0011 |                 android:endY="92.4963"
0012 |                 android:startX="42.9492"
0013 |                 android:startY="49.59793"
0014 |                 android:type="linear">
0015 |                 <item
0016 |                     android:color="#44000000"
0017 |                     android:offset="0.0" />
0018 |                 <item
0019 |                     android:color="#00000000"
0020 |                     android:offset="1.0" />
0021 |                 </gradient>
0022 |             </aapt:attr>
0023 |         </path>
0024 |     <path
0025 |         android:fillColor="#FFFFFF"
0026 |         android:fillType="nonZero"
0027 |         android:pathData="M65.3,45.828l3.8,-6.6c0.2,-0.4 0.1,-0.9 -0.3,-1.1c-0.4,-0.2 -0.9,-0.1
> -1.1,0.3l-3.9,6.7c-6.3,-2.8 -13.4,-2.8 -19.7,0l-3.9,-6.7c-0.2,-0.4 -0.7,-0.5 -1.1,-0.3c38.8,38.328
> 38.7,38.828 38.9,39.228l3.8,6.6c36.2,49.428 31.7,56.028 31,63.928h46c76.3,56.028 71.8,49.428
> 65.3,45.828zM43.4,57.328c-0.8,0 -1.5,-0.5 -1.8,-1.2c-0.3,-0.7 -0.1,-1.5 0.4,-2.1c0.5,-0.5 1.4,-0.7
> 2.1,-0.4c0.7,0.3 1.2,1 1.2,1.8c45.3,56.528 44.5,57.328 43.4,57.328L43.4,57.328zM64.6,57.328c-0.8,0
> -1.5,-0.5 -1.8,-1.2s-0.1,-1.5 0.4,-2.1c0.5,-0.5 1.4,-0.7 2.1,-0.4c0.7,0.3 1.2,1 1.2,1.8c66.5,56.528
> 65.6,57.328 64.6,57.328L64.6,57.328z"
0028 |         android:strokeWidth="1"
0029 |         android:strokeColor="#00000000" />
0030 | </vector>
```

L1: Declares the XML resource or attribute shown here; the file overview explains its Android role.

L2-6: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L7-9: Declares the XML resource or attribute shown here; the file overview explains its Android role.

L10-14: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L15: Declares the XML resource or attribute shown here; the file overview explains its Android role.

L16-17: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L18: Declares the XML resource or attribute shown here; the file overview explains its Android role.

L19-20: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L21-24: Declares the XML resource or attribute shown here; the file overview explains its Android role.

L25-29: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L30: Declares the XML resource or attribute shown here; the file overview explains its Android role.

48. ic_launcher.xml

app/src/main/res/mipmap-anydpi-v26/ic_launcher.xml

Adaptive launcher icon composition.

6 source lines. SHA-256: c83c425cc19d7f44

References foreground/background drawable layers and a monochrome layer. Android uses these layers for supported launcher shapes and themed icons.

Source lines 1-6

```
0001 | <?xml version="1.0" encoding="utf-8"?>
0002 | <adaptive-icon xmlns:android="http://schemas.android.com/apk/res/android">
0003 |     <background android:drawable="@drawable/ic_launcher_background" />
0004 |     <foreground android:drawable="@drawable/ic_launcher_foreground" />
0005 |     <monochrome android:drawable="@drawable/ic_launcher_foreground" />
0006 | </adaptive-icon>
```

L1-6: Declares the XML resource or attribute shown here; the file overview explains its Android role.

49. ic_launcher_round.xml

app/src/main/res/mipmap-anydpi-v26/ic_launcher_round.xml

Adaptive launcher icon composition.

6 source lines. SHA-256: c83c425cc19d7f44

References foreground/background drawable layers and a monochrome layer. Android uses these layers for supported launcher shapes and themed icons.

Source lines 1-6

```
0001 | <?xml version="1.0" encoding="utf-8"?>
0002 | <adaptive-icon xmlns:android="http://schemas.android.com/apk/res/android">
0003 |     <background android:drawable="@drawable/ic_launcher_background" />
0004 |     <foreground android:drawable="@drawable/ic_launcher_foreground" />
0005 |     <monochrome android:drawable="@drawable/ic_launcher_foreground" />
0006 | </adaptive-icon>
```

L1-6: Declares the XML resource or attribute shown here; the file overview explains its Android role.

50. colors.xml

app/src/main/res/values/colors.xml

Legacy XML color resources.

10 source lines. SHA-256: 2ce5321c79b12fa4

Defines named hexadecimal colors for XML resources. Compose has its separate Color.kt palette; an XML color definition does not automatically style a Compose screen.

Source lines 1-10

```
0001 | <?xml version="1.0" encoding="utf-8"?>
0002 | <resources>
0003 |     <color name="purple_200">#FFBB86FC</color>
0004 |     <color name="purple_500">#FF6200EE</color>
0005 |     <color name="purple_700">#FF3700B3</color>
0006 |     <color name="teal_200">#FF03DAC5</color>
0007 |     <color name="teal_700">#FF018786</color>
0008 |     <color name="black">#FF000000</color>
0009 |     <color name="white">#FFFFFFFF</color>
0010 | </resources>
```

L1-10: Declares the XML resource or attribute shown here; the file overview explains its Android role.

51. strings.xml

app/src/main/res/values/strings.xml

Android string resources.

3 source lines. SHA-256: 694d364af8079b38

Defines the launcher app name. Compose screen text currently mostly uses Kotlin literals rather than localized string resources.

Source lines 1-3

```
0001 | <resources>
0002 |     <string name="app_name">Student tracker</string>
0003 | </resources>
```

L1-3: Declares the XML resource or attribute shown here; the file overview explains its Android role.

52. themes.xml

app/src/main/res/values/themes.xml

Native Android window theme.

5 source lines. SHA-256: 5554a431684724d5

Provides a platform Material light no-action-bar theme for the activity window. Compose MaterialTheme controls the actual app content colors separately.

Source lines 1-5

```
0001 | <?xml version="1.0" encoding="utf-8"?>
0002 | <resources>
0003 |
0004 |     <style name="Theme.StudentTracker" parent="android:Theme.Material.Light.NoActionBar" />
0005 | </resources>
```

L1-2: Declares the XML resource or attribute shown here; the file overview explains its Android role.

L4-5: Declares the XML resource or attribute shown here; the file overview explains its Android role.

53. backup_rules.xml

app/src/main/res/xml/backup_rules.xml

Android backup and transfer exclusions.

5 source lines. SHA-256: 90b75e3d8b375425

Exclude DataStore and shared-preference session metadata in the corresponding legacy backup, cloud backup, or device-transfer rules. allowBackup=false also disables app backup in the manifest.

Source lines 1-5

```
0001 | <?xml version="1.0" encoding="utf-8"?>
0002 | <full-backup-content>
0003 |     <exclude domain="file" path="datastore/" />
0004 |     <exclude domain="sharedpref" path="." />
0005 | </full-backup-content>
```

L1-5: Declares the XML resource or attribute shown here; the file overview explains its Android role.

54. data_extraction_rules.xml

app/src/main/res/xml/data_extraction_rules.xml

Android backup and transfer exclusions.

11 source lines. SHA-256: 4be5262ea224fe39

Exclude DataStore and shared-preference session metadata in the corresponding legacy backup, cloud backup, or device-transfer rules. allowBackup=false also disables app backup in the manifest.

Source lines 1-11

```
0001 | <?xml version="1.0" encoding="utf-8"?>
0002 | <data-extraction-rules>
0003 |     <cloud-backup>
0004 |         <exclude domain="file" path="datastore/" />
0005 |         <exclude domain="sharedpref" path="." />
0006 |     </cloud-backup>
0007 |     <device-transfer>
0008 |         <exclude domain="file" path="datastore/" />
0009 |         <exclude domain="sharedpref" path="." />
0010 |     </device-transfer>
0011 | </data-extraction-rules>
```

L1-11: Declares the XML resource or attribute shown here; the file overview explains its Android role.

55. file_paths.xml

app/src/main/res/xml/file_paths.xml

Allowed FileProvider share directory.

4 source lines. SHA-256: 87de4ba1ca286ecd

The cache-path entry exposes only cache/exports for generated CSV sharing, rather than the full private storage tree.

Source lines 1-4

```
0001 | <?xml version="1.0" encoding="utf-8"?>
0002 | <paths>
0003 |     <cache-path name="attendance_exports" path="exports/" />
0004 | </paths>
```

L1-4: Declares the XML resource or attribute shown here; the file overview explains its Android role.

56. CsvExporterTest.kt

app/src/test/java/com/example/studenttracker/CsvExporterTest.kt

CSV correctness and formula-neutralization tests.

20 source lines. SHA-256: 1dc3c82b6d307881

Tests exercise quoted commas and quotes, formula-like prefixes, BOM, and CRLF output. They assert observable output rather than opening a spreadsheet application.

Source lines 1-20

```
0001 | package com.example.studenttracker
0002 |
0003 | import com.example.studenttracker.domain.model.CsvExporter
0004 | import org.junit.Assert.*
0005 | import org.junit.Test
0006 |
0007 | class CsvExporterTest {
0008 |     @Test fun quotesCommasQuotesAndLineBreaks() {
0009 |         assertEquals("\"Doe, \\\"Jane\\\"\\nStudent\"", CsvExporter.cell("Doe, \"Jane\\nStudent"))
0010 |     }
0011 |     @Test fun neutralizesFormulasIncludingWhitespacePrefixes() {
0012 |         listOf("=SUM(A1)", "+123", "-1", "@test", " =HYPERLINK(x)", "\t1").forEach {
0013 |             assertTrue(CsvExporter.cell(it).startsWith("\"'"))
0014 |         }
0015 |         assertEquals("\"Student 1\"", CsvExporter.cell("Student 1"))
0016 |     }
0017 |     @Test fun usesUtf8BomAndCrLfForSpreadsheetCompatibility() {
0018 |         assertEquals("\uFEFF\"Name\", \"Marks\\r\\n\"Anita\", \"0\\r\\n\",
0019 |             > CsvExporter.encode(listOf(listOf("Name", "Marks"), listOf("Anita", "0"))))
0020 |     }
```

L1: Declares the Kotlin namespace; it organizes classes and functions.

L3-5: Imports the named library symbols used by this file.

L7: Declares the named type; constructor properties define its stored data or dependencies.

L8: Marks the next function as a test executed by the test runner.

L9: Checks the displayed actual value against the expected behavior; failure fails the test.

L10: Closes or continues the surrounding block, expression, or call.

L11: Marks the next function as a test executed by the test runner.

L12: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L13: Checks the displayed actual value against the expected behavior; failure fails the test.

L14: Closes or continues the surrounding block, expression, or call.

L15: Checks the displayed actual value against the expected behavior; failure fails the test.

L16: Closes or continues the surrounding block, expression, or call.

L17: Marks the next function as a test executed by the test runner.

L18: Checks the displayed actual value against the expected behavior; failure fails the test.

L19-20: Closes or continues the surrounding block, expression, or call.

57. ExampleUnitTest.kt

app/src/test/java/com/example/studenttracker/ExampleUnitTest.kt

Android Studio arithmetic template test.

12 source lines. SHA-256: a88a0d1b359abcc5

Checks a simple arithmetic assertion. It verifies test setup but does not validate an academic workflow.

Source lines 1-12

```
0001 | package com.example.studenttracker
0002 |
0003 | import org.junit.Test
0004 |
0005 | import org.junit.Assert.*
0006 |
0007 | class ExampleUnitTest {
0008 |     @Test
0009 |     fun addition_isCorrect() {
0010 |         assertEquals(4, 2 + 2)
0011 |     }
0012 | }
```

L1: Declares the Kotlin namespace; it organizes classes and functions.

L3: Imports the named library symbols used by this file.

L5: Imports the named library symbols used by this file.

L7: Declares the named type; constructor properties define its stored data or dependencies.

L8: Marks the next function as a test executed by the test runner.

L9: Defines this Kotlin operation and its parameters; suspend allows asynchronous calls.

L10: Checks the displayed actual value against the expected behavior; failure fails the test.

L11-12: Closes or continues the surrounding block, expression, or call.

58. TrackerInterceptorTest.kt

app/src/test/java/com/example/studenttracker/TrackerInterceptorTest.kt

HTTP authentication and response-envelope unit tests.

56 source lines. SHA-256: cf42312f38c5e3f1

MockWebServer records the actual outbound request and provides controlled responses. Tests check POST conversion, token payload, incompatible API version, failed envelopes, and session clearing.

These are local JVM tests; they do not contact a deployed Google backend.

Source lines 1-32

```
0001 | package com.example.studenttracker
0002 |
0003 | import com.example.studenttracker.data.network.*
0004 | import com.google.gson.JsonParser
0005 | import okhttp3.OkHttpClient
0006 | import okhttp3.Request
0007 | import okhttp3.mockwebserver.MockResponse
0008 | import okhttp3.mockwebserver.MockWebServer
0009 | import org.junit.After
0010 | import org.junit.Assert.*
0011 | import org.junit.Before
0012 | import org.junit.Test
0013 |
0014 | class TrackerInterceptorTest {
0015 |     private lateinit var server: MockWebServer
0016 |     private lateinit var client: OkHttpClient
0017 |     @Before fun setup() {
0018 |         server = MockWebServer().apply { start() }
0019 |         client = OkHttpClient.Builder().addInterceptor(TrackerInterceptor()).build()
0020 |         AuthSession.begin("test-google-token")
0021 |     }
0022 |     @After fun cleanup() { AuthSession.clear(); server.shutdown() }
0023 |     @Test fun movesReadParametersAndCredentialsIntoPostBody() {
0024 |
0025 |         > server.enqueue(MockResponse().setBody("""{"apiVersion":2,"status":"success","students":[]}"""))
0026 |         > client.newCall(Request.Builder().url(server.url("/exec?action=getStudents&subjectCode=CS101")
0027 |         > ).build()).execute().use { assertTrue(it.isSuccessful) }
0028 |         val request = server.takeRequest()
0029 |         assertEquals("POST", request.method)
0030 |         assertEquals("/exec", request.path)
0031 |         val body = JsonParser.parseString(request.body.readUtf8()).asJsonObject
0032 |         assertEquals("test-google-token", body["idToken"].asString)
0033 |         assertEquals("getStudents", body["action"].asString)
0034 |         assertEquals("CS101", body["subjectCode"].asString)
0035 |     }
0036 | }
```

L1: Declares the Kotlin namespace; it organizes classes and functions.

L3-12: Imports the named library symbols used by this file.

L14: Declares the named type; constructor properties define its stored data or dependencies.

L15-16: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L17: Applies the displayed annotation to the declaration.

L18-19: Assigns the displayed expression to a variable or named argument.

L20: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L21: Closes or continues the surrounding block, expression, or call.

L22: Applies the displayed annotation to the declaration.

L23: Marks the next function as a test executed by the test runner.

L24: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L25: Checks the displayed actual value against the expected behavior; failure fails the test.

L26: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L27-28: Checks the displayed actual value against the expected behavior; failure fails the test.

L29: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L30-32: Checks the displayed actual value against the expected behavior; failure fails the test.

Source lines 33-56

```
0033 |     }
0034 |     @Test fun expiresSessionOnBackendAuthenticationFailure() {
0035 |         server.enqueue(MockResponse().setBody("""{"apiVersion":2,"status":"error","code":"UNAUTHENTI
> CATED","message":"Sign in again"}"""))
0036 |         val exception = try {
> client.newCall(Request.Builder().url(server.url("/exec")).build()).execute(); null } catch (e:
> TrackerApiException) { e }
0037 |         assertEquals("UNAUTHENTICATED", exception?.code)
0038 |         assertNull(AuthSession.idToken)
0039 |     }
0040 |     @Test fun refusesOldBackendInsteadOfTrustingEmailOnlyRoles() {
0041 |         server.enqueue(MockResponse().setBody("""{"status":"success","role":"admin"}"""))
0042 |         val exception = try {
> client.newCall(Request.Builder().url(server.url("/exec")).build()).execute(); null } catch (e:
> TrackerApiException) { e }
0043 |         assertEquals("CONFIGURATION", exception?.code)
0044 |     }
0045 |     @Test fun reportsMalformedResponseWithoutCrashingOrAcceptingIt() {
0046 |         server.enqueue(MockResponse().setBody("<html>Deployment error</html>"))
0047 |         val exception = try {
> client.newCall(Request.Builder().url(server.url("/exec")).build()).execute(); null } catch (e:
> TrackerApiException) { e }
0048 |         assertEquals("INVALID_RESPONSE", exception?.code)
0049 |     }
0050 |     @Test fun makesNoNetworkRequestWithoutCredential() {
0051 |         AuthSession.clear()
0052 |         val exception = try {
> client.newCall(Request.Builder().url(server.url("/exec")).build()).execute(); null } catch (e:
> TrackerApiException) { e }
0053 |         assertEquals("UNAUTHENTICATED", exception?.code)
0054 |         assertEquals(0, server.requestCount)
0055 |     }
0056 | }
```

L33: Closes or continues the surrounding block, expression, or call.

L34: Marks the next function as a test executed by the test runner.

L35: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L36: Handles this exception type and translates it into the following failure behavior.

L37-38: Checks the displayed actual value against the expected behavior; failure fails the test.

L39: Closes or continues the surrounding block, expression, or call.

L40: Marks the next function as a test executed by the test runner.

L41: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L42: Handles this exception type and translates it into the following failure behavior.

L43: Checks the displayed actual value against the expected behavior; failure fails the test.

L44: Closes or continues the surrounding block, expression, or call.

L45: Marks the next function as a test executed by the test runner.

L46: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L47: Handles this exception type and translates it into the following failure behavior.

L48: Checks the displayed actual value against the expected behavior; failure fails the test.

L49: Closes or continues the surrounding block, expression, or call.

L50: Marks the next function as a test executed by the test runner.

L51: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L52: Handles this exception type and translates it into the following failure behavior.

L53-54: Checks the displayed actual value against the expected behavior; failure fails the test.

L55-56: Closes or continues the surrounding block, expression, or call.

59. README.md

backend/README.md

Project setup and usage documentation.

130 source lines. SHA-256: 75dc282646005cff

Explains how to build, configure authentication, package/deploy Apps Script, and validate workflows. Follow the backend README for required Script Properties and sheet headers.

Complete document text

```
001 # Google Sheets / Apps Script backend
003 This backend stays entirely on Google Sheets and Google Apps Script. It implements
004 the Android client's version 2 API. It has not been deployed or tested against a
005 live Google project yet. Use a separate staging spreadsheet first.
007 ## Build and verify locally
009 Requires Node.js 22 or later. From this directory:
011 ```powershell
012 npm ci --ignore-scripts
013 npm run build
014 npm test
015 npm audit --omit=dev
016 ```
018 The build produces `dist/Code.gs` and `dist/appsscript.json`. The generated script
019 includes the pinned RSA verification library, application rules and Apps Script
020 adapter. Local tests execute the generated bundle with Apps Script service doubles
021 and independently signed RSA tokens; they do not access Google or live student data.
023 ## Set up a staging deployment
025 1. Create a private Google spreadsheet. Share it only with authorized operators.
026 2. Open Extensions → Apps Script. Replace `Code.gs` with the generated
027 `dist/Code.gs`. Enable showing the manifest in Project Settings and copy
028 `dist/appsscript.json` into `appsscript.json`.
029 3. Add these Script Properties in Project Settings:
030 - `SPREADSHEET_ID`: the spreadsheet's ID, not its URL.
031 - `GOOGLE_WEB_CLIENT_ID`: your Web OAuth client ID, matching the Android build.
032 - Optional `ALLOWED_GOOGLE_DOMAIN`: the exact Workspace domain. This checks the
033 signed `hd` claim; an email suffix alone does not establish domain membership.
034 4. Run `setUpTracker` once in the editor and authorize spreadsheet/external-request
035 access. It adds two tabs without modifying other tabs:
036 - `TrackerUsers`: `email, role, profileId, name, active, googleSubject`.
037 - `TrackerRecords`: an append-only record journal with audit metadata.
038 5. Add your administrator to `TrackerUsers`: Google email, role `admin`, blank
039 `profileId`, display name, `active` = TRUE. `googleSubject` can initially be blank;
040 filling it with the account's Google subject ID additionally pins the identity.
041 6. Deploy as a Web app, executing as the owner, accessible to Anyone if your
042 Workspace policy allows it. The public HTTP endpoint still requires a verified
043 Google ID token and an active user allowlist entry for all data requests.
044 7. Configure the Android base URL as the deployment URL without `exec`, ending
045 with `/`: `https://script.google.com/macros/s/YOUR_DEPLOYMENT_ID/`.
046 8. Configure Android OAuth registration for `com.example.studenttracker` and your
```

047 actual debug/release signing certificate, plus the matching Web client ID.

049 ## First complete workflow

051 1. Sign in as the allowlisted administrator.

052 2. Create faculty and student profiles in the app, then create subjects assigned to

053 active faculty employee IDs.

054 3. In `TrackerUsers`, allowlist faculty using role `faculty` and `profileId` equal

055 to their employee ID. Allowlist students with role `student` and `profileId`

056 equal to their roll number. Adding a profile does not automatically grant access.

057 4. Open **Academic workspace** → **Manage enrolled students** for each subject.

058 5. Sign in as faculty, select an assigned subject, and submit a date/session roster.

059 6. Create an assessment, save marks as a draft, then publish after every entry is

060 scored or absent. Published assessments are locked in this version.

061 7. Sign in as a student and verify that only that student's attendance and published

062 results are visible. Sign in as faculty/admin to review and export attendance.

064 ## API and authorization

066 - All actions use `POST /exec` with JSON containing `action` and `idToken`. GET

067 requests never return student records. Tokens are not placed in URLs or logs.

068 - Responses contain `apiVersion: 2` and `status: success | error`. Errors include

069 a machine-readable `code` and a user-facing `message`; Apps Script's HTTP status

070 is not used to represent domain errors.

071 - RS256 signatures use Google's HTTPS public certificates, with issuer, audience,

072 expiry, issue-time and verified-email checks. Certificate caching respects

073 `Cache-Control`; an unknown signing key triggers one refresh.

074 - The user allowlist is read on every request. Faculty also need an active profile

075 and subject assignment. Students need an active profile and only receive their

076 own progress. Client-provided student/email parameters cannot select a different

077 identity.

078 - Administrator reads: `getDashboard`, `getStudents`, `getStudent`, `getFaculty`,

079 `getFacultyMember`, `getSubject`. Profile actions: `add/update/delete` +

080 `Student/Faculty/Subject`.

081 - Faculty/admin actions: `getSubjects`, `getRoster`, `saveAttendance`,

082 `getAttendanceReport`, `getAssessments`, `saveAssessment`.

083 - Enrollment writes are administrator-only: `saveEnrollment`.

084 - Student progress: `getMyProgress`; student `getSubjects` is limited to enrollment.

086 ## Integrity and policies

088 - `TrackerRecords` holds complete entity revisions, including actor/action/time.

089 A script lock serializes all access. Each mutation appends one aggregate row;

090 attendance/assessment/enrollment retries are deduplicated and stale revisions are

091 rejected. Identifiers and JSON are stored as text to preserve leading zeroes and

092 prevent formula execution. Do not edit or sort this journal manually.

093 - Updating or deactivating profiles requires the last observed `revision`.

094 Deactivation retains historical attendance/marks. Faculty with assigned subjects

095 must be reassigned before deactivation. Identifiers cannot be reused.

096 - Attendance is unique by subject + date + case-insensitive session label. Present

097 and late count as attendance; excused entries are excluded from the denominator.

098 Only sessions with that student in the roster count for that student. No counted

099 sessions means an unavailable percentage, not zero attendance.

100 - The institution-local date follows the manifest timezone (`Asia/Kolkata` by

101 default). Change it for institutions in another timezone.

102 - Marks distinguish zero, absent and pending. Publishing requires all marks to be
103 complete. The current version supports individual assessment scores, not weighted
104 final grades or approved corrections to published results.

105 - Enrollment is limited to 250 students per subject; any aggregate above 40,000 JSON
106 characters is rejected before writing. This is a guard, not a measured capacity
107 guarantee. Every request currently rebuilds projections from the journal; pagination,
108 compaction and load testing are required before a large deployment.

110 ## Existing data and release gates

112 Legacy Students/Faculty/Subjects/Attendance tabs are not automatically imported.

113 Back up the legacy sheet, map its columns explicitly, and rehearse import into a
114 staging spreadsheet before switching the app. No legacy data has been changed by
115 this implementation.

117 Before public release, verify the real Apps Script runtime and Google sign-in flow,
118 perform a backup/restore drill, load-test the expected institution size, set up
119 monitoring and establish a rollback procedure. Google accounts and cloud deployment
120 configuration are required for those checks.

122 The RSA verifier uses pinned `node-forge` 1.4.0, including its corrected RSA signature
123 validation. Review dependencies and run the audit before each deployment.

124 The generated third-party license file must accompany distribution of the backend bundle.

126 References:

127 - [Google ID-token verification](https://developers.google.com/identity/gsi/web/guides/verify-google-id-token)
128 - [Apps Script locks](https://developers.google.com/apps-script/reference/lock/lock-service)
129 - [Forge cryptography library](https://github.com/digitalbazaar/forge)
130 - [Forge RSA signature verification fix](https://github.com/digitalbazaar/forge/security/advisories/GHSA-ppp5-5v6c-4jwp)

60. package-lock.json

backend/package-lock.json

Exact npm dependency resolution.

24 source lines. SHA-256: 4686ec261e4bed5f

Pins package versions, resolved downloads, and integrity hashes for reproducible npm ci installs. The generated lock file is data, not executable application logic.

Source lines 1-24

```
0001 | {
0002 |   "name": "student-tracker-apps-script",
0003 |   "version": "1.0.0",
0004 |   "lockfileVersion": 3,
0005 |   "requires": true,
0006 |   "packages": {
0007 |     "": {
0008 |       "name": "student-tracker-apps-script",
0009 |       "version": "1.0.0",
0010 |       "dependencies": {
0011 |         "node-forge": "1.4.0"
0012 |       }
0013 |     },
0014 |     "node_modules/node-forge": {
0015 |       "version": "1.4.0",
0016 |       "resolved": "https://registry.npmjs.org/node-forge/-/node-forge-1.4.0.tgz",
0017 |       "integrity":
0018 |         > "sha512-LarFH0+6VfriEhqMMcLX2F7SwSxeWwnEAJEsYm5QKWchiVYVvJyV9v7UDvUv+w5H023ZpQTXDv/GxdDdMy0uoQ==",
0019 |       "license": "(BSD-3-Clause OR GPL-2.0)",
0020 |       "engines": {
0021 |         "node": ">= 6.13.0"
0022 |       }
0023 |     }
0024 |   }
```

L1-11: Defines the displayed configuration key, section, dependency, or literal value.

L12-13: Closes or continues the surrounding block, expression, or call.

L14-20: Defines the displayed configuration key, section, dependency, or literal value.

L21-24: Closes or continues the surrounding block, expression, or call.

61. package.json

backend/package.json

Node build/test commands and backend dependency declaration.

12 source lines. SHA-256: 7b40a0fb3d08922c

npm run build executes scripts/build.mjs. npm test executes Node built-in test runner over test/*.test.cjs.

node-forge 1.4.0 supplies RSA certificate/signature verification. Node tooling is used locally; deployed authored code runs in Apps Script V8.

Source lines 1-12

```
0001 | {  
0002 |   "name": "student-tracker-apps-script",  
0003 |   "private": true,  
0004 |   "version": "1.0.0",  
0005 |   "scripts": {  
0006 |     "build": "node scripts/build.mjs",  
0007 |     "test": "node --test test/*.test.cjs"  
0008 |   },  
0009 |   "dependencies": {  
0010 |     "node-forge": "1.4.0"  
0011 |   }  
0012 | }
```

L1-7: Defines the displayed configuration key, section, dependency, or literal value.

L8: Closes or continues the surrounding block, expression, or call.

L9-10: Defines the displayed configuration key, section, dependency, or literal value.

L11-12: Closes or continues the surrounding block, expression, or call.

62. build.mjs

backend/scripts/build.mjs

Packages authored backend and cryptographic dependency for Apps Script.

12 source lines. SHA-256: 988bc2837b48e542

Reads node-forge browser output plus core.js, auth.js, and apps-script.js. A window shim makes the browser bundle available in the Apps Script environment.

Writes dist/Code.gs, THIRD_PARTY_LICENSE.txt, and appsscript.json. The manifest sets V8, Asia/Kolkata, and Sheets/external-request OAuth scopes.

This is a packaging step; running it does not upload or deploy the web app.

Source lines 1-12

```
0001 | import { readFileSync, mkdirSync, writeFileSync } from 'node:fs';
0002 | import { fileURLToPath } from 'node:url';
0003 | import path from 'node:path';
0004 | const root = path.resolve(path.dirname(fileURLToPath(import.meta.url)), '..');
0005 | const read = file => readFileSync(path.join(root, file), 'utf8');
0006 | mkdirSync(path.join(root, 'dist'), { recursive: true });
0007 |
0008 | const crypto = read('node_modules/node-forge/dist/forge.min.js');
0009 | writeFileSync(path.join(root, 'dist', 'Code.gs'), 'var window = {};\n' + crypto + '\nvar forge =
> window.forge;\n' + ['core.js', 'auth.js', 'apps-script.js'].map(f => read('src/' + f)).join('\n'));
0010 | writeFileSync(path.join(root, 'dist', 'THIRD_PARTY_LICENSE.txt'),
> read('node_modules/node-forge/LICENSE'));
0011 | writeFileSync(path.join(root, 'dist', 'appsscript.json'), JSON.stringify({ timeZone: 'Asia/Kolkata',
> runtimeVersion: 'V8', exceptionLogging: 'STACKDRIVER', oauthScopes:
> ['https://www.googleapis.com/auth/spreadsheets',
> 'https://www.googleapis.com/auth/script.external_request'] }, null, 2));
0012 | console.log('Built dist/Code.gs and dist/appsscript.json');
```

L1-3: Imports the named library symbols used by this file.

L4-5: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L6: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L8: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L9: Transforms, selects, or indexes the displayed collection using its callback.

L10: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L11: Serializes structured values to JSON for transport, saveable state, or journal storage.

L12: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

63. apps-script.js

backend/src/apps-script.js

Apps Script HTTP adapter, Google certificates, and Sheet journal.

91 source lines. SHA-256: 1751920d36cfe961

doGet rejects API access; doPost parses a bounded object body, checks required Script Properties, obtains trusted certificates, verifies the ID token, and optionally enforces a Workspace domain.

The adapter acquires the script lock, opens TrackerUsers and TrackerRecords, derives the principal, creates the store/service, executes the action, flushes, and returns an apiVersion=2 envelope. finally releases the lock.

googleCertificates_ caches the Google certificate response within its max-age. An unknown key triggers one refresh. jsonResponse_ uses ContentService JSON MIME type.

tableObjects_ maps header names to row values. sheetStore_ projects the latest journal entries into a Map and appends one text-formatted, JSON-encoded audited row per accepted write.

setupTracker is an editor setup function. It creates required sheets and headers and freezes the header row; deployment and approved account population are separate steps.

Source lines 1-32

```
0001 | function doGet() { return jsonResponse_({ status: 'error', apiVersion: 2, code:
    > 'METHOD_NOT_ALLOWED', message: 'Use authenticated POST requests.' }); }
0002 | function doPost(event) {
0003 |   let lock;
0004 |   try {
0005 |     const body = event && event.postData && event.postData.contents;
0006 |     if (!body || body.length > 100000) fault('VALIDATION', 'Invalid request size.');
```

```
0007 |     let request;
0008 |     try { request = JSON.parse(body); } catch (_) { fault('VALIDATION', 'Invalid JSON request.');
```

```
0009 |     if (!request || typeof request !== 'object' || Array.isArray(request)) fault('VALIDATION',
    > 'Invalid request.');
```

```
0010 |     const config = PropertiesService.getScriptProperties();
0011 |     const audience = config.getProperty('GOOGLE_WEB_CLIENT_ID');
```

```
0012 |     const spreadsheetId = config.getProperty('SPREADSHEET_ID');
```

```
0013 |     if (!audience || !spreadsheetId) fault('CONFIGURATION', 'Backend setup is incomplete.');
```

```
0014 |     let certificates = googleCertificates_();
0015 |
0016 |     try {
0017 |       const header = decodeJwtJson_(String(request.idToken).split('.')[0]);
0018 |       if (header && header.alg === 'RS256' && header.kid &&
    > !Object.prototype.hasOwnProperty.call(certificates, header.kid)) certificates =
    > googleCertificates_(true);
0019 |     } catch (_) { }
0020 |     const claims = verifyGoogleToken(request.idToken, audience, certificates, Math.floor(Date.now()
    > / 1000));
0021 |     const domain = config.getProperty('ALLOWED_GOOGLE_DOMAIN');
```

```
0022 |     if (domain && claims.hd !== domain) fault('FORBIDDEN', 'Use your institutional Google
    > account.');
```

```
0023 |
0024 |     lock = LockService.getScriptLock();
0025 |     if (!lock.tryLock(10000)) fault('BUSY', 'The service is busy. Try again shortly.');
```

```
0026 |     const spreadsheet = SpreadsheetApp.openById(spreadsheetId);
0027 |     const usersSheet = spreadsheet.getSheetByName('TrackerUsers');
```

```
0028 |     const recordSheet = spreadsheet.getSheetByName('TrackerRecords');
```

```
0029 |     if (!usersSheet || !recordSheet) fault('CONFIGURATION', 'Run setupTracker in the Apps Script
    > editor first.');
```

```
0030 |     const users = tableObjects_(usersSheet);
0031 |     const user = principalFor(claims, users);
0032 |     const store = sheetStore_(recordSheet);
```

L1-2: Defines the named JavaScript operation or callback; its body implements the following steps.

L3: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L4: Starts an operation whose failures are handled by the following catch/finally blocks.

L5: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L6: Branches on the displayed condition; the accepted branch performs the following operation.

L7: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L8: Handles this exception type and translates it into the following failure behavior.

L9: Declares the named type; constructor properties define its stored data or dependencies.

L10-12: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L13: Branches on the displayed condition; the accepted branch performs the following operation.

L14: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L16: Starts an operation whose failures are handled by the following catch/finally blocks.

L17: Carries the Google identity token for server verification; the token itself does not assign a role.

L18: Branches on the displayed condition; the accepted branch performs the following operation.

L19: Handles this exception type and translates it into the following failure behavior.

L20: Carries the Google identity token for server verification; the token itself does not assign a role.

L21: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L22: Branches on the displayed condition; the accepted branch performs the following operation.

L24: Assigns the displayed expression to a variable or named argument.

L25: Branches on the displayed condition; the accepted branch performs the following operation.

L26-28: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L29: Branches on the displayed condition; the accepted branch performs the following operation.

L30-32: Declares the displayed value or mutable state; its initializer supplies the initial contents.

Source lines 33-64

```

0033 |     const service = createService(store, user, undefined, () => Utilities.formatDate(new Date(),
    > Session.getScriptTimeZone(), 'yyyy-MM-dd'));
0034 |     const result = service.execute(request);
0035 |     SpreadsheetApp.flush();
0036 |     return jsonResponse_({ status: 'success', apiVersion: 2, ...result });
0037 |   } catch (error) {
0038 |   }
0039 |     return jsonResponse_({ status: 'error', apiVersion: 2, code: error.code || 'INTERNAL', message:
    > error.code ? error.message : 'The service could not complete this request. Try again.' });
0040 |   } finally { if (lock && lock.hasLock()) lock.releaseLock(); }
0041 | }
0042 | function jsonResponse_(value) { return
    > ContentService.createTextOutput(JSON.stringify(value)).setMimeType(ContentService.MimeType.JSON); }
0043 | function googleCertificates_(refresh = false) {
0044 |   const cache = CacheService.getScriptCache();
0045 |   const cached = refresh ? null : cache.get('google-signing-certificates-v1');
0046 |   if (cached) return JSON.parse(cached);
0047 |   const response = UrlFetchApp.fetch('https://www.googleapis.com/oauth2/v1/certs', {
    > muteHttpExceptions: true });
0048 |   if (response.getResponseCode() !== 200) fault('UNAVAILABLE', 'Sign-in verification is temporarily
    > unavailable. ');
0049 |   const body = response.getContentText();
0050 |   const headers = response.getAllHeaders();
0051 |   const cacheHeader = Object.keys(headers).find(k => k.toLowerCase() === 'cache-control');
0052 |   const maxAge = /max-age=(\d+)/.exec(String(cacheHeader ? headers[cacheHeader] : ''));
0053 |   if (maxAge && Number(maxAge[1]) > 0) cache.put('google-signing-certificates-v1', body,
    > Math.min(21600, Number(maxAge[1])));
0054 |   return JSON.parse(body);
0055 | }
0056 | function tableObjects_(sheet) {
0057 |   const rows = sheet.getDataRange().getValues();
0058 |   const headers = rows.shift();
0059 |   return rows.filter(row => row[0]).map(row => Object.fromEntries(headers.map((h, i) => [h,
    > row[i]])));
0060 | }
0061 | function sheetStore_(sheet) {
0062 |   const current = new Map();
0063 |   tableObjects_(sheet).forEach(row => current.set(JSON.stringify([row.kind, JSON.parse(row.id)]),
    > JSON.parse(row.data)));
0064 |   return {

```

L33-34: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L35: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L36: Returns this expression as the operation result; subsequent statements in this path are skipped.

L37: Handles this exception type and translates it into the following failure behavior.

L39: Returns this expression as the operation result; subsequent statements in this path are skipped.

L40: Runs cleanup whether the preceding operation succeeded or failed.

L41: Closes or continues the surrounding block, expression, or call.

L42-43: Defines the named JavaScript operation or callback; its body implements the following steps.

L44-45: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L46: Branches on the displayed condition; the accepted branch performs the following operation.

L47: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L48: Branches on the displayed condition; the accepted branch performs the following operation.

L49-52: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L53: Branches on the displayed condition; the accepted branch performs the following operation.

L54: Returns this expression as the operation result; subsequent statements in this path are skipped.

L55: Closes or continues the surrounding block, expression, or call.

L56: Defines the named JavaScript operation or callback; its body implements the following steps.

L57-58: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L59: Returns this expression as the operation result; subsequent statements in this path are skipped.

L60: Closes or continues the surrounding block, expression, or call.

L61: Defines the named JavaScript operation or callback; its body implements the following steps.

L62: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L63: Serializes structured values to JSON for transport, saveable state, or journal storage.

L64: Returns this expression as the operation result; subsequent statements in this path are skipped.

Source lines 65-91

```
0065 |     get: (kind, id) => current.get(JSON.stringify([kind, id])),
0066 |     list: kind => Array.from(current, ([key, value]) => JSON.parse(key)[0] === kind ? value :
    > null).filter(Boolean),
0067 |     put: (kind, id, data, audit) => {
0068 |
0069 |         const row = [kind, JSON.stringify(id), JSON.stringify(data), audit.at,
    > JSON.stringify(audit.actor), audit.action];
0070 |         const range = sheet.getRange(sheet.getLastRow() + 1, 1, 1, row.length);
0071 |
0072 |         range.setNumberFormat('@');
0073 |         range.setValues([row]);
0074 |         current.set(JSON.stringify([kind, id]), data);
0075 |     }
0076 | };
0077 | }
0078 |
0079 | function setupTracker() {
0080 |     const properties = PropertiesService.getScriptProperties();
0081 |     const spreadsheetId = properties.getProperty('SPREADSHEET_ID');
0082 |     if (!spreadsheetId) throw new Error('Set SPREADSHEET_ID and GOOGLE_WEB_CLIENT_ID in Script
    > Properties.');
```

```
0083 |     const spreadsheet = SpreadsheetApp.openById(spreadsheetId);
0084 |     const schemas = { TrackerUsers: ['email', 'role', 'profileId', 'name', 'active', 'googleSubject'],
    > TrackerRecords: ['kind', 'id', 'data', 'at', 'actor', 'action'] };
0085 |     Object.keys(schemas).forEach(name => {
0086 |         if (!spreadsheet.getSheetByName(name)) {
0087 |             const sheet = spreadsheet.insertSheet(name);
0088 |             sheet.appendRow(schemas[name]); sheet.setFrozenRows(1);
0089 |         }
0090 |     });
0091 | }
```

L65: Serializes structured values to JSON for transport, saveable state, or journal storage.

L66: Parses JSON text into values; surrounding validation determines whether they are acceptable.

L67: Defines the named JavaScript operation or callback; its body implements the following steps.

L69: Serializes structured values to JSON for transport, saveable state, or journal storage.

L70: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L72: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L73: Writes the displayed values through the journal storage adapter.

L74: Serializes structured values to JSON for transport, saveable state, or journal storage.

L75-77: Closes or continues the surrounding block, expression, or call.

L79: Defines the named JavaScript operation or callback; its body implements the following steps.

L80-81: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L82: Branches on the displayed condition; the accepted branch performs the following operation.

L83-84: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L85: Defines the named JavaScript operation or callback; its body implements the following steps.

L86: Branches on the displayed condition; the accepted branch performs the following operation.

L87: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L88: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L89-91: Closes or continues the surrounding block, expression, or call.

64. auth.js

backend/src/auth.js

Cryptographic Google ID-token validation.

24 source lines. SHA-256: 7b1fbf74b57afdb6

decodeJwtPart_ validates base64url characters and padding; decodeJwtJson_ decodes UTF-8 JSON.

verifyGoogleToken accepts a bounded three-part token with RS256 and a known kid, rejecting unsupported critical headers.

The public key comes from trusted Google certificates, not from token-supplied URLs. RSA size and exponent are checked; node-forge verifies SHA-256 over the original encoded header and payload.

Signature success is followed by validateIdentity. Failures become UNAUTHENTICATED rather than accepting untrusted claims. Tests load these functions in a JavaScript VM.

Source lines 1-24

```
0001 | function decodeJwtPart_(part) {
0002 |   if (!/^[A-Za-z0-9_-]+$/ .test(part) || part.length % 4 !== 1) throw new Error('Invalid JWT
    > encoding. ');
0003 |   return forge.util.decode64(part.replace(/-/g, '+').replace(/_/g, '/') + '='.repeat((4 -
    > part.length % 4) % 4));
0004 | }
0005 | function decodeJwtJson_(part) { return JSON.parse(forge.util.decodeUtf8(decodeJwtPart_(part))); }
0006 | function verifyGoogleToken(token, audience, certificates, nowSeconds) {
0007 |   try {
0008 |     if (typeof token !== 'string' || token.length > 16000 || token.split('.').length !== 3) throw
    > new Error();
0009 |     const parts = token.split('.');
0010 |     const header = decodeJwtJson_(parts[0]);
0011 |     if (!header || header.alg !== 'RS256' || header.crit || !header.kid ||
    > !Object.prototype.hasOwnProperty.call(certificates, header.kid)) throw new Error();
0012 |     const pem = certificates[header.kid];
0013 |     const publicKey = pem.includes('BEGIN CERTIFICATE') ?
    > forge.pki.certificateFromPem(pem).publicKey : forge.pki.publicKeyFromPem(pem);
0014 |     if (publicKey.n.bitLength() < 2048 || publicKey.e.toString() !== '65537') throw new Error();
0015 |     const digest = forge.md.sha256.create();
0016 |     digest.update(parts[0] + '.' + parts[1], 'utf8');
0017 |     const valid = publicKey.verify(digest.digest().bytes(), decodeJwtPart_(parts[2]), undefined, {
    > _parseAllDigestBytes: true });
0018 |     if (!valid) throw new Error();
0019 |     const claims = decodeJwtJson_(parts[1]);
0020 |     return validateIdentity(claims, audience, nowSeconds);
0021 |   } catch (_) {
0022 |     fault('UNAUTHENTICATED', 'Your Google session is invalid or expired. Sign in again. ');
0023 |   }
0024 | }
```

L1: Defines the named JavaScript operation or callback; its body implements the following steps.

L2: Branches on the displayed condition; the accepted branch performs the following operation.

L3: Returns this expression as the operation result; subsequent statements in this path are skipped.

L4: Closes or continues the surrounding block, expression, or call.

L5-6: Defines the named JavaScript operation or callback; its body implements the following steps.

L7: Starts an operation whose failures are handled by the following catch/finally blocks.

L8: Branches on the displayed condition; the accepted branch performs the following operation.

L9-10: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L11: Branches on the displayed condition; the accepted branch performs the following operation.

L12-13: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L14: Branches on the displayed condition; the accepted branch performs the following operation.

L15: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L16: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

- L17: Declares the displayed value or mutable state; its initializer supplies the initial contents.
- L18: Branches on the displayed condition; the accepted branch performs the following operation.
- L19: Declares the displayed value or mutable state; its initializer supplies the initial contents.
- L20: Returns this expression as the operation result; subsequent statements in this path are skipped.
- L21: Handles this exception type and translates it into the following failure behavior.
- L22: Rejects the operation with the displayed backend error code and message.
- L23-24: Closes or continues the surrounding block, expression, or call.

65. core.js

backend/src/core.js

Backend academic domain logic and authorization.

229 source lines. SHA-256: bb18d030f1e04e29

fault raises an error with a public code. required, numeric, and emailAddress validate and normalize values before storage.

validateIdentity checks token issuer, audience, timestamps, subject, and verified email. principalFor requires exactly one active allowlisted account, a recognized role, and matching optional Google subject.

createService receives storage, identity, and clock functions. Its helpers load active records, enforce admin or assigned-faculty access, cap JSON record size, and append audited writes.

CRUD dispatch handles student, faculty, and subject actions, immutable identifiers, active state, revisions, safe retries, and assigned-faculty constraints.

saveEnrollment validates distinct active student IDs, revision, and the 250-student limit. saveAttendance validates institution date, full roster, allowed statuses, and unique subject/date/slot.

Attendance reports count present and late as attended, absent in the denominator, and exclude excused. Historical roster snapshots prevent newly enrolled students being penalized for earlier sessions.

saveAssessment validates the full roster, finite mark limits, status/score consistency, revision, request ID, and publication completeness. Published assessments cannot be rewritten.

getMyProgress derives the student profile from the verified principal and exposes only that student attendance and published marks. Unknown actions raise VALIDATION.

Source lines 1-32

```
0001 | function fault(code, message) {
0002 |   const error = new Error(message);
0003 |   error.code = code;
0004 |   throw error;
0005 | }
0006 | function required(value, label, max = 160) {
0007 |   if (typeof value !== 'string' || !value.trim() || value.trim().length > max) {
0008 |     fault('VALIDATION', label + ' is required (maximum ' + max + ' characters).');
0009 |   }
0010 |   return value.trim();
0011 | }
0012 | function numeric(value, label, min, max) {
0013 |   if (!['number', 'string'].includes(typeof value) || (typeof value === 'string' && !value.trim())
    > || !Number.isFinite(Number(value)) || Number(value) < min || Number(value) > max) {
0014 |     fault('VALIDATION', label + ' must be between ' + min + ' and ' + max + '.');
0015 |   }
0016 |   return Number(value);
0017 | }
0018 | function emailAddress(value) {
0019 |   const email = required(value, 'Email', 254).toLowerCase();
0020 |   if (!/^[^\s@]+@[^\s@]+\.[^\s@]+$/\.test(email)) fault('VALIDATION', 'Enter a valid email.');
```

0032 | }

L1: Defines the named JavaScript operation or callback; its body implements the following steps.

L2: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L3: Assigns the displayed expression to a variable or named argument.

L4: Stops the current operation by raising the displayed error.

L5: Closes or continues the surrounding block, expression, or call.

L6: Defines the named JavaScript operation or callback; its body implements the following steps.

L7: Branches on the displayed condition; the accepted branch performs the following operation.

L8: Rejects the operation with the displayed backend error code and message.

L9: Closes or continues the surrounding block, expression, or call.

L10: Returns this expression as the operation result; subsequent statements in this path are skipped.

L11: Closes or continues the surrounding block, expression, or call.

L12: Defines the named JavaScript operation or callback; its body implements the following steps.

L13: Branches on the displayed condition; the accepted branch performs the following operation.

L14: Rejects the operation with the displayed backend error code and message.

L15: Closes or continues the surrounding block, expression, or call.

L16: Returns this expression as the operation result; subsequent statements in this path are skipped.

L17: Closes or continues the surrounding block, expression, or call.

L18: Defines the named JavaScript operation or callback; its body implements the following steps.

L19: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L20: Branches on the displayed condition; the accepted branch performs the following operation.

L21: Returns this expression as the operation result; subsequent statements in this path are skipped.

L22: Closes or continues the surrounding block, expression, or call.

L23: Defines the named JavaScript operation or callback; its body implements the following steps.

L24: Branches on the displayed condition; the accepted branch performs the following operation.

L25-28: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L29: Rejects the operation with the displayed backend error code and message.

L30: Closes or continues the surrounding block, expression, or call.

L31: Returns this expression as the operation result; subsequent statements in this path are skipped.

L32: Closes or continues the surrounding block, expression, or call.

Source lines 33-64

```
0033 | function principalFor(claims, users) {
0034 |   const matches = users.filter(u => String(u.email).toLowerCase() === claims.email.toLowerCase());
0035 |   const user = matches.length === 1 ? matches[0] : null;
0036 |   if (!user || String(user.active).toLowerCase() !== 'true' || ![ 'admin', 'faculty',
    > 'student' ].includes(user.role) ||
0037 |     (user.googleSubject && String(user.googleSubject) !== claims.sub)) {
0038 |     fault('FORBIDDEN', 'This account has not been approved. Contact your administrator.');
```

```
0039 |   }
0040 |   return { email: claims.email.toLowerCase(), role: user.role, profileId: String(user.profileId ||
    > ''), name: String(user.name || claims.name || ''), expiresAt: claims.exp };
0041 | }
0042 |
0043 | function createService(store, user, now = () => new Date().toISOString(), today = () =>
    > now().slice(0, 10)) {
0044 |   const all = kind => store.list(kind).filter(x => x.active !== false);
0045 |   const admin = () => { if (user.role !== 'admin') fault('FORBIDDEN', 'Administrator access
    > required.')} };
0046 |   const get = (kind, id) => {
0047 |     const value = store.get(kind, id);
0048 |     if (!value || value.active === false) fault('NOT_FOUND', 'Record not found or inactive.');
```

```
0049 |     return value;
0050 |   };
0051 |   const subjectFor = id => {
0052 |     const subject = get('subject', id);
0053 |     if (user.role === 'faculty' && subject.facultyId === user.profileId) return subject;
0054 |     if (user.role !== 'admin') fault('FORBIDDEN', 'You are not assigned to this subject.');
```



```

0055 |     return subject;
0056 |   };
0057 |   const save = (kind, id, data, action) => {
0058 |     const value = { ...data, updatedAt: now() };
0059 |     if (JSON.stringify(value).length > 40000) fault('CAPACITY', 'This record exceeds the supported
    > size. Contact your administrator.');
```

L33: Defines the named JavaScript operation or callback; its body implements the following steps.

L34: Transforms, selects, or indexes the displayed collection using its callback.

L35: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L36: Branches on the displayed condition; the accepted branch performs the following operation.

L37: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L38: Rejects the operation with the displayed backend error code and message.

L39: Closes or continues the surrounding block, expression, or call.

L40: Returns this expression as the operation result; subsequent statements in this path are skipped.

L41: Closes or continues the surrounding block, expression, or call.

L43: Defines the named JavaScript operation or callback; its body implements the following steps.

L44: Reads current projected journal records for this type or identifier.

L45-46: Defines the named JavaScript operation or callback; its body implements the following steps.

L47: Reads current projected journal records for this type or identifier.

L48: Branches on the displayed condition; the accepted branch performs the following operation.

L49: Returns this expression as the operation result; subsequent statements in this path are skipped.

L50: Closes or continues the surrounding block, expression, or call.

L51: Defines the named JavaScript operation or callback; its body implements the following steps.

L52: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L53-54: Branches on the displayed condition; the accepted branch performs the following operation.

L55: Returns this expression as the operation result; subsequent statements in this path are skipped.

L56: Closes or continues the surrounding block, expression, or call.

L57: Defines the named JavaScript operation or callback; its body implements the following steps.

L58: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L59: Branches on the displayed condition; the accepted branch performs the following operation.

L60: Writes the displayed values through the journal storage adapter.

L61: Returns this expression as the operation result; subsequent statements in this path are skipped.

L62: Closes or continues the surrounding block, expression, or call.

L63: Defines the named JavaScript operation or callback; its body implements the following steps.

L64: Reads current projected journal records for this type or identifier.

Source lines 65-96

```

0065 |     return (enrollment ? enrollment.studentIds : []).map(id => store.get('student', id)).filter(s =>
    > s && s.active !== false);
0066 |   };
0067 |   const minimal = s => ({ studentId: s.rollNumber, name: s.name });
0068 |   const validateRoster = (code, records) => {
0069 |     const members = roster(code);
0070 |     if (!members.length) fault('VALIDATION', 'No students are enrolled in this subject.');
```

```

0079 |     const students = new Map(roster(code).map(s => [s.rollNumber, s.name]));
0080 |     sessions.forEach(s => s.records.forEach(r => students.set(r.studentId, r.name)));
0081 |     return Array.from(students, ([studentId, name]) => {
0082 |         const entries = sessions.flatMap(s => s.records.filter(r => r.studentId === studentId &&
> r.status !== 'excused'));
0083 |         const present = entries.filter(r => r.status === 'present' || r.status === 'late').length;
0084 |         return { studentId, name, totalPresent: String(present), totalClasses: String(entries.length),
> percentage: entries.length ? (100 * present / entries.length).toFixed(1) : '' };
0085 |     });
0086 | };
0087 | function crud(action, request) {
0088 |     admin();
0089 |     const match = /^(add|update|delete)(Student|Faculty|Subject)$/.exec(action);
0090 |     if (!match) fault('VALIDATION', 'Unknown action.');
```

L65: Returns this expression as the operation result; subsequent statements in this path are skipped.

L66: Closes or continues the surrounding block, expression, or call.

L67: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L68: Defines the named JavaScript operation or callback; its body implements the following steps.

L69: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L70: Branches on the displayed condition; the accepted branch performs the following operation.

L71: Declares the named type; constructor properties define its stored data or dependencies.

L72: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L73: Declares the named type; constructor properties define its stored data or dependencies.

L74: Closes or continues the surrounding block, expression, or call.

L75: Returns this expression as the operation result; subsequent statements in this path are skipped.

L76: Closes or continues the surrounding block, expression, or call.

L77: Defines the named JavaScript operation or callback; its body implements the following steps.

L78-79: Transforms, selects, or indexes the displayed collection using its callback.

L80: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L81: Defines the named JavaScript operation or callback; its body implements the following steps.

L82-83: Transforms, selects, or indexes the displayed collection using its callback.

L84: Returns this expression as the operation result; subsequent statements in this path are skipped.

L85-86: Closes or continues the surrounding block, expression, or call.

L87: Defines the named JavaScript operation or callback; its body implements the following steps.

L88: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L89: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L90: Branches on the displayed condition; the accepted branch performs the following operation.

L91-94: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L95: Reads current projected journal records for this type or identifier.

L96: Branches on the displayed condition; the accepted branch performs the following operation.

Source lines 97-128

```

0097 |     if (operation !== 'add' && (!Number.isInteger(request.revision) || request.revision < 0))
> fault('VALIDATION', 'Invalid record revision.');
```

```

0103 |         save(kind, id, { ...old, active: false, revision: (old.revision || 0) + 1 }, action);
0104 |         return { message: type + ' deactivated. Historical records were retained.' };
0105 |     }
0106 |     const fields = {
0107 |         student: ['rollNumber', 'name', 'email', 'phone', 'department', 'batch', 'year', 'semester', 'section'
> , 'parentName', 'parentPhone', 'parentEmail'],
0108 |         faculty: ['employeeId', 'name', 'email', 'phone', 'department', 'designation', 'joiningDate'],
0109 |         subject: ['subjectCode', 'subjectName', 'facultyId', 'semester', 'totalClasses']
0110 |     }[kind];
0111 |     const data = {};
0112 |     fields.forEach(field => { data[field] = String(request[field] || '').trim(); if
> (data[field].length > 254) fault('VALIDATION', field + ' is too long.')});
0113 |     if (operation === 'update') {
0114 |         get(kind, id);
0115 |
0116 |         if (old.revision === request.revision + 1 && fields.every(field => old[field] ===
> data[field])) return { message: type + ' already saved.' };
0117 |         if ((old.revision || 0) !== request.revision) fault('CONFLICT', 'This record changed. Reload
> before saving. ');
0118 |     }
0119 |     required(data[kind === 'subject' ? 'subjectName' : 'name'], 'Name');
0120 |     if (kind === 'subject') {
0121 |         get('faculty', required(data.facultyId, 'Faculty'));
0122 |         if (!Number.isInteger(numeric(data.totalClasses, 'Planned classes', 1, 1000)))
> fault('VALIDATION', 'Planned classes must be a whole number. ');
0123 |         required(data.semester, 'Semester');
0124 |     } else {
0125 |         data.email = emailAddress(data.email);
0126 |         required(data.department, 'Department');
0127 |         if (all(kind).some(v => v[key] !== id && v.email.toLowerCase() === data.email))
> fault('CONFLICT', 'This email is already registered. ');
0128 |         if (data.parentEmail) emailAddress(data.parentEmail);

```

L97-99: Branches on the displayed condition; the accepted branch performs the following operation.

L100: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L101-102: Branches on the displayed condition; the accepted branch performs the following operation.

L103: Carries or checks the record version used to detect stale edits or matching retries.

L104: Returns this expression as the operation result; subsequent statements in this path are skipped.

L105: Closes or continues the surrounding block, expression, or call.

L106: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L107-110: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L111: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L112: Defines the named JavaScript operation or callback; its body implements the following steps.

L113: Branches on the displayed condition; the accepted branch performs the following operation.

L114: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L116-117: Branches on the displayed condition; the accepted branch performs the following operation.

L118: Closes or continues the surrounding block, expression, or call.

L119: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L120: Branches on the displayed condition; the accepted branch performs the following operation.

L121: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L122: Branches on the displayed condition; the accepted branch performs the following operation.

L123-124: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L125: Assigns the displayed expression to a variable or named argument.

L126: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L127-128: Branches on the displayed condition; the accepted branch performs the following operation.

Source lines 129-160

```

0129 |     }
0130 |     save(kind, id, { ...data, active: true, revision: (old?.revision || 0) + 1 }, action);
0131 |     return { message: type + ' saved.' };
0132 | }
0133 | function execute(request) {

```

```

0134 |     const action = required(request.action, 'Action', 60);
0135 |     if (user.role !== 'admin') get(user.role, user.profileId);
0136 |     switch (action) {
0137 |       case 'getUserRole': return { ...user };
0138 |       case 'getStudent': admin(); return { student: get('student', request.rollNumber) };
0139 |       case 'getFacultyMember': admin(); return { facultyMember: get('faculty', request.employeeId)
> };
0140 |       case 'getSubject': admin(); return { subject: get('subject', request.subjectCode) };
0141 |       case 'getStudents': admin(); return { students: all('student') };
0142 |       case 'getFaculty': admin(); return { faculty: all('faculty') };
0143 |       case 'getSubjects': {
0144 |         if (user.role === 'student') return { subjects: all('subject').filter(s =>
> (store.get('enrollment', s.subjectCode)?.studentIds || []).includes(user.profileId)) };
0145 |         return { subjects: all('subject').filter(s => user.role === 'admin' || s.facultyId ===
> user.profileId) };
0146 |       }
0147 |       case 'getDashboard': {
0148 |         admin();
0149 |         return { dashboard: { students: all('student').length, faculty: all('faculty').length,
> subjects: all('subject').length, departments: new Set(all('student').map(s => s.department)).size }
> };
0150 |       }
0151 |       case 'getRoster': {
0152 |         subjectFor(request.subjectCode);
0153 |         const enrollment = store.get('enrollment', request.subjectCode);
0154 |         return { students: roster(request.subjectCode).map(minimal), revision: enrollment?.revision
> || 0 };
0155 |       }
0156 |       case 'saveEnrollment': {
0157 |         admin(); get('subject', request.subjectCode);
0158 |         const old = store.get('enrollment', request.subjectCode);
0159 |         if (!Array.isArray(request.studentIds) || request.studentIds.length > 250 || new
> Set(request.studentIds).size !== request.studentIds.length) fault('VALIDATION', 'Select up to 250
> distinct students.');
```

L129: Closes or continues the surrounding block, expression, or call.

L130: Carries or checks the record version used to detect stale edits or matching retries.

L131: Returns this expression as the operation result; subsequent statements in this path are skipped.

L132: Closes or continues the surrounding block, expression, or call.

L133: Defines the named JavaScript operation or callback; its body implements the following steps.

L134: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L135-136: Branches on the displayed condition; the accepted branch performs the following operation.

L137-143: Selects the backend action shown here; subsequent statements handle that request.

L144: Branches on the displayed condition; the accepted branch performs the following operation.

L145: Returns this expression as the operation result; subsequent statements in this path are skipped.

L146: Closes or continues the surrounding block, expression, or call.

L147: Selects the backend action shown here; subsequent statements handle that request.

L148: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L149: Returns this expression as the operation result; subsequent statements in this path are skipped.

L150: Closes or continues the surrounding block, expression, or call.

L151: Selects the backend action shown here; subsequent statements handle that request.

L152: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L153: Reads current projected journal records for this type or identifier.

L154: Returns this expression as the operation result; subsequent statements in this path are skipped.

L155: Closes or continues the surrounding block, expression, or call.

L156: Selects the backend action shown here; subsequent statements handle that request.

L157: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L158: Reads current projected journal records for this type or identifier.

L159-160: Branches on the displayed condition; the accepted branch performs the following operation.

Source lines 161-192

```
0161 |         if (old && old.revision === request.revision + 1 &&  
    > JSON.stringify([...old.studentIds].sort()) === JSON.stringify([...request.studentIds].sort()))  
    > return { message: 'Enrollment already saved.', revision: old.revision };  
0162 |         if ((old?.revision || 0) !== request.revision) fault('CONFLICT', 'Enrollment changed. Reload  
    > and try again.');
```

```
0163 |         request.studentIds.forEach(id => get('student', id));  
0164 |         const enrollment = save('enrollment', request.subjectCode, { studentIds: request.studentIds,  
    > revision: (old?.revision || 0) + 1 }, action);  
0165 |         return { message: 'Enrollment saved.', revision: enrollment.revision };  
0166 |     }  
0167 |     case 'saveAttendance': {  
0168 |         subjectFor(request.subjectCode);  
0169 |         const date = required(request.date, 'Date', 10);  
0170 |         const parsed = new Date(date + 'T00:00:00Z');  
0171 |         if (!/^\d{4}-\d{2}-\d{2}$/.test(date) || !Number.isFinite(parsed.getTime()) ||  
    > parsed.toISOString().slice(0, 10) !== date || date > today()) fault('VALIDATION', 'Enter a valid  
    > class date that is not in the future.');
```

```
0172 |         const slot = required(request.slot, 'Session', 40);  
0173 |         const requestId = required(request.requestId, 'Request ID', 80);  
0174 |         const id = JSON.stringify([request.subjectCode, date, slot.toLowerCase()]);  
0175 |         const old = store.get('attendance', id);  
0176 |         if (old) {  
0177 |             if (old.requestId === requestId && Array.isArray(request.records) &&  
    > JSON.stringify(old.records.map(r => [r.studentId, r.status]).sort()) ===  
    > JSON.stringify(request.records.map(r => r && [r.studentId, r.status]).sort())) return { message:  
    > 'Attendance already submitted.' };  
0178 |             fault('CONFLICT', 'Attendance for this session has already been submitted.');
```

```
0179 |         }  
0180 |         const members = validateRoster(request.subjectCode, request.records);  
0181 |         if (request.records.some(r => ![ 'present', 'absent', 'late', 'excused' ].includes(r.status)))  
    > fault('VALIDATION', 'Choose a status for every student.');
```

```
0182 |         save('attendance', id, { subjectCode: request.subjectCode, date, slot, requestId, records:  
    > request.records.map(r => ({ studentId: r.studentId, name: members.find(s => s.rollNumber ===  
    > r.studentId).name, status: r.status })), action);  
0183 |         return { message: 'Attendance submitted.' };  
0184 |     }  
0185 |     case 'getAttendanceReport': subjectFor(request.subjectCode); return { report:  
    > attendanceReport(request.subjectCode) };  
0186 |     case 'getAssessments': {  
0187 |         subjectFor(request.subjectCode);  
0188 |         return { assessments: all('assessment').filter(a => a.subjectCode === request.subjectCode)  
    > };  
0189 |     }  
0190 |     case 'saveAssessment': {  
0191 |         subjectFor(request.subjectCode);  
0192 |         const id = required(request.assessmentId, 'Assessment ID', 80);
```

L161-162: Branches on the displayed condition; the accepted branch performs the following operation.

L163: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L164: Carries or checks the record version used to detect stale edits or matching retries.

L165: Returns this expression as the operation result; subsequent statements in this path are skipped.

L166: Closes or continues the surrounding block, expression, or call.

L167: Selects the backend action shown here; subsequent statements handle that request.

L168: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L169-170: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L171: Declares the named type; constructor properties define its stored data or dependencies.

L172-173: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L174: Serializes structured values to JSON for transport, saveable state, or journal storage.

L175: Reads current projected journal records for this type or identifier.

L176-177: Branches on the displayed condition; the accepted branch performs the following operation.

L178: Rejects the operation with the displayed backend error code and message.

L179: Closes or continues the surrounding block, expression, or call.

L180: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L181: Branches on the displayed condition; the accepted branch performs the following operation.

L182: Transforms, selects, or indexes the displayed collection using its callback.

L183: Returns this expression as the operation result; subsequent statements in this path are skipped.

L184: Closes or continues the surrounding block, expression, or call.

L185-186: Selects the backend action shown here; subsequent statements handle that request.

L187: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L188: Returns this expression as the operation result; subsequent statements in this path are skipped.

L189: Closes or continues the surrounding block, expression, or call.

L190: Selects the backend action shown here; subsequent statements handle that request.

L191: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L192: Declares the displayed value or mutable state; its initializer supplies the initial contents.

Source lines 193-224

```
0193 |         const old = store.get('assessment', id);
0194 |         if (old && old.subjectCode !== request.subjectCode) fault('FORBIDDEN', 'Assessment belongs
> to another subject.');
```

```
0195 |
0196 |         const requestId = required(request.requestId, 'Request ID', 80);
0197 |         if (!Number.isInteger(request.revision) || request.revision < 0 || typeof request.published
> !== 'boolean') fault('VALIDATION', 'Invalid assessment revision or publication state.');
```

```
0198 |         if (old && old.requestId === requestId) {
0199 |             const entries = rows => rows.map(m => m && [m.studentId, m.status, m.status === 'scored' ?
> Number(m.score) : null]).sort();
0200 |             if (typeof request.title !== 'string' || request.title.trim() !== old.title ||
> Number(request.maxMarks) !== old.maxMarks || request.published !== old.published ||
> !Array.isArray(request.marks) || JSON.stringify(entries(request.marks)) !==
> JSON.stringify(entries(old.marks))) fault('CONFLICT', 'This request ID was already used with
> different data.');
```

```
0201 |             return { message: 'Assessment already saved.', assessment: old };
0202 |         }
0203 |         if ((old?.revision || 0) !== request.revision) fault('CONFLICT', 'Assessment changed. Reload
> before saving.');
```

```
0204 |         if (old?.published) fault('CONFLICT', 'Published assessments are locked.');
```

```
0205 |         const title = required(request.title, 'Assessment title');
```

```
0206 |         const maxMarks = numeric(request.maxMarks, 'Maximum marks', 1, 10000);
0207 |         const members = validateRoster(request.subjectCode, request.marks);
0208 |         const marks = request.marks.map(m => {
0209 |             if (!['scored', 'absent', 'pending'].includes(m.status)) fault('VALIDATION', 'Invalid marks
> status.');
```

```
0210 |             if (request.published && m.status === 'pending') fault('VALIDATION', 'Complete all marks
> before publishing.');
```

```
0211 |             return { studentId: m.studentId, name: members.find(s => s.rollNumber ===
> m.studentId).name, status: m.status, score: m.status === 'scored' ? numeric(m.score, 'Marks', 0,
> maxMarks) : null };
0212 |         });
0213 |         const assessment = save('assessment', id, { assessmentId: id, subjectCode:
> request.subjectCode, title, maxMarks, marks, published: request.published === true, revision:
> (old?.revision || 0) + 1, requestId }, action);
0214 |         return { message: request.published ? 'Results published.' : 'Draft saved.', assessment };
0215 |     }
0216 |     case 'getMyProgress': {
0217 |         if (user.role !== 'student') fault('FORBIDDEN', 'Student access required.');
```

```
0218 |         const courses = store.list('subject').filter(s => (store.get('enrollment',
> s.subjectCode)?.studentIds || []).includes(user.profileId) || all('attendance').some(a =>
> a.subjectCode === s.subjectCode && a.records.some(r => r.studentId === user.profileId)));
0219 |         return { attendance: courses.map(s => ({ subjectCode: s.subjectCode, subjectName:
> s.subjectName, ...(attendanceReport(s.subjectCode).find(r => r.studentId === user.profileId) || {
> totalPresent: '0', totalClasses: '0', percentage: '' }))), results: all('assessment').filter(a =>
> a.published && a.marks.some(m => m.studentId === user.profileId)).map(a => {
0220 |             const mark = a.marks.find(m => m.studentId === user.profileId);
0221 |             return { assessmentId: a.assessmentId, title: a.title, subjectCode: a.subjectCode,
> maxMarks: a.maxMarks, score: mark.score, status: mark.status };
0222 |         }) };
0223 |     }
0224 |     default: return crud(action, request);
```

L193: Reads current projected journal records for this type or identifier.

L194: Branches on the displayed condition; the accepted branch performs the following operation.

L196: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L197-198: Branches on the displayed condition; the accepted branch performs the following operation.

L199: Transforms, selects, or indexes the displayed collection using its callback.

L200: Branches on the displayed condition; the accepted branch performs the following operation.

L201: Returns this expression as the operation result; subsequent statements in this path are skipped.

L202: Closes or continues the surrounding block, expression, or call.

L203-204: Branches on the displayed condition; the accepted branch performs the following operation.

L205-207: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L208: Defines the named JavaScript operation or callback; its body implements the following steps.

L209-210: Branches on the displayed condition; the accepted branch performs the following operation.

L211: Returns this expression as the operation result; subsequent statements in this path are skipped.

L212: Closes or continues the surrounding block, expression, or call.

L213: Carries or checks the record version used to detect stale edits or matching retries.

L214: Returns this expression as the operation result; subsequent statements in this path are skipped.

L215: Closes or continues the surrounding block, expression, or call.

L216: Selects the backend action shown here; subsequent statements handle that request.

L217: Branches on the displayed condition; the accepted branch performs the following operation.

L218: Reads current projected journal records for this type or identifier.

L219: Defines the named JavaScript operation or callback; its body implements the following steps.

L220: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L221: Returns this expression as the operation result; subsequent statements in this path are skipped.

L222: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L223: Closes or continues the surrounding block, expression, or call.

L224: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

Source lines 225-229

```

0225 |     }
0226 |   }
0227 |   return { execute };
0228 | }
0229 | if (typeof module !== 'undefined') module.exports = { createService, validateIdentity, principalFor,
    >   fault };

```

L225-226: Closes or continues the surrounding block, expression, or call.

L227: Returns this expression as the operation result; subsequent statements in this path are skipped.

L228: Closes or continues the surrounding block, expression, or call.

L229: Branches on the displayed condition; the accepted branch performs the following operation.

66. apps-script.test.cjs

backend/test/apps-script.test.cjs

Dispatcher and Sheet-adapter integration tests with Apps Script mocks.

65 source lines. SHA-256: 88d238aa52c2c668

Mocks Google certificate fetches, cache, lock, Sheet ranges, and ContentService. Runs the actual adapter through a JavaScript VM.

Tests inspect JSON envelopes and journal cells, including locking, setup, identity, scoping, and forged payload rejection. Mocks do not replace live deployment acceptance.

Source lines 1-32

```
0001 | const test = require('node:test');
0002 | const assert = require('node:assert/strict');
0003 | const vm = require('node:vm');
0004 | const fs = require('node:fs');
0005 | const { generateKeyPairSync, sign } = require('node:crypto');
0006 | const bundle = fs.readFileSync('dist/Code.gs', 'utf8');
0007 | const { publicKey, privateKey } = generateKeyPairSync('rsa', { modulusLength: 2048 });
0008 | function fixture() {
0009 |   let locked = false, writes = 0, fetches = 0;
0010 |   const sheets = {
0011 |     TrackerUsers: [['email', 'role', 'profileId', 'name', 'active', 'googleSubject'],
0012 |       > ['admin@example.edu', 'admin', '', 'Admin', true, 'admin-id']],
0013 |     TrackerRecords: [['kind', 'id', 'data', 'at', 'actor', 'action']]
0014 |   };
0015 |   const sheet = name => ({
0016 |     getDataRange: () => ({ getValues: () => structuredClone(sheets[name]) }),
0017 |     getLastRow: () => sheets[name].length,
0018 |     getRange: (row, column, count, width) => ({
0019 |       setNumberFormat(format) { assert.equal(format, '@'); },
0020 |       setValues(rows) { assert.equal(locked, true); assert.equal(column, 1); assert.equal(count, 1);
0021 |       > assert.equal(rows[0].length, width); sheets[name][row - 1] = rows[0]; writes++; }
0022 |     });
0023 |   });
0024 |   const certificates = JSON.stringify({ 'test-key': publicKey.export({ type: 'spki', format: 'pem'
0025 |   > }) });
0026 |   const context = vm.createContext({
0027 |     ContentService: { MimeType: { JSON: 'application/json' }, createTextOutput: text => ({
0028 |       > setMimeType: () => JSON.parse(text) }) },
0029 |     PropertiesService: { getScriptProperties: () => ({ getProperty: key => ({ SPREADSHEET_ID:
0030 |       > 'test-sheet', GOOGLE_WEB_CLIENT_ID: 'web-client' })[key] }) },
0031 |     CacheService: { getScriptCache: () => ({ get: () => certificates, put: () => {} }) },
0032 |     UrlFetchApp: { fetch: () => { fetches++; throw new Error('Unexpected external request'); } },
0033 |     LockService: { getScriptLock: () => ({ tryLock: () => { locked = true; return true; }, hasLock:
0034 |       > () => locked, releaseLock: () => { locked = false; } }) },
0035 |     SpreadsheetApp: { openById: id => { assert.equal(id, 'test-sheet'); assert.equal(locked, true);
0036 |       > return { getSheetByName: name => sheet(name) }; }, flush: () => {} },
0037 |     Utilities: { formatDate: () => new Date().toISOString().slice(0, 10) },
0038 |     Session: { getScriptTimeZone: () => 'Asia/Kolkata' }
0039 |   });
0040 | }
```

L1: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L2: Checks the displayed actual value against the expected behavior; failure fails the test.

L3-7: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L8: Defines the named JavaScript operation or callback; its body implements the following steps.

L9-10: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L11-12: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L13: Closes or continues the surrounding block, expression, or call.

L14: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L15-17: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L18-19: Checks the displayed actual value against the expected behavior; failure fails the test.

L20-21: Closes or continues the surrounding block, expression, or call.

L22: Serializes structured values to JSON for transport, saveable state, or journal storage.

L23: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L24: Parses JSON text into values; surrounding validation determines whether they are acceptable.

L25: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L26-29: Defines the named JavaScript operation or callback; its body implements the following steps.

L30-31: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L32: Closes or continues the surrounding block, expression, or call.

Source lines 33-64

```
0033 |   vm.runInContext(bundle, context);
0034 |   const now = Math.floor(Date.now() / 1000);
0035 |   const body = [{ alg: 'RS256', kid: 'test-key' }, { iss: 'https://accounts.google.com', aud:
    > 'web-client', sub: 'admin-id', email: 'admin@example.edu', email_verified: true, exp: now + 3600,
    > iat: now - 10 }].map(x => Buffer.from(JSON.stringify(x)).toString('base64url')).join('.');
0036 |   const idToken = body + '.' + sign('RSA-SHA256', Buffer.from(body),
    > privateKey).toString('base64url');
0037 |   return { context, sheets, post: data => context.doPost({ postData: { contents: JSON.stringify({
    > idToken, ...data }) } }), get writes() { return writes; }, get locked() { return locked; }, get
    > fetches() { return fetches; } };
0038 | }
0039 | test('complete dispatcher authenticates, persists an audited record and reads its projection', () =>
    > {
0040 |   const f = fixture();
0041 |   const result = f.post({ action: 'addStudent', rollNumber: '001', name: 'Student', email:
    > 'student@example.edu', department: 'CS' });
0042 |   assert.equal(result.status, 'success'); assert.equal(result.apiVersion, 2);
0043 |   assert.equal(f.writes, 1); assert.equal(f.locked, false); assert.equal(f.fetches, 0);
0044 |   assert.equal(f.sheets.TrackerRecords[1][1], '"001"');
0045 |   assert.equal(JSON.parse(f.sheets.TrackerRecords[1][4]), 'admin@example.edu');
0046 |   assert.equal(f.post({ action: 'getStudents' }).students[0].rollNumber, '001');
0047 | });
0048 | test('unauthenticated POST and every GET are denied without touching spreadsheet records', () => {
0049 |   const f = fixture();
0050 |   assert.equal(f.post({ action: 'getStudents', idToken: 'forged' }).code, 'UNAUTHENTICATED');
0051 |   assert.equal(f.context.doGet().code, 'METHOD_NOT_ALLOWED');
0052 |   assert.equal(f.writes, 0); assert.equal(f.locked, false);
0053 | });
0054 | test('malformed JSON is rejected and lock is released on domain validation errors', () => {
0055 |   const f = fixture();
0056 |   assert.equal(f.context.doPost({ postData: { contents: '{bad-json}' }).code, 'VALIDATION');
0057 |   assert.equal(f.post({ action: 'addStudent', rollNumber: '001' }).code, 'VALIDATION');
0058 |   assert.equal(f.locked, false); assert.equal(f.writes, 0);
0059 | });
0060 | test('spreadsheet identifiers preserve leading zeroes and cannot become executable formulas', () =>
    > {
0061 |   const f = fixture();
0062 |   assert.equal(f.post({ action: 'addStudent', rollNumber: '=IMPORTXML("x")', name: '=DANGER', email:
    > 's@example.edu', department: 'CS' }).status, 'success');
0063 |   assert.ok(!f.sheets.TrackerRecords[1].some(cell => /^[=+@%-]/.test(String(cell))));
0064 |   assert.equal(f.post({ action: 'getStudents' }).students[0].rollNumber, '=IMPORTXML("x")');
```

L33: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L34: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L35: Serializes structured values to JSON for transport, saveable state, or journal storage.

L36: Carries the Google identity token for server verification; the token itself does not assign a role.

L37: Returns this expression as the operation result; subsequent statements in this path are skipped.

L38: Closes or continues the surrounding block, expression, or call.

L39: Defines the named JavaScript operation or callback; its body implements the following steps.

L40-41: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L42-46: Checks the displayed actual value against the expected behavior; failure fails the test.

L47: Closes or continues the surrounding block, expression, or call.

L48: Defines the named JavaScript operation or callback; its body implements the following steps.

L49: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L50-52: Checks the displayed actual value against the expected behavior; failure fails the test.

L53: Closes or continues the surrounding block, expression, or call.

L54: Defines the named JavaScript operation or callback; its body implements the following steps.

L55: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L56-58: Checks the displayed actual value against the expected behavior; failure fails the test.

L59: Closes or continues the surrounding block, expression, or call.

L60: Defines the named JavaScript operation or callback; its body implements the following steps.

L61: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L62-64: Checks the displayed actual value against the expected behavior; failure fails the test.

Source lines 65-65

0065 | });

L65: Closes or continues the surrounding block, expression, or call.

67. auth.test.cjs

backend/test/auth.test.cjs

Identity-verification tests with independently generated RSA signatures.

47 source lines. SHA-256: 60d1327345278740

Node crypto signs test tokens; the application node-forge verifier must accept valid ones and reject altered or invalid tokens.

Coverage includes Unicode claims, bad signatures/headers, timestamps, audience checks, and allowlist rules. This avoids testing a verifier only against its own signing implementation.

Source lines 1-32

```
0001 | const test = require('node:test');
0002 | const assert = require('node:assert/strict');
0003 | const vm = require('node:vm');
0004 | const fs = require('node:fs');
0005 | const { generateKeyPairSync, sign } = require('node:crypto');
0006 |
0007 | const context = vm.createContext({ console });
0008 | vm.runInContext(fs.readFileSync('dist/Code.gs', 'utf8'), context);
0009 | const { privateKey, publicKey } = generateKeyPairSync('rsa', { modulusLength: 2048 });
0010 | const pem = publicKey.export({ type: 'spki', format: 'pem' });
0011 | const now = Math.floor(Date.now() / 1000);
0012 | const claims = { iss: 'https://accounts.google.com', aud: 'web-client', sub: 'google-id', email:
    > 'student@example.edu', email_verified: true, iat: now - 10, exp: now + 3600 };
0013 | function token(payload = claims, header = { alg: 'RS256', kid: 'test-key' }, key = privateKey) {
0014 |   const body = [header, payload].map(x =>
    > Buffer.from(JSON.stringify(x)).toString('base64url')).join('.');
0015 |   return body + '.' + sign('RSA-SHA256', Buffer.from(body), key).toString('base64url');
0016 | }
0017 | function verify(jwt) { return context.verifyGoogleToken(jwt, 'web-client', { 'test-key': pem },
    > now); }
0018 | test('generated Apps Script bundle verifies an independently signed RSA token', () => {
0019 |   assert.equal(verify(token()).sub, claims.sub);
0020 | });
0021 | test('signature tampering, wrong signing key and algorithm confusion are rejected', () => {
0022 |   const valid = token();
0023 |   const parts = valid.split('.');
0024 |   parts[1] = Buffer.from(JSON.stringify({ ...claims, email: 'admin@example.edu'
    > })).toString('base64url');
0025 |   const otherKey = generateKeyPairSync('rsa', { modulusLength: 2048 }).privateKey;
0026 |   for (const jwt of [parts.join('.'), token(claims, { alg: 'RS256', kid: 'test-key' }, otherKey),
    > token(claims, { alg: 'none', kid: 'test-key' }), token(claims, { alg: 'HS256', kid: 'test-key' }),
    > token(claims, { alg: 'RS256', kid: 'unknown' }), 'not-a-token']) {
0027 |     assert.throws(() => verify(jwt), e => e.code === 'UNAUTHENTICATED');
0028 |   }
0029 | });
0030 | test('signed but invalid audience, issuer and time claims are rejected', () => {
0031 |   for (const override of [{ aud: 'other-client' }, { iss: 'evil' }, { exp: now }, { exp: undefined
    > }, { iat: now + 100 }, { nbf: now + 100 }, { email_verified: false }]) {
0032 |     assert.throws(() => verify(token({ ...claims, ...override })), e => e.code ===
    > 'UNAUTHENTICATED');
```

L1: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L2: Checks the displayed actual value against the expected behavior; failure fails the test.

L3-5: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L7: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L8: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L9-12: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L13: Defines the named JavaScript operation or callback; its body implements the following steps.

L14: Serializes structured values to JSON for transport, saveable state, or journal storage.

L15: Returns this expression as the operation result; subsequent statements in this path are skipped.

L16: Closes or continues the surrounding block, expression, or call.

L17-18: Defines the named JavaScript operation or callback; its body implements the following steps.

L19: Checks the displayed actual value against the expected behavior; failure fails the test.

L20: Closes or continues the surrounding block, expression, or call.

L21: Defines the named JavaScript operation or callback; its body implements the following steps.

L22-23: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L24: Serializes structured values to JSON for transport, saveable state, or journal storage.

L25: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L26: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L27: Checks the displayed actual value against the expected behavior; failure fails the test.

L28-29: Closes or continues the surrounding block, expression, or call.

L30: Defines the named JavaScript operation or callback; its body implements the following steps.

L31: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L32: Checks the displayed actual value against the expected behavior; failure fails the test.

Source lines 33-47

```
0033 |   }
0034 | });
0035 | test('Google-style X509 certificates and Unicode claims are supported', () => {
0036 |   const forge = require('node-forge');
0037 |   const certificate = forge.pki.createCertificate();
0038 |   certificate.publicKey = forge.pki.publicKeyFromPem(pem);
0039 |   certificate.serialNumber = '01';
0040 |   certificate.validity.notBefore = new Date(now * 1000);
0041 |   certificate.validity.notAfter = new Date((now + 86400) * 1000);
0042 |   certificate.setSubject([{ name: 'commonName', value: 'Test signing key' }]);
0043 |   certificate.setIssuer([{ name: 'commonName', value: 'Test signing key' }]);
0044 |   certificate.sign(forge.pki.privateKeyFromPem(privateKey.export({ type: 'pkcs8', format: 'pem' })),
    >   forge.md.sha256.create());
0045 |   const jwt = token({ ...claims, name: ' ' });
0046 |   assert.equal(context.verifyGoogleToken(jwt, 'web-client', { 'test-key':
    >   forge.pki.certificateToPem(certificate) }, now).name, ' ');
0047 | });
```

L33-34: Closes or continues the surrounding block, expression, or call.

L35: Defines the named JavaScript operation or callback; its body implements the following steps.

L36-37: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L38-41: Assigns the displayed expression to a variable or named argument.

L42-44: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L45: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L46: Checks the displayed actual value against the expected behavior; failure fails the test.

L47: Closes or continues the surrounding block, expression, or call.

68. core.test.cjs

backend/test/core.test.cjs

Domain-rule tests using an in-memory journal store.

176 source lines. SHA-256: 1803d1406459dbcb

Tests create users, records, and subjects to exercise authorization, CRUD revisions, attendance rosters, retry rules, publication, and student privacy.

The fake store isolates business behavior from Google quotas and real network availability. Inspect each test title and assertion below for the exact expected behavior.

Source lines 1-32

```
0001 | const test = require('node:test');
0002 | const assert = require('node:assert/strict');
0003 | const { createService, principalFor, validateIdentity } = require('../src/core.js');
0004 |
0005 | function fixture() {
0006 |   const records = new Map();
0007 |   const events = [];
0008 |   const key = (kind, id) => JSON.stringify([kind, id]);
0009 |   const store = {
0010 |     get: (kind, id) => records.get(key(kind, id)),
0011 |     list: kind => [...records].filter([k] => JSON.parse(k)[0] === kind).map([, v] => v),
0012 |     put(kind, id, data, audit) { records.set(key(kind, id), structuredClone(data)); events.push({
0013 |       > kind, id, data, audit }); }
0014 |   };
0015 |   const service = (role, profileId = '') => createService(store, { role, profileId, email: role +
0016 |     > '@example.edu' }, () => '2026-10-01T12:00:00.000Z');
0017 |   const admin = service('admin');
0018 |   admin.execute({ action: 'addFaculty', employeeId: 'F1', name: 'Teacher', email:
0019 |     > 'teacher@example.edu', department: 'CS' });
0020 |   ['S1', 'S2', 'S3'].forEach(id => admin.execute({ action: 'addStudent', rollNumber: id, name: id,
0021 |     > email: id + '@example.edu', department: 'CS' }));
0022 |   admin.execute({ action: 'addSubject', subjectCode: 'CS101', subjectName: 'Computing', facultyId:
0023 |     > 'F1', semester: '1', totalClasses: '40' });
0024 |   admin.execute({ action: 'saveEnrollment', subjectCode: 'CS101', studentIds: ['S1', 'S2'],
0025 |     > revision: 0 });
0026 |   return { store, events, service, admin, faculty: service('faculty', 'F1'), student:
0027 |     > service('student', 'S1') };
0028 | }
0029 | function fails(code, operation) { assert.throws(operation, error => error.code === code); }
0030 | function attendance(overrides = {}) {
0031 |   return { action: 'saveAttendance', subjectCode: 'CS101', date: '2026-09-30', slot: 'Period 1',
0032 |     > requestId: 'attendance-1', records: [{ studentId: 'S1', status: 'present' }, { studentId: 'S2',
0033 |     > status: 'absent' }], ...overrides };
0034 | }
0035 | function assessment(overrides = {}) {
0036 |   return { action: 'saveAssessment', subjectCode: 'CS101', assessmentId: 'A1', title: 'Quiz',
0037 |     > maxMarks: 20, revision: 0, requestId: 'assessment-1', published: false, marks: [{ studentId: 'S1',
0038 |     > status: 'scored', score: 0 }, { studentId: 'S2', status: 'pending' }], ...overrides };
0039 | }
0040 | test('server roles deny student and faculty administration or another teacher's subject', () => {
0041 |   const { student, faculty, service } = fixture();
0042 |   for (const action of ['getStudents', 'getFaculty', 'getDashboard', 'saveEnrollment',
0043 |     > 'deleteStudent']) {
```

L1: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L2: Checks the displayed actual value against the expected behavior; failure fails the test.

L3: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L5: Defines the named JavaScript operation or callback; its body implements the following steps.

L6-7: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L8: Serializes structured values to JSON for transport, saveable state, or journal storage.

L9: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L10: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L11: Parses JSON text into values; surrounding validation determines whether they are acceptable.

L12: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L13: Closes or continues the surrounding block, expression, or call.

L14-15: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L16-18: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L19: Carries or checks the record version used to detect stale edits or matching retries.

L20: Returns this expression as the operation result; subsequent statements in this path are skipped.

L21: Closes or continues the surrounding block, expression, or call.

L22-23: Defines the named JavaScript operation or callback; its body implements the following steps.

L24: Returns this expression as the operation result; subsequent statements in this path are skipped.

L25: Closes or continues the surrounding block, expression, or call.

L26: Defines the named JavaScript operation or callback; its body implements the following steps.

L27: Returns this expression as the operation result; subsequent statements in this path are skipped.

L28: Closes or continues the surrounding block, expression, or call.

L30: Defines the named JavaScript operation or callback; its body implements the following steps.

L31: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L32: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

Source lines 33-64

```
0033 |     fails('FORBIDDEN', () => student.execute({ action, subjectCode: 'CS101', rollNumber: 'S2' }));
0034 |     fails('FORBIDDEN', () => faculty.execute({ action, subjectCode: 'CS101', rollNumber: 'S2' }));
0035 | }
0036 | fails('FORBIDDEN', () => student.execute({ action: 'getRoster', subjectCode: 'CS101' }));
0037 | fails('NOT_FOUND', () => service('faculty', 'OTHER').execute({ action: 'getSubjects' }));
0038 | fails('FORBIDDEN', () => faculty.execute({ action: 'getMyProgress' }));
0039 | });
0040 | test('faculty roster is explicit enrollment and excludes private contact fields', () => {
0041 |   const { faculty } = fixture();
0042 |   const result = faculty.execute({ action: 'getRoster', subjectCode: 'CS101' });
0043 |   assert.deepEqual(result.students, [{ studentId: 'S1', name: 'S1' }, { studentId: 'S2', name: 'S2'
    > }]);
0044 | });
0045 | test('enrollment rejects duplicates, nonexistent students and stale writes', () => {
0046 |   const { admin } = fixture();
0047 |   fails('VALIDATION', () => admin.execute({ action: 'saveEnrollment', subjectCode: 'CS101',
    > studentIds: ['S1', 'S1'], revision: 1 }));
0048 |   fails('NOT_FOUND', () => admin.execute({ action: 'saveEnrollment', subjectCode: 'CS101',
    > studentIds: ['MISSING'], revision: 1 }));
0049 |   fails('CONFLICT', () => admin.execute({ action: 'saveEnrollment', subjectCode: 'CS101',
    > studentIds: ['S3'], revision: 0 }));
0050 | });
0051 | test('attendance retries do not append a second session and alternate submissions conflict', () => {
0052 |   const { faculty, store, events } = fixture();
0053 |   faculty.execute(attendance());
0054 |   const writes = events.length;
0055 |   faculty.execute(attendance());
0056 |   assert.equal(events.length, writes);
0057 |   assert.equal(store.list('attendance').length, 1);
0058 |   fails('CONFLICT', () => faculty.execute(attendance({ requestId: 'other-request', slot: 'period 1'
    > })));
0059 | });
0060 | test('attendance validates complete distinct roster and real nonfuture dates', () => {
0061 |   const { faculty } = fixture();
0062 |   for (const date of ['2026-02-30', '2026-13-01', '2026-10-02', 'yesterday']) fails('VALIDATION', ()
    > => faculty.execute(attendance({ date })));
0063 |   fails('CONFLICT', () => faculty.execute(attendance({ records: [{ studentId: 'S1', status:
    > 'present' }] })));
0064 |   fails('CONFLICT', () => faculty.execute(attendance({ records: [{ studentId: 'S1', status:
    > 'present' }, { studentId: 'S1', status: 'absent' }] })));

```

L33-34: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L35: Closes or continues the surrounding block, expression, or call.

L36-38: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L39: Closes or continues the surrounding block, expression, or call.

L40: Defines the named JavaScript operation or callback; its body implements the following steps.

L41-42: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L43: Checks the displayed actual value against the expected behavior; failure fails the test.

L44: Closes or continues the surrounding block, expression, or call.

L45: Defines the named JavaScript operation or callback; its body implements the following steps.

L46: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L47-49: Carries or checks the record version used to detect stale edits or matching retries.

L50: Closes or continues the surrounding block, expression, or call.

L51: Defines the named JavaScript operation or callback; its body implements the following steps.

L52: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L53: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L54: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L55: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L56-57: Checks the displayed actual value against the expected behavior; failure fails the test.

L58: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L59: Closes or continues the surrounding block, expression, or call.

L60: Defines the named JavaScript operation or callback; its body implements the following steps.

L61: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L62-64: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

Source lines 65-96

```
0065 |   fails('VALIDATION', () => faculty.execute(attendance({ records: [{ studentId: 'S1', status:
    > 'unknown' }, { studentId: 'S2', status: 'present' }] })));
0066 | });
0067 | test('attendance percentage counts late, excludes excused and never fabricates a zero percentage',
    > () => {
0068 |   const { faculty } = fixture();
0069 |   const initially = faculty.execute({ action: 'getAttendanceReport', subjectCode: 'CS101' }).report;
0070 |   assert.equal(initially[0].percentage, '');
0071 |   faculty.execute(attendance());
0072 |   faculty.execute(attendance({ slot: 'Period 2', requestId: 'attendance-2', records: [{ studentId:
    > 'S1', status: 'late' }, { studentId: 'S2', status: 'excused' }] })));
0073 |   const report = faculty.execute({ action: 'getAttendanceReport', subjectCode: 'CS101' }).report;
0074 |   assert.deepEqual(report[0], { studentId: 'S1', name: 'S1', totalPresent: '2', totalClasses: '2',
    > percentage: '100.0' });
0075 |   assert.equal(report[1].totalClasses, '1'); assert.equal(report[1].percentage, '0.0');
0076 | });
0077 | test('newly enrolled students are not penalized for sessions they were never on the roster for', ()
    > => {
0078 |   const { admin, faculty } = fixture();
0079 |   faculty.execute(attendance());
0080 |   admin.execute({ action: 'saveEnrollment', subjectCode: 'CS101', studentIds: ['S1', 'S2', 'S3'],
    > revision: 1 });
0081 |   const report = faculty.execute({ action: 'getAttendanceReport', subjectCode: 'CS101' }).report;
0082 |   assert.equal(report.find(r => r.studentId === 'S3').totalClasses, '0');
0083 | });
0084 | test('draft marks are private, zero is a score, publishing requires complete entries', () => {
0085 |   const { faculty, student } = fixture();
0086 |   faculty.execute(assessment());
0087 |   assert.equal(student.execute({ action: 'getMyProgress' }).results.length, 0);
0088 |   fails('VALIDATION', () => faculty.execute(assessment({ revision: 1, requestId: 'publish-1',
    > published: true })));
0089 |   faculty.execute(assessment({ revision: 1, requestId: 'publish-2', published: true, marks: [{
    > studentId: 'S1', status: 'scored', score: 0 }, { studentId: 'S2', status: 'absent' }] })));
0090 |   const results = student.execute({ action: 'getMyProgress' }).results;
0091 |   assert.equal(results.length, 1); assert.equal(results[0].score, 0);
0092 |   assert.equal('marks' in results[0], false); assert.equal('studentId' in results[0], false);
0093 | });
0094 | test('marks reject invalid scores, stale edits and edits to published results', () => {
0095 |   const { faculty } = fixture();
```



```
0096 |   for (const score of [-1, 21, null, '', Infinity, true, []]) fails('VALIDATION', () =>
    > faculty.execute(assessment({ marks: [{ studentId: 'S1', status: 'scored', score }, { studentId:
    > 'S2', status: 'absent' }] })));
```

L65: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L66: Closes or continues the surrounding block, expression, or call.

L67: Defines the named JavaScript operation or callback; its body implements the following steps.

L68-69: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L70: Checks the displayed actual value against the expected behavior; failure fails the test.

L71-72: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L73: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L74-75: Checks the displayed actual value against the expected behavior; failure fails the test.

L76: Closes or continues the surrounding block, expression, or call.

L77: Defines the named JavaScript operation or callback; its body implements the following steps.

L78: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L79: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L80: Carries or checks the record version used to detect stale edits or matching retries.

L81: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L82: Checks the displayed actual value against the expected behavior; failure fails the test.

L83: Closes or continues the surrounding block, expression, or call.

L84: Defines the named JavaScript operation or callback; its body implements the following steps.

L85: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L86: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L87: Checks the displayed actual value against the expected behavior; failure fails the test.

L88-89: Carries or checks the record version used to detect stale edits or matching retries.

L90: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L91-92: Checks the displayed actual value against the expected behavior; failure fails the test.

L93: Closes or continues the surrounding block, expression, or call.

L94: Defines the named JavaScript operation or callback; its body implements the following steps.

L95: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L96: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

Source lines 97-128

```
0097 |   faculty.execute(assessment());
0098 |   fails('CONFLICT', () => faculty.execute(assessment({ requestId: 'stale' })));
0099 |   faculty.execute(assessment({ revision: 1, requestId: 'publish', published: true, marks: [{
    > studentId: 'S1', status: 'scored', score: 10 }, { studentId: 'S2', status: 'absent' }] })));
0100 |   fails('CONFLICT', () => faculty.execute(assessment({ revision: 2, requestId: 'change' })));
0101 | });
0102 | test('assessment retry is idempotent and student records stay isolated', () => {
0103 |   const { faculty, service, events } = fixture();
0104 |   const request = assessment({ published: true, marks: [{ studentId: 'S1', status: 'scored', score:
    > 8 }, { studentId: 'S2', status: 'scored', score: 19 }] });
0105 |   faculty.execute(request); const count = events.length;
0106 |   faculty.execute(request); assert.equal(events.length, count);
0107 |   assert.equal(service('student', 'S1').execute({ action: 'getMyProgress', studentId: 'S2'
    > }).results[0].score, 8);
0108 | });
0109 | test('deactivation preserves history and blocks access to inactive profiles', () => {
0110 |   const { admin, faculty, store, student } = fixture();
0111 |   faculty.execute(attendance());
0112 |   admin.execute({ action: 'deleteStudent', rollNumber: 'S1', revision: 1 });
0113 |   assert.equal(store.get('student', 'S1').active, false);
0114 |   assert.equal(admin.execute({ action: 'getStudents' }).students.length, 2);
0115 |   assert.equal(faculty.execute({ action: 'getAttendanceReport', subjectCode: 'CS101'
    > }).report.find(r => r.studentId === 'S1').totalPresent, '1');
0116 |   fails('NOT_FOUND', () => student.execute({ action: 'getMyProgress' }));
0117 |   fails('CONFLICT', () => admin.execute({ action: 'deleteFaculty', employeeId: 'F1', revision: 1
    > }));
0118 | });
```



```

0119 | test('dashboard counts come from active records and every mutation has an audit actor', () => {
0120 |   const { admin, events } = fixture();
0121 |   assert.deepEqual(admin.execute({ action: 'getDashboard' }).dashboard, { students: 3, faculty: 1,
    > subjects: 1, departments: 1 });
0122 |   assert.ok(events.every(e => e.audit.actor === 'admin@example.edu' && e.audit.action &&
    > e.audit.at));
0123 | });
0124 | test('user allowlist fails closed on inactive, duplicated, unrecognized or subject-mismatched
    > users', () => {
0125 |   const claims = { email: 'student@example.edu', sub: 'google-id', name: 'Student', exp: 1000 };
0126 |   const user = { email: claims.email, role: 'student', profileId: 'S1', active: true };
0127 |   assert.equal(principalFor(claims, [user]).profileId, 'S1');
0128 |   for (const users of [[], [user, user], [{ ...user, active: false }], [{ ...user, role: 'owner' }],
    > [{ ...user, googleSubject: 'other-id' }]]) fails('FORBIDDEN', () => principalFor(claims, users));

```

L97-98: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L99-100: Carries or checks the record version used to detect stale edits or matching retries.

L101: Closes or continues the surrounding block, expression, or call.

L102: Defines the named JavaScript operation or callback; its body implements the following steps.

L103-104: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L105: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L106-107: Checks the displayed actual value against the expected behavior; failure fails the test.

L108: Closes or continues the surrounding block, expression, or call.

L109: Defines the named JavaScript operation or callback; its body implements the following steps.

L110: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L111: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L112: Carries or checks the record version used to detect stale edits or matching retries.

L113-115: Checks the displayed actual value against the expected behavior; failure fails the test.

L116: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L117: Carries or checks the record version used to detect stale edits or matching retries.

L118: Closes or continues the surrounding block, expression, or call.

L119: Defines the named JavaScript operation or callback; its body implements the following steps.

L120: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L121-122: Checks the displayed actual value against the expected behavior; failure fails the test.

L123: Closes or continues the surrounding block, expression, or call.

L124: Defines the named JavaScript operation or callback; its body implements the following steps.

L125-126: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L127: Checks the displayed actual value against the expected behavior; failure fails the test.

L128: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

Source lines 129-160

```

0129 | });
0130 | test('identity claims reject wrong audience/issuer, expiration, future issue time or unverified
    > email', () => {
0131 |   const claims = { iss: 'https://accounts.google.com', aud: 'web-client', sub: 'google-id', email:
    > 'student@example.edu', email_verified: true, exp: 2000, iat: 900 };
0132 |   assert.equal(validateIdentity(claims, 'web-client', 1000), claims);
0133 |   for (const override of [{ aud: 'attacker' }, { iss: 'evil' }, { exp: 1000 }, { iat: 1100 }, {
    > email_verified: false }, { exp: undefined }]) fails('UNAUTHENTICATED', () => validateIdentity({
    > ...claims, ...override }, 'web-client', 1000));
0134 | });
0135 |
0136 | test('profile updates and deactivation detect conflicts and retry committed revisions safely', () =>
    > {
0137 |   const { admin, store, events } = fixture();
0138 |   const student = store.get('student', 'S1');
0139 |   const update = { ...student, action: 'updateStudent', name: 'Updated Name', revision: 1 };
0140 |   admin.execute(update);
0141 |   const writes = events.length;
0142 |   admin.execute(update);
0143 |   assert.equal(events.length, writes);

```

```

0144 |   fails('CONFLICT', () => admin.execute({ ...update, name: 'Stale Writer' }));
0145 |   fails('CONFLICT', () => admin.execute({ action: 'deleteStudent', rollNumber: 'S1', revision: 1
> }));
0146 |   admin.execute({ action: 'deleteStudent', rollNumber: 'S1', revision: 2 });
0147 |   admin.execute({ action: 'deleteStudent', rollNumber: 'S1', revision: 2 });
0148 |   assert.equal(store.get('student', 'S1').revision, 3);
0149 | });
0150 | test('enrollment retries acknowledge a previously committed identical selection', () => {
0151 |   const { admin, events } = fixture();
0152 |   const request = { action: 'saveEnrollment', subjectCode: 'CS101', studentIds: ['S1', 'S2', 'S3'],
> revision: 1 };
0153 |   admin.execute(request); const writes = events.length;
0154 |   admin.execute(request); assert.equal(events.length, writes);
0155 | });
0156 | test('retry operation identifiers cannot hide a changed attendance or marks payload', () => {
0157 |   const { faculty } = fixture();
0158 |   faculty.execute(attendance());
0159 |   fails('CONFLICT', () => faculty.execute(attendance({ records: [{ studentId: 'S1', status: 'absent'
> }, { studentId: 'S2', status: 'absent' }] })));
0160 |   faculty.execute(assessment());

```

L129: Closes or continues the surrounding block, expression, or call.

L130: Defines the named JavaScript operation or callback; its body implements the following steps.

L131: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L132: Checks the displayed actual value against the expected behavior; failure fails the test.

L133: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L134: Closes or continues the surrounding block, expression, or call.

L136: Defines the named JavaScript operation or callback; its body implements the following steps.

L137: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L138: Reads current projected journal records for this type or identifier.

L139: Carries or checks the record version used to detect stale edits or matching retries.

L140: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L141: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L142: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L143: Checks the displayed actual value against the expected behavior; failure fails the test.

L144: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L145-147: Carries or checks the record version used to detect stale edits or matching retries.

L148: Checks the displayed actual value against the expected behavior; failure fails the test.

L149: Closes or continues the surrounding block, expression, or call.

L150: Defines the named JavaScript operation or callback; its body implements the following steps.

L151: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L152: Carries or checks the record version used to detect stale edits or matching retries.

L153: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L154: Checks the displayed actual value against the expected behavior; failure fails the test.

L155: Closes or continues the surrounding block, expression, or call.

L156: Defines the named JavaScript operation or callback; its body implements the following steps.

L157: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L158-160: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

Source lines 161-176

```

0161 |   fails('CONFLICT', () => faculty.execute(assessment({ title: 'Different title' })));
0162 | });
0163 | test('faculty cannot read or write another faculty member's assigned subject', () => {
0164 |   const { admin, service } = fixture();
0165 |   admin.execute({ action: 'addFaculty', employeeId: 'F2', name: 'Other teacher', email:
> 'other@example.edu', department: 'CS' });
0166 |   const other = service('faculty', 'F2');
0167 |   assert.equal(other.execute({ action: 'getSubjects' }).subjects.length, 0);
0168 |   for (const action of ['getRoster', 'getAttendanceReport', 'getAssessments']) fails('FORBIDDEN', ()
> => other.execute({ action, subjectCode: 'CS101' }));

```

```

0169 |   fails('FORBIDDEN', () => other.execute(attendance()));
0170 |   fails('FORBIDDEN', () => other.execute(assessment()));
0171 | });
0172 | test('institution-local date allows attendance after local midnight before UTC midnight', () => {
0173 |   const { store } = fixture();
0174 |   const faculty = createService(store, { role: 'faculty', profileId: 'F1', email:
    > 'teacher@example.edu' }, () => '2026-09-30T20:00:00.000Z', () => '2026-10-01');
0175 |   assert.equal(faculty.execute(attendance({ date: '2026-10-01' })).message, 'Attendance
    > submitted.');
```

L161: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L162: Closes or continues the surrounding block, expression, or call.

L163: Defines the named JavaScript operation or callback; its body implements the following steps.

L164: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L165: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L166: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L167: Checks the displayed actual value against the expected behavior; failure fails the test.

L168-170: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L171: Closes or continues the surrounding block, expression, or call.

L172: Defines the named JavaScript operation or callback; its body implements the following steps.

L173-174: Declares the displayed value or mutable state; its initializer supplies the initial contents.

L175: Checks the displayed actual value against the expected behavior; failure fails the test.

L176: Closes or continues the surrounding block, expression, or call.

69. build.gradle.kts

build.gradle.kts

Root Gradle plugin declaration.

4 source lines. SHA-256: 846e90c2c4aa5bee

Version-catalog plugin aliases are declared with `apply false` so subprojects can apply the chosen versions. This file does not define application screens.

Source lines 1-4

```
0001 | plugins {  
0002 |     alias(libs.plugins.android.application) apply false  
0003 |     alias(libs.plugins.kotlin.compose) apply false  
0004 | }
```

L1-3: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L4: Closes or continues the surrounding block, expression, or call.

70. STAGING_CHECKLIST.md

docs/STAGING_CHECKLIST.md

Live deployment and acceptance checklist.

48 source lines. SHA-256: 54ba408bfea6ef02

Lists configuration, role, security, academic, export, and deployment checks that must be run against a real institution setup. Checklist text is a requirement, not proof of completion.

Complete document text

001 # Staging acceptance checklist

003 Run against a private staging spreadsheet and three dedicated test accounts.

004 Record the deployment version, app build, device/API level and results before release.

006 ## Identity and permissions

007 - Approved administrator signs in; an unknown or inactive account is rejected.

008 - Student and faculty accounts cannot access administrator actions, including when

009 requests are made directly rather than through the app.

010 - A faculty member cannot access another faculty member's subject roster or marks.

011 - A student cannot select another student's records using a request parameter.

012 - A tampered/expired token is rejected; expiration and logout return the app to login.

013 - Closing/restarting the app requires sign-in (tokens currently stay in memory).

015 ## Records and enrollment

016 - Create faculty, students and subjects; counts reflect active records.

017 - Duplicate IDs/emails and nonexistent faculty assignments are rejected.

018 - Enroll distinct students in a subject; verify the faculty roster matches exactly.

019 - Two administrators editing the same record cannot overwrite a newer revision.

020 - Deactivate a student; previous submitted records remain. Faculty assignment blocks

021 deactivation until subjects are reassigned.

022 - Rotate the device on create/edit forms; field values survive and navigation remains correct.

024 ## Attendance

025 - Submit present/absent/late/excused entries for a full roster and a valid date/session.

026 - Missing/duplicate students and future/invalid dates are rejected.

027 - Submit the same request twice and simulate a lost response; no second session is created.

028 - Verify present + late numerator, excused exclusion, zero denominator and newly enrolled students.

029 - Switch subjects quickly during report loading; export must match the displayed subject.

030 - Export names containing commas/quotes/newlines and formula-like prefixes; open the CSV

031 in a spreadsheet app and check both formatting and formula neutralization.

033 ## Marks

034 - Save a draft with zero, absent and pending; students see no draft marks.

035 - Reject values outside 0..maximum and attempts to publish pending marks.

036 - Publish complete marks; each student sees their own result only.

037 - Retry a save after losing the response; no duplicate assessment or revision is created.

038 - Concurrent draft edits detect stale revisions. Published results reject changes.

040 ## Mobile and operations

041 - Check narrow/large screens, large font sizes, TalkBack and keyboard navigation.

042 - Check network loss, service errors, retries, back while saving and unsaved-change dialogs.

043 - Confirm live Apps Script certificate fetch, cache rotation and lock behavior.

044 - Measure response times and quotas at expected journal size/concurrency.

045 - Restore a backup into a separate sheet and confirm records/audit metadata are intact.

046 - Rehearse deployment rollback and verify monitoring/support procedures.

048 No checks in this file have been marked complete solely from local mocks or compilation.

71. gradle.properties

gradle.properties

Gradle build property values.

3 source lines. SHA-256: 1952aa605790ec2a

The main file controls memory, encoding, configuration cache, and Kotlin style. The example supplies placeholders for the institution Apps Script base URL and Google web client ID.

Copy real values to a private configuration or environment. The base URL ends in / before Retrofit appends exec. Placeholders do not enable live login.

Source lines 1-3

```
0001 | org.gradle.jvmargs=-Xmx2048m -Dfile.encoding=UTF-8
0002 | org.gradle.configuration-cache=true
0003 | kotlin.code.style=official
```

L1-3: Defines the displayed configuration key, section, dependency, or literal value.

72. gradle.properties.example

gradle.properties.example

Gradle build property values.

2 source lines. SHA-256: 8901b3de1a5eae38

The main file controls memory, encoding, configuration cache, and Kotlin style. The example supplies placeholders for the institution Apps Script base URL and Google web client ID.

Copy real values to a private configuration or environment. The base URL ends in / before Retrofit appends exec. Placeholders do not enable live login.

Source lines 1-2

```
0001 | TRACKER_BASE_URL=https://script.google.com/macros/s/YOUR_VERSION_2_DEPLOYMENT_ID/  
0002 | GOOGLE_WEB_CLIENT_ID=YOUR_WEB_CLIENT_ID.apps.googleusercontent.com
```

L1-2: Defines the displayed configuration key, section, dependency, or literal value.

73. gradle-daemon-jvm.properties

gradle/gradle-daemon-jvm.properties

Gradle daemon JDK toolchain specification.

11 source lines. SHA-256: c70b10204afd88c3

Pins Java 21 and download locations by operating system and architecture. It governs the build process, not the app device runtime.

Source lines 1-11

```
0001 | toolchainUrl.FREE_BSD.AARCH64=https\://api.foojay.io/disco/v3.0/ids/ec7520a1e057cd116f9544c42142a16b
    > /redirect
0002 | toolchainUrl.FREE_BSD.X86_64=https\://api.foojay.io/disco/v3.0/ids/4c4f879899012ff0a8b2e2117df03b0e/
    > redirect
0003 | toolchainUrl.LINUX.AARCH64=https\://api.foojay.io/disco/v3.0/ids/ec7520a1e057cd116f9544c42142a16b/re
    > direct
0004 | toolchainUrl.LINUX.X86_64=https\://api.foojay.io/disco/v3.0/ids/4c4f879899012ff0a8b2e2117df03b0e/red
    > irect
0005 | toolchainUrl.MAC_OS.AARCH64=https\://api.foojay.io/disco/v3.0/ids/73bcfb608d1fde9fb62e462f834a3299/r
    > edirect
0006 | toolchainUrl.MAC_OS.X86_64=https\://api.foojay.io/disco/v3.0/ids/846ee0d876d26a26f37aa1ce8de73224/re
    > direct
0007 | toolchainUrl.UNIX.AARCH64=https\://api.foojay.io/disco/v3.0/ids/ec7520a1e057cd116f9544c42142a16b/red
    > irect
0008 | toolchainUrl.UNIX.X86_64=https\://api.foojay.io/disco/v3.0/ids/4c4f879899012ff0a8b2e2117df03b0e/redi
    > rect
0009 | toolchainUrl.WINDOWS.AARCH64=https\://api.foojay.io/disco/v3.0/ids/9482dddec596298c84656d31d16652665/
    > redirect
0010 | toolchainUrl.WINDOWS.X86_64=https\://api.foojay.io/disco/v3.0/ids/39701d92e1756bb2f141eb67cd4c660e/r
    > edirect
0011 | toolchainVersion=21
```

L1-11: Defines the displayed configuration key, section, dependency, or literal value.

74. libs.versions.toml

gradle/libs.versions.toml

Gradle version catalog.

31 source lines. SHA-256: 0cb81760ffe02ed1

[versions] defines shared versions. [libraries] maps an alias to a Maven module and version. [plugins] maps an alias to a plugin ID and version.

Gradle generates the libs accessors used in the build scripts. Update compatible versions deliberately; the catalog is not runtime data.

Source lines 1-31

```
0001 | [versions]
0002 | agp = "9.2.1"
0003 | coreKtx = "1.19.0"
0004 | junit = "4.13.2"
0005 | junitVersion = "1.1.5"
0006 | espressoCore = "3.5.1"
0007 | lifecycleRuntimeKtx = "2.11.0"
0008 | activityCompose = "1.13.0"
0009 | kotlin = "2.2.10"
0010 | composeBom = "2026.02.01"
0011 |
0012 | [libraries]
0013 | androidx-core-ktx = { group = "androidx.core", name = "core-ktx", version.ref = "coreKtx" }
0014 | junit = { group = "junit", name = "junit", version.ref = "junit" }
0015 | androidx-junit = { group = "androidx.test.ext", name = "junit", version.ref = "junitVersion" }
0016 | androidx-espresso-core = { group = "androidx.test.espresso", name = "espresso-core", version.ref =
> "espressoCore" }
0017 | androidx-lifecycle-runtime-ktx = { group = "androidx.lifecycle", name = "lifecycle-runtime-ktx",
> version.ref = "lifecycleRuntimeKtx" }
0018 | androidx-activity-compose = { group = "androidx.activity", name = "activity-compose", version.ref =
> "activityCompose" }
0019 | androidx-compose-bom = { group = "androidx.compose", name = "compose-bom", version.ref =
> "composeBom" }
0020 | androidx-compose-ui = { group = "androidx.compose.ui", name = "ui" }
0021 | androidx-compose-ui-graphics = { group = "androidx.compose.ui", name = "ui-graphics" }
0022 | androidx-compose-ui-tooling = { group = "androidx.compose.ui", name = "ui-tooling" }
0023 | androidx-compose-ui-tooling-preview = { group = "androidx.compose.ui", name = "ui-tooling-preview" }
0024 | androidx-compose-ui-test-manifest = { group = "androidx.compose.ui", name = "ui-test-manifest" }
0025 | androidx-compose-ui-test-junit4 = { group = "androidx.compose.ui", name = "ui-test-junit4" }
0026 | androidx-compose-material3 = { group = "androidx.compose.material3", name = "material3" }
0027 |
0028 | [plugins]
0029 | android-application = { id = "com.android.application", version.ref = "agp" }
0030 | kotlin-compose = { id = "org.jetbrains.kotlin.plugin.compose", version.ref = "kotlin" }
0031 |
```

L1-10: Defines the displayed configuration key, section, dependency, or literal value.

L12-26: Defines the displayed configuration key, section, dependency, or literal value.

L28-30: Defines the displayed configuration key, section, dependency, or literal value.

75. gradle-wrapper.properties

gradle/wrapper/gradle-wrapper.properties

Reproducible Gradle wrapper distribution settings.

8 source lines. SHA-256: 1c7e57bf89dc168e

Selects Gradle 9.4.1 binary distribution, cache locations, URL validation, and network timeout. The wrapper downloads this version rather than relying on a global installation.

Source lines 1-8

```
0001 | #Wed Jul 22 13:14:11 IST 2026
0002 | distributionBase=GRADLE_USER_HOME
0003 | distributionPath=wrapper/dists
0004 | distributionUrl=https\://services.gradle.org/distributions/gradle-9.4.1-bin.zip
0005 | networkTimeout=10000
0006 | validateDistributionUrl=true
0007 | zipStoreBase=GRADLE_USER_HOME
0008 | zipStorePath=wrapper/dists
```

L1: Infrastructure documentation or retained upstream notice; not an application operation.

L2-8: Defines the displayed configuration key, section, dependency, or literal value.

76. gradlew

gradlew

Standard Gradle launcher script.

251 source lines. SHA-256: 3238afb2aed5cb16

Resolves the project and Java executable, assembles JVM options and classpath, and launches GradleWrapperMain from gradle-wrapper.jar.

gradlew is for Unix shells; gradlew.bat is for Windows. Quoted Windows paths support this project directory containing spaces and an ampersand.

Upstream license notices and wrapper documentation are retained. These are build infrastructure, not application business logic.

Source lines 1-32

```
0001 | #!/bin/sh
0002 |
0003 | #
0004 | # Copyright © 2015 the original authors.
0005 | #
0006 | # Licensed under the Apache License, Version 2.0 (the "License");
0007 | # you may not use this file except in compliance with the License.
0008 | # You may obtain a copy of the License at
0009 | #
0010 | #     https://www.apache.org/licenses/LICENSE-2.0
0011 | #
0012 | # Unless required by applicable law or agreed to in writing, software
0013 | # distributed under the License is distributed on an "AS IS" BASIS,
0014 | # WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
0015 | # See the License for the specific language governing permissions and
0016 | # limitations under the License.
0017 | #
0018 | # SPDX-License-Identifier: Apache-2.0
0019 | #
0020 |
0021 | #####
0022 | #
0023 | #   Gradle start up script for POSIX generated by Gradle.
0024 | #
0025 | #   Important for running:
0026 | #
0027 | #   (1) You need a POSIX-compliant shell to run this script. If your /bin/sh is
0028 | #       noncompliant, but you have some other compliant shell such as ksh or
0029 | #       bash, then to run this script, type that shell name before the whole
0030 | #       command line, like:
0031 | #
0032 | #           ksh Gradle
```

L1: Infrastructure documentation or retained upstream notice; not an application operation.

L3-19: Infrastructure documentation or retained upstream notice; not an application operation.

L21-32: Infrastructure documentation or retained upstream notice; not an application operation.

Source lines 33-64

```
0033 | #
0034 | #   Busybox and similar reduced shells will NOT work, because this script
0035 | #   requires all of these POSIX shell features:
0036 | #       * functions;
0037 | #       * expansions «$var», «${var}», «${var:-default}», «${var+SET}»,
0038 | #         «${var#prefix}», «${var%suffix}», and «$( cmd )»;
0039 | #       * compound commands having a testable exit status, especially «case»;
0040 | #       * various built-in commands including «command», «set», and «ulimit».
0041 | #
0042 | #   Important for patching:
0043 | #
0044 | #   (2) This script targets any POSIX shell, so it avoids extensions provided
```

```

0045 | #      by Bash, Ksh, etc; in particular arrays are avoided.
0046 | #
0047 | #      The "traditional" practice of packing multiple parameters into a
0048 | #      space-separated string is a well documented source of bugs and security
0049 | #      problems, so this is (mostly) avoided, by progressively accumulating
0050 | #      options in "$@", and eventually passing that to Java.
0051 | #
0052 | #      Where the inherited environment variables (DEFAULT_JVM_OPTS, JAVA_OPTS,
0053 | #      and GRADLE_OPTS) rely on word-splitting, this is performed explicitly;
0054 | #      see the in-line comments for details.
0055 | #
0056 | #      There are tweaks for specific operating systems such as AIX, CygWin,
0057 | #      Darwin, MinGW, and NonStop.
0058 | #
0059 | #      (3) This script is generated from the Groovy template
0060 | #      https://github.com/gradle/gradle/blob/HEAD/platforms/jvm/plugins-application/src/main/resour
0061 | #      ces/org/gradle/api/internal/plugins/unixStartScript.txt
0062 | #      within the Gradle project.
0063 | #      You can find Gradle at https://github.com/gradle/gradle/.
0064 | #

```

L33-64: Infrastructure documentation or retained upstream notice; not an application operation.

Source lines 65-96

```

0065 | #####
0066 |
0067 | # Attempt to set APP_HOME
0068 |
0069 | # Resolve links: $0 may be a link
0070 | app_path=$0
0071 |
0072 | # Need this for daisy-chained symlinks.
0073 | while
0074 |     APP_HOME=${app_path%"${app_path##*/}"} # leaves a trailing /; empty if no leading path
0075 |     [ -h "$app_path" ]
0076 | do
0077 |     ls=$( ls -ld "$app_path" )
0078 |     link=${ls#*' -> '}
0079 |     case $link in
0080 |         /*)   app_path=$link ;;
0081 |         *)    app_path=$APP_HOME$link ;;
0082 |     esac
0083 | done
0084 |
0085 | # This is normally unused
0086 | # shellcheck disable=SC2034
0087 | APP_BASE_NAME=${0##*/}
0088 | # Discard cd standard output in case $CDPATH is set (https://github.com/gradle/gradle/issues/25036)
0089 | APP_HOME=$( cd -P "${APP_HOME:-.}" > /dev/null && printf '%s\n' "$PWD" ) || exit
0090 |
0091 | # Use the maximum available, or set MAX_FD != -1 to use that value.
0092 | MAX_FD=maximum
0093 |
0094 | warn () {
0095 |     echo "$*"
0096 | } >&2

```

L65: Infrastructure documentation or retained upstream notice; not an application operation.

L67: Infrastructure documentation or retained upstream notice; not an application operation.

L69: Infrastructure documentation or retained upstream notice; not an application operation.

L70: Participates in Java/path discovery, option handling, or the wrapper launch described in this file overview.

L72: Infrastructure documentation or retained upstream notice; not an application operation.

L73-79: Participates in Java/path discovery, option handling, or the wrapper launch described in this file overview.

L80-81: Infrastructure documentation or retained upstream notice; not an application operation.

L82-83: Participates in Java/path discovery, option handling, or the wrapper launch described in this file overview.

L85-86: Infrastructure documentation or retained upstream notice; not an application operation.

L87: Participates in Java/path discovery, option handling, or the wrapper launch described in this file overview.

L88: Infrastructure documentation or retained upstream notice; not an application operation.

L89: Participates in Java/path discovery, option handling, or the wrapper launch described in this file overview.

L91: Infrastructure documentation or retained upstream notice; not an application operation.

L92: Participates in Java/path discovery, option handling, or the wrapper launch described in this file overview.

L94-96: Participates in Java/path discovery, option handling, or the wrapper launch described in this file overview.

Source lines 97-128

```
0097 |  
0098 | die () {  
0099 |     echo  
0100 |     echo "$*"  
0101 |     echo  
0102 |     exit 1  
0103 | } >&2  
0104 |  
0105 | # OS specific support (must be 'true' or 'false').  
0106 | cygwin=false  
0107 | msys=false  
0108 | darwin=false  
0109 | nonstop=false  
0110 | case "$( uname )" in           #(   
0111 |     CYGWIN* )                 cygwin=true  ;; #(   
0112 |     Darwin* )                 darwin=true  ;; #(   
0113 |     MSYS* | MINGW* )          msys=true   ;; #(   
0114 |     NONSTOP* )                nonstop=true ;;   
0115 | esac  
0116 |  
0117 | CLASSPATH="\\\\"  
0118 |  
0119 |  
0120 | # Determine the Java command to use to start the JVM.  
0121 | if [ -n "$JAVA_HOME" ] ; then  
0122 |     if [ -x "$JAVA_HOME/jre/sh/java" ] ; then  
0123 |         # IBM's JDK on AIX uses strange locations for the executables  
0124 |         JAVACMD="$JAVA_HOME/jre/sh/java"  
0125 |     else  
0126 |         JAVACMD="$JAVA_HOME/bin/java"  
0127 |     fi  
0128 |     if [ ! -x "$JAVACMD" ] ; then
```

L98-103: Participates in Java/path discovery, option handling, or the wrapper launch described in this file overview.

L105: Infrastructure documentation or retained upstream notice; not an application operation.

L106-115: Participates in Java/path discovery, option handling, or the wrapper launch described in this file overview.

L117: Participates in Java/path discovery, option handling, or the wrapper launch described in this file overview.

L120: Infrastructure documentation or retained upstream notice; not an application operation.

L121-122: Branches on the displayed condition; the accepted branch performs the following operation.

L123: Infrastructure documentation or retained upstream notice; not an application operation.

L124-127: Participates in Java/path discovery, option handling, or the wrapper launch described in this file overview.

L128: Branches on the displayed condition; the accepted branch performs the following operation.

Source lines 129-160

```
0129 |         die "ERROR: JAVA_HOME is set to an invalid directory: $JAVA_HOME"  
0130 |  
0131 | Please set the JAVA_HOME variable in your environment to match the  
0132 | location of your Java installation."  
0133 |     fi  
0134 | else  
0135 |     JAVACMD=java  
0136 |     if ! command -v java >/dev/null 2>&1  
0137 |     then  
0138 |         die "ERROR: JAVA_HOME is not set and no 'java' command could be found in your PATH."  
0139 |  
0140 | Please set the JAVA_HOME variable in your environment to match the  
0141 | location of your Java installation."  
0142 |     fi  
0143 | fi
```

```

0144 |
0145 | # Increase the maximum file descriptors if we can.
0146 | if ! "$cygwin" && ! "$darwin" && ! "$nonstop" ; then
0147 |     case $MAX_FD in #(
0148 |         max*)
0149 |             # In POSIX sh, ulimit -H is undefined. That's why the result is checked to see if it worked.
0150 |             # shellcheck disable=SC2039,SC3045
0151 |             MAX_FD=$( ulimit -H -n ) ||
0152 |                 warn "Could not query maximum file descriptor limit"
0153 |         esac
0154 |     case $MAX_FD in #(
0155 |         '' | soft) ;; #(
0156 |         *)
0157 |             # In POSIX sh, ulimit -n is undefined. That's why the result is checked to see if it worked.
0158 |             # shellcheck disable=SC2039,SC3045
0159 |             ulimit -n "$MAX_FD" ||
0160 |                 warn "Could not set maximum file descriptor limit to $MAX_FD"

```

L129: Participates in Java/path discovery, option handling, or the wrapper launch described in this file overview.

L131-135: Participates in Java/path discovery, option handling, or the wrapper launch described in this file overview.

L136: Branches on the displayed condition; the accepted branch performs the following operation.

L137-138: Participates in Java/path discovery, option handling, or the wrapper launch described in this file overview.

L140-143: Participates in Java/path discovery, option handling, or the wrapper launch described in this file overview.

L145: Infrastructure documentation or retained upstream notice; not an application operation.

L146: Branches on the displayed condition; the accepted branch performs the following operation.

L147-148: Participates in Java/path discovery, option handling, or the wrapper launch described in this file overview.

L149-150: Infrastructure documentation or retained upstream notice; not an application operation.

L151-155: Participates in Java/path discovery, option handling, or the wrapper launch described in this file overview.

L156-158: Infrastructure documentation or retained upstream notice; not an application operation.

L159-160: Participates in Java/path discovery, option handling, or the wrapper launch described in this file overview.

Source lines 161-192

```

0161 |     esac
0162 | fi
0163 |
0164 | # Collect all arguments for the java command, stacking in reverse order:
0165 | # * args from the command line
0166 | # * the main class name
0167 | # * -classpath
0168 | # * -D...appname settings
0169 | # * --module-path (only if needed)
0170 | # * DEFAULT_JVM_OPTS, JAVA_OPTS, and GRADLE_OPTS environment variables.
0171 |
0172 | # For Cygwin or MSYS, switch paths to Windows format before running java
0173 | if "$cygwin" || "$msys" ; then
0174 |     APP_HOME=$( cygpath --path --mixed "$APP_HOME" )
0175 |     CLASSPATH=$( cygpath --path --mixed "$CLASSPATH" )
0176 |
0177 |     JAVACMD=$( cygpath --unix "$JAVACMD" )
0178 |
0179 |     # Now convert the arguments - kludge to limit ourselves to /bin/sh
0180 |     for arg do
0181 |         if
0182 |             case $arg in
0183 |                 -*) false ;;
0184 |                 /*) t=${arg#/} t=${t%/*}
0185 |                     [ -e "$t" ] ;;
0186 |                 *) false ;;
0187 |             esac
0188 |         then
0189 |             arg=$( cygpath --path --ignore --mixed "$arg" )
0190 |         fi
0191 |         # Roll the args list around exactly as many times as the number of
0192 |         # args, so each arg winds up back in the position where it started, but

```

L161-162: Participates in Java/path discovery, option handling, or the wrapper launch described in this file overview.

L164-165: Infrastructure documentation or retained upstream notice; not an application operation.

L166: Declares the named type; constructor properties define its stored data or dependencies.

L167-170: Infrastructure documentation or retained upstream notice; not an application operation.

L172: Infrastructure documentation or retained upstream notice; not an application operation.

L173: Branches on the displayed condition; the accepted branch performs the following operation.

L174-175: Participates in Java/path discovery, option handling, or the wrapper launch described in this file overview.

L177: Participates in Java/path discovery, option handling, or the wrapper launch described in this file overview.

L179: Infrastructure documentation or retained upstream notice; not an application operation.

L180: Participates in Java/path discovery, option handling, or the wrapper launch described in this file overview.

L181: Branches on the displayed condition; the accepted branch performs the following operation.

L182-185: Participates in Java/path discovery, option handling, or the wrapper launch described in this file overview.

L186: Infrastructure documentation or retained upstream notice; not an application operation.

L187-190: Participates in Java/path discovery, option handling, or the wrapper launch described in this file overview.

L191-192: Infrastructure documentation or retained upstream notice; not an application operation.

Source lines 193-224

```
0193 |         # possibly modified.
0194 |         #
0195 |         # NB: a `for` loop captures its iteration list before it begins, so
0196 |         # changing the positional parameters here affects neither the number of
0197 |         # iterations, nor the values presented in `arg`.
0198 |         shift                # remove old arg
0199 |         set -- "$@" "$arg"    # push replacement arg
0200 |     done
0201 | fi
0202 |
0203 |
0204 | # Add default JVM options here. You can also use JAVA_OPTS and GRADLE_OPTS to pass JVM options to
    > this script.
0205 | DEFAULT_JVM_OPTS="-Xmx64m" "-Xms64m"
0206 |
0207 | # Collect all arguments for the java command:
0208 | # * DEFAULT_JVM_OPTS, JAVA_OPTS, and optsEnvironmentVar are not allowed to contain shell
    > fragments,
0209 | #   and any embedded shellness will be escaped.
0210 | # * For example: A user cannot expect ${Hostname} to be expanded, as it is an environment variable
    > and will be
0211 | #   treated as '${Hostname}' itself on the command line.
0212 |
0213 | set -- \
0214 |     "-Dorg.gradle.appname=$APP_BASE_NAME" \
0215 |     -classpath "$CLASSPATH" \
0216 |     -jar "$APP_HOME/gradle/wrapper/gradle-wrapper.jar" \
0217 |     "$@"
0218 |
0219 | # Stop when "xargs" is not available.
0220 | if ! command -v xargs >/dev/null 2>&1
0221 | then
0222 |     die "xargs is not available"
0223 | fi
0224 |
```

L193-197: Infrastructure documentation or retained upstream notice; not an application operation.

L198-201: Participates in Java/path discovery, option handling, or the wrapper launch described in this file overview.

L204: Infrastructure documentation or retained upstream notice; not an application operation.

L205: Participates in Java/path discovery, option handling, or the wrapper launch described in this file overview.

L207-211: Infrastructure documentation or retained upstream notice; not an application operation.

L213-217: Participates in Java/path discovery, option handling, or the wrapper launch described in this file overview.

L219: Infrastructure documentation or retained upstream notice; not an application operation.

L220: Branches on the displayed condition; the accepted branch performs the following operation.

L221-223: Participates in Java/path discovery, option handling, or the wrapper launch described in this file overview.

Source lines 225-251


```

0225 | # Use "xargs" to parse quoted args.
0226 | #
0227 | # With -n1 it outputs one arg per line, with the quotes and backslashes removed.
0228 | #
0229 | # In Bash we could simply go:
0230 | #
0231 | #   readarray ARGS < <( xargs -n1 <<<"$var" ) &&
0232 | #   set -- "${ARGS[@]}" "$@"
0233 | #
0234 | # but POSIX shell has neither arrays nor command substitution, so instead we
0235 | # post-process each arg (as a line of input to sed) to backslash-escape any
0236 | # character that might be a shell metacharacter, then use eval to reverse
0237 | # that process (while maintaining the separation between arguments), and wrap
0238 | # the whole thing up as a single "set" statement.
0239 | #
0240 | # This will of course break if any of these variables contains a newline or
0241 | # an unmatched quote.
0242 | #
0243 |
0244 | eval "set -- $(
0245 |     printf '%s\n' "$DEFAULT_JVM_OPTS $JAVA_OPTS $GRADLE_OPTS" |
0246 |     xargs -n1 |
0247 |     sed ' s~^[^[:alnum:]]+,./*:=@_~\\&~g; ' |
0248 |     tr '\n' ' '
0249 | )" "$@"
0250 |
0251 | exec "$JAVACMD" "$@"

```

L225-242: Infrastructure documentation or retained upstream notice; not an application operation.

L244-249: Participates in Java/path discovery, option handling, or the wrapper launch described in this file overview.

L251: Participates in Java/path discovery, option handling, or the wrapper launch described in this file overview.

77. gradlew.bat

gradlew.bat

Standard Gradle launcher script.

94 source lines. SHA-256: f8a59bd1f2a48f47

Resolves the project and Java executable, assembles JVM options and classpath, and launches GradleWrapperMain from gradle-wrapper.jar.

gradlew is for Unix shells; gradlew.bat is for Windows. Quoted Windows paths support this project directory containing spaces and an ampersand.

Upstream license notices and wrapper documentation are retained. These are build infrastructure, not application business logic.

Source lines 1-32

```
0001 | @rem
0002 | @rem Copyright 2015 the original author or authors.
0003 | @rem
0004 | @rem Licensed under the Apache License, Version 2.0 (the "License");
0005 | @rem you may not use this file except in compliance with the License.
0006 | @rem You may obtain a copy of the License at
0007 | @rem
0008 | @rem      https://www.apache.org/licenses/LICENSE-2.0
0009 | @rem
0010 | @rem Unless required by applicable law or agreed to in writing, software
0011 | @rem distributed under the License is distributed on an "AS IS" BASIS,
0012 | @rem WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
0013 | @rem See the License for the specific language governing permissions and
0014 | @rem limitations under the License.
0015 | @rem
0016 | @rem SPDX-License-Identifier: Apache-2.0
0017 | @rem
0018 |
0019 | @if "%DEBUG%"==" " @echo off
0020 | @rem #####
0021 | @rem
0022 | @rem Gradle startup script for Windows
0023 | @rem
0024 | @rem #####
0025 |
0026 | @rem Set local scope for the variables with windows NT shell
0027 | if "%OS%"=="Windows_NT" setlocal
0028 |
0029 | set "DIRNAME=%~dp0"
0030 | if "%DIRNAME%"==" " set DIRNAME=.
0031 | @rem This is normally unused
0032 | set "APP_BASE_NAME=%~n0"
```

L1-17: Applies the displayed annotation to the declaration.

L19-24: Applies the displayed annotation to the declaration.

L26: Applies the displayed annotation to the declaration.

L27: Branches on the displayed condition; the accepted branch performs the following operation.

L29: Participates in Java/path discovery, option handling, or the wrapper launch described in this file overview.

L30: Branches on the displayed condition; the accepted branch performs the following operation.

L31: Applies the displayed annotation to the declaration.

L32: Participates in Java/path discovery, option handling, or the wrapper launch described in this file overview.

Source lines 33-64

```
0033 | set "APP_HOME=%DIRNAME%"
0034 |
0035 | @rem Resolve any "." and ".." in APP_HOME to make it shorter.
0036 | for %i in ("%APP_HOME%") do set "APP_HOME=%~fi"
0037 |
```

```

0038 | @rem Add default JVM options here. You can also use JAVA_OPTS and GRADLE_OPTS to pass JVM options to
    > this script.
0039 | set DEFAULT_JVM_OPTS="-Xmx64m" "-Xms64m"
0040 |
0041 | @rem Find java.exe
0042 | if defined JAVA_HOME goto findJavaFromJavaHome
0043 |
0044 | set JAVA_EXE=java.exe
0045 | %JAVA_EXE% -version >NUL 2>&1
0046 | if %ERRORLEVEL% equ 0 goto execute
0047 |
0048 | echo. 1>&2
0049 | echo ERROR: JAVA_HOME is not set and no 'java' command could be found in your PATH. 1>&2
0050 | echo. 1>&2
0051 | echo Please set the JAVA_HOME variable in your environment to match the 1>&2
0052 | echo location of your Java installation. 1>&2
0053 |
0054 | goto fail
0055 |
0056 | :findJavaFromJavaHome
0057 | set JAVA_HOME=%JAVA_HOME:="%\%"
0058 | set "JAVA_EXE=%JAVA_HOME%/bin/java.exe"
0059 |
0060 | if exist "%JAVA_EXE%" goto execute
0061 |
0062 | echo. 1>&2
0063 | echo ERROR: JAVA_HOME is set to an invalid directory: %JAVA_HOME% 1>&2
0064 | echo. 1>&2

```

L33: Participates in Java/path discovery, option handling, or the wrapper launch described in this file overview.

L35: Applies the displayed annotation to the declaration.

L36: Participates in Java/path discovery, option handling, or the wrapper launch described in this file overview.

L38: Applies the displayed annotation to the declaration.

L39: Participates in Java/path discovery, option handling, or the wrapper launch described in this file overview.

L41: Applies the displayed annotation to the declaration.

L42: Branches on the displayed condition; the accepted branch performs the following operation.

L44-45: Participates in Java/path discovery, option handling, or the wrapper launch described in this file overview.

L46: Branches on the displayed condition; the accepted branch performs the following operation.

L48-52: Participates in Java/path discovery, option handling, or the wrapper launch described in this file overview.

L54: Participates in Java/path discovery, option handling, or the wrapper launch described in this file overview.

L56-58: Participates in Java/path discovery, option handling, or the wrapper launch described in this file overview.

L60: Branches on the displayed condition; the accepted branch performs the following operation.

L62-64: Participates in Java/path discovery, option handling, or the wrapper launch described in this file overview.

Source lines 65-94

```

0065 | echo Please set the JAVA_HOME variable in your environment to match the 1>&2
0066 | echo location of your Java installation. 1>&2
0067 |
0068 | goto fail
0069 |
0070 | :execute
0071 | @rem Setup the command line
0072 |
0073 | set CLASSPATH=
0074 |
0075 |
0076 | @rem Execute Gradle
0077 | "%JAVA_EXE%" %DEFAULT_JVM_OPTS% %JAVA_OPTS% %GRADLE_OPTS% "-Dorg.gradle.appname=%APP_BASE_NAME%"
    > -classpath "%CLASSPATH%" -jar "%APP_HOME%\gradle\wrapper\gradle-wrapper.jar" %*
0078 |
0079 | :end
0080 | @rem End local scope for the variables with windows NT shell
0081 | if %ERRORLEVEL% equ 0 goto mainEnd
0082 |
0083 | :fail
0084 | rem Set variable GRADLE_EXIT_CONSOLE if you need the _script_ return code instead of

```

```

0085 | rem the _cmd.exe /c_ return code!
0086 | set EXIT_CODE=%ERRORLEVEL%
0087 | if %EXIT_CODE% equ 0 set EXIT_CODE=1
0088 | if not ""=="%GRADLE_EXIT_CONSOLE%" exit %EXIT_CODE%
0089 | exit /b %EXIT_CODE%
0090 |
0091 | :mainEnd
0092 | if "%OS%"=="Windows_NT" endlocal
0093 |
0094 | :omega

```

L65-66: Participates in Java/path discovery, option handling, or the wrapper launch described in this file overview.

L68: Participates in Java/path discovery, option handling, or the wrapper launch described in this file overview.

L70: Participates in Java/path discovery, option handling, or the wrapper launch described in this file overview.

L71: Applies the displayed annotation to the declaration.

L73: Participates in Java/path discovery, option handling, or the wrapper launch described in this file overview.

L76: Applies the displayed annotation to the declaration.

L77: Participates in Java/path discovery, option handling, or the wrapper launch described in this file overview.

L79: Participates in Java/path discovery, option handling, or the wrapper launch described in this file overview.

L80: Applies the displayed annotation to the declaration.

L81: Branches on the displayed condition; the accepted branch performs the following operation.

L83: Participates in Java/path discovery, option handling, or the wrapper launch described in this file overview.

L84-85: Infrastructure documentation or retained upstream notice; not an application operation.

L86: Participates in Java/path discovery, option handling, or the wrapper launch described in this file overview.

L87-88: Branches on the displayed condition; the accepted branch performs the following operation.

L89: Participates in Java/path discovery, option handling, or the wrapper launch described in this file overview.

L91: Participates in Java/path discovery, option handling, or the wrapper launch described in this file overview.

L92: Branches on the displayed condition; the accepted branch performs the following operation.

L94: Participates in Java/path discovery, option handling, or the wrapper launch described in this file overview.

78. settings.gradle.kts

settings.gradle.kts

Gradle project topology and dependency repositories.

27 source lines. SHA-256: 4fd15c967b837578

pluginManagement configures plugin repositories. The Foojay resolver supports Java toolchain discovery.

dependencyResolutionManagement centralizes repositories and rejects project-level repositories. rootProject.name names the project; include(":app") adds the Android module.

Source lines 1-27

```
0001 | pluginManagement {
0002 |     repositories {
0003 |         google {
0004 |             content {
0005 |                 includeGroupByRegex("com\\.android.*")
0006 |                 includeGroupByRegex("com\\.google.*")
0007 |                 includeGroupByRegex("androidx.*")
0008 |             }
0009 |         }
0010 |         mavenCentral()
0011 |         gradlePluginPortal()
0012 |     }
0013 | }
0014 | plugins {
0015 |     id("org.gradle.toolchains.foojay-resolver-convention") version "1.0.0"
0016 | }
0017 | dependencyResolutionManagement {
0018 |     repositoriesMode.set(RepositoriesMode.FAIL_ON_PROJECT_REPOS)
0019 |     repositories {
0020 |         google()
0021 |         mavenCentral()
0022 |         maven { url = uri("https://jitpack.io") }
0023 |     }
0024 | }
0025 |
0026 | rootProject.name = "Student tracker"
0027 | include(":app")
```

L1-7: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L8-9: Closes or continues the surrounding block, expression, or call.

L10-11: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L12-13: Closes or continues the surrounding block, expression, or call.

L14-15: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L16: Closes or continues the surrounding block, expression, or call.

L17-22: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

L23-24: Closes or continues the surrounding block, expression, or call.

L26: Assigns the displayed expression to a variable or named argument.

L27: Continues the surrounding call, condition, declaration, or data structure using the displayed value.

Binary assets and generated infrastructure

Binary files contain encoded assets or compiled tooling rather than editable source statements. Every project binary asset is listed here.

app/src/main/res/mipmap-hdpi/ic_launcher.webp (1,404 bytes): Raster launcher icon. The directory suffix is its Android density bucket; round variants supply the round icon.

app/src/main/res/mipmap-hdpi/ic_launcher_round.webp (2,898 bytes): Raster launcher icon. The directory suffix is its Android density bucket; round variants supply the round icon.

app/src/main/res/mipmap-mdpi/ic_launcher.webp (982 bytes): Raster launcher icon. The directory suffix is its Android density bucket; round variants supply the round icon.

app/src/main/res/mipmap-mdpi/ic_launcher_round.webp (1,772 bytes): Raster launcher icon. The directory suffix is its Android density bucket; round variants supply the round icon.

app/src/main/res/mipmap-xhdpi/ic_launcher.webp (1,900 bytes): Raster launcher icon. The directory suffix is its Android density bucket; round variants supply the round icon.

app/src/main/res/mipmap-xhdpi/ic_launcher_round.webp (3,918 bytes): Raster launcher icon. The directory suffix is its Android density bucket; round variants supply the round icon.

app/src/main/res/mipmap-xxhdpi/ic_launcher.webp (2,884 bytes): Raster launcher icon. The directory suffix is its Android density bucket; round variants supply the round icon.

app/src/main/res/mipmap-xxhdpi/ic_launcher_round.webp (5,914 bytes): Raster launcher icon. The directory suffix is its Android density bucket; round variants supply the round icon.

app/src/main/res/mipmap-xxxhdpi/ic_launcher.webp (3,844 bytes): Raster launcher icon. The directory suffix is its Android density bucket; round variants supply the round icon.

app/src/main/res/mipmap-xxxhdpi/ic_launcher_round.webp (7,778 bytes): Raster launcher icon. The directory suffix is its Android density bucket; round variants supply the round icon.

gradle/wrapper/gradle-wrapper.jar (45,457 bytes): Compiled Gradle bootstrap classes used by the launcher scripts.

backend/dist/Code.gs: generated deployable script. Authored core.js, auth.js, and apps-script.js are completely reproduced in their chapters; the node-forge browser dependency retains its upstream license.

backend/dist/appsscript.json: generated manifest uses timeZone Asia/Kolkata, runtimeVersion V8, exceptionLogging STACKDRIVER, and spreadsheet/external-request scopes.

backend/dist/THIRD_PARTY_LICENSE.txt: generated node-forge attribution. Do not remove required third-party notices.

node_modules/: installed npm dependencies. package.json and package-lock.json are explained; dependency implementation is not project-authored code.

app/build/ and build/: compiled APK, generated BuildConfig, resource intermediates, test and lint reports. These are regenerated by Gradle.

.gradle/ and .kotlin/: tool caches. .git/: revision-control data. .idea/: personal Android Studio settings, including existing user changes; not academic logic.

local.properties: private machine SDK location, intentionally omitted from listings. tmp/pdfs/: guide-generation and visual QA artifacts. output/pdf/: final presentation documents. docs/PRESENTATION_SCRIPT.md: editable companion to the separate presentation PDF.

Explain a file in four sentences

Say what the file owns, which values it receives, what it does with those values, and how the result or failure is used. For example: AttendanceScreen owns the attendance form. It receives a subject code and loads the roster. It collects statuses and sends a validated session request. Success locks the form; failure displays an error, while server validation protects the stored record.

Questions to practice

Trace login from CredentialManager to principalFor. Trace attendance from button to journal row. Explain why a token must be signed and why its audience matters. Explain UI state versus database storage. Explain revisions, locking, and request IDs separately. Explain zero versus pending versus absent. Explain why append-only history costs more at scale. State which checks passed and which live checks remain.

Use Presentation_Script.pdf for the speaking script and concise viva answers. This guide is a source reference, not a claim that the institution deployment has been completed.